

UDP Ethernet Performance Server using VITIS 2024.2 for VCU118 Board

תוכן עניינים:

2	מבוא
2	דרישות קדם
2	שלב 1: יצירת פרויקט חדש ב-Vivado
3	שלב 2: הגדרת שם ומיקום הפרויקט
3	שלב 3: בחירת סוג פרויקט RTL
4	שלב 4: בחירת לוח VCU118
4	שלב 5: יצירת Block Design
5	שלב 6: הוספת MicroBlaze
5	שלב 7: Run Block Automation והגדרת הזיכרון המקומי
6	שלב 8: הגדרת Clocking Wizard
8	שלב 9: Run Connection Automation
8	שלב 10: הוספת Tri Mode Ethernet MAC ו-PCS/PMA
9	שלב 11: הגדרת רכיב ה-PCS/PMA
14	שלב 12: הגדרת ה-MAC, השעונים והאיפוס
17	שלב 13: הוספת AXI DMA
21	שלב 14: אריזת מודול ה-UOE כרכיב IP
38	שלב 15: עדכון תשתיות השרטוט וה-Interconnect
41	שלב 16: יצירת רכיב TUSER מותאם
49	שלב 17: הוספת AXI Timer ו-UART
55	שלב 18: הוספת רכיב ה-Splitter
61	שלב 19: חיבור ה-MAC לרכיב ה-PCS/PMA
64	שלב 20: הכנות לבדיקה ורכיב ה-ILA
81	שלב 21: יצירת מעטפת ראשית והוספת קובץ אילוצים
84	שלב 22: הפעלת JTAG UART ברכיב ה-MDM
98	שלב 23: ייבוא החומרה ל-Vitis וכתיבת קוד הרשת
112	שלב 24: חיבור פיזי והגדרת כתובת IP סטטית
114	שלב 25: אימות מקדים של פענוח החבילות
116	שלב 26: בדיקות עומס ו-Loopback
136	שלב 27: ניטור אותות באמצעות ה-ILA

מבוא:

פרוטוקול UDP הוא פרוטוקול תקשורת מהיר ויעיל המאפשר שידור נתונים ללא צורך בהקמת חיבור או שליחת אישורי קבלה, מה שהופך אותו לאידיאלי להזרמת נתונים בקצבים עצומים. במדריך זה נשתמש בלוח הפיתוח המתקדם VCU118 של חברת AMD/Xilinx המבוסס על ארכיטקטורת Ultrascale+ בניגוד למערכות מבוססות Zynq שבהן הרשת מנוהלת על ידי מעבד קשיח, PS, בפרויקט זה נממש את כל מסלול הנתונים והתקשורת ישירות על גבי הלוגיקה המתוכנתת PL באמצעות MicroBlaze ומנוע UDP Offload Engine ייעודי. המטרה שלנו היא לנצל את כוח העיבוד המקבילי של ה-FPGA - כדי להגיע למהירות תקשורת מקסימלית של כמעט 1Gbps.

דרישות קדם:

חומרה:

לוח פיתוח מבוסס Xilinx VCU118

כבל רשת תקני RJ45 Gigabit

כבל צריבה Xilinx JTAG

כבל חיבור סריאלי USB-to-UART למחשב

ספק כוח מתאים ללוח 12וולט

תוכנה:

Vivado Design Suite 2024.2

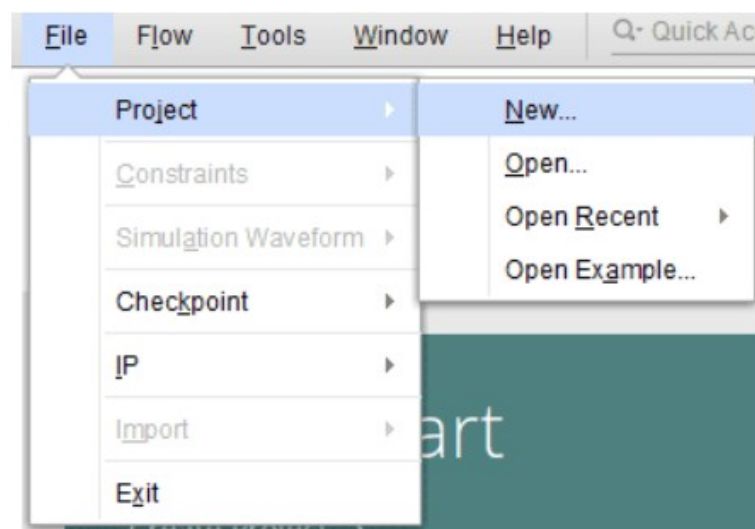
Vitis Unified IDE 2024.2

תוכנת טרמינל כמו PuTTY או Tera Term

תוכנת Wireshark לניתוח תעבורת רשת לצורך אימות קבלת הנתונים

שלב 1:

נפתח את תוכנת Vivado, ניגש לתפריט File ונבחר Project ואז New. חלון יצירת פרויקט חדש ייפתח. נלחץ Next.



שלב 2:

נזין שם לפרויקט ונשמור אותו במיקום מתאים במחשב. נסמן את האפשרות ליצירת תת תיקייה לפרויקט. נלחץ Next.

New Project

Project Name
Enter a name for your project and specify a directory where the project data files will be stored.

Project name: UDP_Eth_System_v3

Project location: N:/412/412_Comp_DevTeam/Yair_Osh/Yair_Tayri

☒ Create project subdirectory

Project will be created at: N:/412/412_Comp_DevTeam/Yair_Osh/Yair_Tayri/UDP_Eth_System_v3

< Back Next > Finish Cancel

שלב 3:

בחלון סוג הפרויקט, נבחר RTL Project ונסמן את האפשרות שלא להגדיר קבצי מקור כרגע. נלחץ Next.

New Project

Project Type
Specify the type of project to create.

☒ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.
☒ Do not specify sources at this time
☐ Project is an extensible Ylts platform

☐ **Post-synthesis Project**
You will be able to add sources, view device resources, run design analysis, planning and implementation.
☐ Do not specify sources at this time

☐ **I/O Planning Project**
Do not specify design sources. You will be able to view part/package resources.

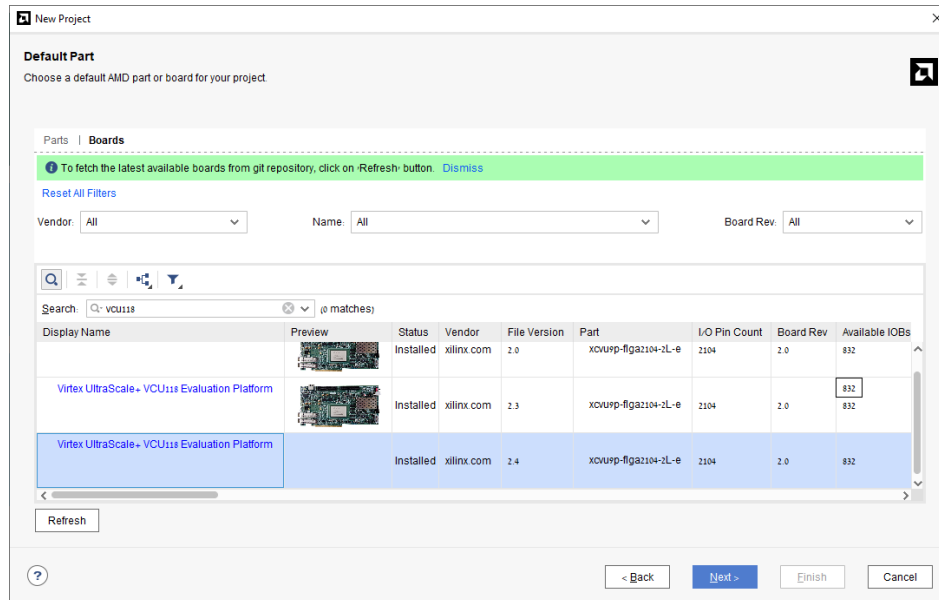
☐ **Imported Project**
Create a Vivado project from a Synplify Project File.

☐ **Example Project**
Create a new Vivado project from a predefined template.

< Back Next > Finish Cancel

שלב 4:

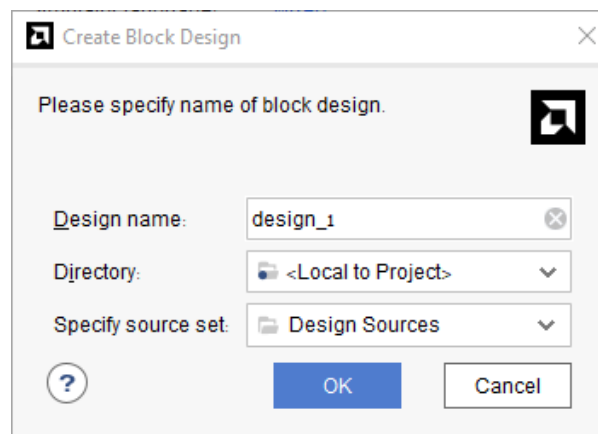
בחלון בחירת החלק נוודא שאנחנו נמצאים בלשונית ה- Boards. מתוך הרשימה שתופיע נחפש ונבחר את לוח ה- VCU118 Evaluation Platform. בחירה בלוח במקום ברכיב הספציפי מאפשרת ל- Vivado לטעון מראש הגדרות חומרה רלוונטיות כמו מתחי זיכרון ותדירים בסיסיים.



נלחץ Next ואז Finish כדי לסיים. תוכנת Vivado תייצר כעת את הפרויקט עם ההגדרות שבחרנו.

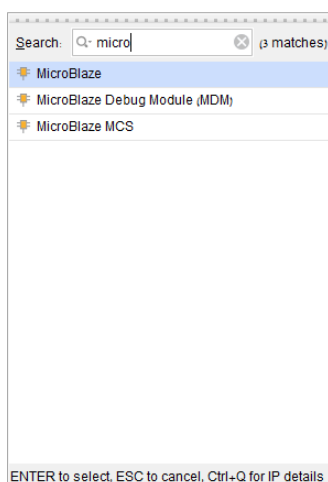
שלב 5:

בסרגל הכלים השמאלי תחת IP INTEGRATOR נבחר Create Block Design. ניתן שם לשרטוט ונלחץ OK. ייוצר שרטוט ריק לחלוטין.



שלב 6:

ניגש לחלון השרטוט, נלחץ קליק ימני ונבחר Add IP. נחפש את הרכיב MicroBlaze ונוסיף אותו לשרטוט על ידי לחיצה כפולה.



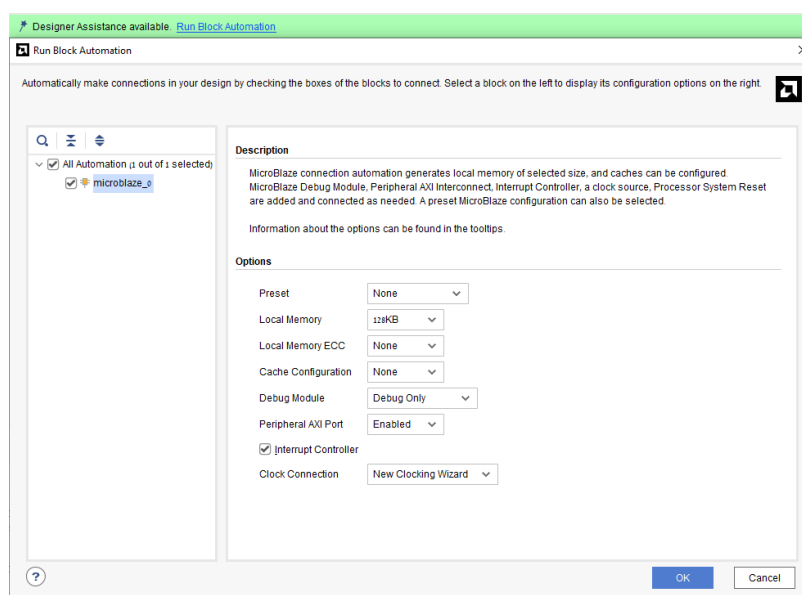
שלב 7:

בחלון העליון יופיע פס ירוק. נלחץ על Run Block Automation. בחלון שיקפוצ, נגדיר את תצורת המעבד לפני לחיצה על OK באופן הבא:

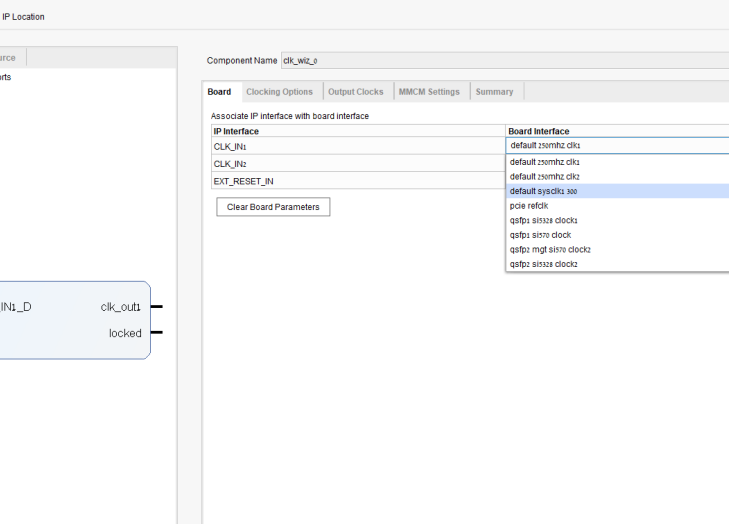
בשדה Local Memory נשנה את הערך ל- 128 KB. מכיוון שה-MicroBlaze מסתמך על זיכרון פנימי ב-PL (בניגוד ל-Zynq שמשמש ב-DDR חיצוני), הגדלה זו הכרחית כדי למנוע שגיאות חוסר זיכרון בעת קימפול התוכנה ב-Vitis.

נוודא שתיבת Interrupt Controller מסומנת ב-V, כדי לייצר בקר פסיקות שיאפשר למעבד לקבל התראות מרכיבים היקפיים.

את שאר ההגדרות נשאיר כברירת מחדל.



נלחץ לחיצה כפולה על בלוק ה-Clocking Wizard.



The screenshot displays the Xilinx Clocking Wizard (v.0) interface. The top bar shows the title "Re-customize IP" and a "Clocking Wizard (v.0)" button. Below the title bar, there are links for "Documentation" and "IP Location".

The main window is divided into two tabs: "IP Symbol" and "Resource". The "IP Symbol" tab is active, showing a block diagram of the CLK_WIZ_0 component. The block diagram includes a "reset" input, a "locked" output, and a "clk_out1" output. A plus sign (+) is visible next to the "CLK_IN1_D" input.

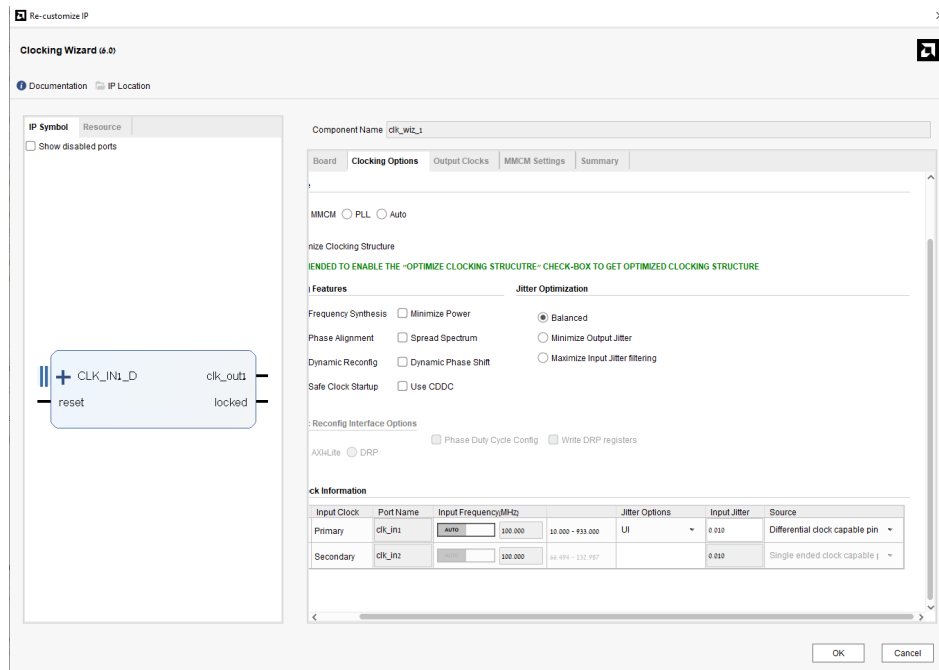
The "Resource" tab is also visible, showing the component name "clk_wiz_0" and the board interface settings. The "Board" tab is selected, displaying the "Associate IP interface with board interface" section. This section includes a table with the following columns: "IP Interface" and "Board Interface".

IP Interface	Board Interface
CLK_IN1s	default0 250mhz clk1
CLK_IN2s	default1 250mhz clk1
EXT_RESET_IN	default2 250mhz clk2
	default3 sysclk1 300
	pcie refclk
	qsfps s1s32a clock1
	qsfps s1s70 clock
	qsfps mgl s1s70 clock2
	qsfps s1s32a clock2

Below the table, there is a "Clear Board Parameters" button. The "Board Interface" dropdown menu is open, showing the list of board interfaces.

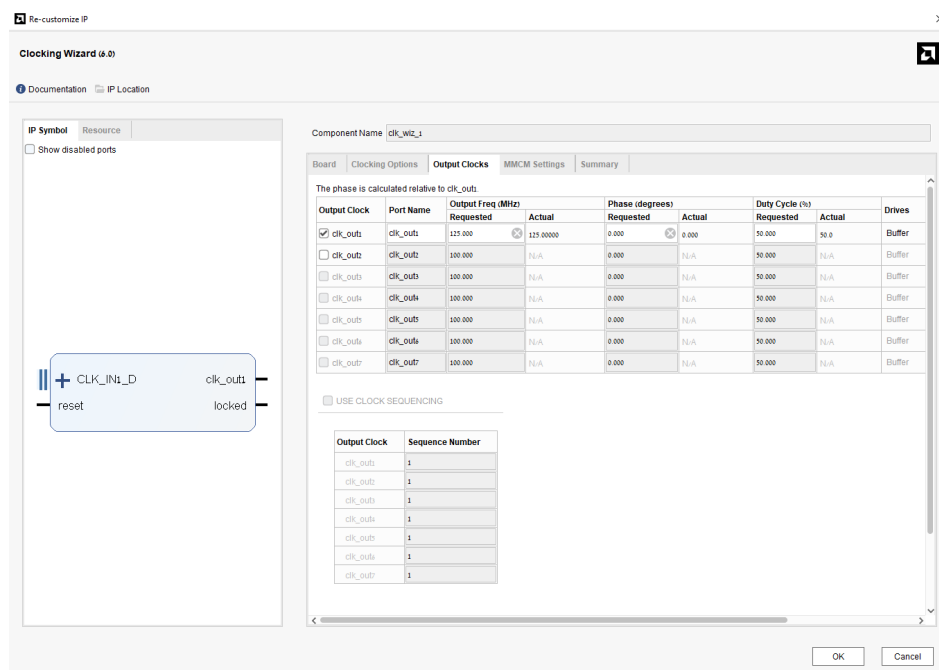
At the bottom right of the window, there are "OK" and "Cancel" buttons.

בלשונית Clocking Options, נוודא שהכניסה (Input Clock) מוגדרת כ- Differential. הגדרה זו הכרחית מאחר שלוח ה-VCU118 מספק שעון מערכת יציב דרך זוג פנים דיפרנציאליים להפחתת רעשים.



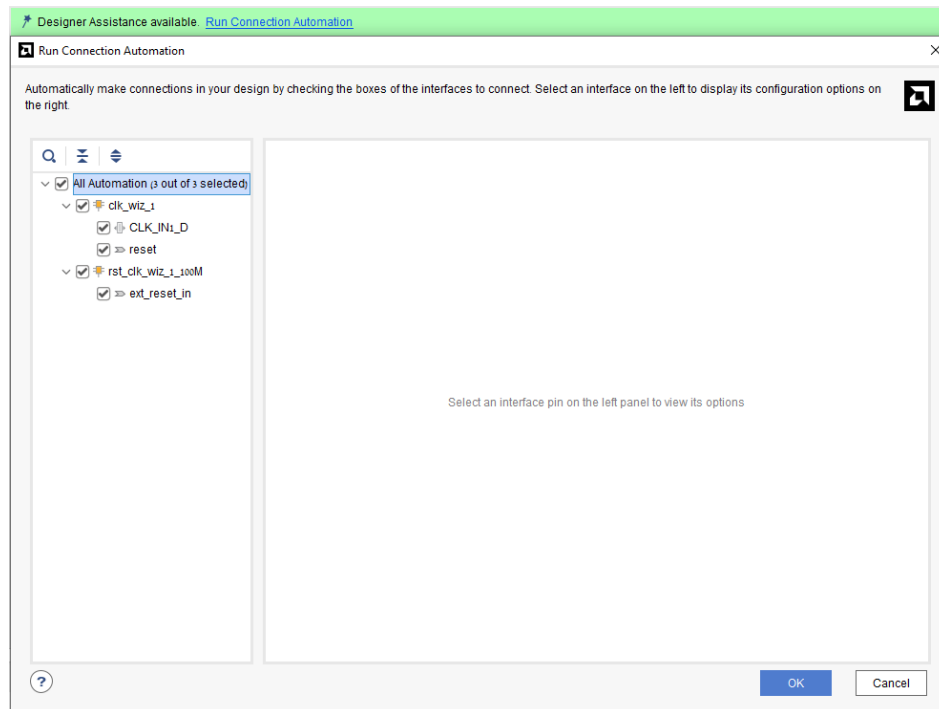
בלשונית Output Clocks, נגדיר את התדר של clk_out1 ל- 125 MHz. תדר זה מהווה את דרישת הסף המחייבת לעבודה בתקן Gigabit Ethernet, המאפשר סנכרון מדויק של העברת הנתונים בקצב של 1 Gbps.

בסיום, נלחץ על OK.



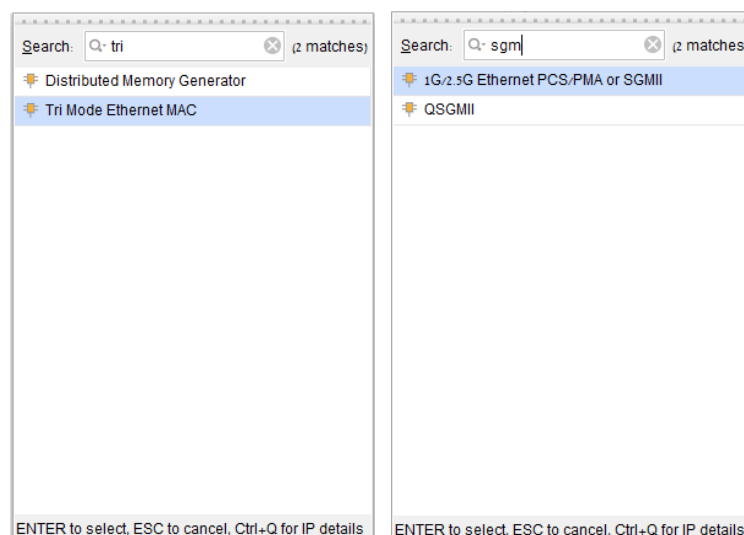
שלב 9:

נלחץ על הפס הירוק שיזכיר את: Run Connection Automation. בחלון שייפתח נסמן את כל התיבות `All Automation` ונלחץ OK. בצענו זאת מכיוון שפעולה זו מחברת אוטומטית את אפיקי ה-AXI בין המעבד לבקר הפסיקות, ויוצרת את רשת ה-Interconnect הנדרשת לשליטה ברכיבי הקצה. בנוסף, המערכת תחבר את אותות ה-Reset לכלל הבלוקים כדי להבטיח אתחול מסונכרן של החומרה.

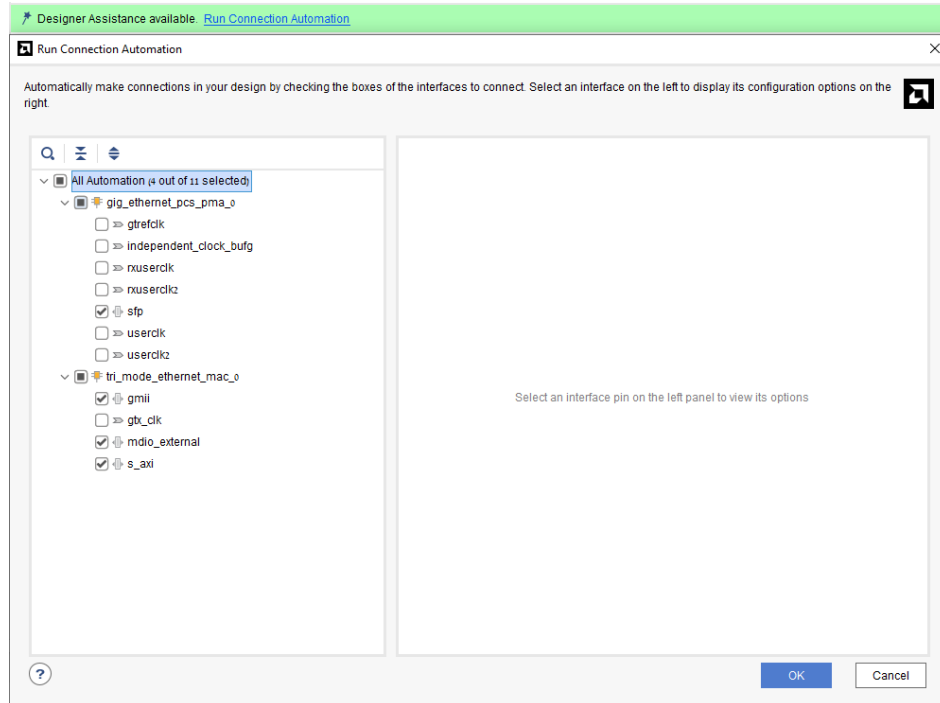


שלב 10:

נוסיף לשרטוט את רכיבי ה-IP הבאים: Tri Mode Ethernet MAC ו-1G/2.5G Ethernet PCS/PMA or SGMII.

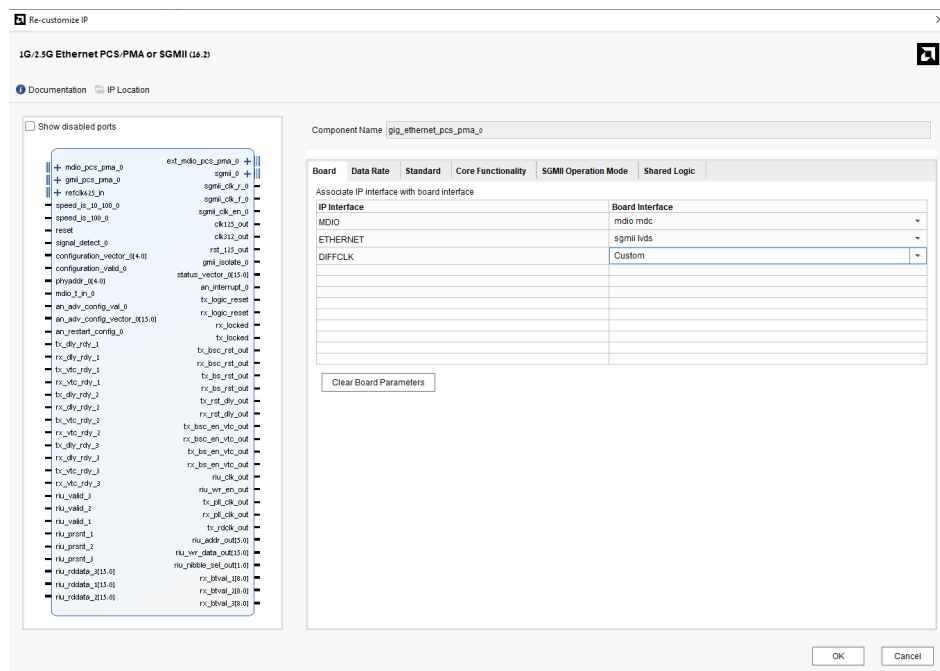


נפעיל עבורם את ה-Run Connection Automation, אך נסמן בתיבות ה-V רק את ממשקי הנתונים והבקרה (sfp, gmii, s_axi) ללא סימון כניסות השעון. ביצענו זאת מכיוון שבניגוד ללוחות Zynq, בלוח VCU118 עלינו לממש את שכבות ה-MAC וה-PHY ב-PL, והימנעות מאוטומציית שעונים מאפשרת לנו שליטה מלאה בחיבורם למקור יחיד ומסונכרן למניעת שגיאות.

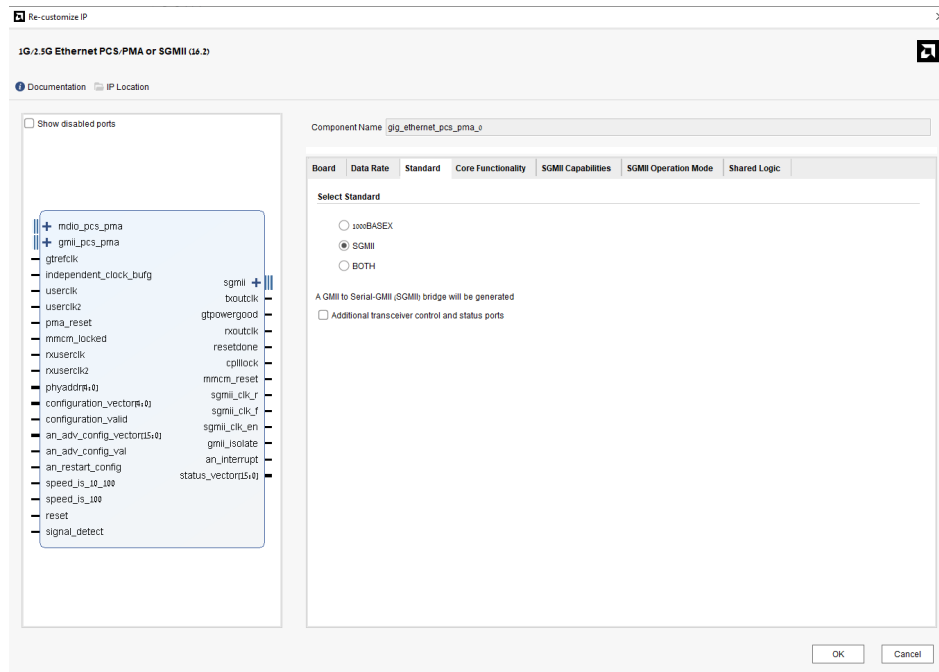


שלב 11:

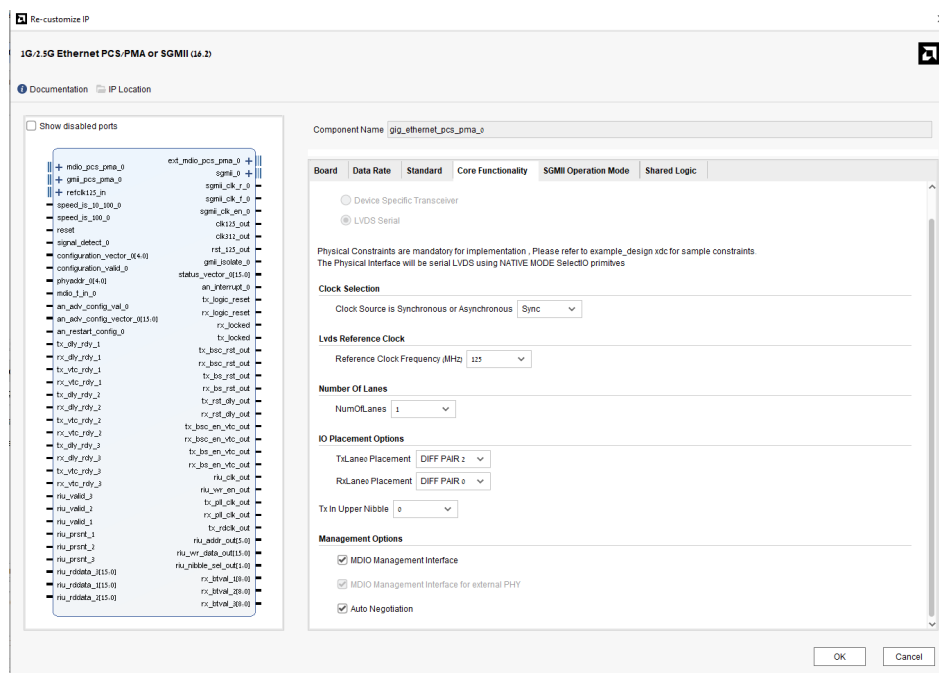
נגדיר את רכיב ה-Ethernet PCS/PMA בהתאם לדרישות החומרה הספציפיות של לוח הפיתוח. בלשונית Board נשייך את ה-MDIO וה-ETHERNET הלוח (mdio mdc ו-sgmii lvds), אך נגדיר את ה-DIFFCLK כ-Custom כדי לאפשר עבודה בתדר 125 MHz.



בלשונית Standard נבחר בתקן SGMII.

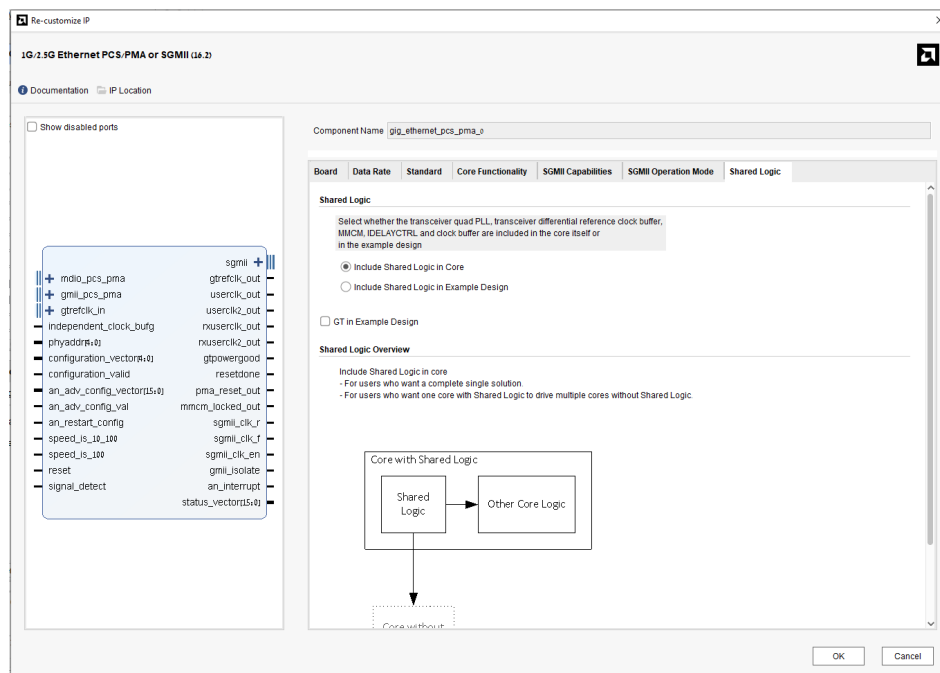


בלשונית Core Functionality, נגדיר את הממשק הפיזי כ-LVDS Serial עם התאמה לאילוצי הפינים של הלוח: Tx Lane0 ל-DIFF PAIR 2- ו-Tx In Upper Nibble ל-0, תוך הפעלת מנגנון ה-Auto Negotiation.



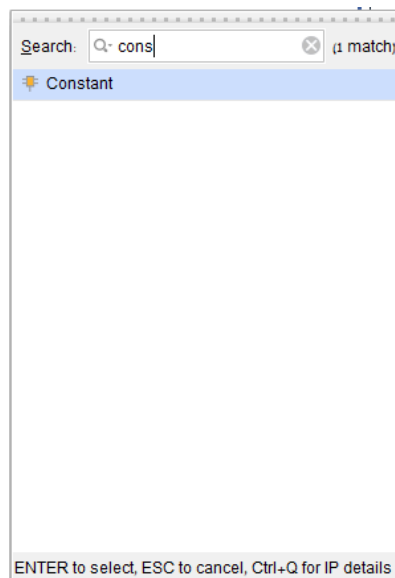
בלשונית Shared Logic נבחר באופציה Include Shared Logic in Core כדי לחשוף את אותות הסנכרון והאיפוס הפנימיים.

בסיום, נלחץ OK.

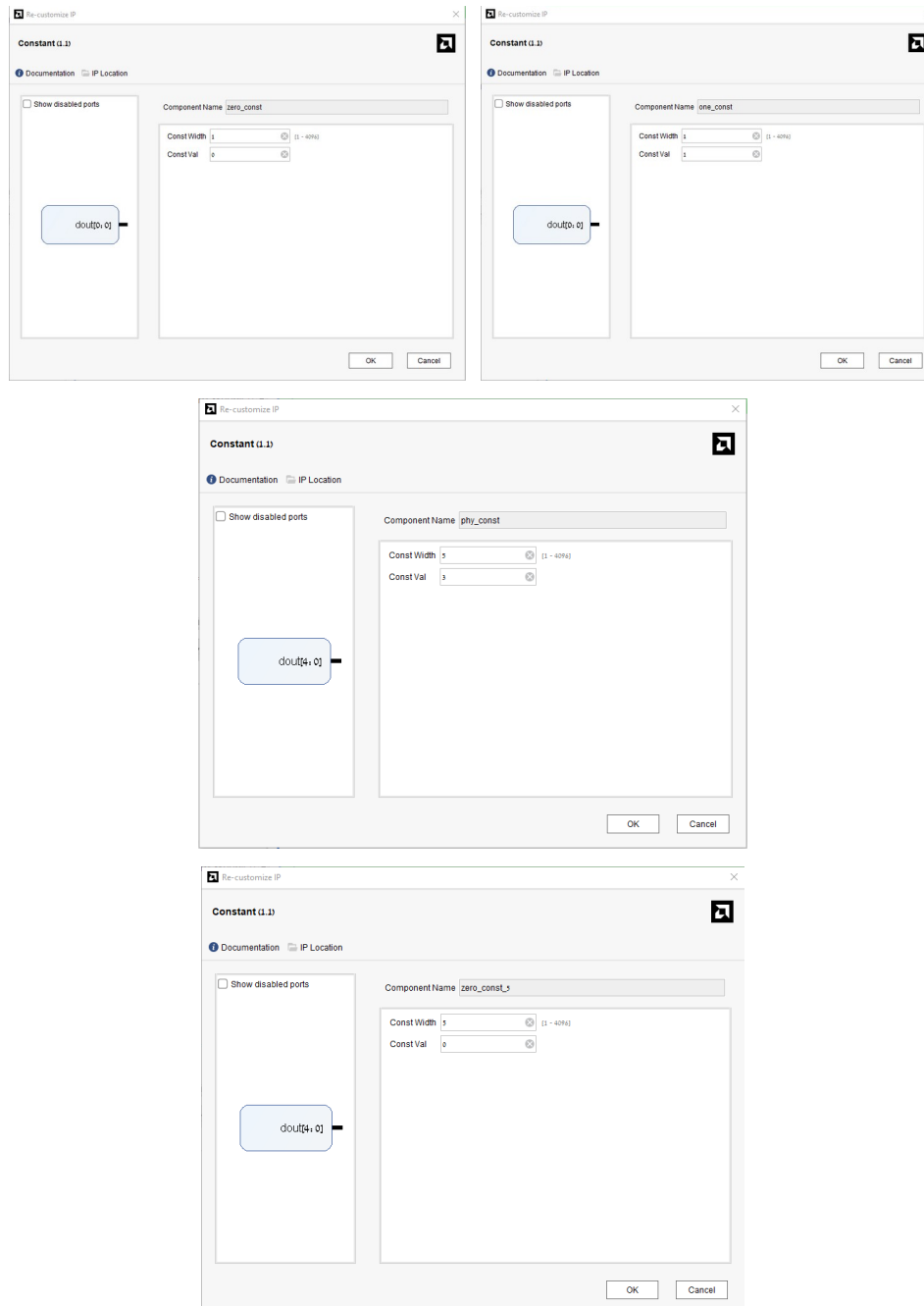


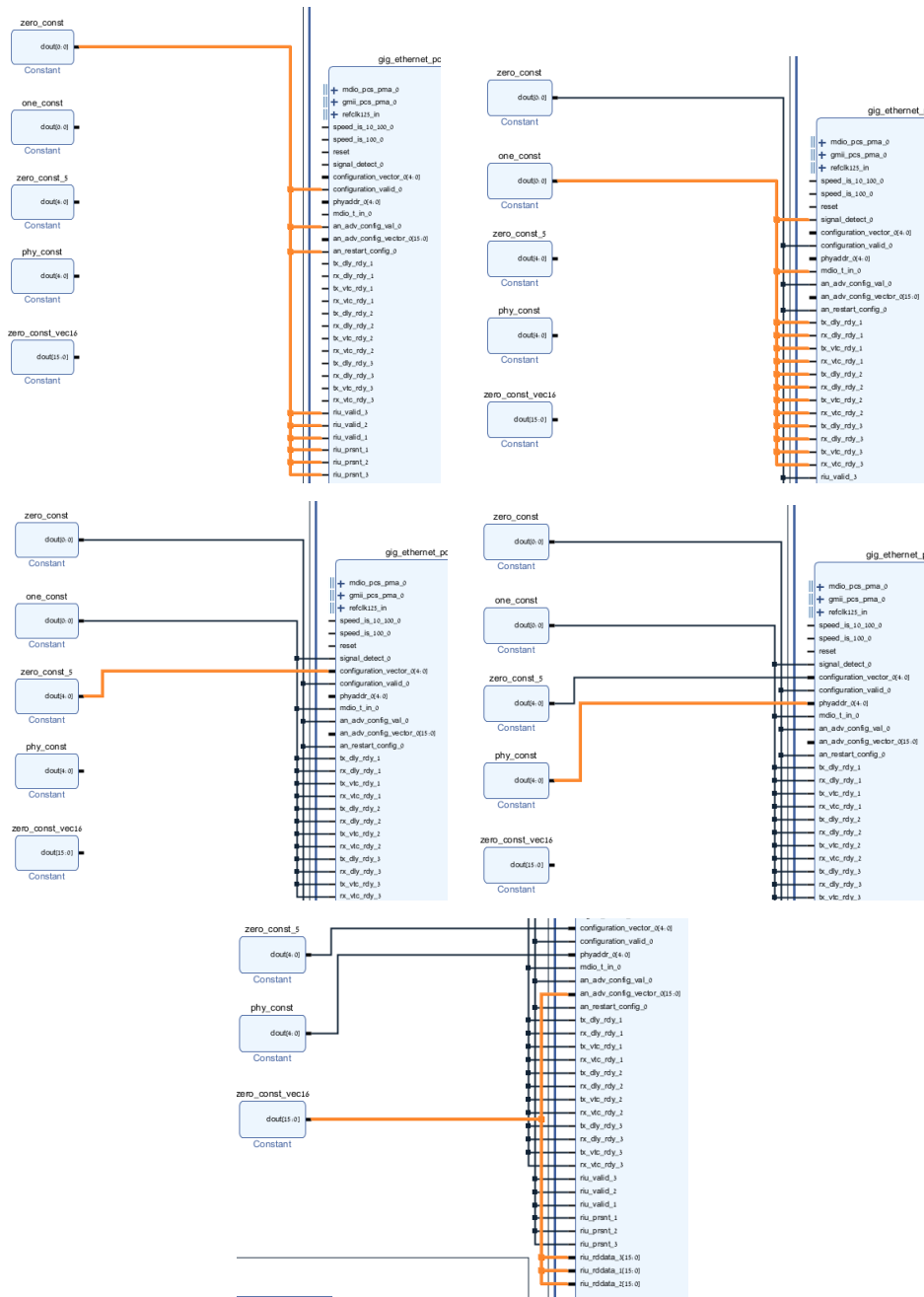
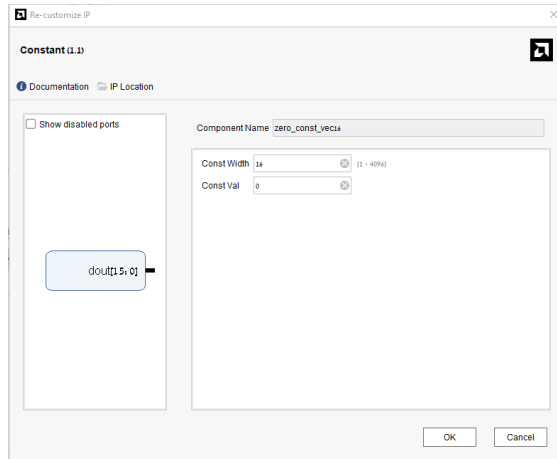
המטרה של ביצוע הגדרות אלו היא בכדי להבטיח שהלוגיקה ב- PL, תפעל בסנכרון מלא מול רכיב ה-PHY המובנה בלוח ה- VCU118 המשויך ליציאת RJ45, ותאפשר תקשורת יציבה בקצב Gigabit Ethernet.

נוסיף רכיבי xconstant ונעניק להם שמות משמעותיים לשמירה על סדר.

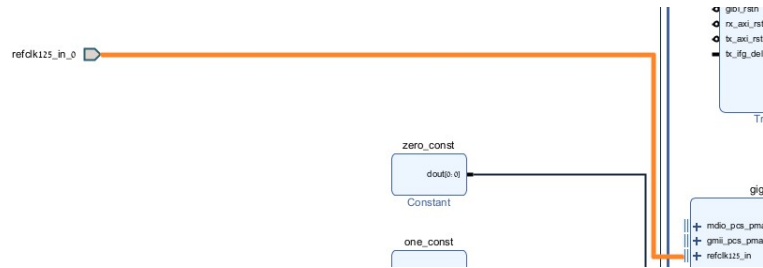


נשתמש ב- zero_const (ברוחב 5 ביטים) וב- phy_const (ערך 3) כדי לקבע את כתובת ה-PHY, וב- zero_const_vec16 כדי לאפס את וקטור הקונפיגורציה. פעולה זו מבטיחה שהמערכת תעלה במצב עבודה ידוע ללא צורך בניהול תוכנה מורכב בשלב האתחול.





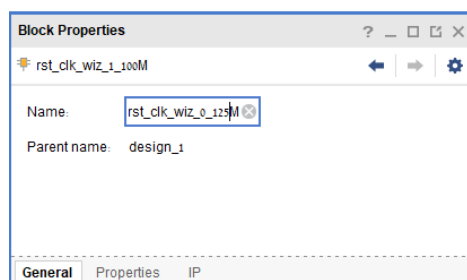
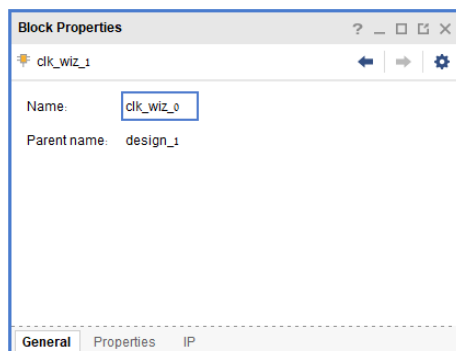
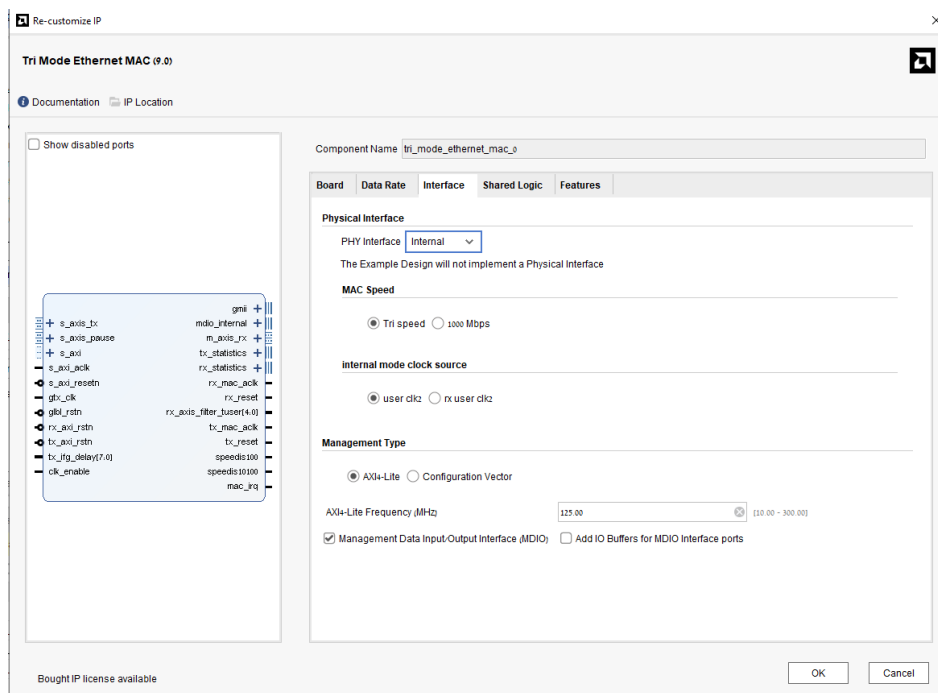
נגדיר את פורט refclk125_in כפורט חיצוני בשם refclk125_in_0, המקבל שעון ייחוס בתדר 125 MHz ישירות מהמתנד הפיזי שעל הלוח.



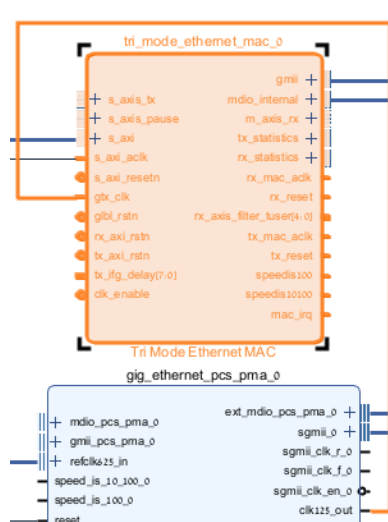
שלב 12:

בשלב זה נגדיר את תצורת ה-MAC, ונבצע את החיבורים הנדרשים להפעלת רכיבי ה-MAC וה-PHY על לוח ה- VCU118, בדגש על סנכרון שעונים ולוגיקת איפוס (Reset).

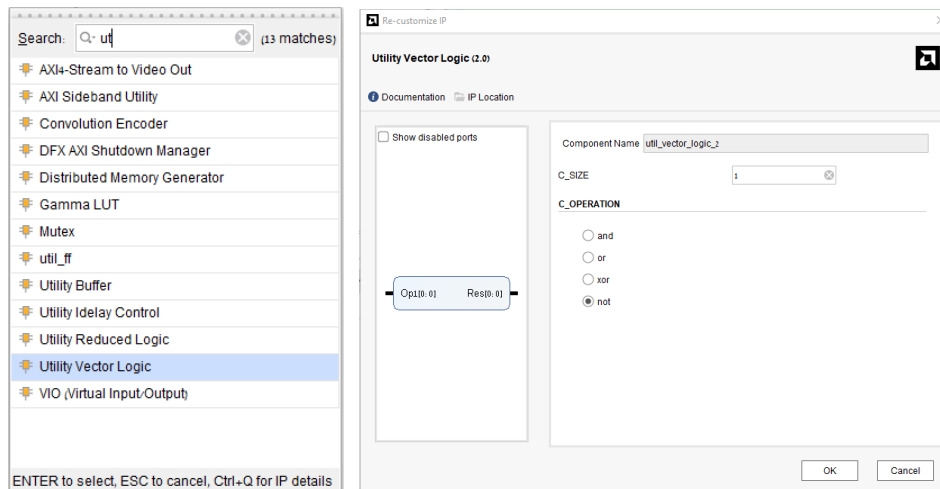
ניכנס להגדרות בלוק ה-Tri Mode Ethernet MAC ונשנה את ממשק ה-PHY הפיזי ל- Internal במקום GMII. בנוסף, נעדכן את תדר ממשק הניהול AXI4-Lite ל- 125 MHz כדי שיהיה מסונכרן לחלוטין עם שעון המערכת. נשנה גם את שמו של בלוק הריסט המרכזי ל- rst_clk_wiz_0_125M במקום ה- M 100 שהיה מוגדר כברירת מחדל, כדי לשקף את התדר המעודכן של הפרויקט.



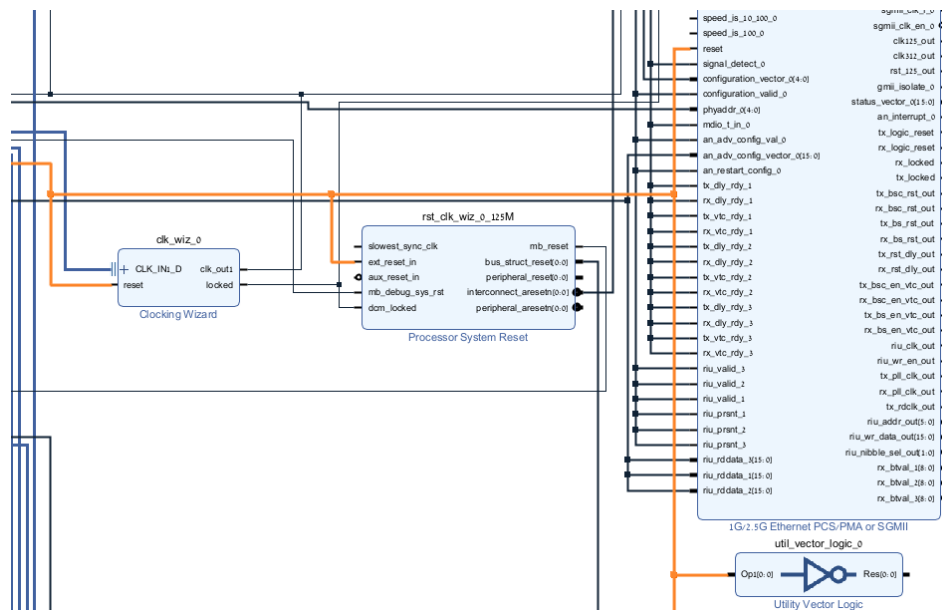
נחבר את מוצא השעון clk125_out של ה-PCS/PMA בבלוק Internal Clocking מביטח סנכרון פאזה מושלם של ה-gtx_clk. שימוש בשיטת ה- Internal Clocking מבטיח סנכרון פאזה מושלם בין השכבות, מונע שגיאות תזמון (Timing) ומבטל את הצורך במקורות שעון חיצוניים נוספים ל-MAC.



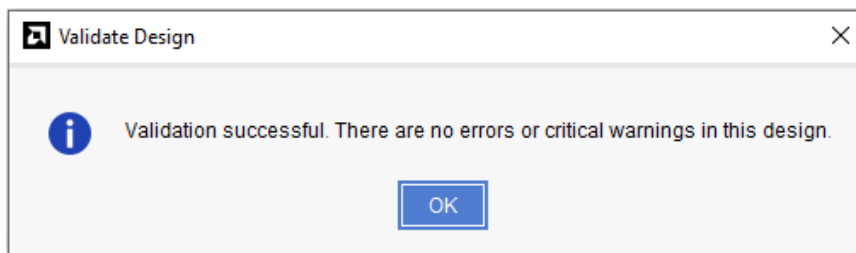
מאחר שפין ה- reset של ה-PCS/PMA דורש לוגיקה הפוכה - Active High, נוסיף רכיב Utility Vector Logic (בשם util_vector_logic_2) המוגדר כשער NOT. השער יהפוך את סיגנל האיפוס המרכזי, מה שיבטיח שכלל הרכיבים ישתחררו מאיפוס בתיאום מלא עם עליית המערכת.



כעת נחבר את כניסות ה- Reset של ה- MAC לסיגנל ה- peripheral aresetn בלוגיקת .Active Low

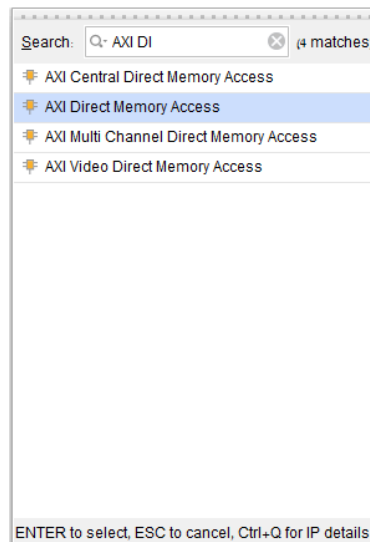


בסיום החיבורים, אם נדרש ונרץ Connection Automation להשלמת החיבורים. לאחר מכן נבצע Validate Design ונוודא קבלת הודעת "Validation successful", המאשרת כי תשתית התקשורת תקינה ומוכנה לשילוב רכיב ה- DMA.

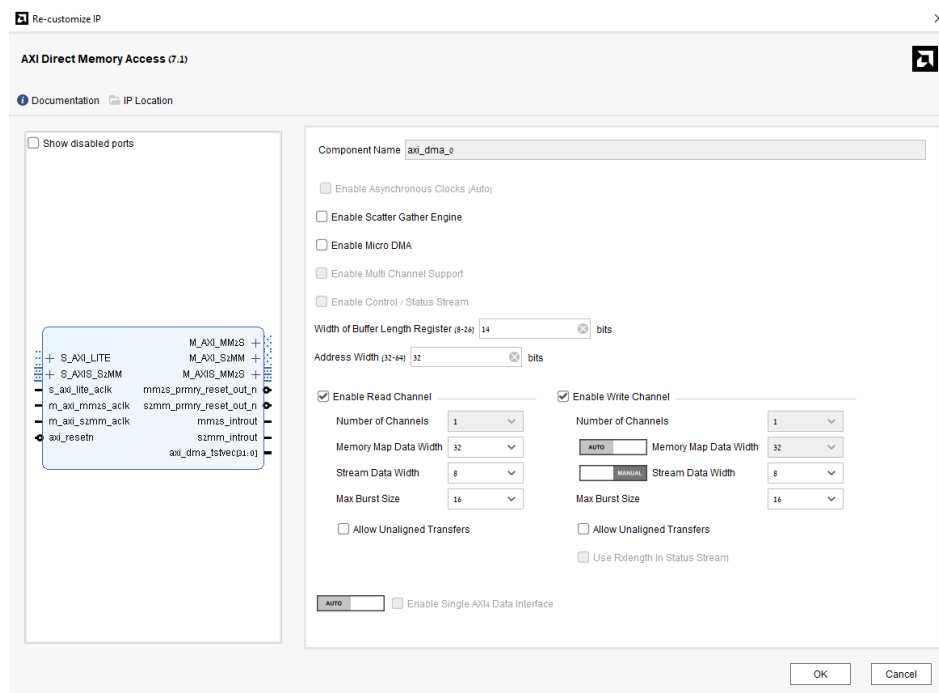


שלב 13:

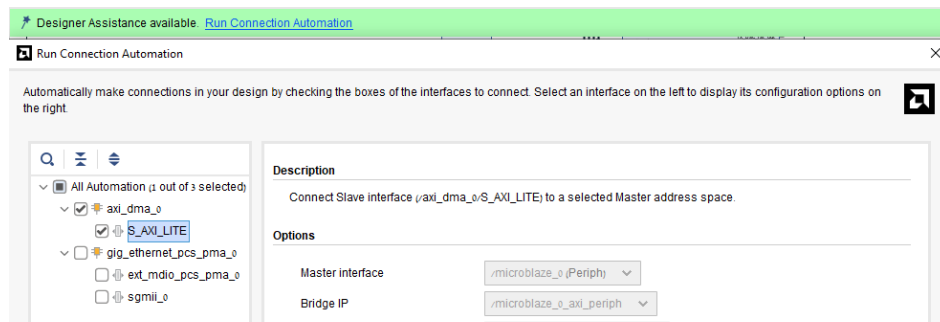
בשלב זה הוספנו למערכת את רכיב ה-AXI Direct Memory Access (DMA), המשמש כגשר גישה ישירה לזיכרון (Memory Map) עבור תעבורת הרשת (AXI-Stream). שימוש ברכיב זה מאפשר העברת נתונים בקצבים גבוהים בין מנוע ה-UDP לזיכרון הראשי, ללא העמסה על משאבי העיבוד של ה-MicroBlaze.



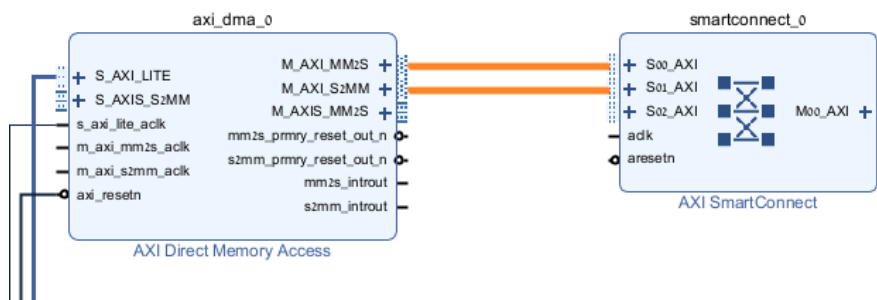
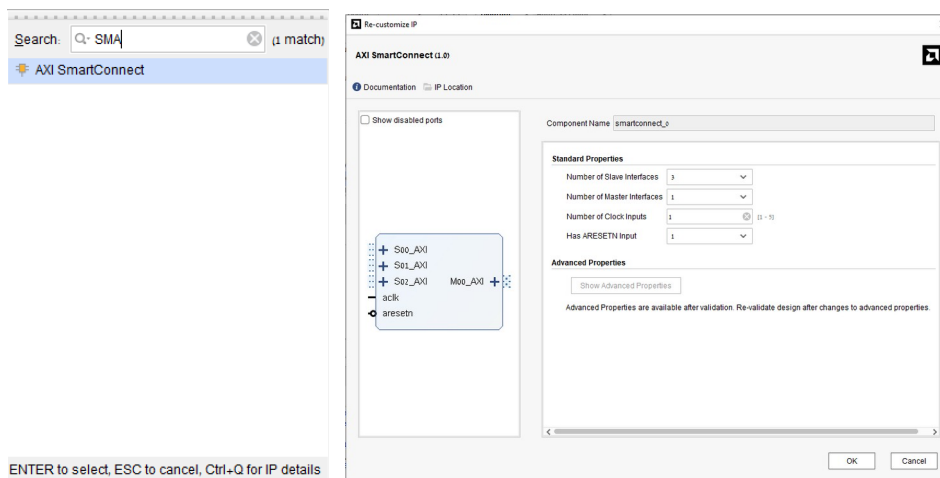
ביטלנו את מנגנון ה-Scatter Gather לטובת עבודה במצב Simple DMA, המפשט את ארכיטקטורת ניהול הזיכרון ברמת התוכנה ומקל על פיתוח מנהל ההתקן (Driver) בהמשך. רוחב אוגר אורך החוצץ Buffer Length Register הוגדר ל-14 ביטים, המאפשר העברת רצפי נתונים של עד כ-16 KB. רוחב נתוני הסטרים Stream Data Width הוגדר ל-8 ביטים להתאמה לקצב עבודה ה-MAC (בייט למחזור שעון בתדר 125 MHz).



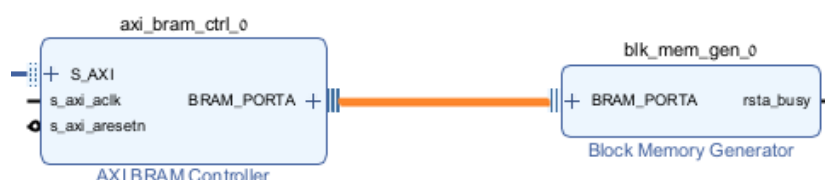
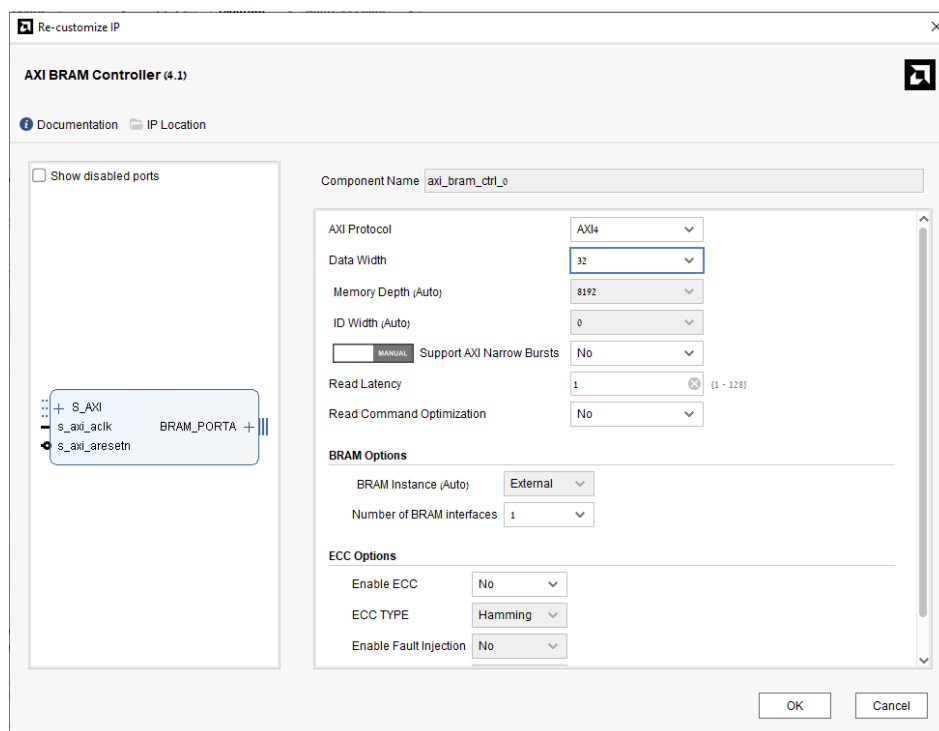
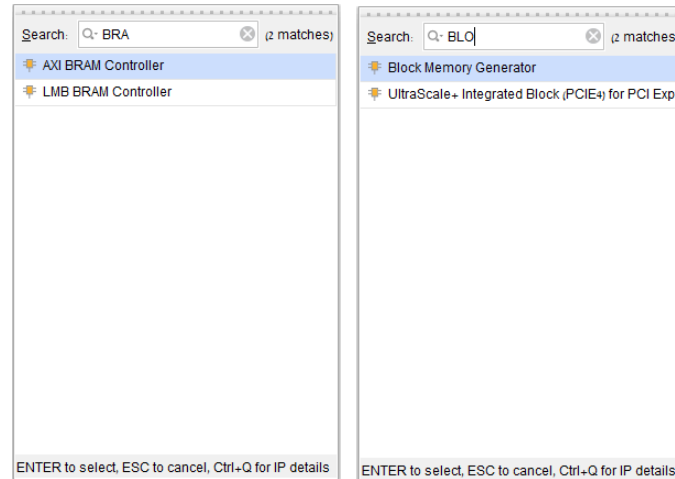
ממשק ה-S_AXI_LITE חובר אל ה-AXI Interconnect ההיקפי של המעבד, לצורך קינפוג אוגרי ה-DMA.



ממשקי הקריאה והכתיבה לזיכרון M_AXI_MM2S ו-M_AXI_S2MM חוברים לרכיב ניתוב מרכזי מסוג AXI SmartConnect. ה-AXI SmartConnect קונפג עם 3 כניסות Slave (קריאת DMA, כתיבת DMA וגישת מעבד) כדי לנהל את הבוררות לזיכרון ביעילות.



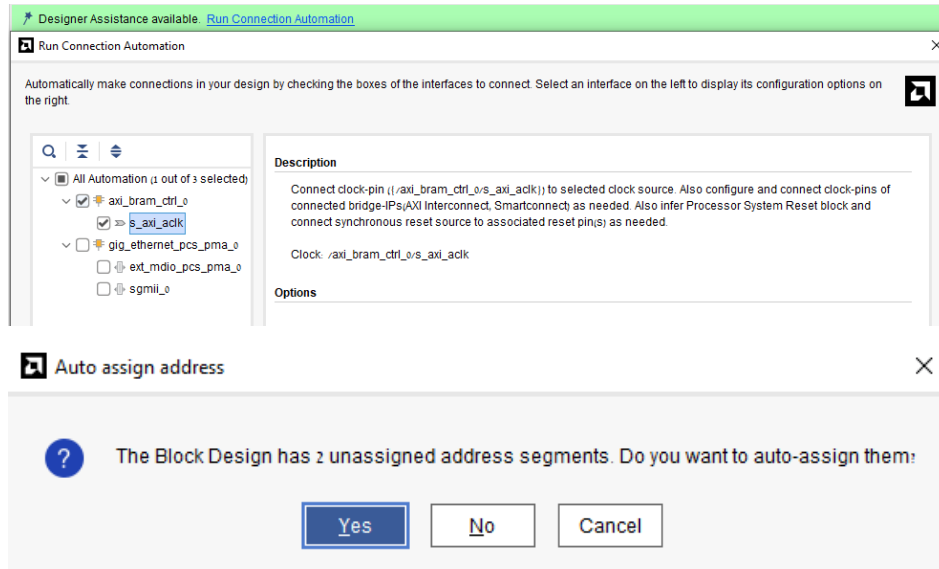
במקום לנתב את העברות ה- DMA לזיכרון ה- DDR החיצוני, בחרנו למפות את תעבורת הרשת לזיכרון פנימי מהיר מסוג BRAM (Block RAM) בנפח של 8 KB. לשם כך הוספנו רכיב AXI BRAM Controller במצב Single Interface וללא תמיכה ב-Narrow Bursts. פתרון זה מספק חוצץ Buffer פנימי עם השהייה מינימלית המושלם לאגירת פאקטים של UDP. שעוני הרכיבים חוברו ידנית לעץ השעונים המרכזי למניעת שגיאות תזמון.



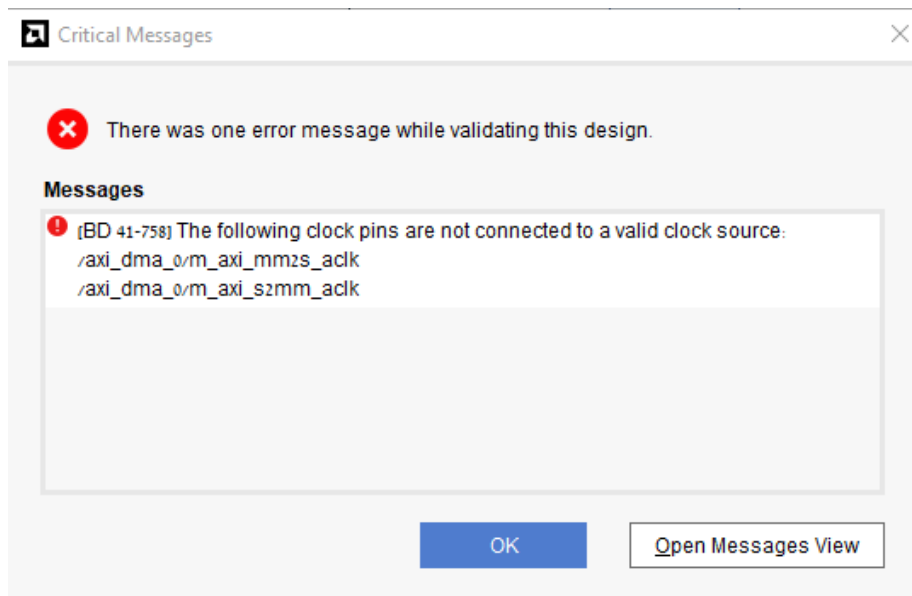
כדי שה- DMA יוכל לאותת למעבד על סיום העברת נתונים, ניתבנו את אותות הפסיקה שלו. הפינים mm2s_introut (סיום שידור) ו-s2mm_introut (סיום קליטה) חוברו לכניסות של

רכיב Concat, אשר מאגד אותם לאפיק אחד ומעביר אותם לכניסת בקר הפסיקות המרכזי של המעבד axi_intc.

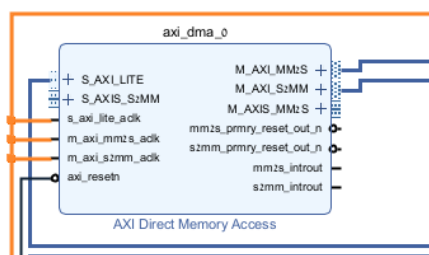
בשלב הבא, הרצנו אוטומציה לחיבור שעוני ה-DMA בעזרת Run Connection Automation.



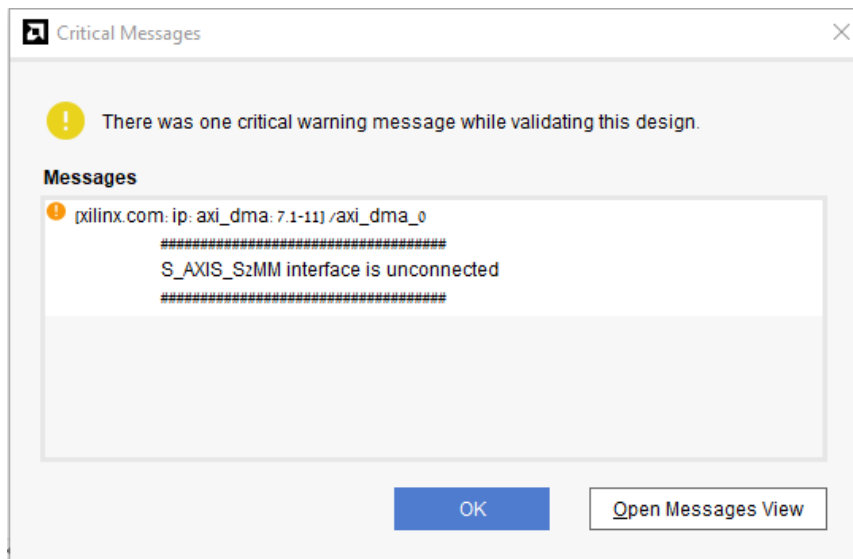
לאחר ביצוע האוטומציה, הרצת Validate Design חשפה שגיאה שלפיה פני השעון m_axi_mm2s_aclk ו-m_axi_s2mm_aclk נותרו מנותקים כפי שניתן לראות בתמונה.



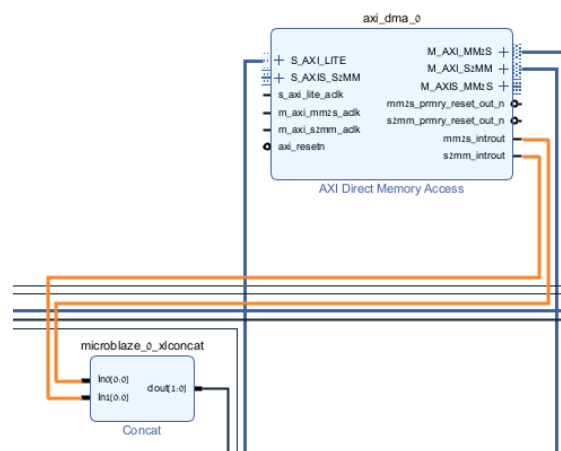
כדי לפתור זאת חיברנו ידנית שני הפינים הללו לאותו מקור שעון מרכזי שמזין את כלל המערכת, ובכך הבטחנו סנכרון מלא של ערוצי הקריאה והכתיבה.



בעת הרצה חוזרת של הבדיקה, התקבלה אזהרה על כך שממשק הקליטה S_AXIS_S2MM עדיין מנותק, אך זוהי אזהרה צפויה בשלב זה, שכן ממשק זה יחובר בהמשך ישירות לפלטפורמת התקשורת.



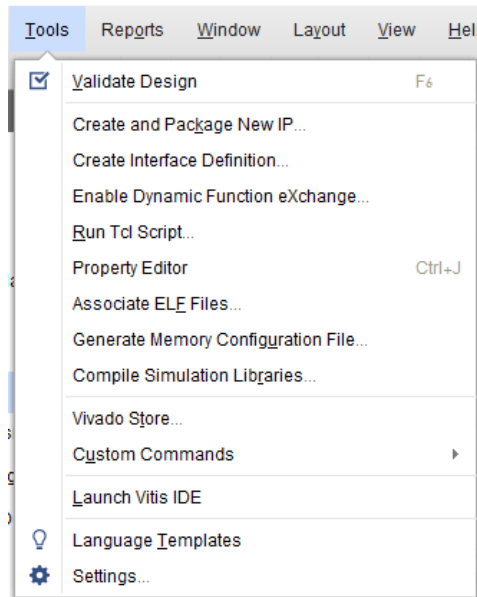
לבסוף על מנת ה-DMA יוכל לאותת ל-MicroBlaze על סיום העברת נתוני רשת, עלינו לנתב אותות הפסיקה שלו. הפינים mm2s_introut (סיום שידור) ו-s2mm_introut (סיום קליטה) חוברים לכניסות של רכיב ה-Concat. רכיב זה מגד את שתי הפסיקות לאפיק יחיד ומעביר אותן בצורה מסודרת אל בקר הפסיקות המרכזי של המעבד axi_intc.



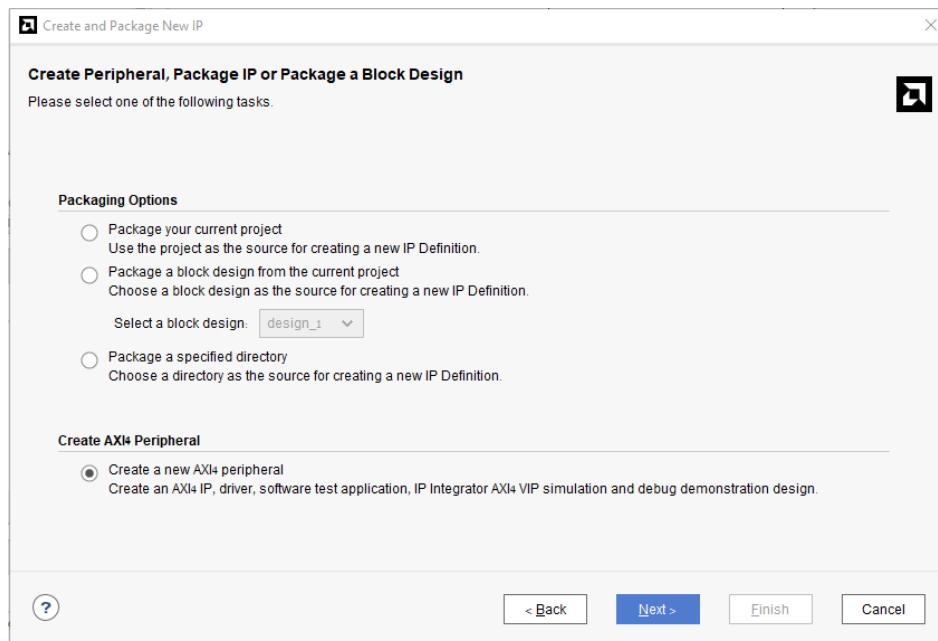
שלב 14:

בשלב זה ניקח את קבצי ה-VHDL של מודול ה-UOE, אותם הורדנו מראש מ-Git. נארוז אותם כרכיב IP תקני של Vivado, ונשלב את הרכיב במערכת מבוססת המעבד שלנו. תחילה נייבא את קבצי הליבה שהורדנו מ-Git אל תוך סביבת העבודה ונגדיר אותם כבלוק חומרה עצמאי.

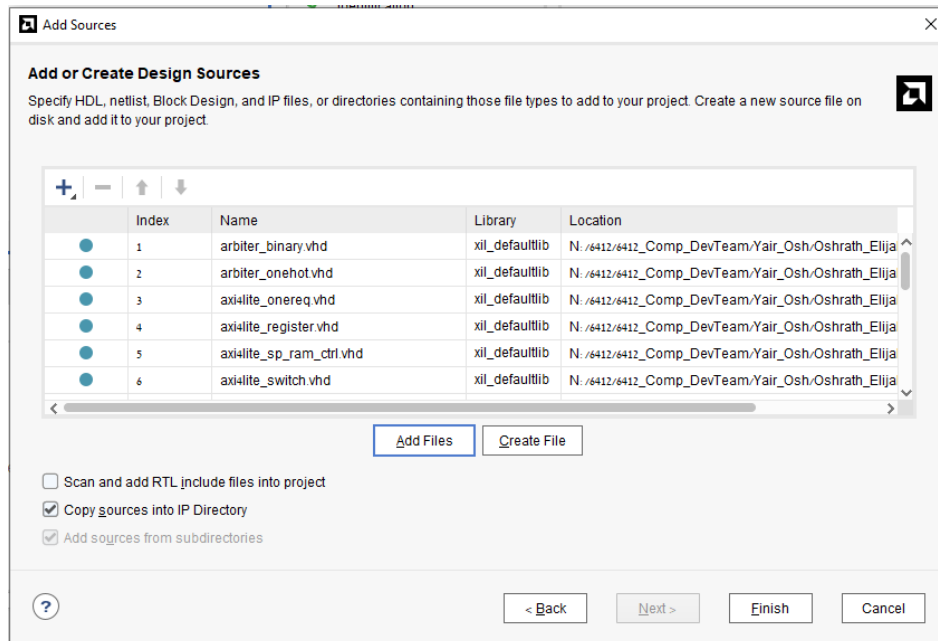
בסרגל הכלים העליון נלחץ על תפריט Tools ונבחר באפשרות Create and Package New IP



באשף שייפתח נבחר באפשרות לארוז את הפרויקט הנוכחי ונלחץ Next עד לסיום התהליך.



נוודא שקובץ ה-Top Module מוגדר כראוי, ונוסיף את כל קבצי ה-VHDL שהורדנו מ-Git אל תוך ה-IP Packager תחת תיקיית המקורות.



מכיוון שליבת ה-UOE כוללת ספריות וטיפוסי נתונים מתקדמים, עלינו להגדיר לתוכנה לקרוא את כל הקבצים בתקן השפה המעודכן. כדי להבטיח שהשינוי יישמר בתוך הגדרות ה-IP ולא יתאפס, נבצע עדכון גורף דרך חלון ה-Tcl Console בעזרת שלוש הפקודות הבאות:

תחילה נגדיר את כל קבצי ה-VHDL לתקן החדש:

```
set_property file_type {VHDL 2008} [get_files *.vhd]
```

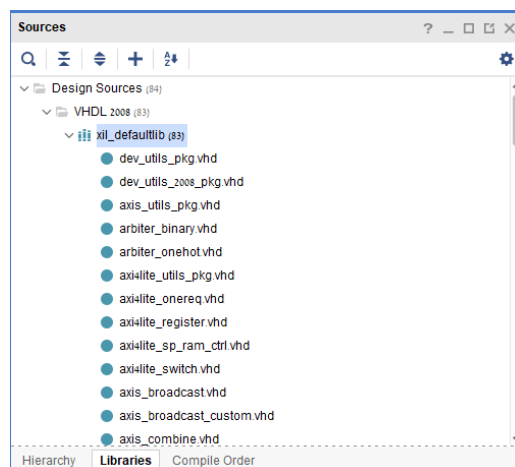
כעת נבצע סנכרון של השינויים אל תוך הגדרות הליבה:

```
ipx::merge_project_changes files [ipx::current_core]
```

לסיום נשמור את השינויים בתוך ה-Metadata של הרכיב:

```
ipx::save_core [ipx::current_core]
```

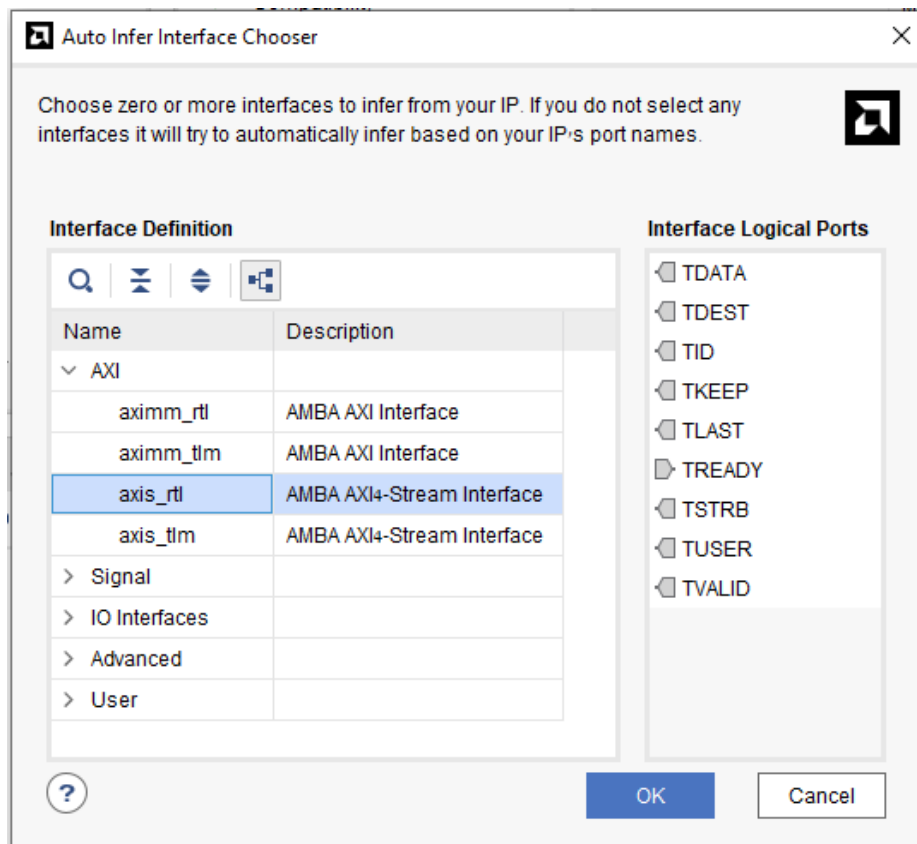
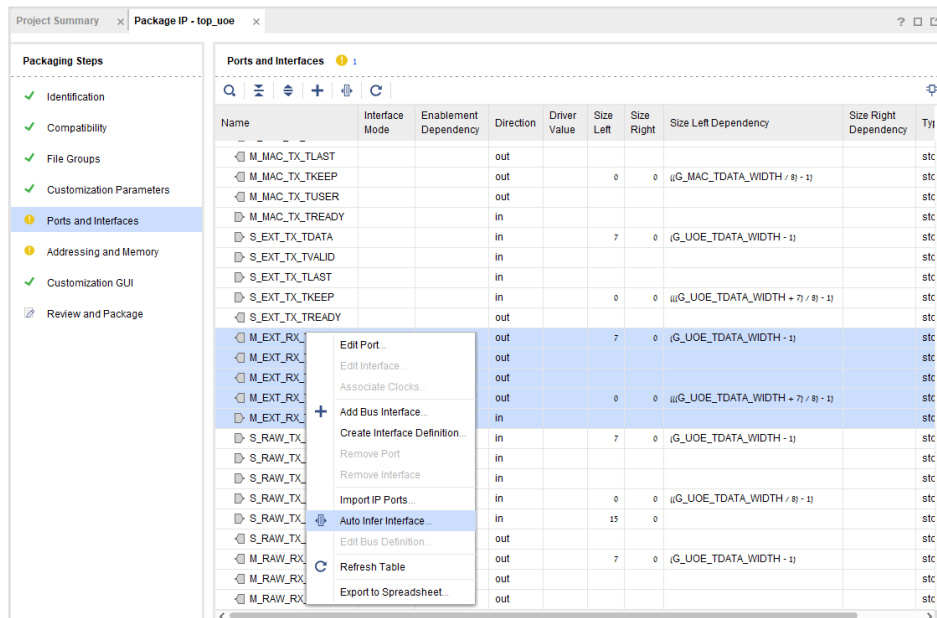
ביצוע רצף פקודות זה מבטיח שתקן השפה יישאר מקובע כ-VHDL 2008 גם לאחר סגירת פרויקט העריכה ועדכון ה-IP בשרטוט הראשי.

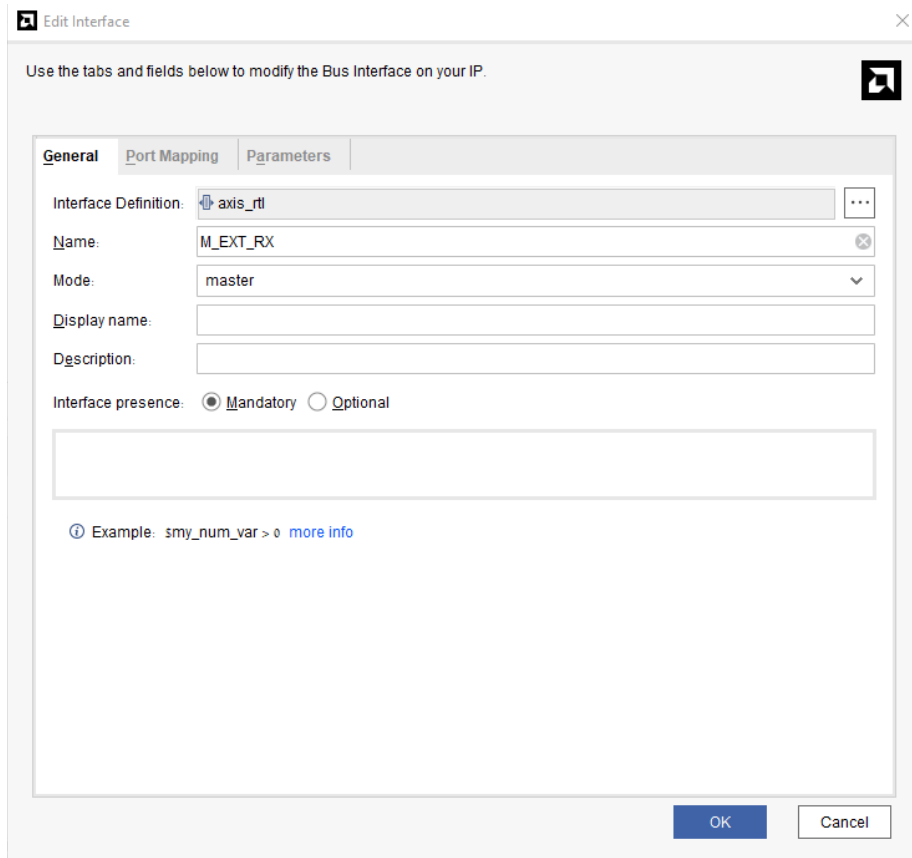


כדי שהבלוק יתקשר בצורה חלקה עם שאר רכיבי המערכת, עלינו לאגד את הפינים הבדידים לממשקים סטנדרטיים.

נעבור ללשונית Ports and Interfaces.

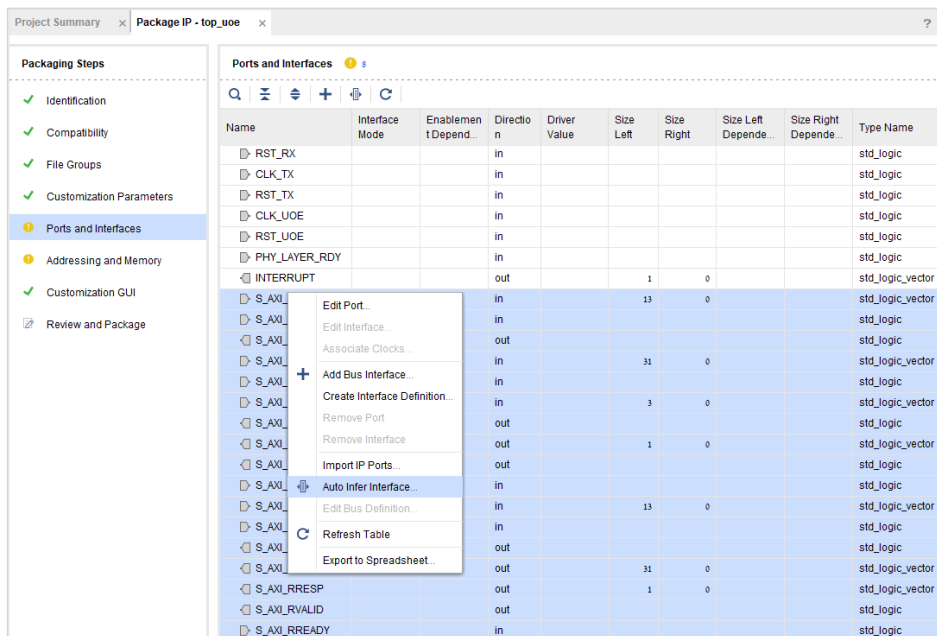
עבור כל קבוצת פינים המייצגת ערוץ נתונים, נלחץ קליק ימני, נבחר באפשרות Auto Infer Interface ונשייך אותם לממשק מסוג `axis\_rtl` פרוטוקול AXI4-Stream.

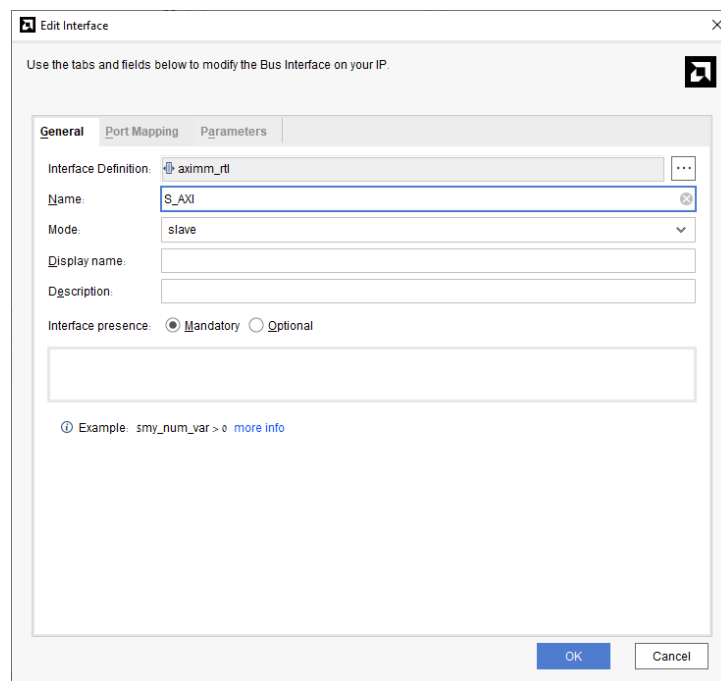
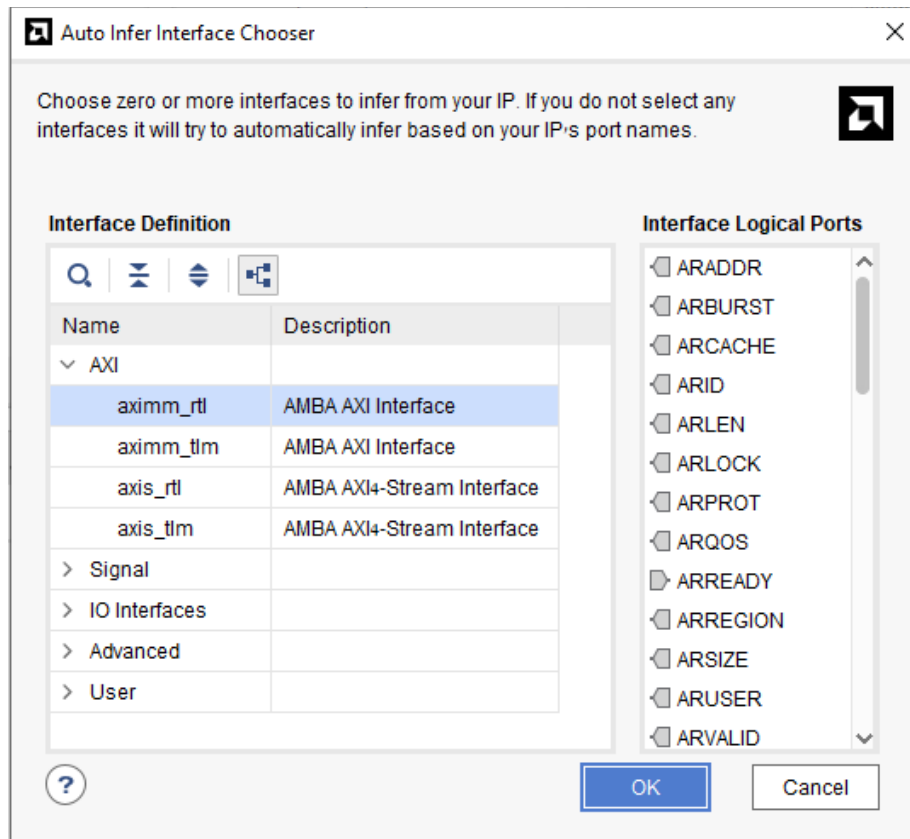


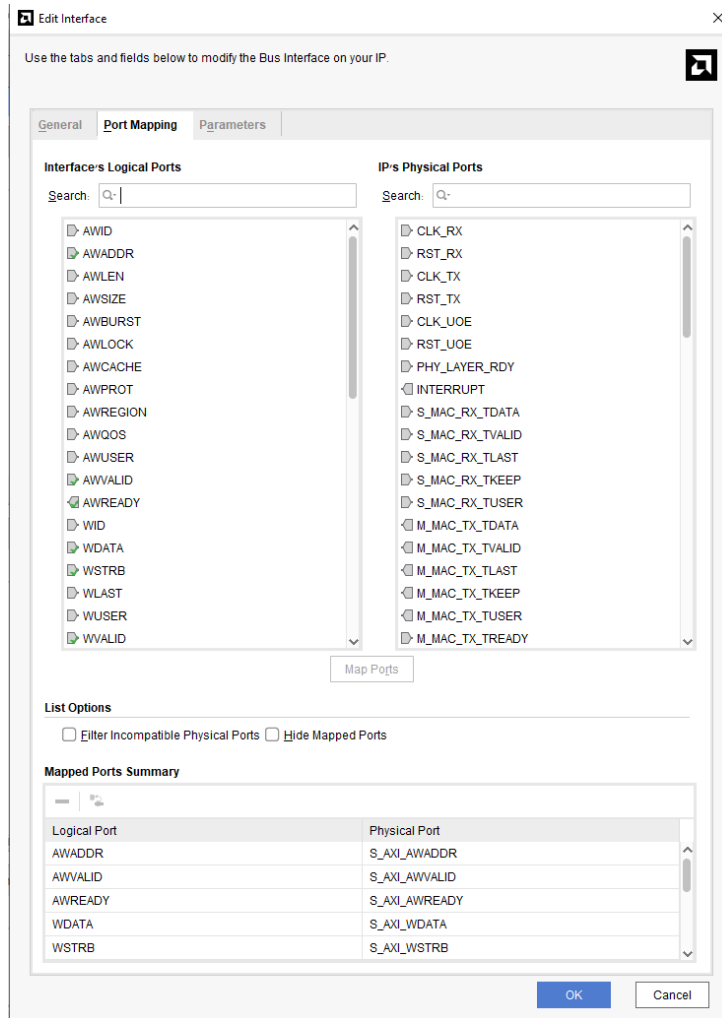


נחזור על הפעולה עבור כל ממשקי התקשורת השונים: חיבורי ה-MAC, חיבורי ה-UDP, חיבורי ה-RAM וממשק התקשורת החיצוני.

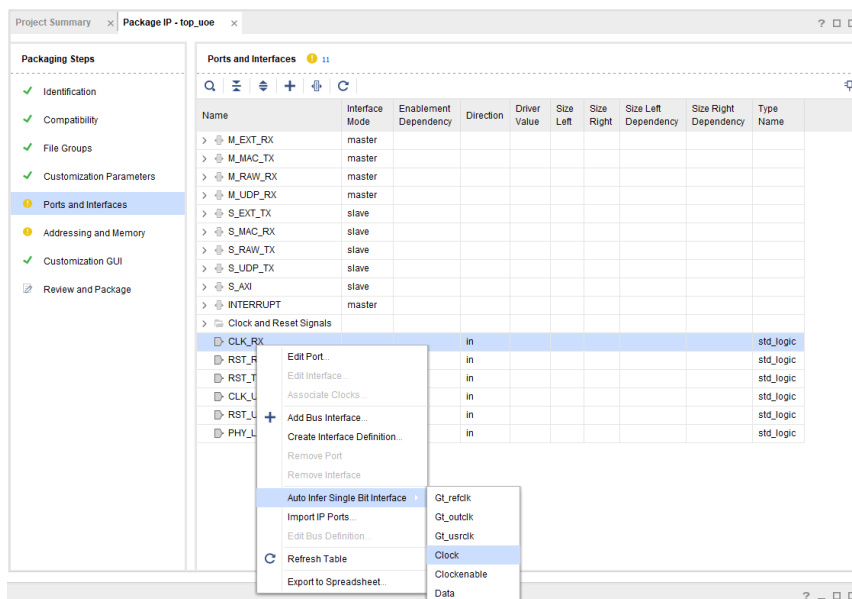
עבור ממשק הבקרה של המעבד, נאגד את הפינים הרלוונטיים לממשק מסוג `aximm\_rtl`

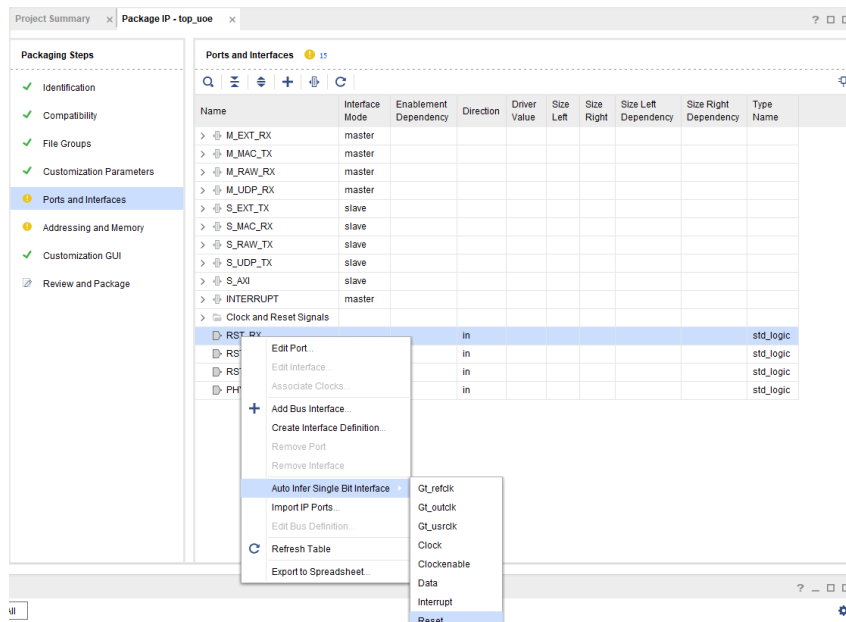




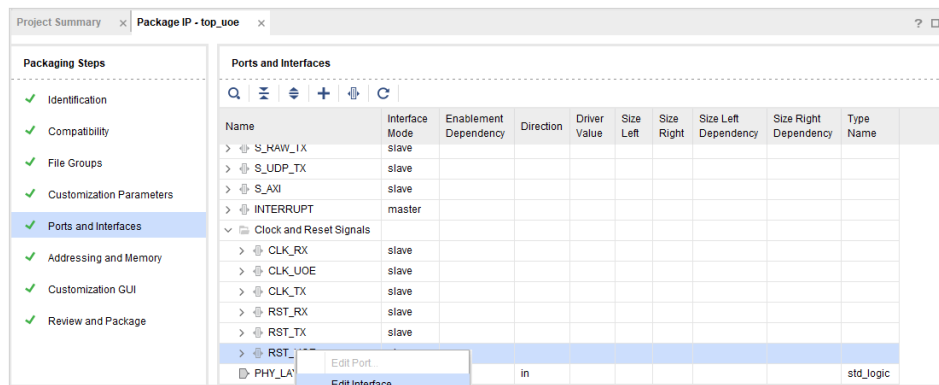


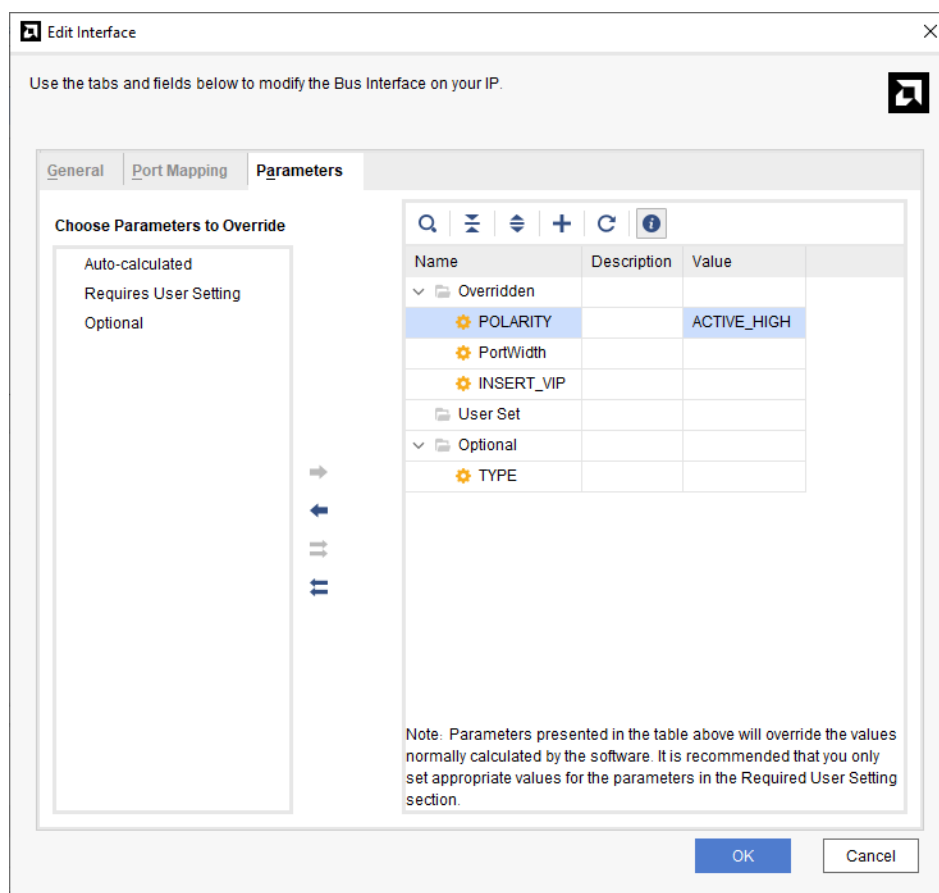
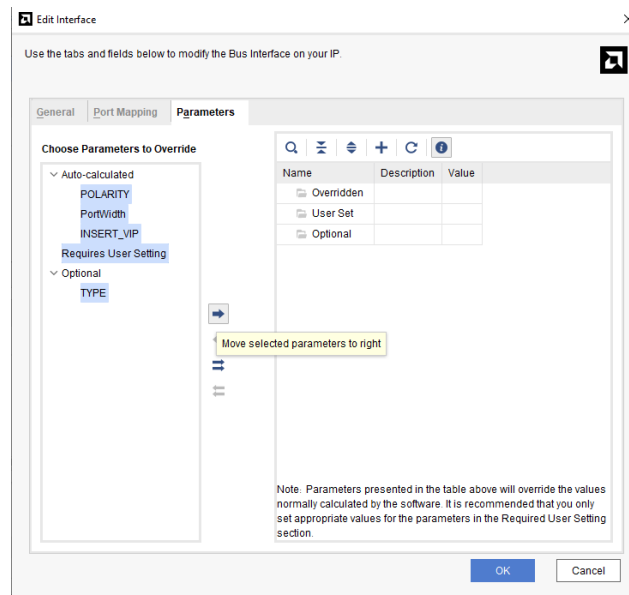
עבור אותות בודדים כמו שעונים וריסטים, נבצע Auto Infer Single Bit Interface ונגדיר אותם כ-Clock או כ-Reset בהתאמה.





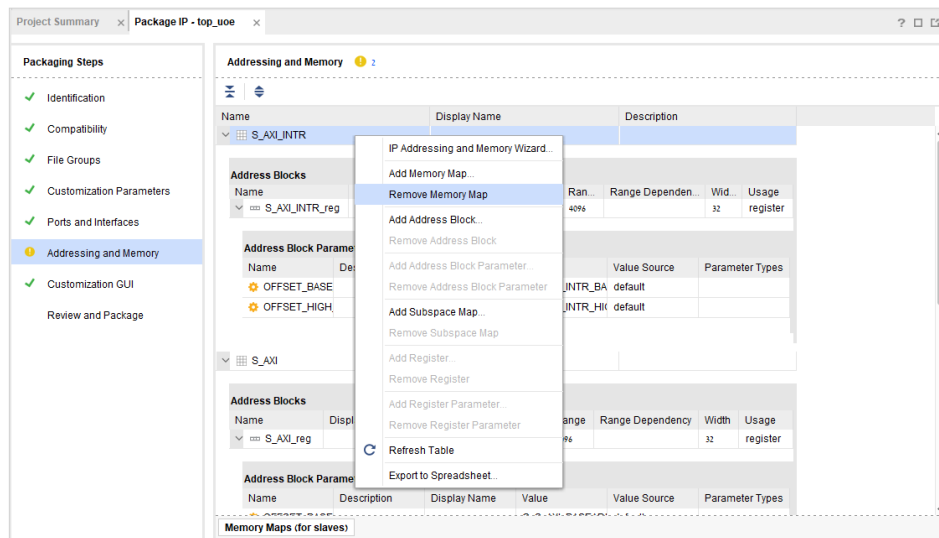
הגדרה קריטית נוספת שביצענו בשלב זה היא קביעת הקוטביות של אותות האיפוס. מכיוון שהלוגיקה הפנימית של ה- UOE דורשת "1" לוגי כדי להתאפס ($G_{Active Rst} = 1$), עלינו לוודא שהממשק מוגדר כ- Active High. נכנסנו להגדרות של כל ממשק ריסט (RST_UOE, RST_RX, RST_TX) על ידי לחיצה ימנית ובחירה 'Edit Interface' - בלשונית Parameters העברנו את הפרמטר POLARITY לעריכה ידנית, והגדרנו אותו במפורש כ- ACTIVE_HIGH. פעולה זו מבטיחה שיוואדו תזהה את הפינים נכונה ולא תוסיף להם היפוך לוגי בשרטוט המערכת.





נעבור ללשונית Addressing and Memory. עלינו להגדיר את טווח הכתובות שממשק ה-AXI יתפוס במרחב הזיכרון של המעבד.

נגדיר את ערך ה- Range להיות 4096 בתים, כפי שמוצג בחלון ההגדרות.



IP Address Block Properties

reg0

Name: reg0

Display name: reg0

Description:

Base address: 0x0

Base address format: bitString

Base address bit string length: 1

Base address resolve type: immediate

IP Address Block Properties

reg0

Base address format: bitString

Base address bit string length: 1

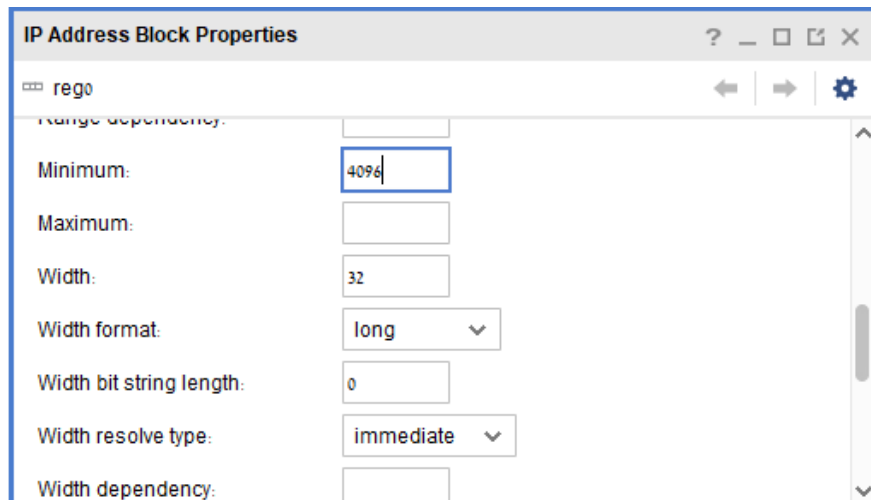
Base address resolve type: immediate

Base address dependency:

Range: 0x4000

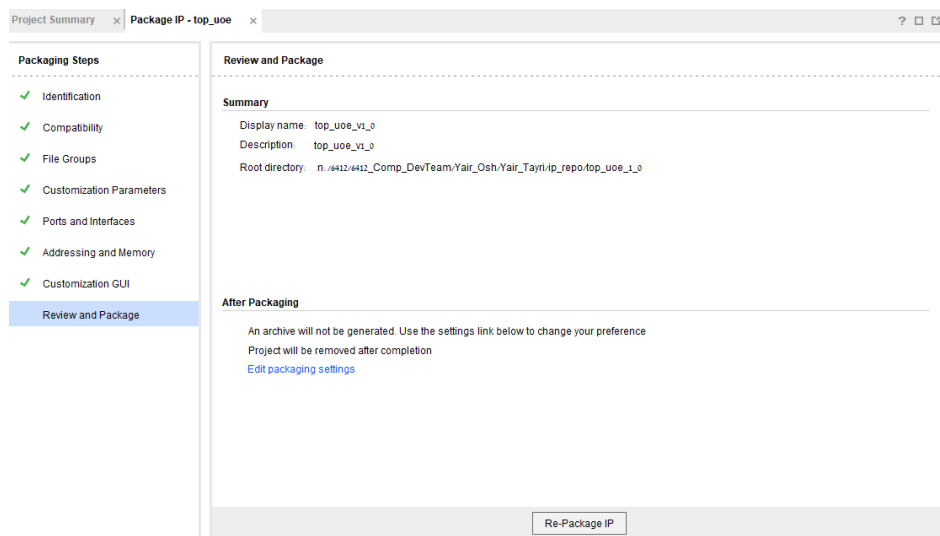
Range format: bitString

Range bit string length: 15

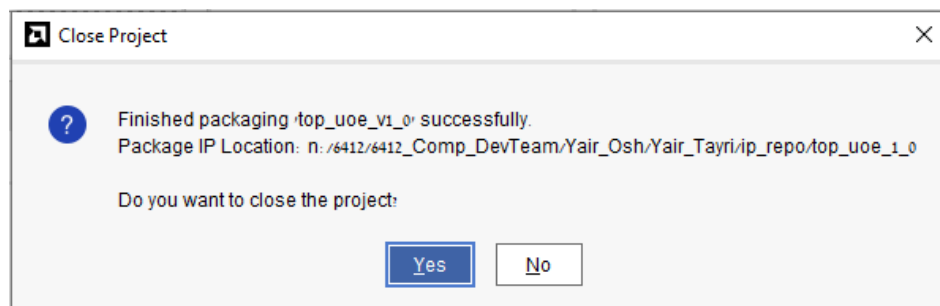


The image shows a dialog box titled "IP Address Block Properties". It contains several input fields and a dropdown menu. The "Minimum" field is highlighted with a blue border and contains the value "4096". The "Maximum" field is empty. The "Width" field contains the value "32". The "Width format" dropdown menu is set to "long". The "Width bit string length" field contains the value "0". The "Width resolve type" dropdown menu is set to "immediate". The "Width dependency" field is empty.

לסיום תהליך האריזה, נעבור ללשונית Review and Package ונלחץ על הכפתור Re-
Package IP. הודעת סיום תאשר שהתהליך עבר בהצלחה, כפי שמוצג.



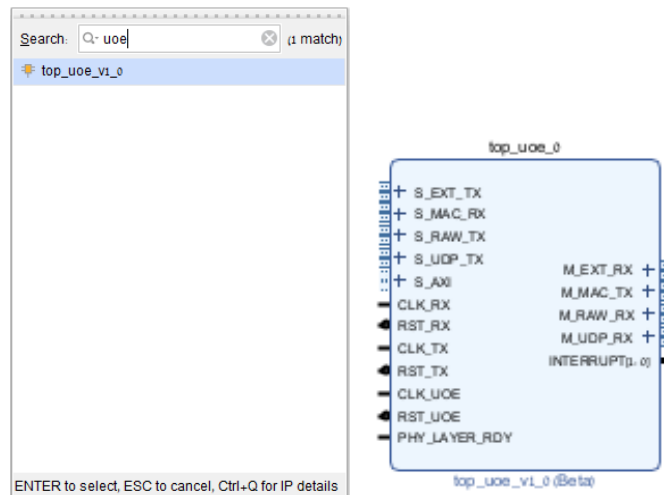
The image shows a dialog box titled "Review and Package". It has a left sidebar with "Packaging Steps" and a main area with "Review and Package" details. The "Packaging Steps" list includes: Identification, Compatibility, File Groups, Customization Parameters, Ports and Interfaces, Addressing and Memory, Customization GUI, and Review and Package (which is selected). The "Review and Package" section shows a "Summary" with the following details: Display name: top_uoe_v1_0, Description: top_uoe_v1_0, Root directory: n:/6412/6412_Comp_DevTeam/Yair_Osh/Yair_Tayri/ip_repo/top_uoe_v1_0. Below the summary, there is an "After Packaging" section with the text: "An archive will not be generated. Use the settings link below to change your preference. Project will be removed after completion. [Edit packaging settings](#)". At the bottom right, there is a button labeled "Re-Package IP".



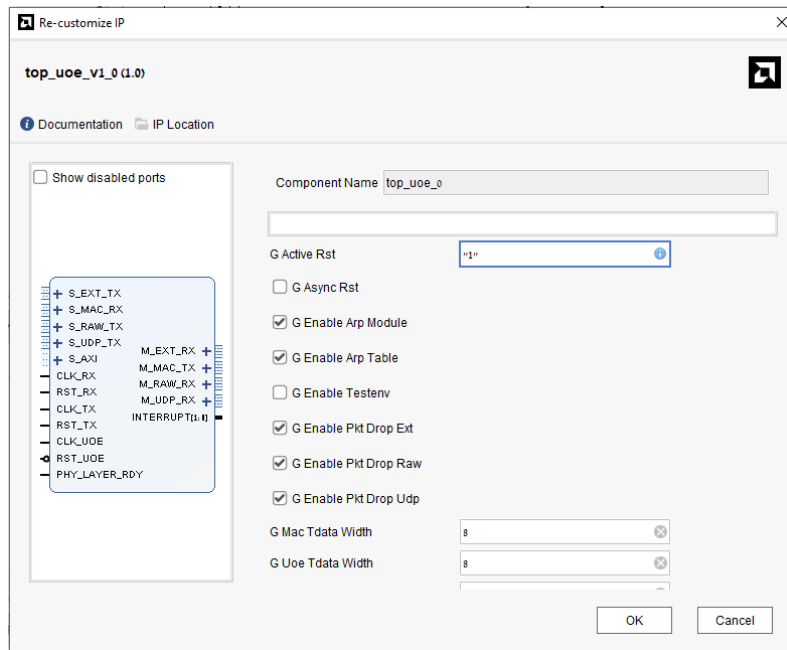
The image shows a dialog box titled "Close Project". It contains a message: "Finished packaging 'top_uoe_v1_0' successfully. Package IP Location: n:/6412/6412_Comp_DevTeam/Yair_Osh/Yair_Tayri/ip_repo/top_uoe_v1_0". Below the message, it asks: "Do you want to close the project:". There are two buttons: "Yes" and "No".

כעת הבלוק מוכן להוספה לשרטוט החומרה הראשי.

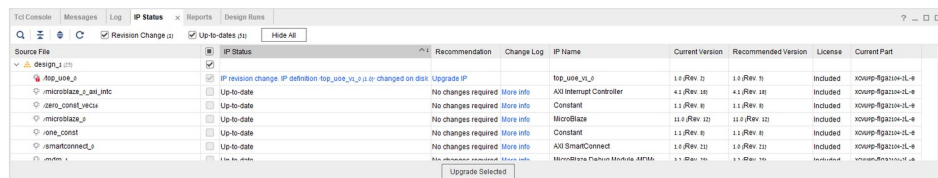
נפתח את שרטוט המערכת הראשי ונוסיף את בלוק ה-`top\_uoe` שיצרנו אל קנבס העבודה.



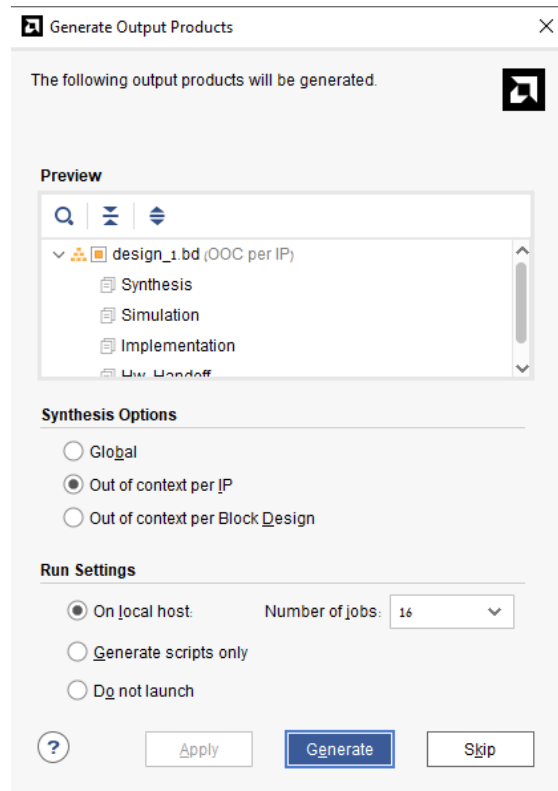
נכנסנו להגדרות הרכיב ושינינו את הפרמטר G Active Rst לערך 1, במטרה להתאים את קוטביות האיפוס של הבלוק ללוגיקת ה- Active High שהוגדרה במערכת.



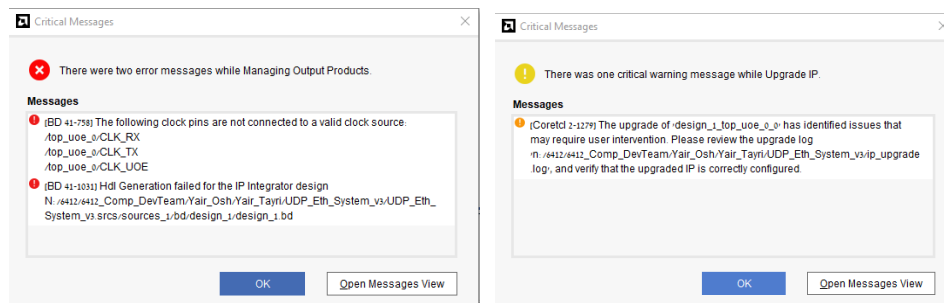
ביצענו עדכון לגרסאות ה- IP במערכת דרך לשונית IP Status ולחצנו על Upgrade Selected.



הפעלנו את תהליך יצירת קבצי התכנון Generate Output Products.

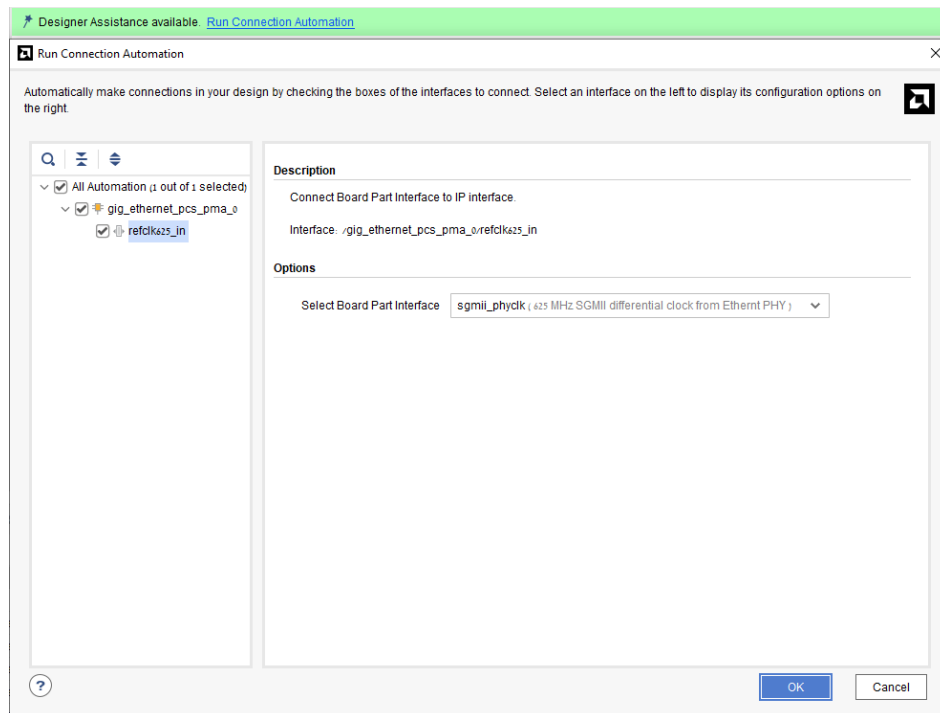


תהליך זה נכשל והציג שגיאה, מכיוון שפיני השעון של בלוק ה- CLK_RX (UOE) ו-CLK_TX_CLK_UOE טרם חוברו לענן שער תקין במערכת.



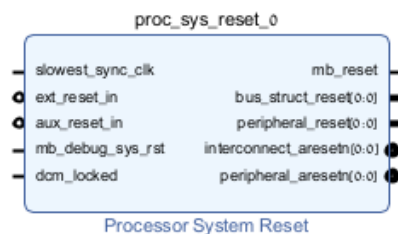
בחלק העליון של המסך מופיע פס ירוק המציע סיוע אוטומטי בחיבורים. נלחץ על Run Connection Automation.

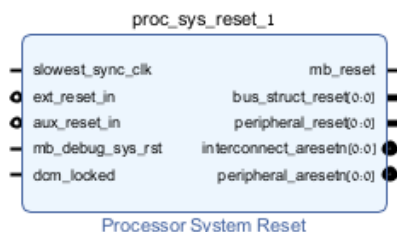
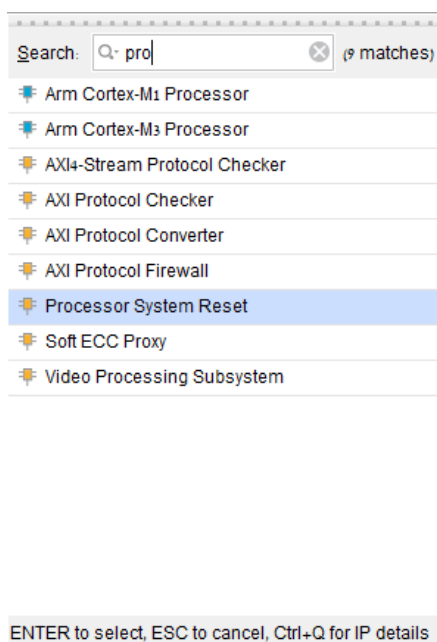
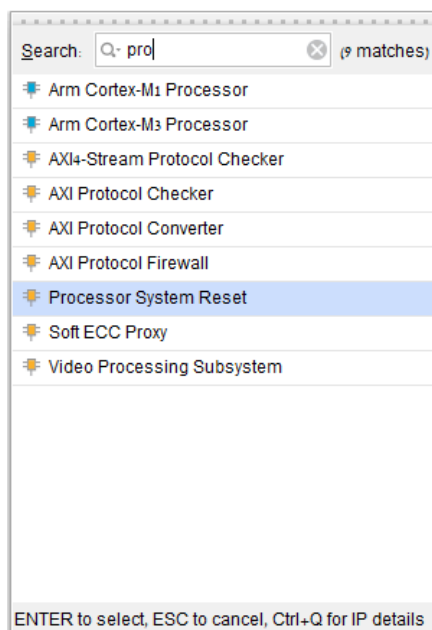
נסמן את ממשק `S\_AXI` של ה-UOE, ונאשר, כדי שיוואדו יחבר אותו אוטומטית לערוץ הבקרה של ה-MicroBlaze.



שלב זה קריטי לסנכרון המערכת לתדר העבודה הנדרש של 125 MHz.

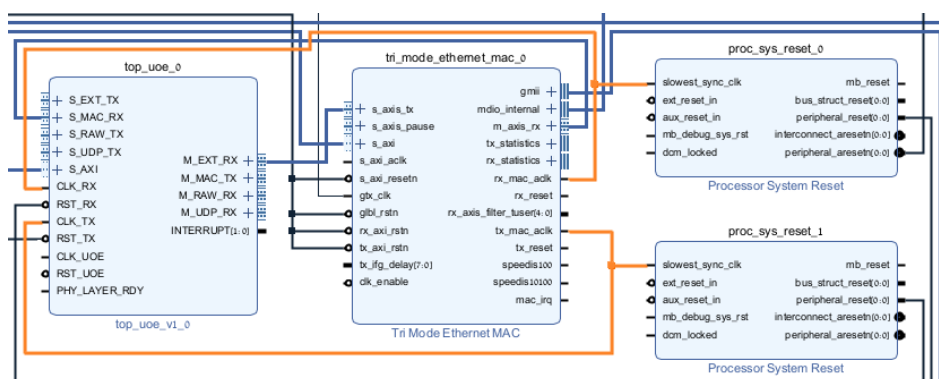
נוסיף בלוק מסוג `Processor System Reset` כדי לטפל באותות האתחול של תדר ה-125 MHz.

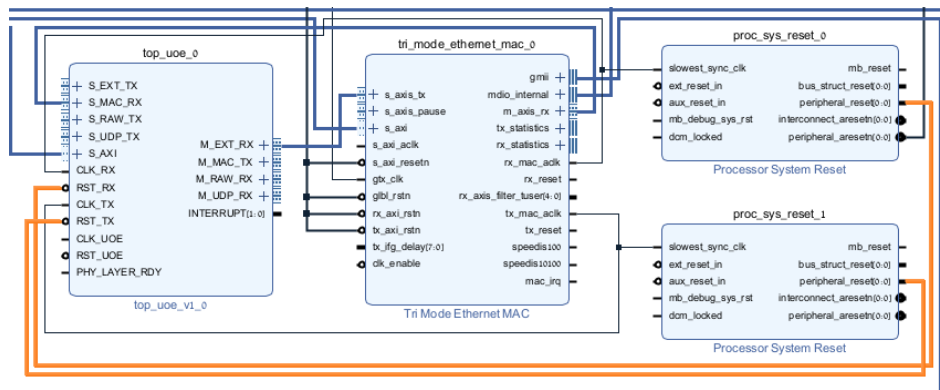




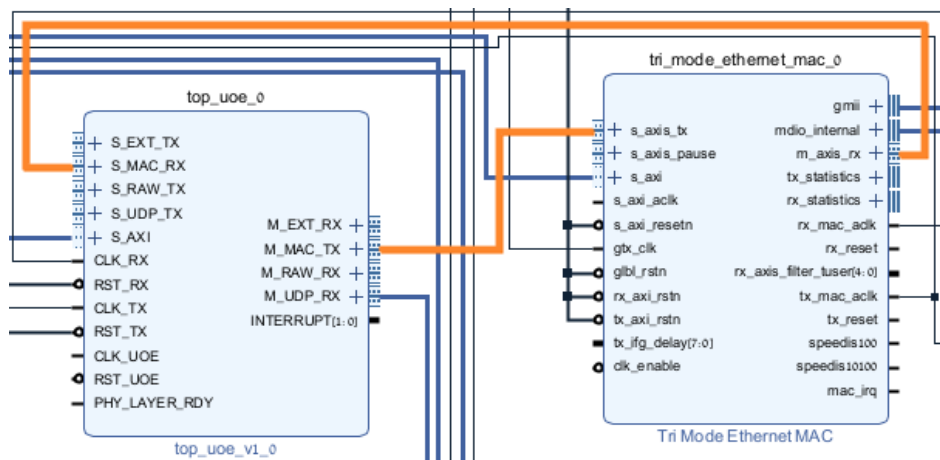
נחבר את כל כניסות השעון של ליבת ה-UOE לקו השעון המרכזי של 125 MHz.

במקביל נחבר את כניסות הריסט השונות של ה-UOE למוצא ה-peripheral_reset (הפעיל בגבוה) של בלוק ה-Processor System Reset שחוספנו.

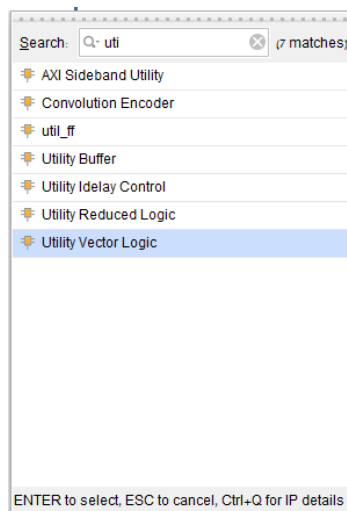




נחבר את ערוץ 'M_MAC_TX' של ה-UOE לכניסת השידור ב-MAC, ואת ערוץ 'S_MAC_RX' ליציאת הקליטה מה-MAC.

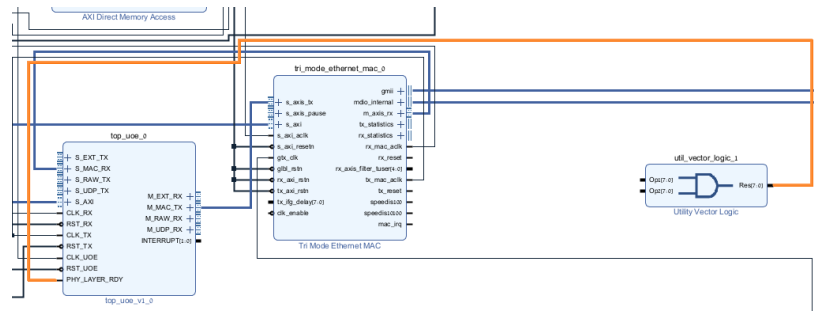


נוסיף בלוק מסוג 'Utility Vector Logic' ונגדיר אותו כשער AND.

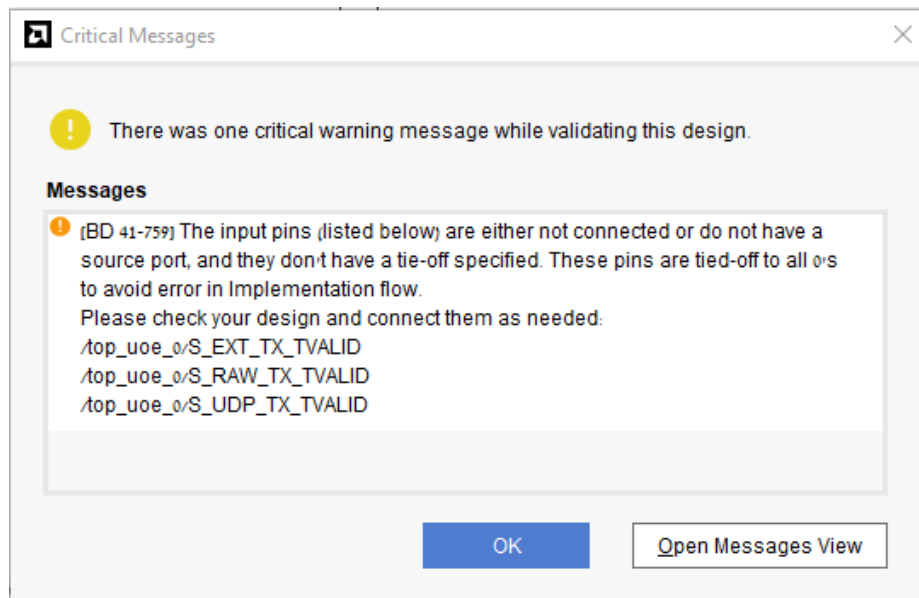


שער זה יקבל את אותות הנעילה והמוכנות של רכיב ה-PHY הפיזי, ורק כאשר כולם תקינים, הוא יעביר אות '1' לוגי אל פין 'PHY_LAYER_RDY' בליבת ה-UOE.

מראה השרטוט הסופי של המערכת, הכולל את ליבת ה-UOE מחוברת כראוי ל-MAC, לשעונים ולשער ה-AND.

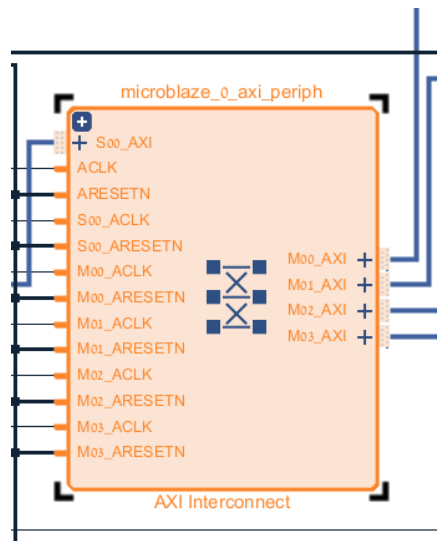


נלחץ על מקש F6 להרצת כלי ה- Validate Design כדי לוודא שאין טעויות מיפוי או שעונים. ויוודאו יציג אזהרה קריטית המציינת שחלק מפיני ה- TVALID קשורים לאפס עקב אי-חיבורם למקורות נתונים חיצוניים. אזהרה זו צפויה ותקינה לחלוטין לשלב הנוכחי בתכנון, כפי שמוצג בהודעת המערכת.



העיצוב נמצא כעת במצב יציב ומאומת.

כדי לאפשר ל-Microblaze לתקשר עם הבלוקים החדשים שהוספנו כגון ה-UOE וה-MAC, ערוצי התקשורת בתוך ה- (microblaze_0_axi_periph) AXI Interconnect הורחבו אוטמטית. פעולה זו הוסיפה פורטי (M0x_AXI) Master המאפשרים למעבד לכתוב ולקרוא מרשמי הבקרה של הבלוקים השונים.

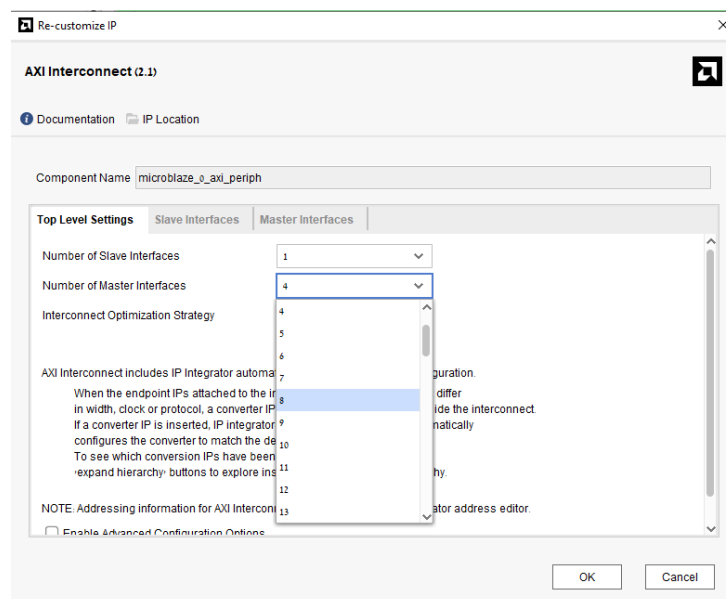


שלב 15:

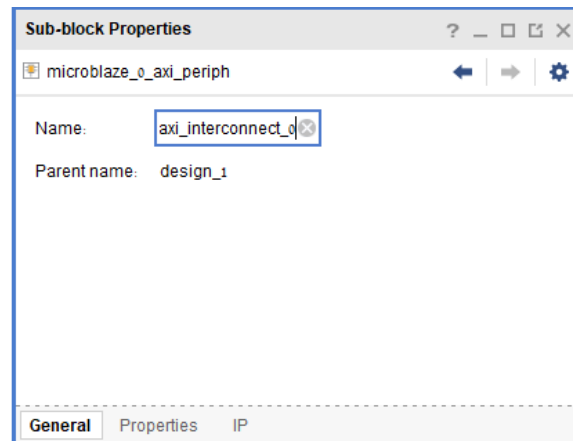
בשלב זה ביצענו יישור קו של תשתיות השרטוט כדי להבטיח עבודה תקינה מול רכיבי החומרה על הלוח וסנכרון מלא בין המעבד לליבת ה-UOE.

בקר ה-Interconnect משמש כצומת המרכזי של המערכת. כדי לאפשר חיבור של כלל הפריפריות, ביצענו את השינויים הבאים:

פתחנו את חלון ההגדרות של הבלוק ושינינו את ערך Number of Master Interfaces מ-4 ל-8. פעולה זו יצרה פורטים חדשים (M04 עד M07) עבור רכיבי הזיכרון והתקשורת הנוספים.



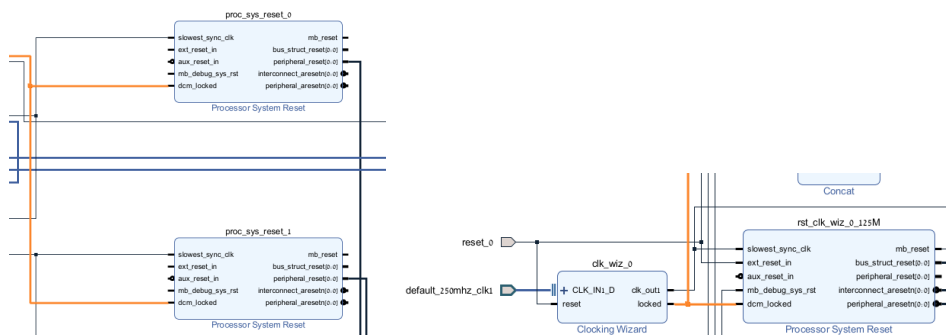
שינינו את שם הרכיב ל- axi_interconnect_0 בחלון ה-Properties כדי לשמור על עקביות מול תיעוד הפרויקט.



כדי למנוע מצב שבו רכיבי המערכת מתחילים לפעול לפני שהשעונים יציבים, השתמשנו באות ה- Locked:

מתחנו קו מציאת ה- locked של ה- Clocking Wizard וחברנו אותו לכניסת ה- dcm_locked בכל שלושת בלוקי ה- Reset הקיימים בשרטוט:
`\proc_sys_reset_0`
`\rst_clk_wiz_0_125M`,
`\proc_sys_reset_1`.

חיבור זה מבטיח שכלל הלוגיקה במעבד ובתקשורת תצא ממצב Reset רק לאחר סנכרון מלא של המתנדמים.

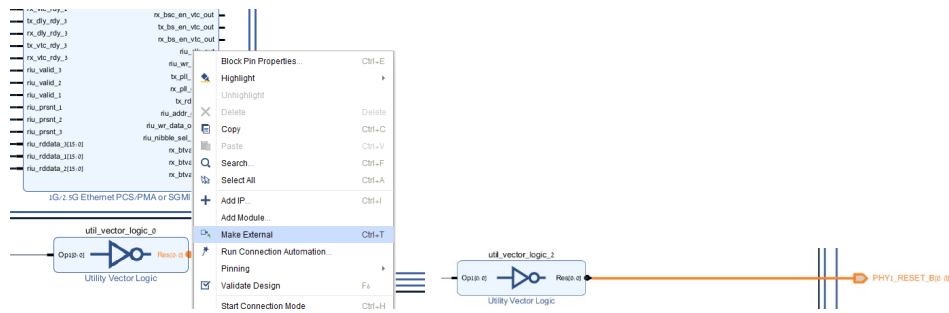


חומרת ה- Ethernet על הלוח דורשת אות אתחול בשיטה של Active Low:

השתמשנו בבלוק Utility Vector Logic שהוגדר כשער NOT.

הכניסה לשער מגיעה מאות הריסט הפנימי, והמוצא שלו חובר לפורט חיצוני חדש שיצרנו בשם PHY1_RESET_B.

מנגנון זה מאפשר למערכת לאפס את רכיב ה-PHY הפיזי בצורה מבוקרת בעת אתחול הפרויקט.

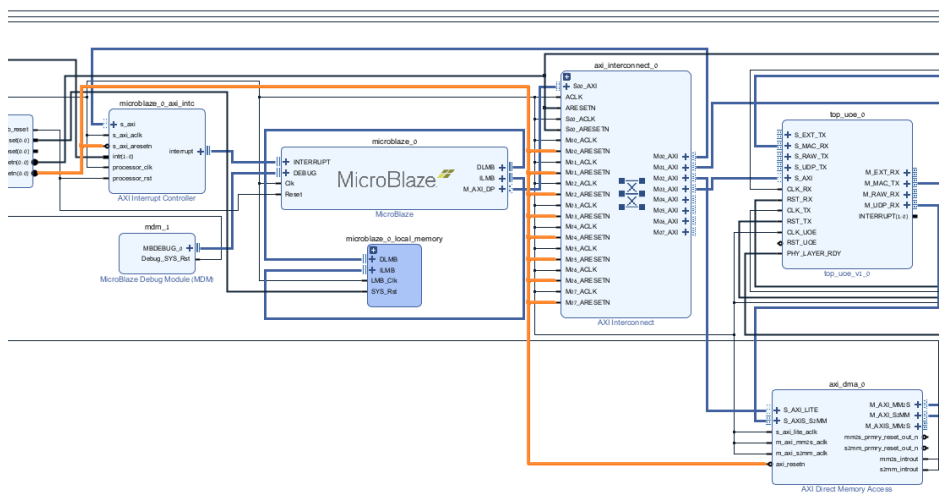


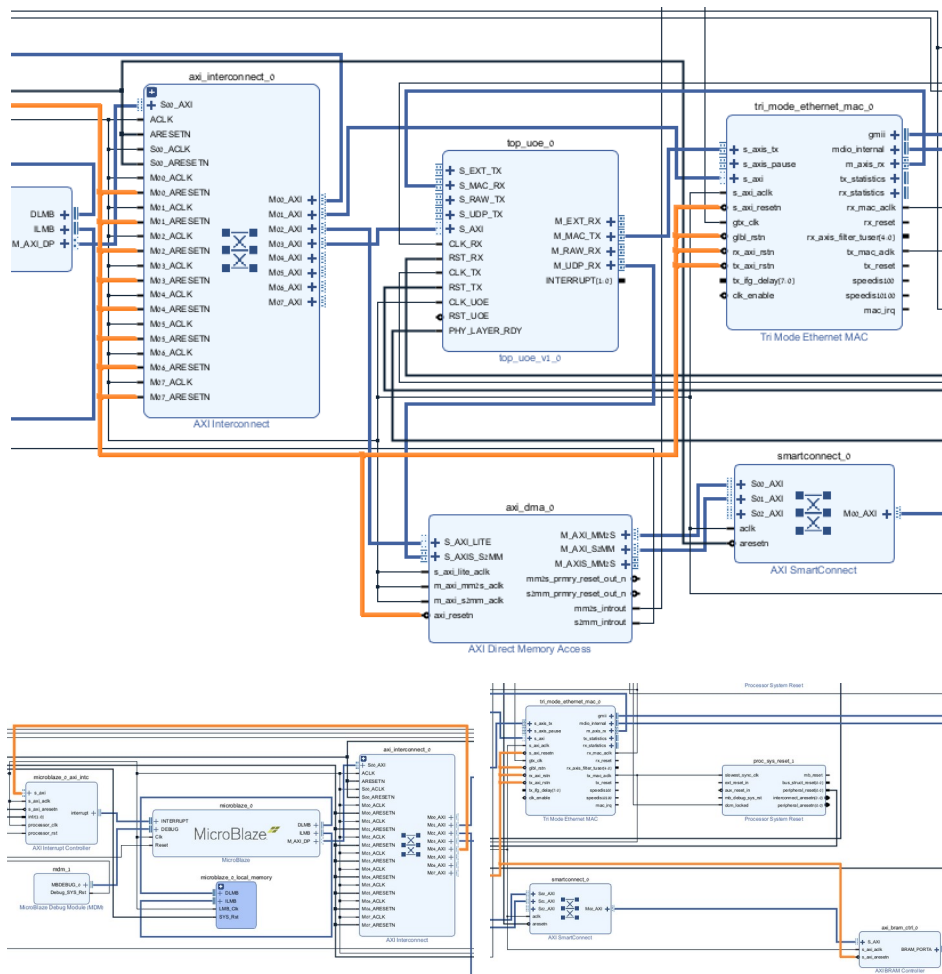
בשלב האחרון חיברנו את הרכיבים לפורטים החדשים ב-Interconnect:

חיברנו את ממשק ה-`S\_AXI` של `axi\_bram\_ctrl\_0` לפורט ה-M04 ב-Interconnect.

וידאנו שכלל הפסיקות מה-DMA נאספות בבלוק ה-Concat ומועברות לבקר הפסיקות (INTC) המחובר למעבד.

וידאנו שכל הפינים החדשים ב-Interconnect (`ACLK` ו-`ARESETN`) מחוברים לקווי השעון של 125 MHz ולקווי הריסט המתאימים.



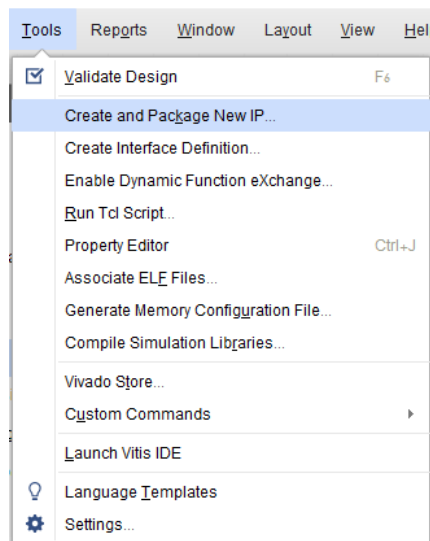


שלב 16:

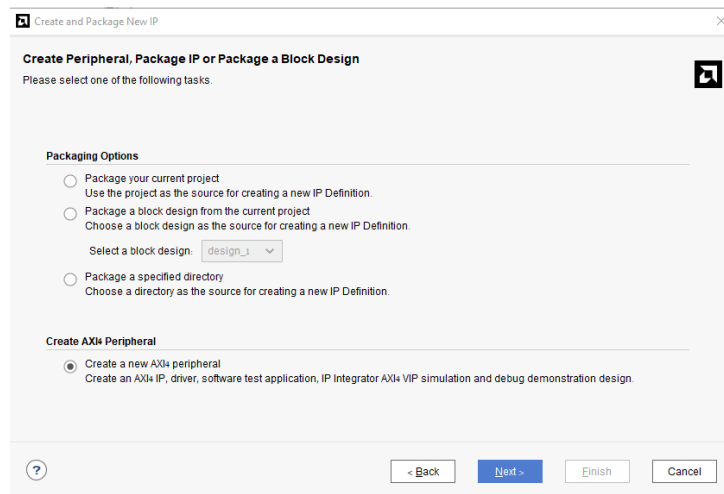
בשלב זה ניצור רכיב חומרה מותאם אישית. התפקיד של הרכיב הזה הוא לאסוף מידע חשוב מהמעבד כמו כתובות MAC, כתובות IP ומספרי פורטים ולארוז אותם לתוך ערוץ מידע הנקרא TUSER. המידע הזה הכרחי כדי שחבילות הרשת יגיעו ליעדן בצורה תקינה.

תחילה נייצר תבנית בסיסית של רכיב תקשורת מול המעבד.

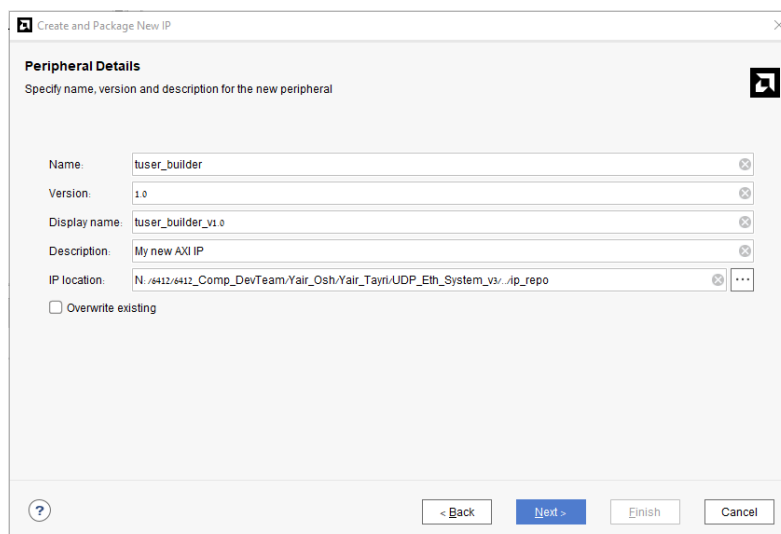
בתפריט העליון נלחץ על Tools ובחר באפשרות Create and Package New IP.



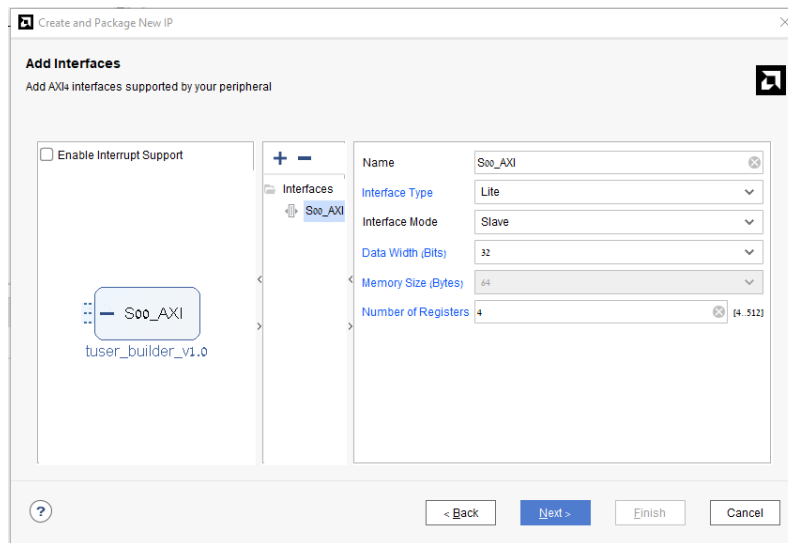
באשף שייפתח נבחר ליצור רכיב חדש מסוג Create a new AXI4 peripheral.



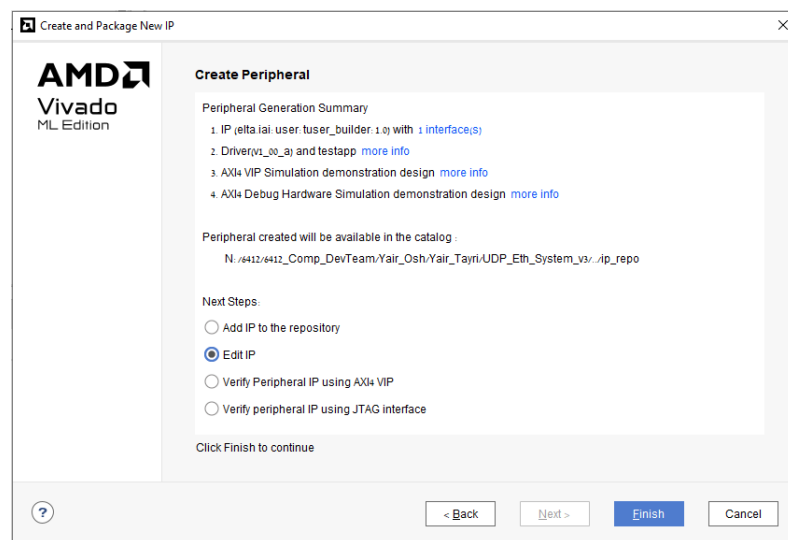
בחלון ההגדרות נמלא את פרטי הזיהוי של הרכיב, בשדה השם נכתוב tuser_builder.



נגדיר את ממשק התקשורת כממשק מסוג AXI4 Lite המוגדר כ-Slave, עם רוחב נתונים של 32 סיביות ו-4 רגיסטרים.



בסיום האשף נבחר באפשרות Edit IP ונלחץ על Finish. פעולה זו תפתח פרויקט חדש וזמני לעריכת קוד הרכיב.



כעת נשנה את קבצי ה-VHDL שנוצרו כדי להוסיף להם את הפעילות הייחודית של הרכיב שלנו.

פתחנו את הקובץ המרכזי והרחבנו את הזיכרון שלו מ- 4 רגיסטרים ל- 5 רגיסטרים, כדי שיהיה מקום לאותות נוספים כמו מצב המוכנות של הרכיב הפיזי. בנוסף, הרחבנו את קווי הכתובות כדי שהמערכת תוכל לגשת לרגיסטר החמישי.

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity tuser_builder_v1_0_S00_AXI is
6      generic (
7          -- Users to add parameters here
8
9          -- User parameters ends
10         -- Do not modify the parameters beyond this line
11
12         -- Width of S_AXI data bus
13         C_S_AXI_DATA_WIDTH  : integer := 32;
14         -- Width of S_AXI address bus
15         C_S_AXI_ADDR_WIDTH  : integer := 5;
16     );

```

בסוף הקובץ מיפינו את המידע מתוך הרגיסטרים השונים אל תוך וקטור אחד ארוך של 80 סיביות.

```

18  -- Users to add ports here
19  | tuser_reg_out      : out std_logic_vector(79 downto 0);
20  phy_layer_rdy_reg_out : out std_logic;
21  -- User ports ends

```

עדכנו את הקבוע OPT_MEM_ADDR_BITS לערך 2. שינוי זה מרחיב את מרחב הכתובות הפנימי של הבלוק, מה שמאפשר פנייה ל- 5 עד 8 רגיסטרים.

הגדרנו את האות החדש slv_reg4 כווקטור בגודל 32 סיביות, בדומה לשאר הרגיסטרים של הממשק.

```

107  constant OPT_MEM_ADDR_BITS : integer := 2;

111  ---- Number of Slave Registers 4
112  signal slv_reg0 : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
113  signal slv_reg1 : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
114  signal slv_reg2 : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
115  signal slv_reg3 : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
116  signal slv_reg4 : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
117  signal slv_reg_rden : std_logic;
118  signal slv_reg_wren : std_logic;
119  signal reg_data_out : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
120  signal byte_index  : integer;
121  signal aw_en       : std_logic;

```

הוספנו את slv_reg4 ללוגיקת האיפוס של הבלוק, כך שיקבל ערך התחלתי של אפס כאשר אות ה- S_AXI_ARESETN מפעיל.

```

217  if rising_edge(S_AXI_ACLK) then
218  | if S_AXI_ARESETN = '0' then
219      slv_reg0 <= (others => '0');
220      slv_reg1 <= (others => '0');
221      slv_reg2 <= (others => '0');
222      slv_reg3 <= (others => '0');
223  | slv_reg4 <= (others => '0');
224  else

```

הוספנו תנאי ("when b"000") בתוך תהליך הכתיבה של ה-AXI. קטע קוד זה מפענח את הכתובת הנכנסת ומאפשר למעבד לכתוב נתונים לתוך הרגיסטר החדש slv_reg4 בהיסט 4.

```

228 when b"000" =>
229   for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
230     if ( S_AXI_WSTRB(byte_index) = '1' ) then
231       -- Respective byte enables are asserted as per write strobes
232       -- slave register 0
233       slv_reg0(byte_index*8+7 downto byte_index*8) <= S_AXI_WDATA(byte_index*8+7 downto byte_index*8);
234     end if;
235   end loop;
236 when b"001" =>
237   for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
238     if ( S_AXI_WSTRB(byte_index) = '1' ) then
239       -- Respective byte enables are asserted as per write strobes
240       -- slave register 1
241       slv_reg1(byte_index*8+7 downto byte_index*8) <= S_AXI_WDATA(byte_index*8+7 downto byte_index*8);
242     end if;
243   end loop;
244 when b"010" =>
245   for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
246     if ( S_AXI_WSTRB(byte_index) = '1' ) then
247       -- Respective byte enables are asserted as per write strobes
248       -- slave register 2
249       slv_reg2(byte_index*8+7 downto byte_index*8) <= S_AXI_WDATA(byte_index*8+7 downto byte_index*8);
250     end if;
251   end loop;
252 when b"011" =>
253   for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
254     if ( S_AXI_WSTRB(byte_index) = '1' ) then
255       -- Respective byte enables are asserted as per write strobes
256       -- slave register 3
257       slv_reg3(byte_index*8+7 downto byte_index*8) <= S_AXI_WDATA(byte_index*8+7 downto byte_index*8);
258     end if;
259   end loop;
260 when b"100" =>
261   for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
262     if ( S_AXI_WSTRB(byte_index) = '1' ) then
263       -- Respective byte enables are asserted as per write strobes
264       -- slave register 4
265       slv_reg4(byte_index*8+7 downto byte_index*8) <= S_AXI_WDATA(byte_index*8+7 downto byte_index*8);
266     end if;
267   end loop;
268 when others =>
269   slv_reg0 <= slv_reg0;
270   slv_reg1 <= slv_reg1;
271   slv_reg2 <= slv_reg2;
272   slv_reg3 <= slv_reg3;
273   slv_reg4 <= slv_reg4;
274 end case;
275 end if;
276 end if;
277 end if;
278 end process;

```

הוספנו את slv_reg4 לרשימת הרגישות של תהליך הקריאה, כך שהתהליך יתעדכן בכל שינוי בערך הרגיסטר.

```

361 process (slv_reg0, slv_reg1, slv_reg2, slv_reg3, slv_reg4, axi_araddr, S_AXI_ARESETN, slv_reg_rden)

```

הוספנו תנאי פיענוח ("when b"100") בתהליך הקריאה, המאפשר למעבד לדגום ולקרוא בחזרה את התוכן של slv_reg4.

```

364 -- Address decoding for reading registers
365 loc_addr := axi_araddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto ADDR_LSB);
366 case loc_addr is
367   when b"000" =>
368     reg_data_out <= slv_reg0;
369   when b"001" =>
370     reg_data_out <= slv_reg1;
371   when b"010" =>
372     reg_data_out <= slv_reg2;
373   when b"011" =>
374     reg_data_out <= slv_reg3;
375   when b"100" =>
376     reg_data_out <= slv_reg4;
377   when others =>
378     reg_data_out <= (others => '0');
379 end case;
380 end process;

```

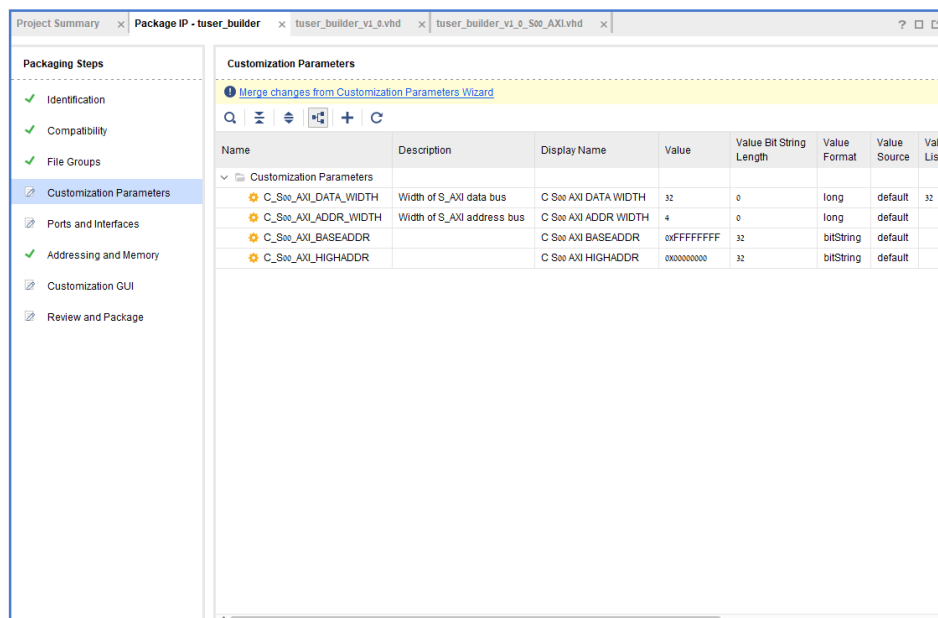
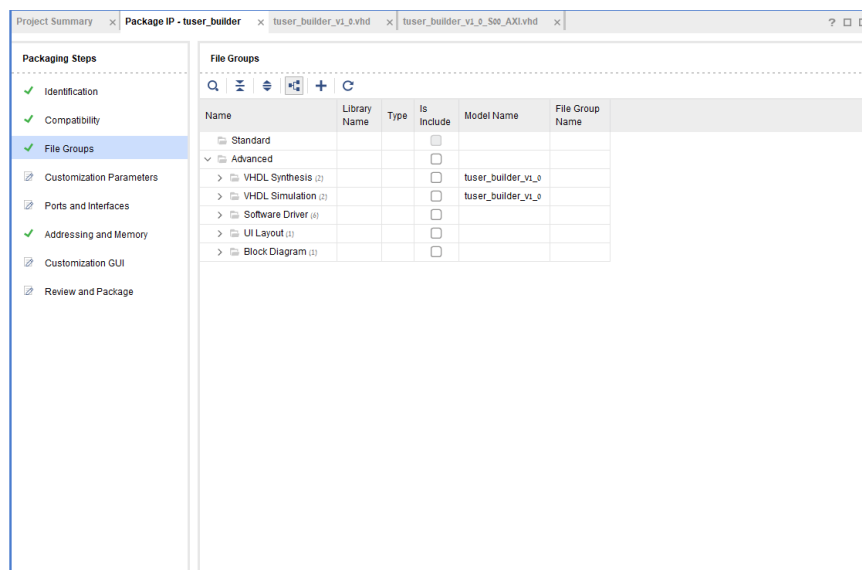
בשלב זה ניתבנו את הנתונים מתוך הרגיסטרים אל הווקטור הכולל tuser_reg_out ברוחב 80 סיביות, תוך חלוקה מדויקת לשדות המידע.

```

401 | -- Add user logic here
402 | tuser_reg_out(31 downto 0) <= slv_reg0(31 downto 0); --IP destination
403 | tuser_reg_out(47 downto 32) <= slv_reg1(15 downto 0); --lenght
404 | tuser_reg_out(63 downto 48) <= slv_reg2(15 downto 0); --src port
405 | tuser_reg_out(79 downto 64) <= slv_reg3(15 downto 0); --des port
406 | phy_layer_rdy_reg_out <= slv_reg4(0);
407 |
408 | -- User logic ends

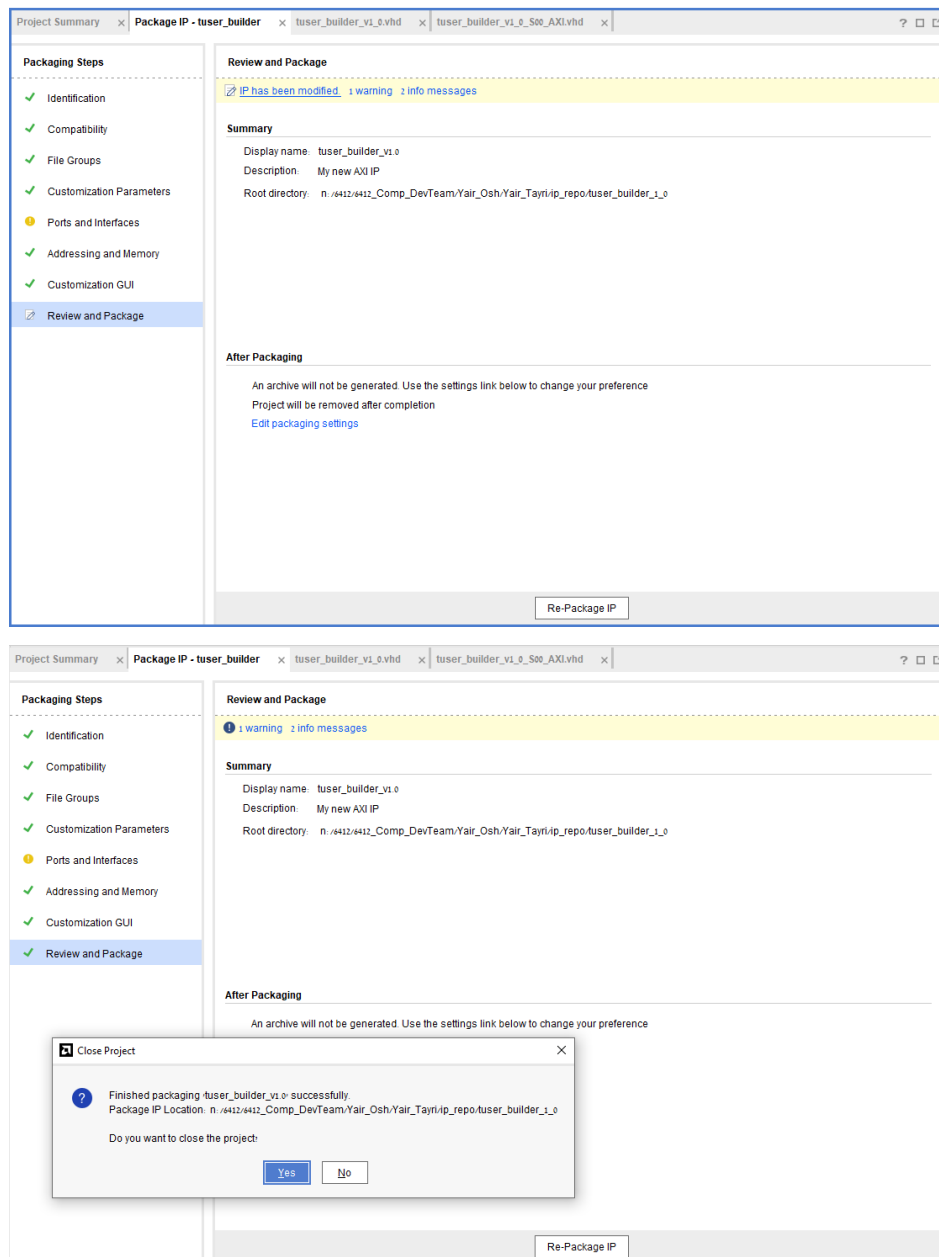
```

בקובץ הראשי של הרכיב הוספנו את הפינים החיצוניים שיתחברו לשאר המערכת. כמו כן הוספנו מנגנון נעילה חכם. מנגנון זה מזהה מתי חבילת נתונים מתחילה להישלח ונועל את הנתונים, כדי למנוע מצב שבו המעבד משנה נתונים באמצע שידור והורס את החבילה.

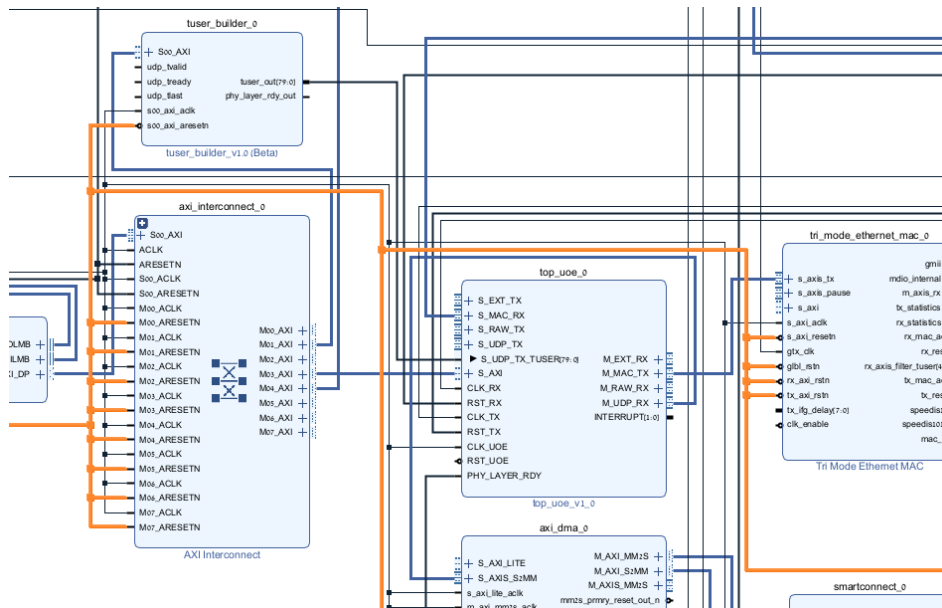


לאחר סיום כתיבת הקוד, נשמור את הקבצים ונחזור לחלון ה- Package IP כדי לעדכן את אריזת הרכיב. נעבור על הלשוניות השונות ונלחץ על הקישור הצהוב Merge changes כדי לסנכרן את התוכנה עם הפורטים והפרמטרים החדשים שהוספנו לקוד VHDL.

נעבור ללשונית האחרונה ונלחץ על הכפתור Re-Package IP. הפרויקט הזמני ייסגר והרכיב שלנו יהיה מוכן לשימוש.

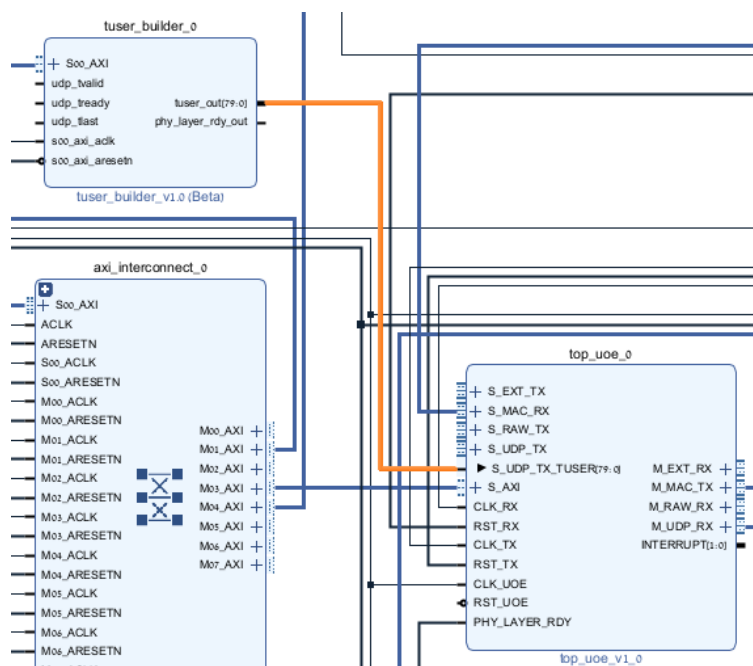


השלב האחרון הוא לשים את הרכיב המוגמר בתוך השרטוט הראשי ולחבר אותו.



כדי שהרכיב יידע מתי מתחיל שידור, נמתח חוטים מהפינים של תוקף הנתונים וסיום החבילה אל הפינים המקבילים בליבת ה-UOE.

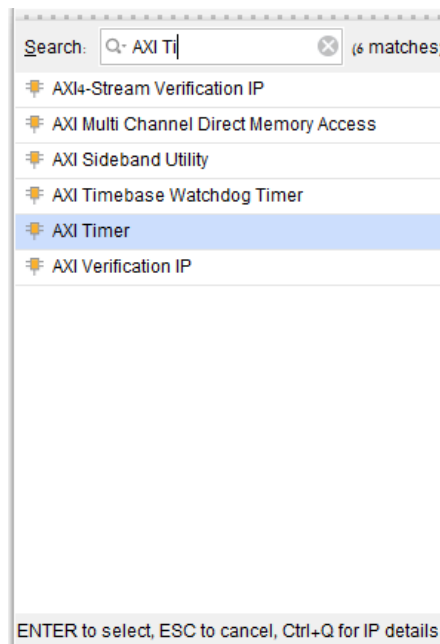
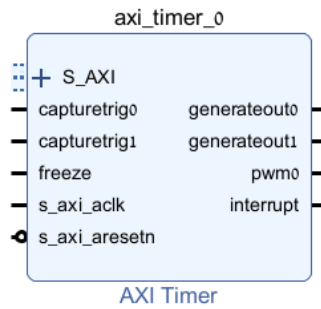
ביצענו את החיבור הייעודי בין הבלוק החדש למנוע ה-UOE. חיווטנו את אפיק הנתונים tuser_out[79:0] המכיל את כתובת ה-IP, הפורטים והאורך ישירות לכניסת S_UDP_TX_TUSER של בלוק ה-top_uoe_0. ה-UOE ישתמש בוקטור זה כדי לעטוף ולנתב נכון כל חבילת UDP שיוצאת מה-DMA החוצה לרשת.



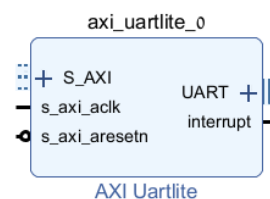
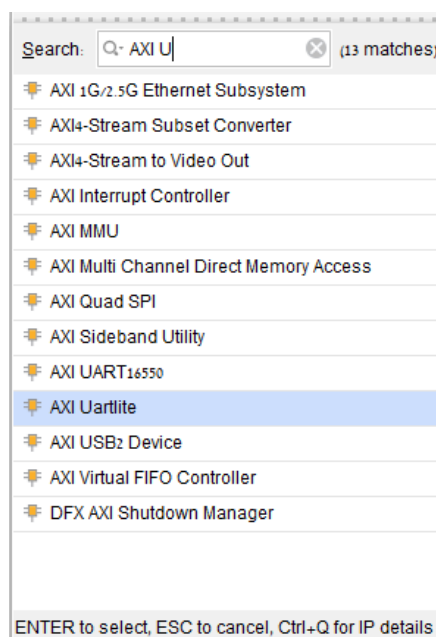
שלב 17:

בשלב זה נוסיף למערכת שני רכיבים חדשים ונחבר אותם אל בקר התקשורת המרכזי. הרכיבים שנוסיף הם טיימר למדידת זמנים ורכיב תקשורת טורית שמאפשר להדפיס הודעות למחשב.

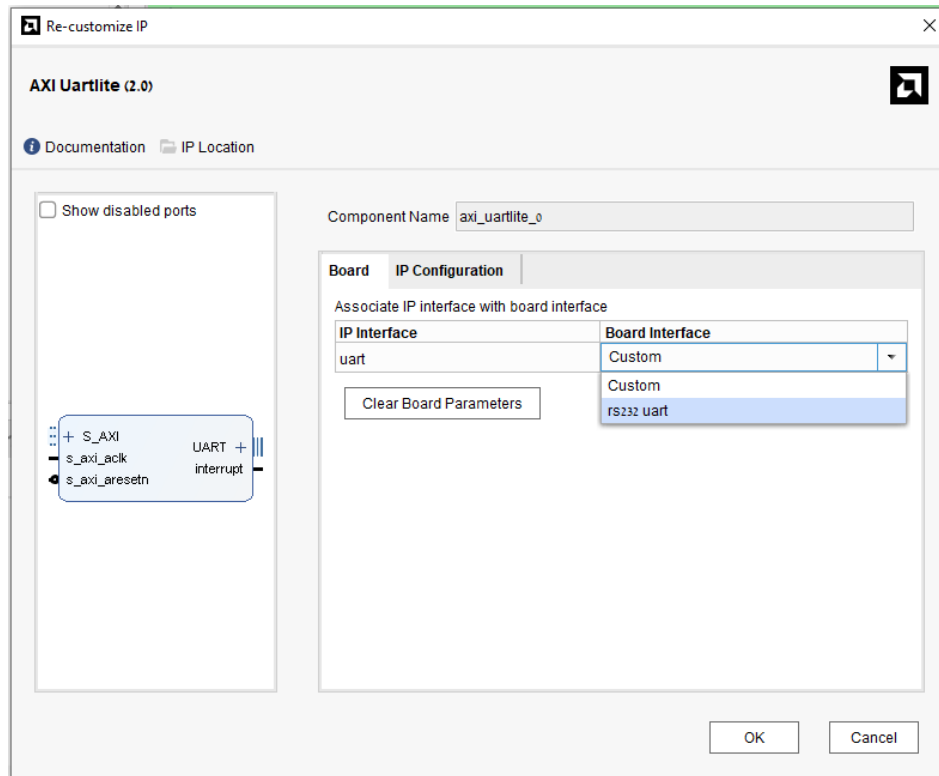
נפתח את קטלוג הרכיבים ונחפש את הרכיב הראשון שנקרא AXI Timer.



לאחר מכן נחפש ונוסיף את הרכיב השני שנקרא AXI Uartlite.

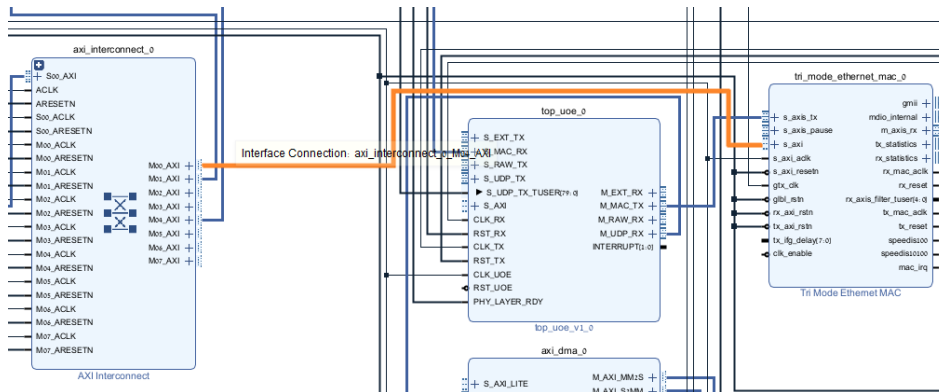


כדי שהרכיב יתאים ללוח שלנו נלחץ עליו פעמיים. בחלון שנפתח נעבור ללשונית Board ונבחר באפשרות rs232 uart במקום המילה Custom.

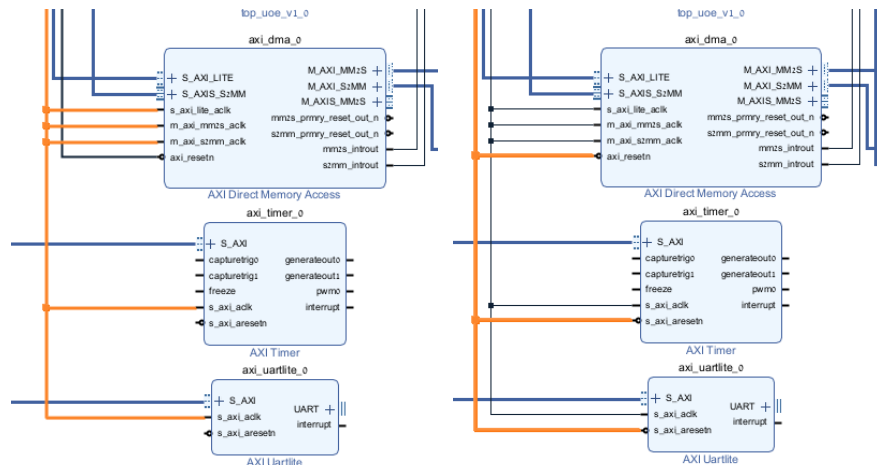


פעולה זו מבטיחה שהתוכנה תכיר את החיבורים הפיזיים של הלוח.

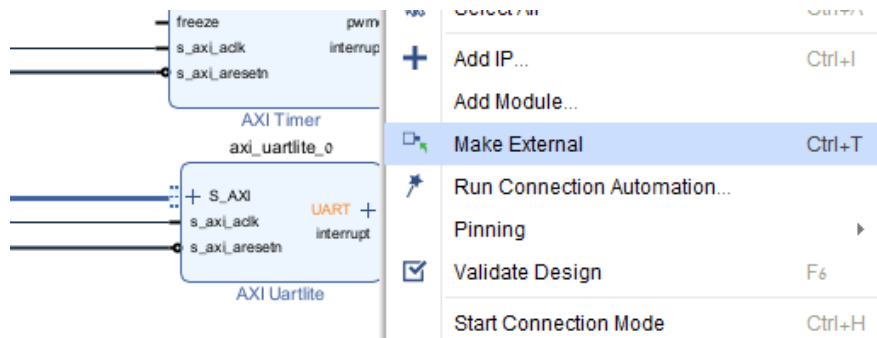
כדי שהמעבד יוכל לשלוט בטיימר ולקרוא את המידע מה- UART, חיברנו את ממשקי ה- S_AXI שלהם אל פורטים פנויים בבקר התקשורת המרכזי axi_interconnect_0.



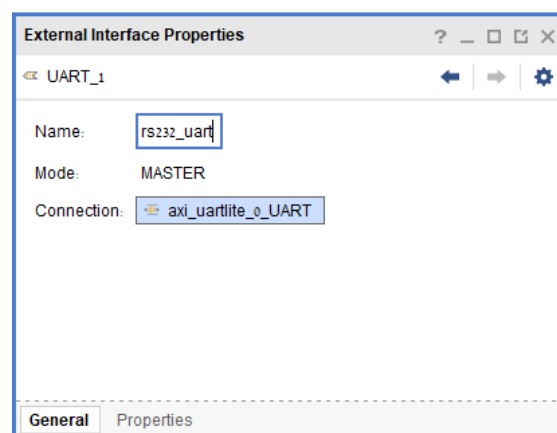
כל הרכיבים במערכת חייבים לעבוד בסנכרון. חיווטנו את קווי השעון `s_axi_aclk` והאיפוס `s_axi_aresetn` של Timer ה- וה- UART לאותו מקור שעון מרכזי המזין את ה- DMA ושאר המערכת.



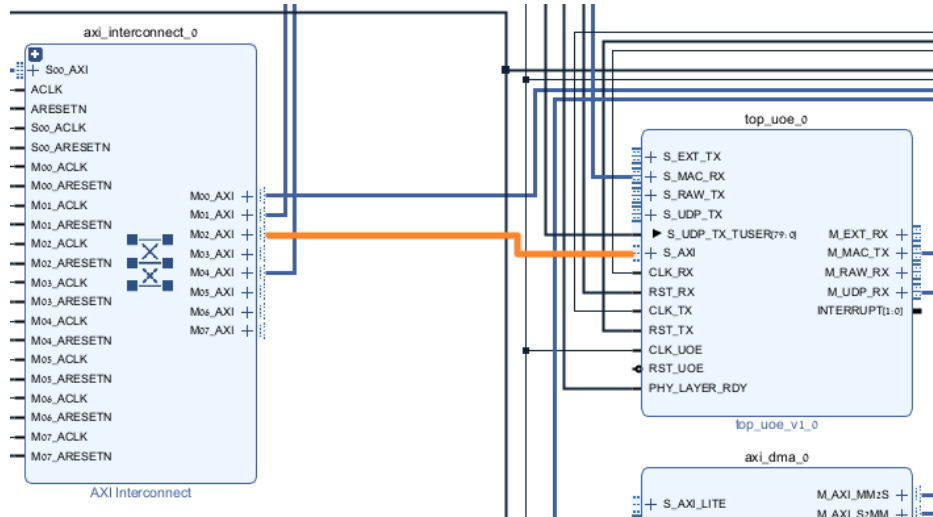
כדי שנוכל לחבר כבל פיזי Serial מהלוח למחשב ולהדפיס הודעות, עלינו לנתב את אותות ה- UART החוצה מהשבב. לחצנו קליק ימני על ממשק ה- UART בבלוק ה- `axi_uartlite_0` ובחרנו באפשרות `Make External`.



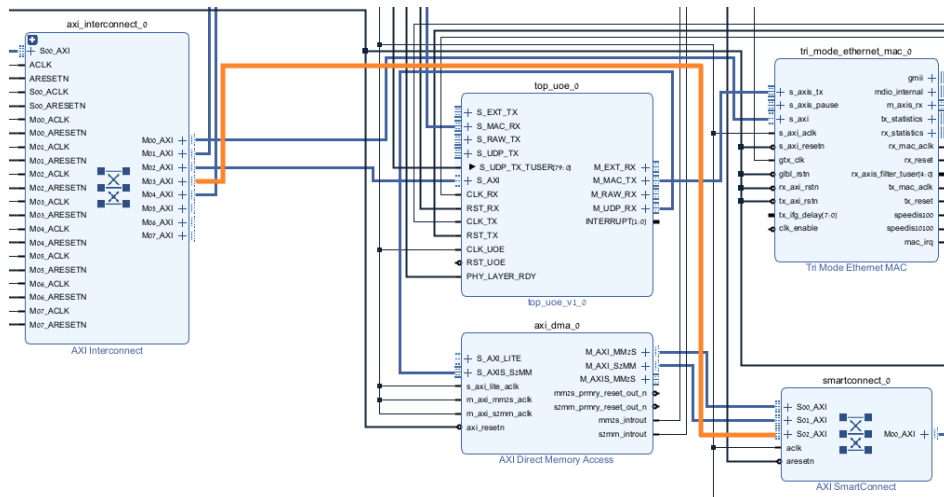
לאחר מכן, נכנסנו להגדרות הפורט החיצוני שנוצר ושינינו את שמו ל- `rs232_uart` כדי שיתאים להגדרות הפרוייקט.



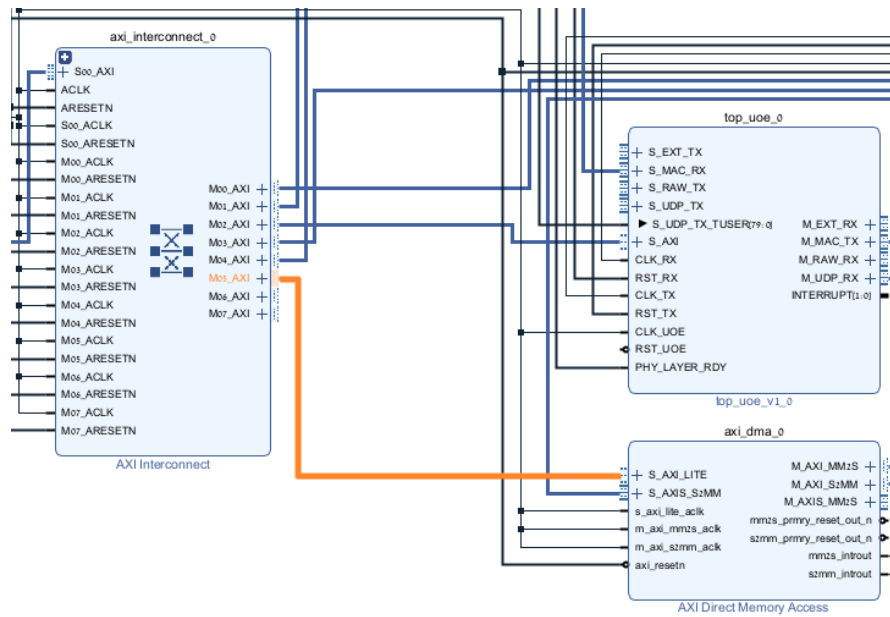
לסיום תכנון החומרה, אנו מוודאים את החיווט של בקר ה- AXI Interconnect. הבקר מנתב ערוץ תקשורת ייעודי לכל רכיב: ערוץ M02_AXI מחובר לממשק הבקרה של מנוע ה- UOE.



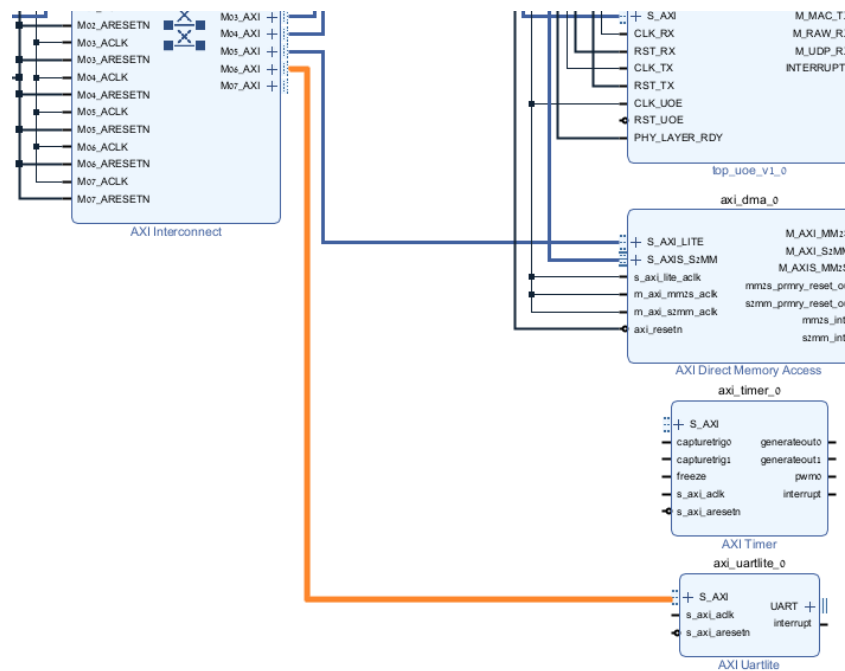
ערוץ M03_AXI מחובר לרכיב ה- AXI SmartConnect המקשר אל בקר ה- MAC.



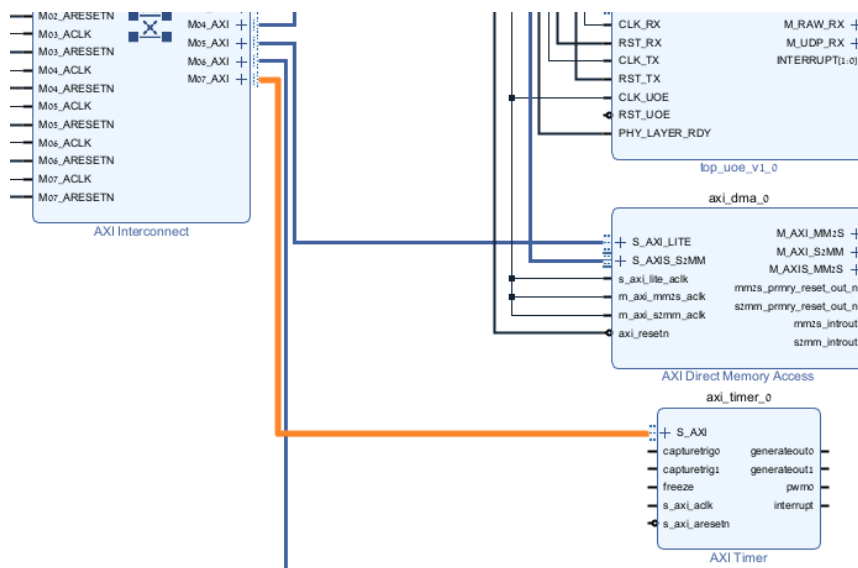
ערוץ M05_AX מחובר לממשק ה-AXI Lite של ה-DMA.



ערוץ M06_AX מחובר אל ה-UART.



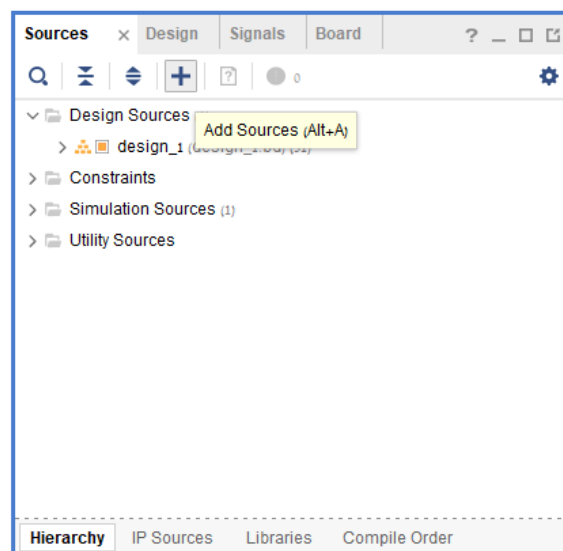
ערוץ M07_AXI מחובר אל ה-Timer.

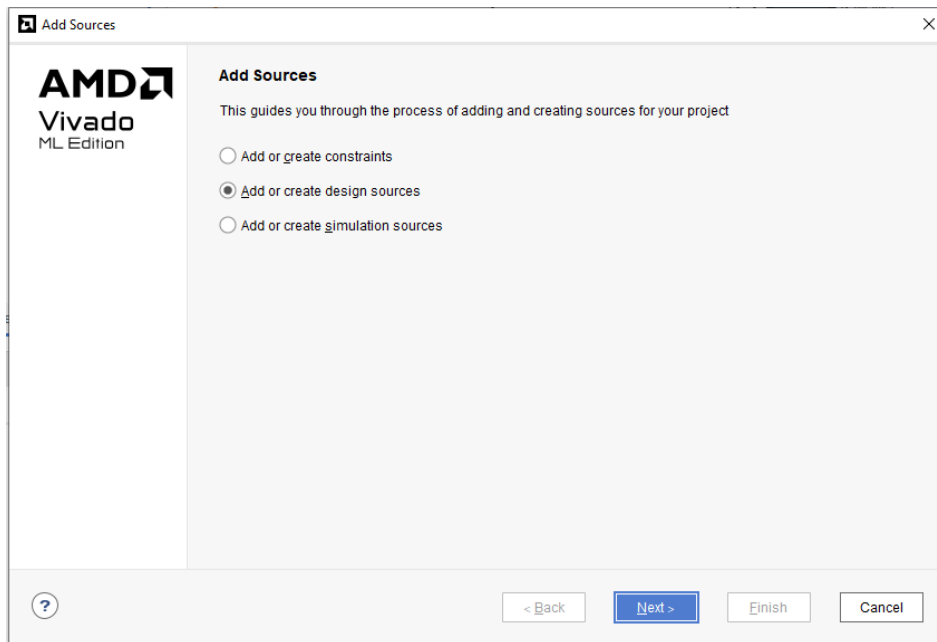


פריסה זו מבטיחה שלמעבד יש גישה ישירה ונפרדת לרגיסטרי הבקרה של כל רכיב במערכת, ללא התנגשויות.

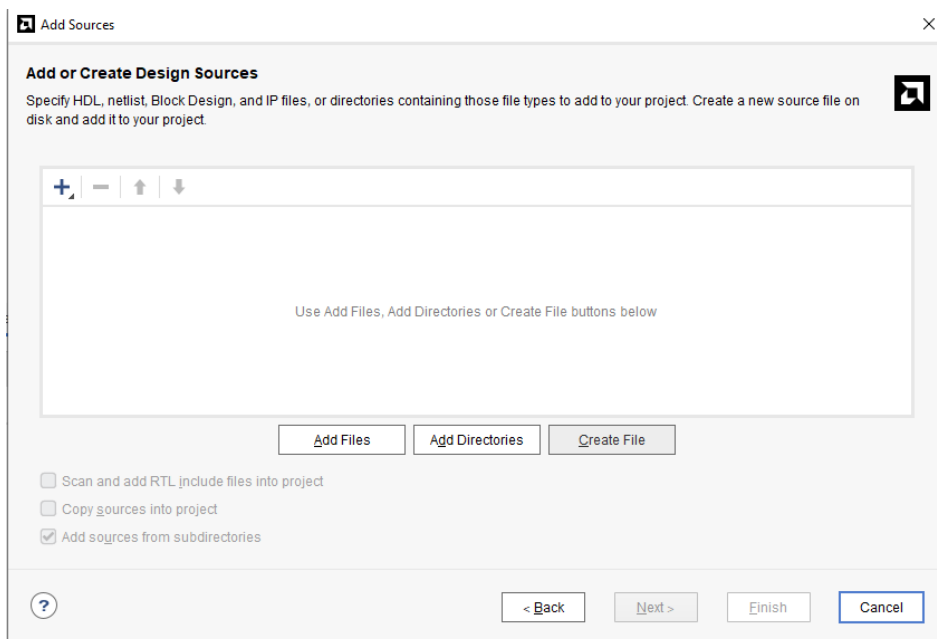
שלב 18:

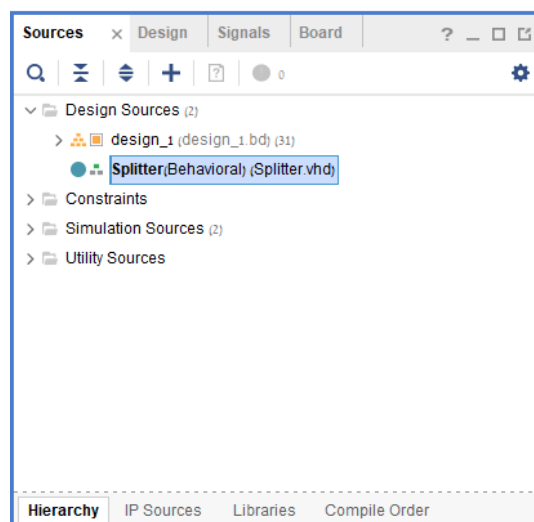
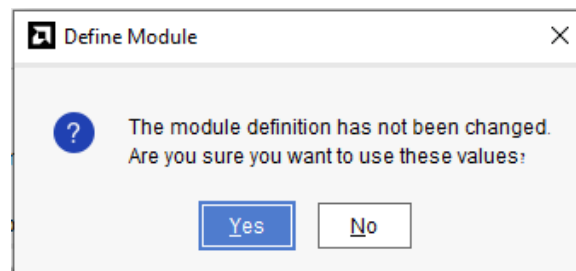
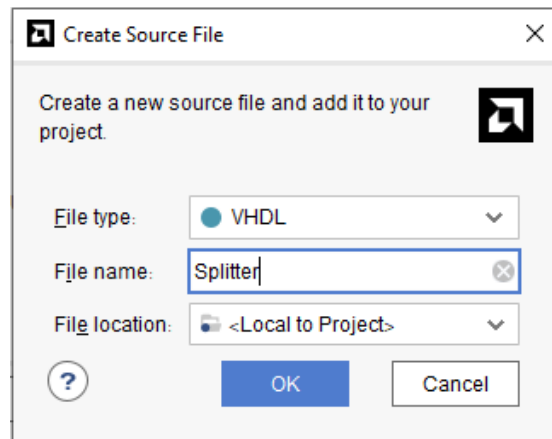
בשלב זה נוסף רכיב שתפקידו לגשר בין יציאת הנתונים של ה-DMA לבין כניסת הנתונים של ה-UOE. הרכיב מאפשר לנו לשכפל את אותות הבקרה של זרימת המידע ולהעביר אותם גם לרכיב ה-tuser_builder כדי להבטיח סנכרון מושלם בשידור החבילות.





תחילה הוספנו קובץ מקור חדש מסוג VHDL בשם Splitter. פתחנו את עורך הקוד והזנו את הלוגיקה הבאה:





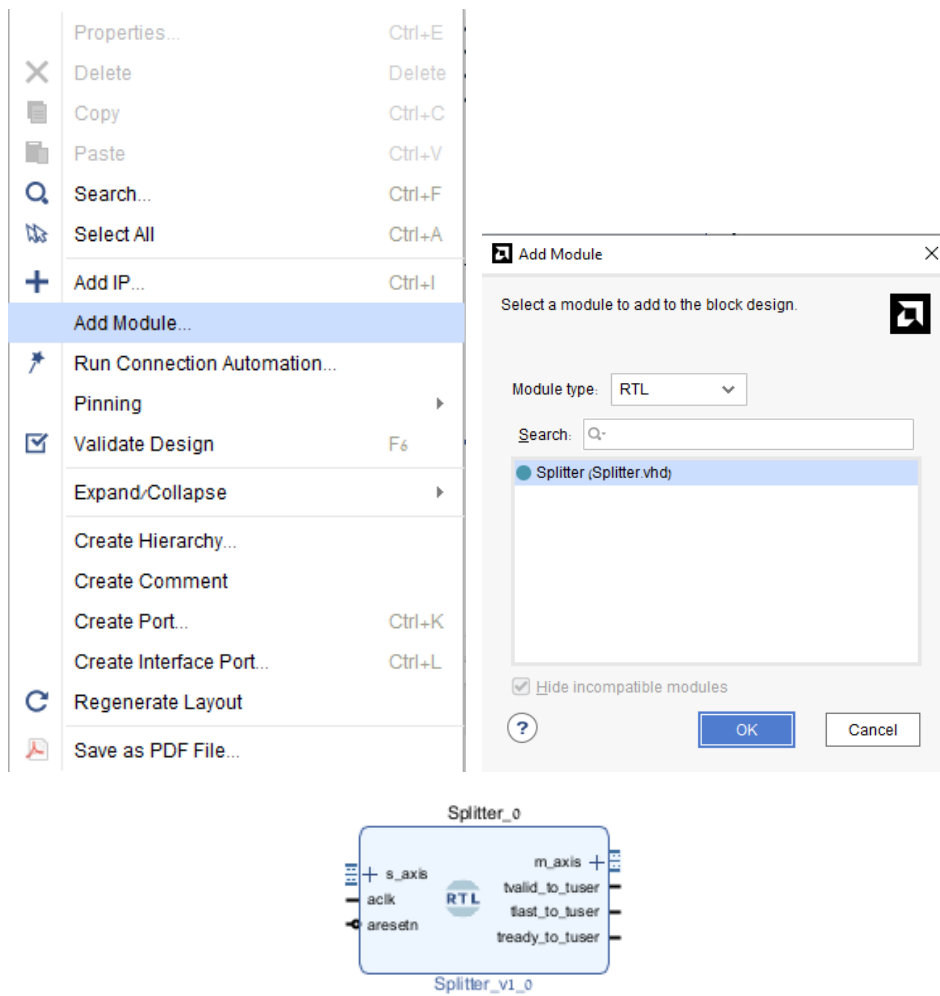
הקוד מגדיר ממשק AXI-Stream מלא הכולל ערוץ נתונים, אותות תוקף, אותות סיום חבילה ואותות מוכנות. הלוגיקה שבנינו מעבירה את כל המידע מהכניסה ישירות ליציאה ללא שינוי, ובמקביל יוצרת עותקים נפרדים לאותות הבקרה עבור רכיב ה-
tuser_builder.

```

1  -----
2  -- Company:
3  -- Engineer:
4  --
5  -- Create Date: 05/04/2026 07:40:49 AM
6  -- Design Name:
7  -- Module Name: Splitter - Behavioral
8  -- Project Name:
9  -- Target Devices:
10 -- Tool Versions:
11 -- Description:
12 --
13 -- Dependencies:
14 --
15 -- Revision:
16 -- Revision 0.01 - File Created
17 -- Additional Comments:
18 --
19 -----
20
21
22 library IEEE;
23 use IEEE.STD_LOGIC_1164.ALL;
24
25 -- Uncomment the following library declaration if using
26 -- arithmetic functions with Signed or Unsigned values
27 --use IEEE.NUMERIC_STD.ALL;
28
29 -- Uncomment the following library declaration if instantiating
30 -- any Xilinx leaf cells in this code.
31 --library UNISIM;
32 --use UNISIM.VComponents.all;
33
34 entity Splitter is
35     Port (
36         aclk: in std_logic;
37
38         aresetn: in std_logic;
39         --DMA
40         s_axis_tdata: in std_logic_vector(7 downto 0);
41         s_axis_tkeep: in std_logic;
42         s_axis_tvalid: in std_logic;
43         s_axis_tlast: in std_logic;
44         s_axis_tready: out std_logic;
45         --UOE
46         m_axis_tdata: out std_logic_vector(7 downto 0);
47         m_axis_tkeep: out std_logic;
48         m_axis_tvalid: out std_logic;
49         m_axis_tlast: out std_logic;
50         m_axis_tready: in std_logic;
51         --seperate outputs for tuser builder module
52         tvalid_to_tuser : out std_logic;
53         tlast_to_tuser : out std_logic;
54         tready_to_tuser : out std_logic;
55     );
56 end Splitter;
57
58 architecture Behavioral of Splitter is
59     begin
60         m_axis_tdata <= s_axis_tdata;
61         m_axis_tkeep <= s_axis_tkeep;
62         m_axis_tvalid <= s_axis_tvalid;
63         m_axis_tlast <= s_axis_tlast;
64         s_axis_tready <= m_axis_tready;
65
66         tvalid_to_tuser <= s_axis_tvalid;
67         tlast_to_tuser <= s_axis_tlast;
68         tready_to_tuser <= m_axis_tready;
69     end Behavioral;

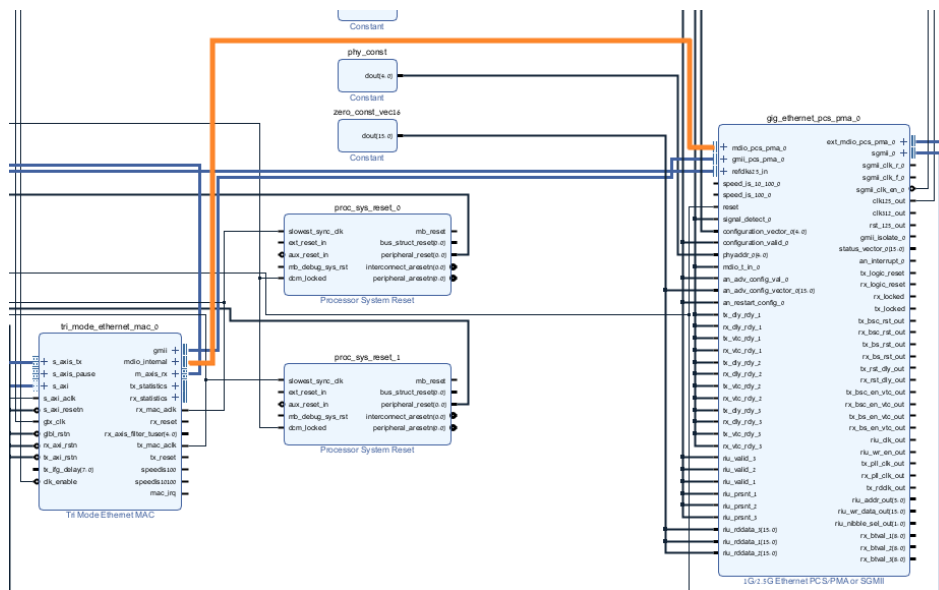
```

לאחר שמירת הקוד הוספנו את הבלוק לשרטוט בעזרת פקודת Add Module.



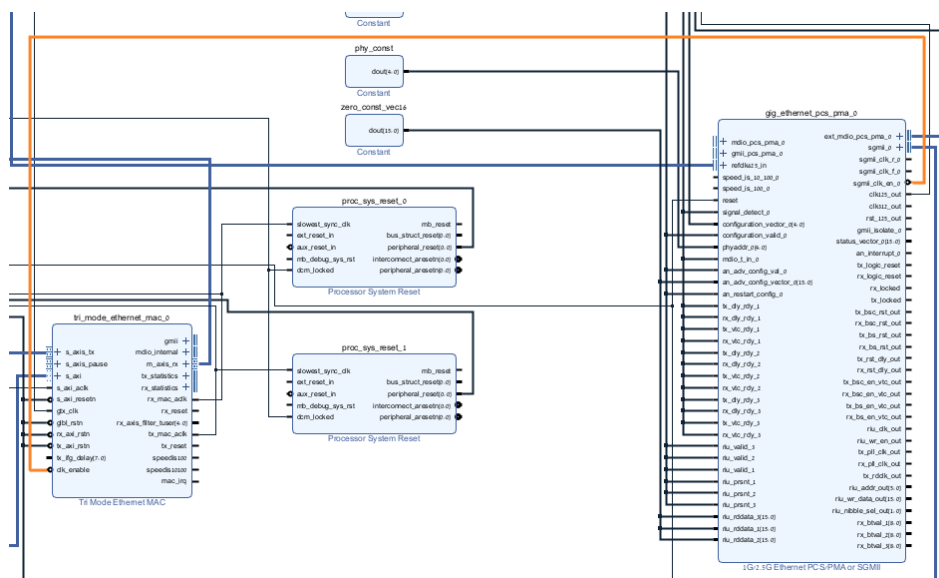
הרכיב ממוקם כחוליה מקשרת בין יציאת ה-DMA לבין כניסת ה-UOE.

בנוסף לנתונים עלינו לאפשר לשני הרכיבים להחליף ביניהם מידע לגבי מצב החיבור. נמתח חוט מהפורט mdio_internal בב्लוק ה-MAC אל הפורט mdio_pcs_pma_0 בב्लוק ה-PCS.

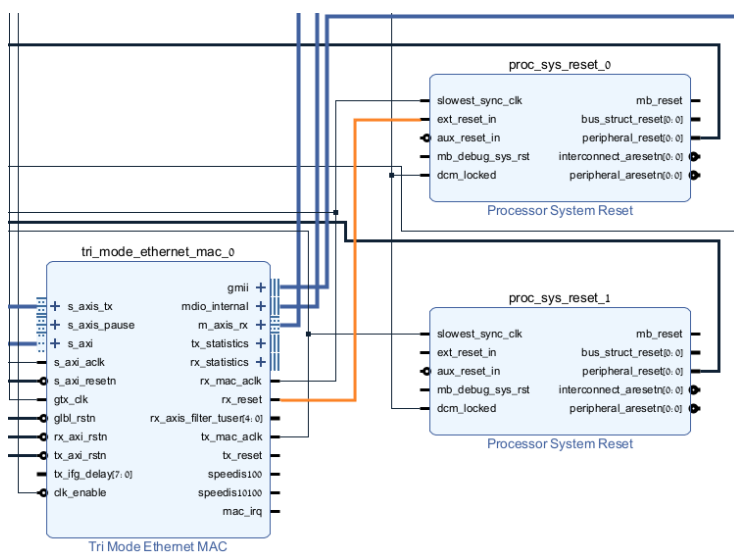


חיבור זה מבטיח שה- MAC יוכל לשלוט בהגדרות הפנימיות של השכבה הפיזית בזמן אמת.

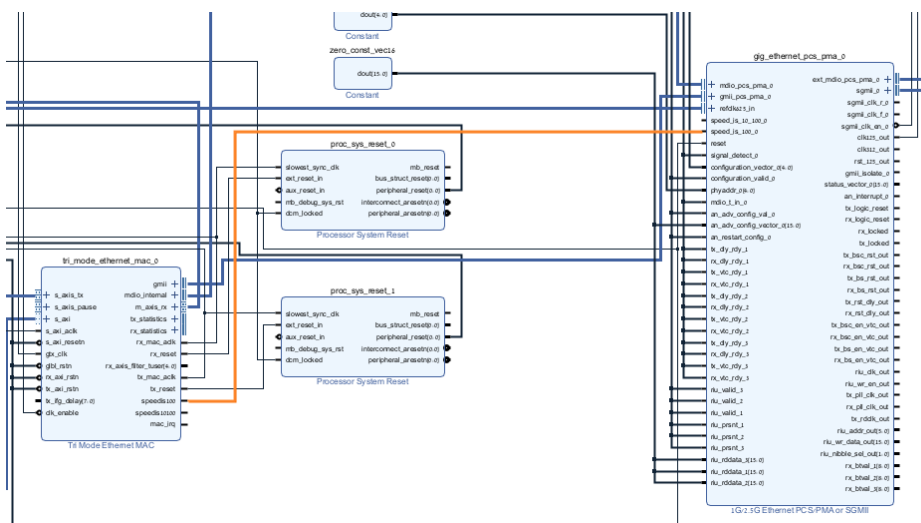
השלב האחרון בחיבור הוא העברת אות אישור השעון כדי ששני הרכיבים יעבדו באותו הקצב בדיוק. נמתח חוט מהפין clk_enable בב्लок ה-MAC ישירות אל הפין sgmiі_clk_en_0 בחלק התחתון של בלוק ה-PCS.

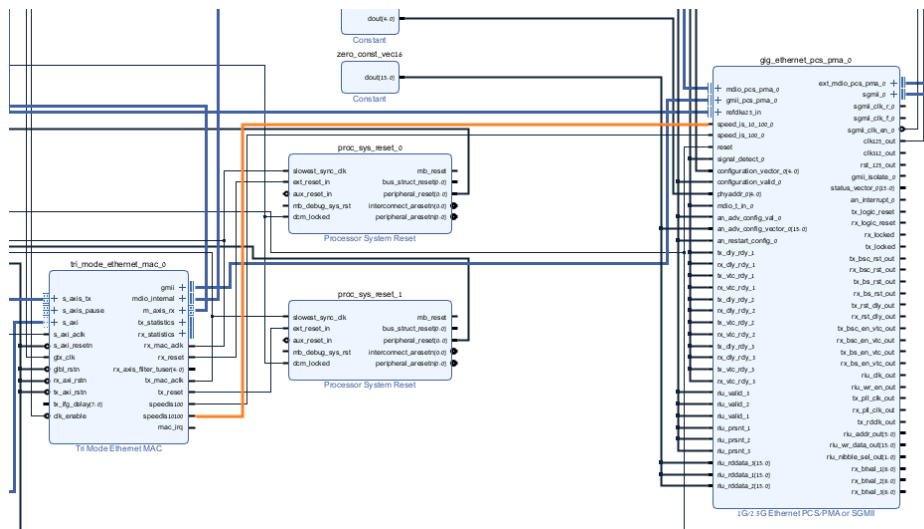


וידאנו כי כניסת ה- reset של בלוק ה- PCS/PMA מחוברת אל אות האיפוס המסונכרן שיוצא מבלוק ה- Processor System Reset הייעודי שלו. חיבור זה מבטיח כי השכבה הפיזית של הרשת תאופס במדויק יחד עם ה-MAC, מה שמונע מצב "תקיעה" (Lock-up) בזמן אתחול המערכת.



על מנת ששני הבלוקים יעבדו תמיד באותו קצב תקשורת ויוכלו להסתגל לשינויים ברשת, ניתבנו את יציאות חיויי המהירות של ה-MAC speedis100 ו-
speedis10100 ישירות לכניסות התואמות בבלוק ה-PCS/PMA
speed_is_100 ו-speed_is_100_100.





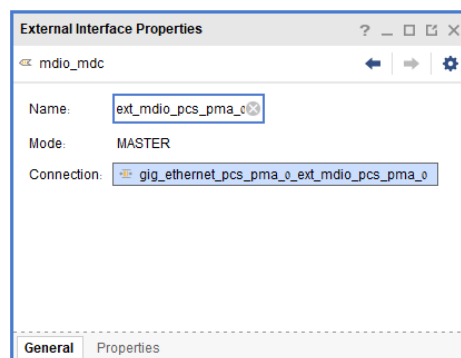
עם סיום חיבורים אלו כל נתיב הנתונים מהמעבד ועד ליציאת הרשת הפיזית מחובר ומוכן לעבודה. כעת נותר רק ללחוץ על כפתור ה- Validate Design כדי לוודא שכל המערכת תקינה.

שלב 20:

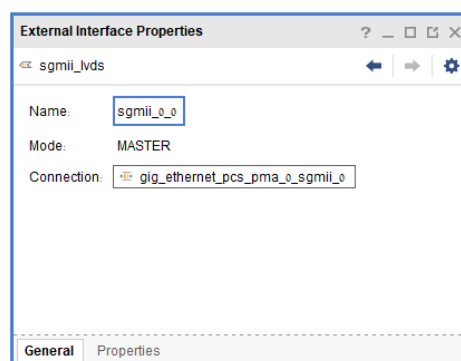
בשלב זה נבצע את ההכנות הסופיות לבדיקת המערכת על גבי הלוח. נשנה את שמות הממשקים החיצוניים, נוסיף לוגיקה לבדיקת תקינות התקשורת, ונגדיר רכיבי דיבאג מתקדמים מסוג ILA למעקב אחר האותות בזמן אמת.

הממשקים החיצוניים נוצרו בשלבים הקודמים וכעת עלינו להתאים את שמם כדי שידהו נכון מול קובץ האילוצים.

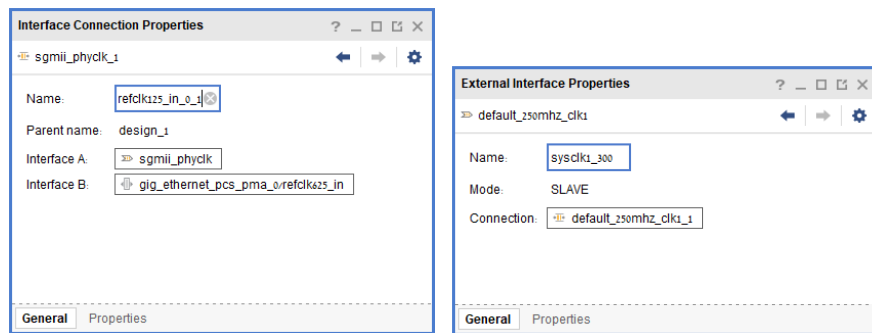
בחלון מאפייני הממשק שינינו את שם ממשק הניהול ל-`ext_mdio_pcs_pma_0`.



את ממשק הנתונים הטורי שינינו ל-`sgmii_0_0`.

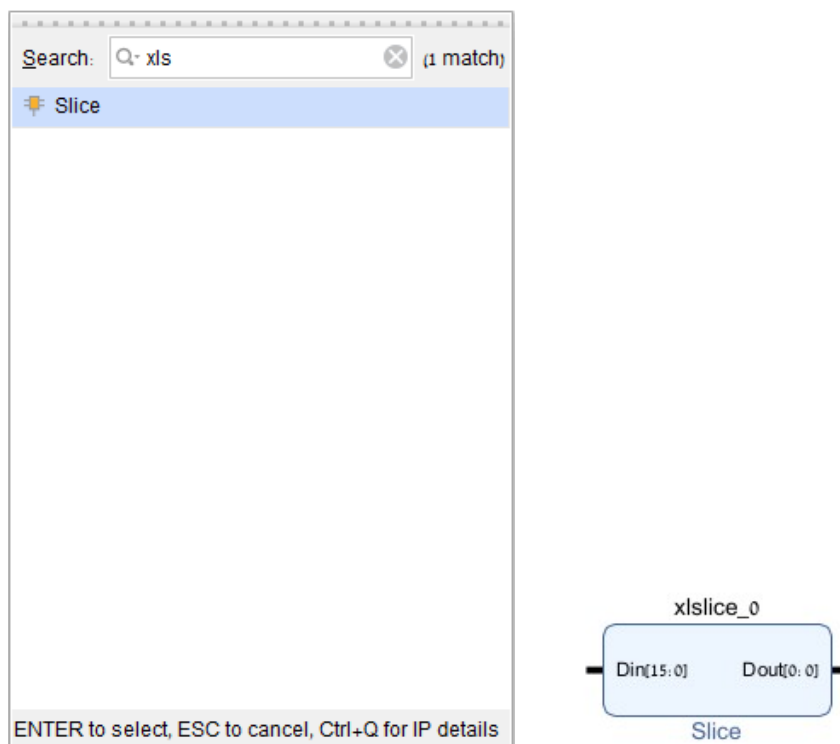


את שעון הייחוס שינינו ל-refclk125_in_0_1 וכן את שם פורט הייחוס ל-sysclk1_300.

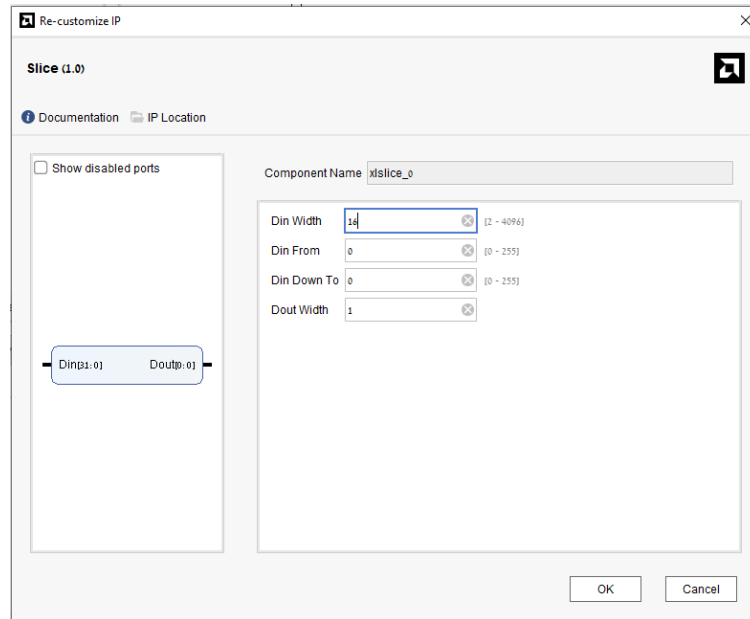


עדכון זה מבטיח שהתוכנה תדע לקשר בין הבלוקים בשרטוט לבין הפינים הפיזיים בלוח ה-VCU118.

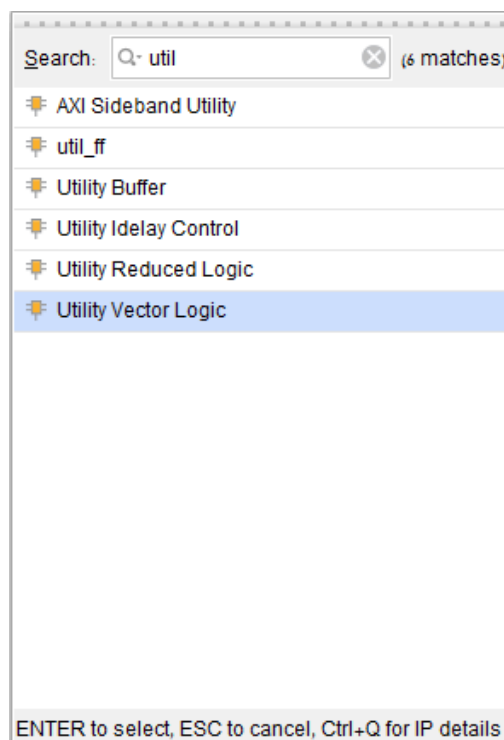
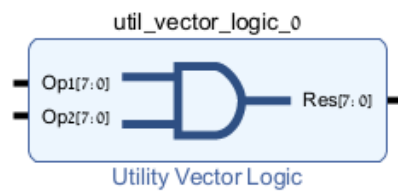
כדי לזהות מתי הקישור הפיזי עובד באופן תקין הוספנו רכיב חיתוך הנקרא Slice.

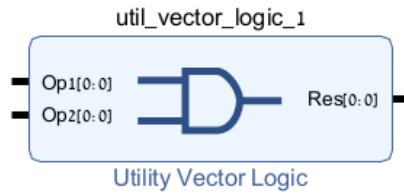


בהגדרות הרכיב שינינו את רוחב כניסת הנתונים Din Width מרוחב 32 לרוחב 16.

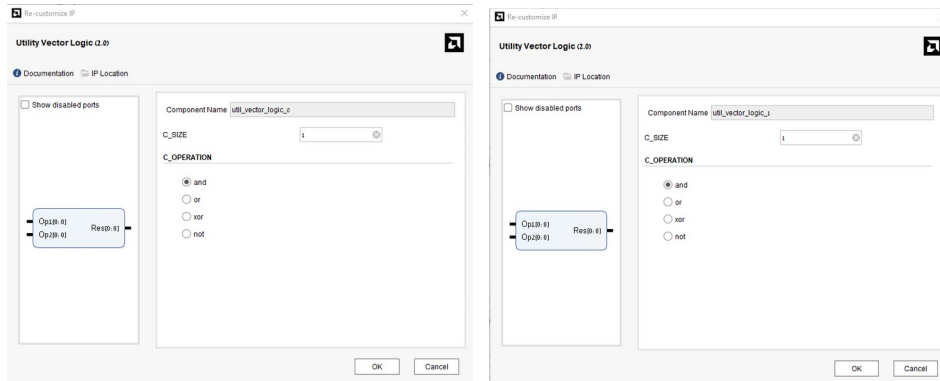


רכיב זה גוזר את המידע הרלוונטי מתוך וקטור הסטטוס היוצא מהשכבה הפיזית. לאחר מכן העברנו את המידע החתוך דרך שני שערים לוגיים מסוג AND בעזרת רכיבי Utility Vector Logic.



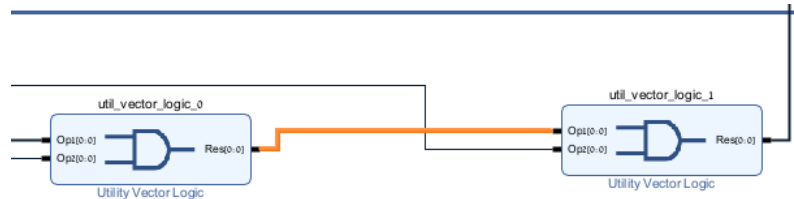


מכיוון שרכיבים אלו מוגדרים כברירת מחדל לעבוד עם נתונים ברוחב של 8 ביטים, נכנסנו להגדרות של כל אחד משני הבלוקים ושינינו את גודל הווקטור מ-8 לביט אחד בודד.



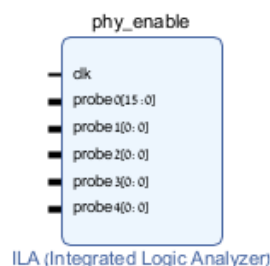
הגדרה זו קריטית כדי למנוע שגיאות של אי התאמה ברוחב הנתונים, שכן המידע היוצא מרכיב החיתוך ואותות הנעילה של השעונים הם בני ביט אחד בלבד.

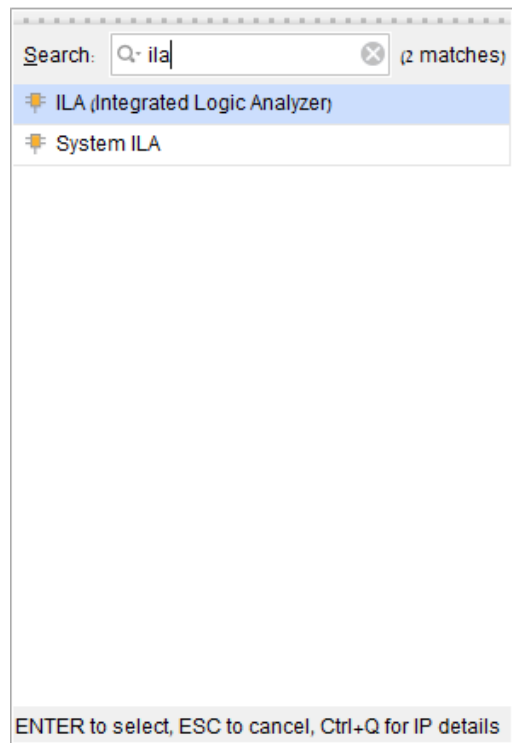
בסיום ההגדרות חיווטנו את השערים בשרשרת יחד עם אותות נעילת השעון כדי לייצר אות מוכנות אחד סופי ואמין המעיד שהמערכת מוכנה לשידור.



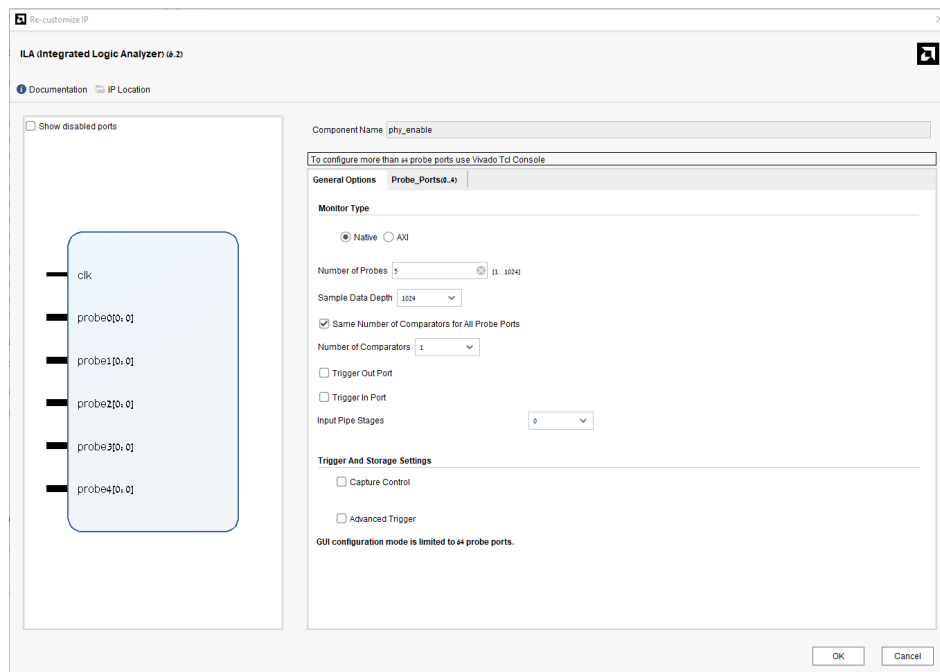
כדי לבצע דיבאג מעמיק ולנתח את הנתונים בזמן ריצה בחומרה, תכננו מערך ניטור מקיף הכולל חמישה רכיבי Integrated Logic Analyzer (ILA). המערך מחולק לניטור של אותות בקרה פשוטים (Native) ולניטור של צינורות תקשורת רחבים בפרוטוקול AXI Stream.

הוספנו למערכת רכיב ILA סטנדרטי שתפקידו לנטר את קווי הבקרה של הרכיב הפיזי PHY.

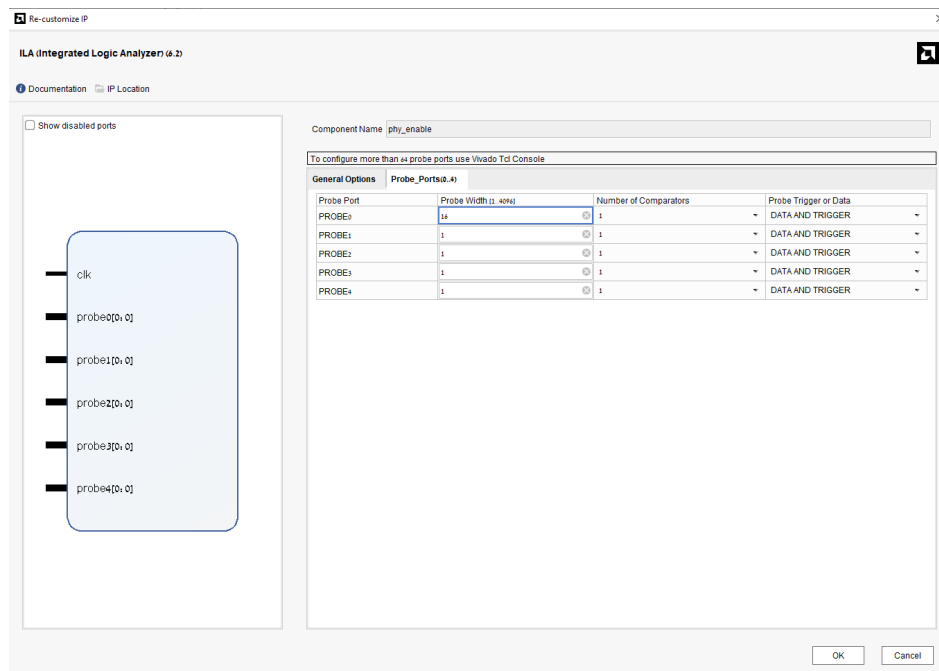




בהגדרות הרכיב, תחת General Options, קבענו את סוג הניטור ל- Native, והגדרנו 5 ערוצי דגימה.



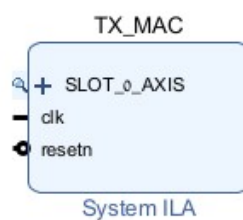
כדי לאפשר דגימה של וקטור סטטוס שלם יחד עם אותות בקרה בודדים, שינינו בלשונית ה-Probe_Ports את רוחב הפרוב הראשון (probe0) ל-16 ביט, בעוד שאר הפרובים הושארו ברוחב של ביט בודד.

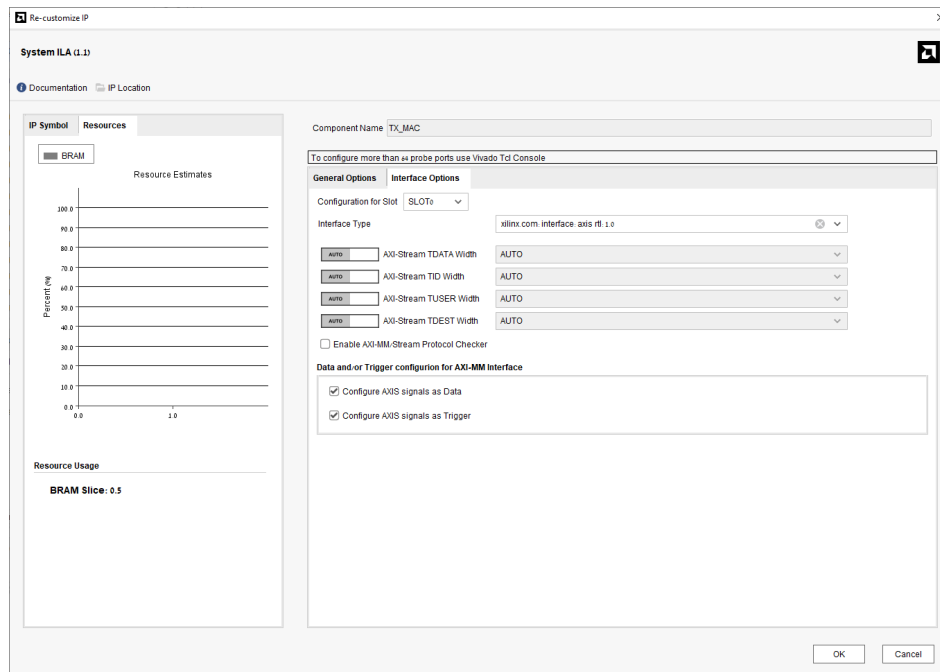
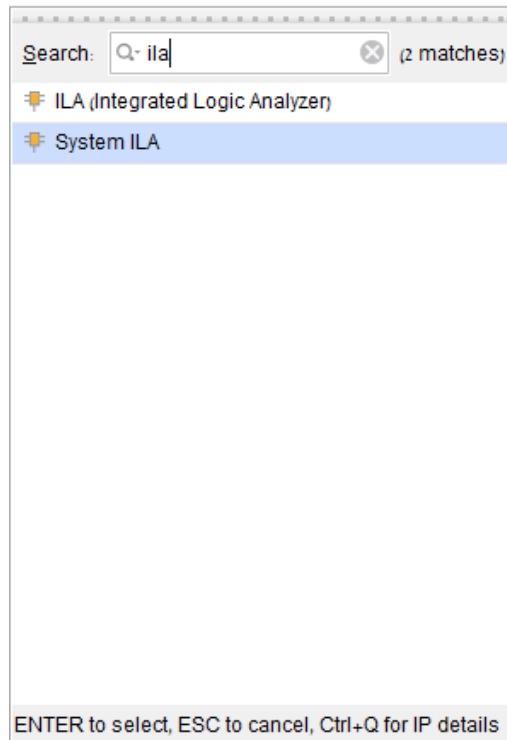


בנוסף לאותות הסטטוס, הוספנו ארבעה רכיבי System ILA מתקדמים שנועדו להאזין ישירות לזרם הנתונים בין ה-MAC ל-UOE, הן במסלול השידור TX והן במסלול הקליטה RX. רכיבים אלו מסוגלים לפענח אוטומטית את פרוטוקול ה-AXI Stream ולהציג את הפאקטות בצורה קריאה.

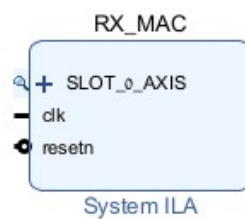
עבור ניטור הפלט והקלט של רכיב ה-MAC (TX_MAC ו-RX_MAC), הגדרנו את סוג המוניטור כ-Interface בלבד.

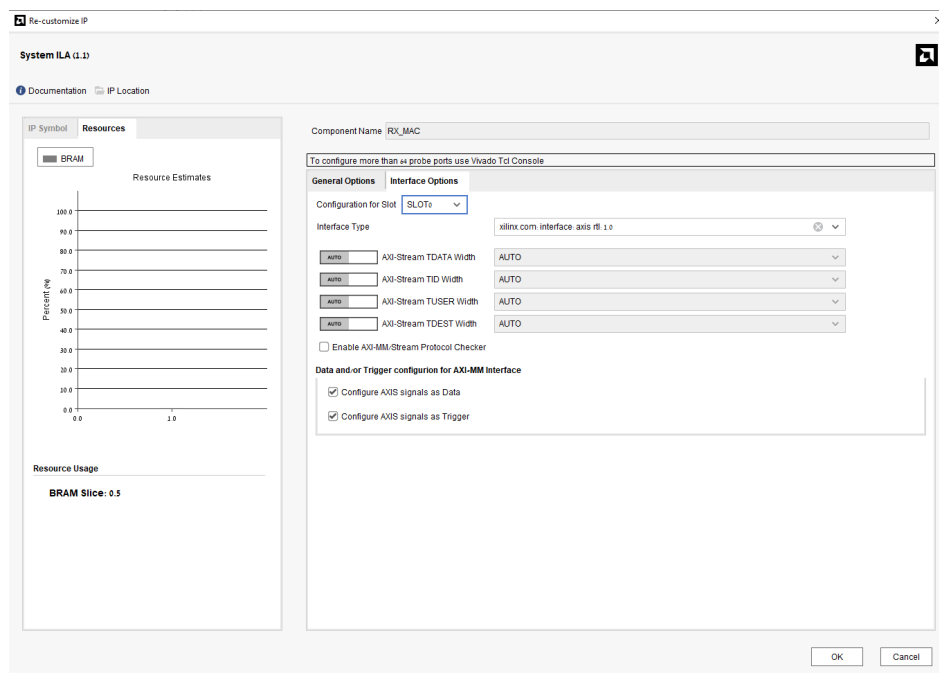
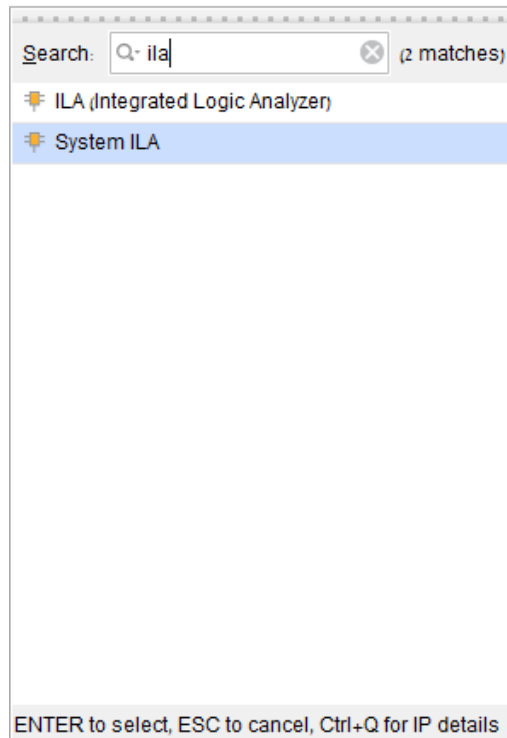
עבור TX_MAC.





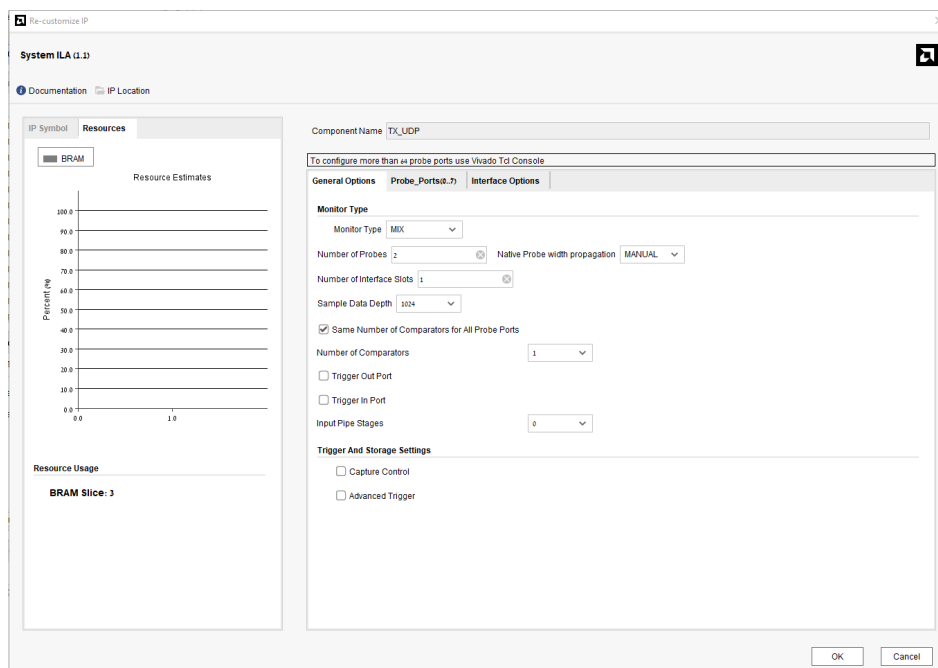
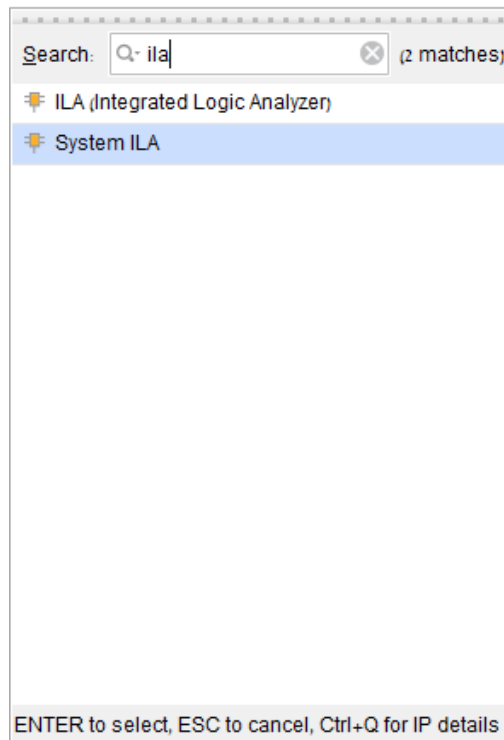
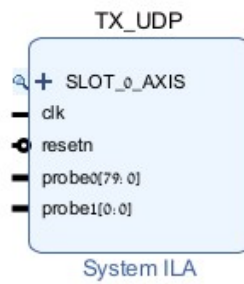
עבור RX_MAC.

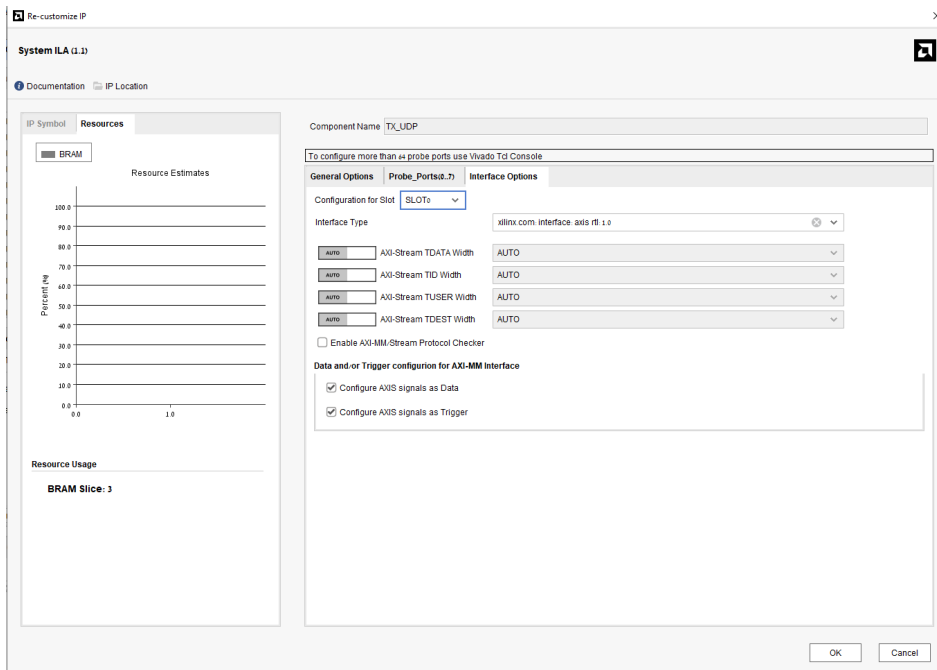
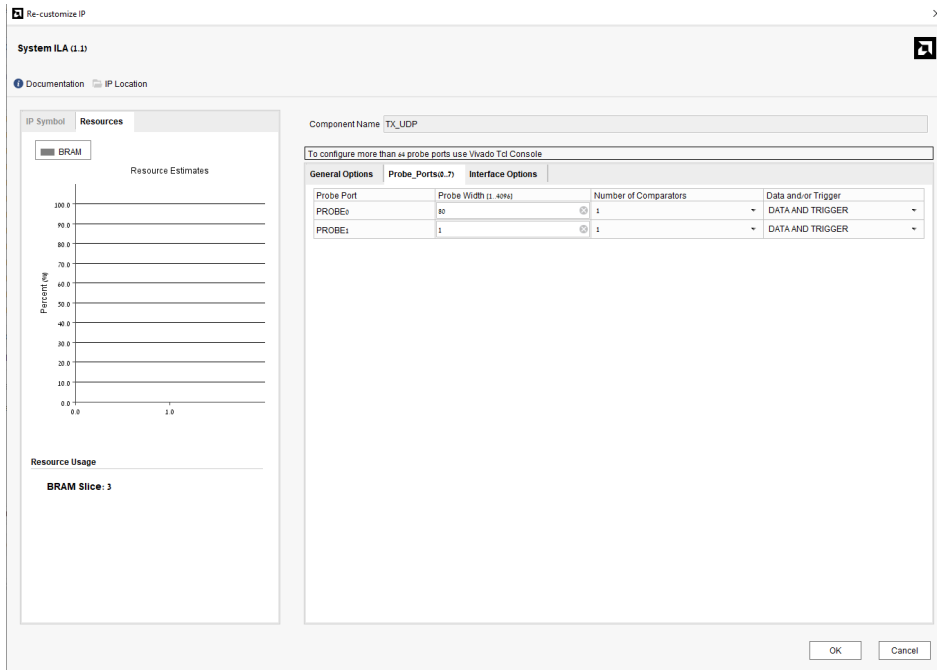




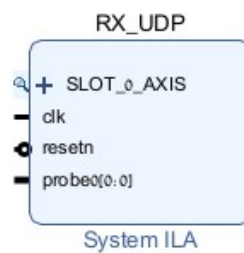
לעומת זאת, רכיבי הניטור הצמודים ל-UOE (TX_UDP ו-RX_UDP) דרשו ניטור משולב. בלשונית General Options קבענו את סוג הניטור ל-MIX, מה שמאפשר לרכיב להאזין גם לאפיק ה-AXI וגם לקווי Native רגילים (כמו אותות ה-TUSER שמעבירים את נתוני הניתוב). הגדרנו להם פרובים נוספים בהתאם לצורך (למשל, probe0 ברוחב 80 ביט עבור ה-TX_UDP).

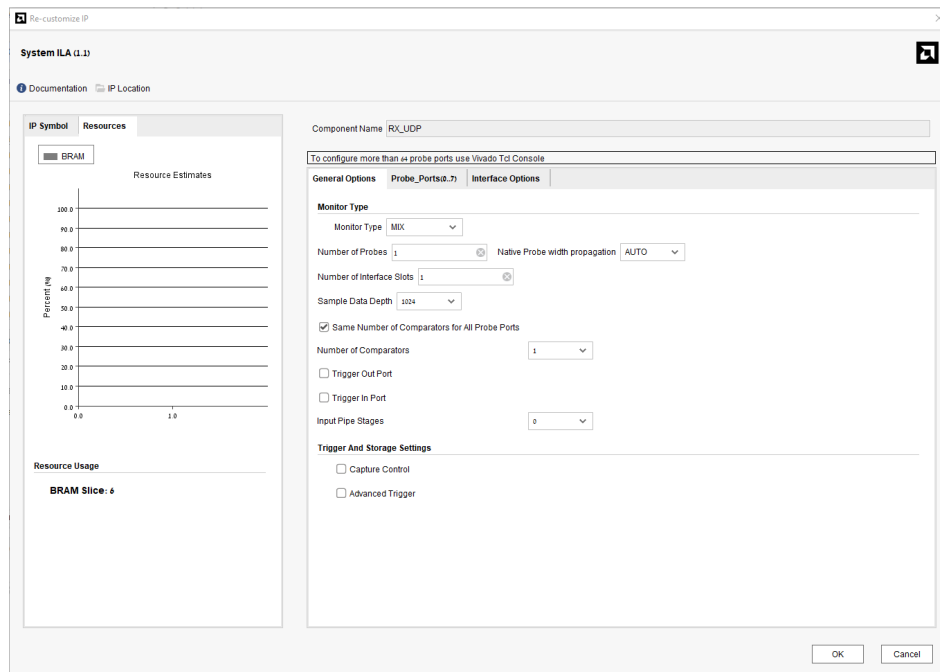
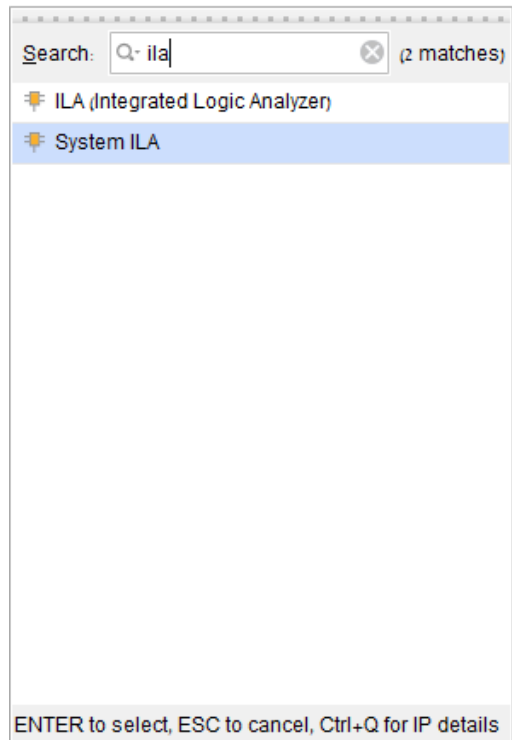
עבור TX_UDP.

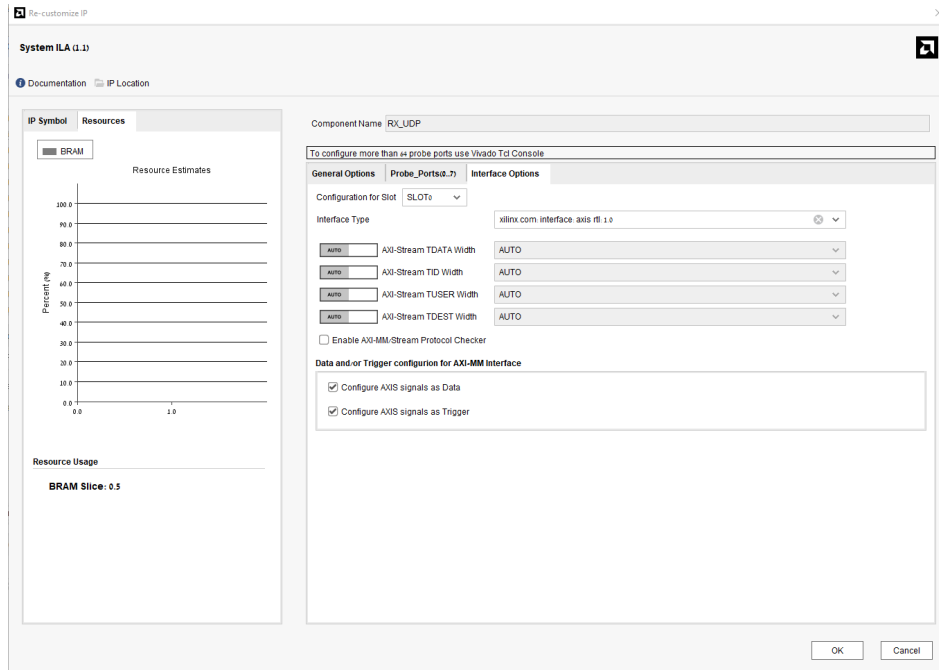




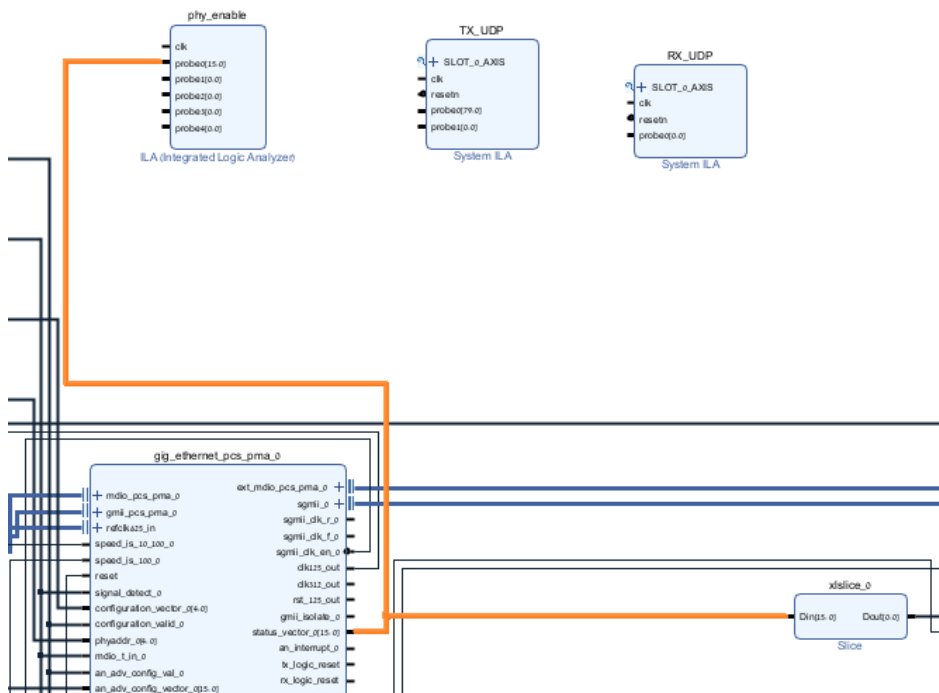
עבור RX_UDP.

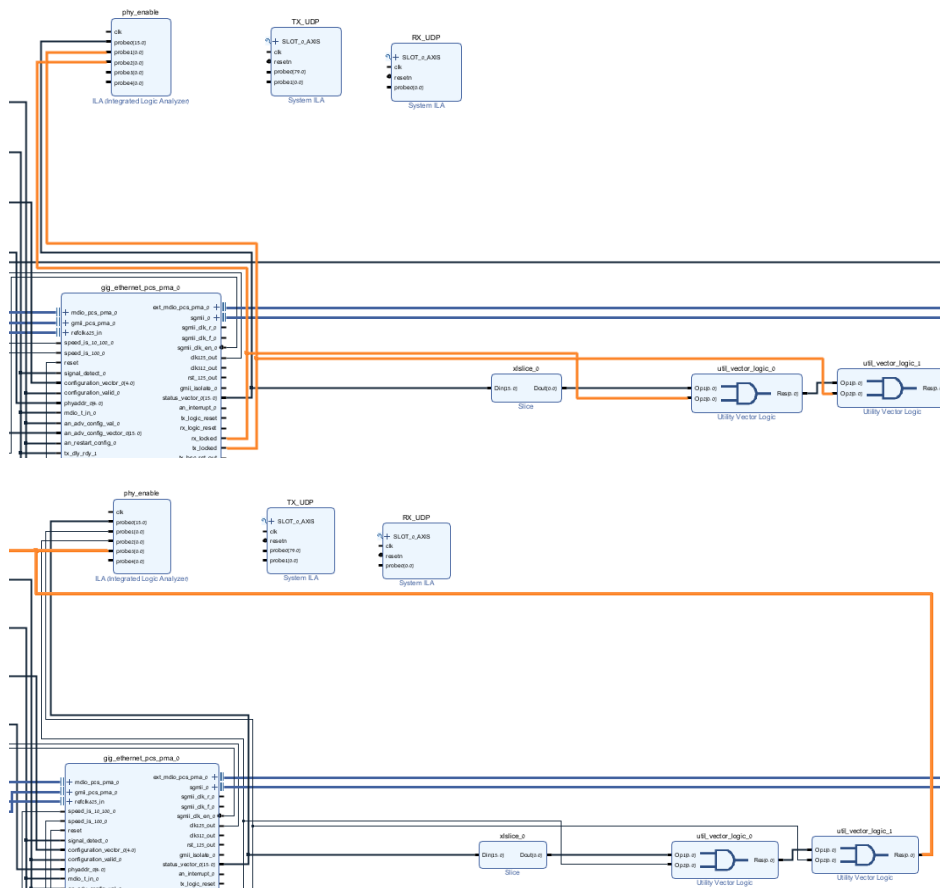




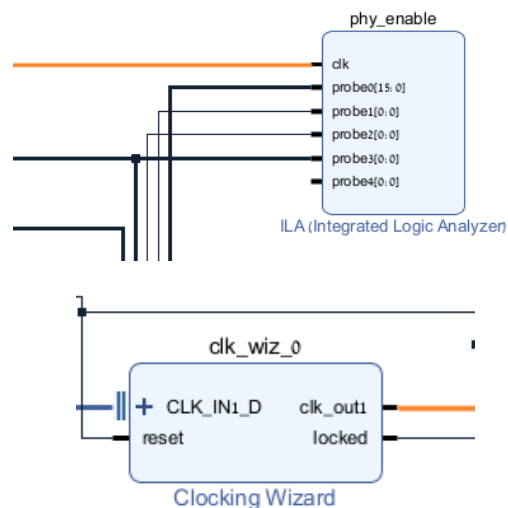


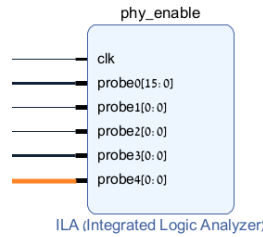
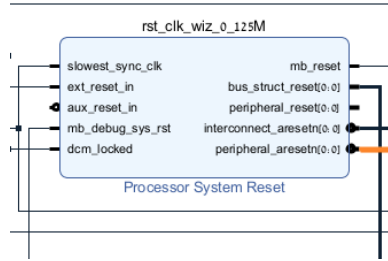
חיווטנו את רכיב ה-ILA הסטנדרטי `phy_enable` שנועד לנטר את תהליך האתחול של השכבה הפיזית. חיברנו את ערוצי הדגימה שלו (`probe0` עד `probe4`) אל אותות המפתח: וקטור הסטטוס (`status_vector`) של בלוק ה-PCS/PMA, יציאות הלוגיקה של שערי ה-AND, ואות ה-`PHY_LAYER_RDY` הסופי. ניטור זה הכרחי כדי לוודא ששלב ה-`Auto-Negotiation` מסתיים בהצלחה טרם תחילת השידור.



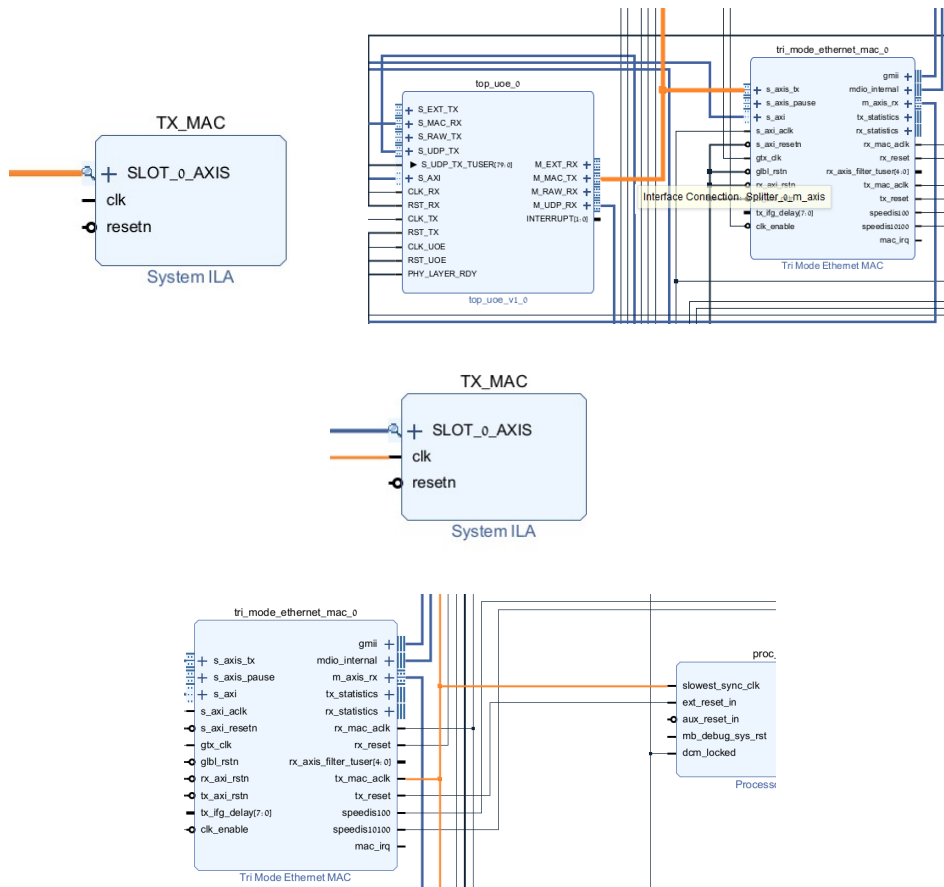


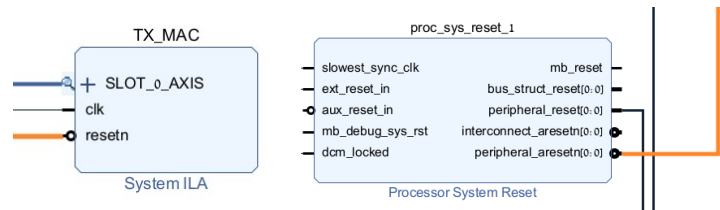
לסנכרון מושלם מול המערכת הנדגמת, חיווטנו את כניסת השעון של ה-ILA אל רשת ה-125MHz. בנוסף, הקצינו את ערוץ probe4 כדי לדגום ולנטר את אות ה-peripheral_aresetn, מה שיאפשר לנו לראות את שחרור האיפוס של המערכת בזמן אמת.





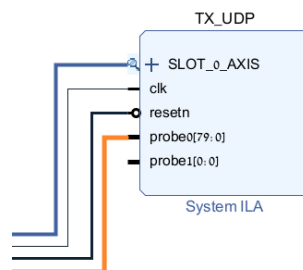
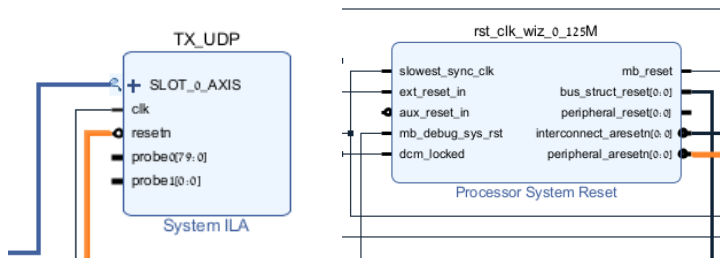
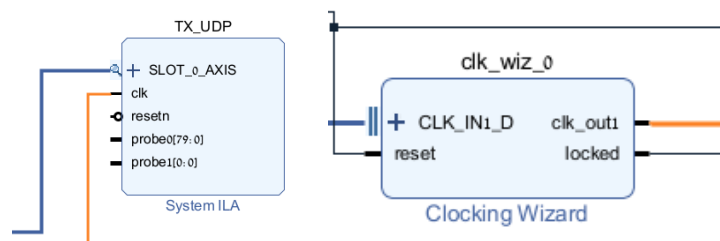
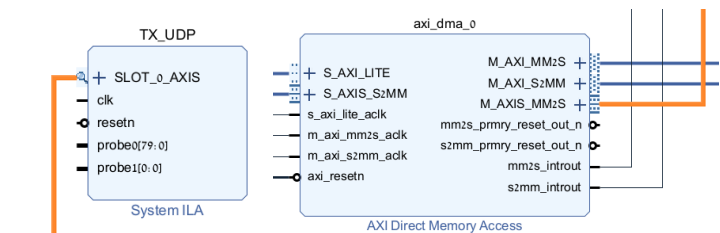
חיווטנו את ערוץ הנתונים של רכיב הניטור TX_MAC (כניסת SLOT_0_AXIS). התחברנו במקביל אל אפיק ה- AXI-Stream המקשר בין יציאת M_MAC_TX של מנוע ה-UOE לכניסת השידור s_axis_tx של בקר ה-MAC. דגימה בנקודה זו תאפשר לנו לנטר את חבילות הנתונים המלאות רגע לפני שהן עוברות לשכבה הפיזית. לסנכרון מושלם מול המערכת הנדגמת, חיווטנו את אותות השעון clk והאיפוס resetn של הרכיב אל תשתית ה- 125 MHz ובלוק ה- Processor System Reset המרכזיים

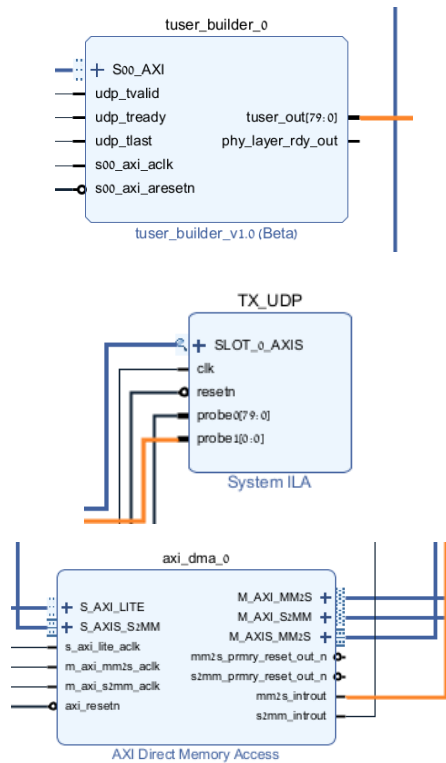




עבור רכיב הניטור TX_UDP שהוגדר קודם לכן במצב MIX, חיברנו את אפיק ה- AXI- Stream המרכזי אל הקו המקשר בין ה-DMA לכניסת מנוע ה-UOE. בנוסף, חיווטנו את כניסת ה-probe0 הבדידה ברוחב 80 ביט ישירות אל וקטור ה-TUSER, המכיל את נתוני הניתוב כגון כתובת ה-IP ופורטי המקור והיעד.

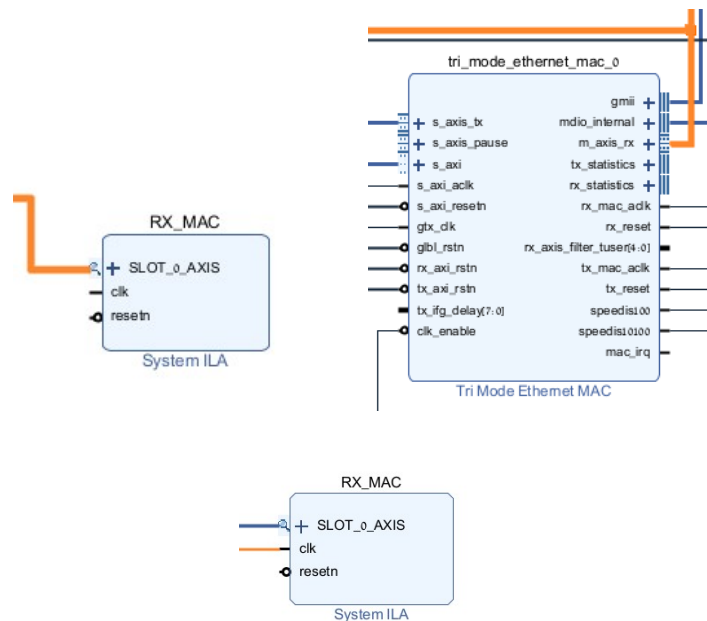
לסנכרון מושלם מול המערכת הנדגמת, חיווטנו את אותות השעון clk והאיפוס resetn של הרכיב אל תשתית ה-125 MHz ובלוק ה-Processor System Reset המרכזיים.

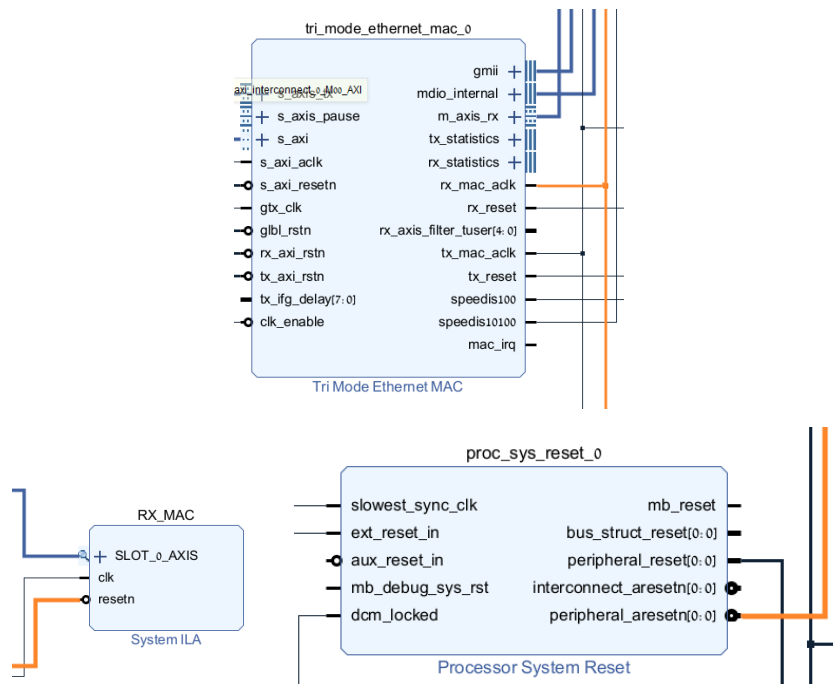




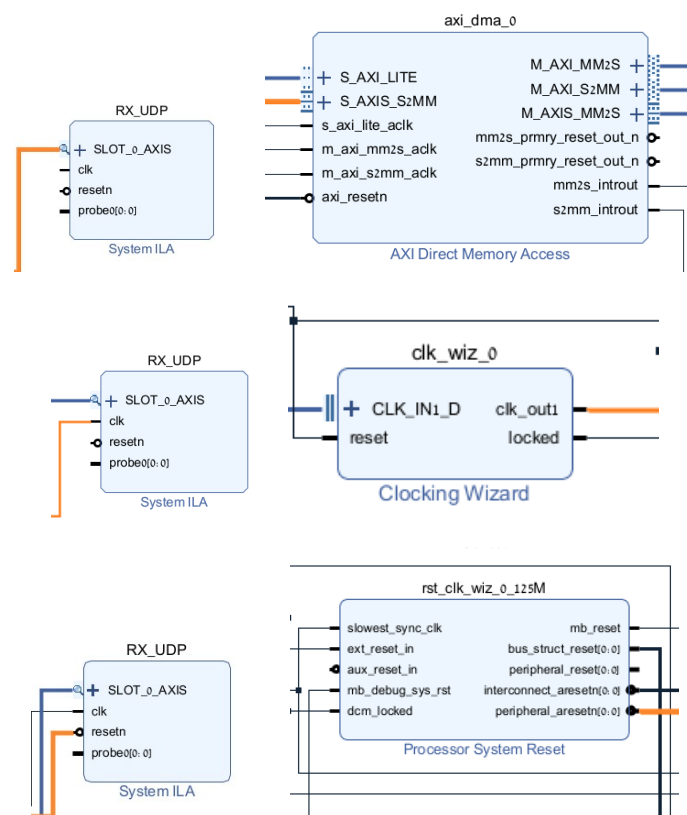
בשלב הבא, חיווטנו את רכיב הניטור RX_MAC. SLOT_0_AXIS של RX_MAC חובר את כניסת SLOT_0_AXIS של RX_MAC. אפיק הנתונים המקשר בין יציאת הקליטה rx_axis_mm של RX_MAC לבין כניסת מנוע ה-UOE. דגימה זו תאפשר לנו לעקוב אחרי חבילות המידע הנכנסות מהרשת החיצונית טרם עיבודן.

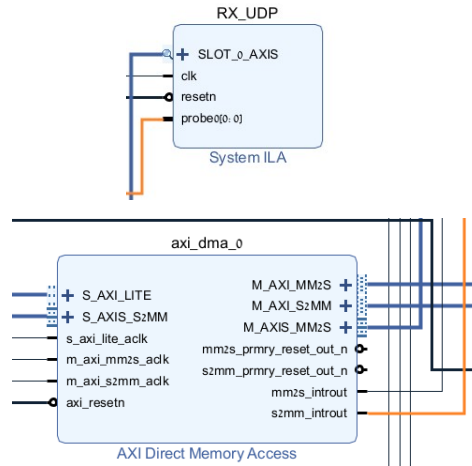
לסנכרון מושלם מול המערכת הנדגמת, חיווטנו את אותות השעון clk והאיפוס resetn של הרכיב אל תשתית ה-125 MHz ובזמן ה-Processor System Reset המרכזיים.





באופן דומה, חיווטנו את רכיב הניטור RX_UDP אל אפיק הנתונים היוצא ממנוע ה- UOE לאחר סינון החבילה ופיענוח פרוטוקול ה- UDP בדרכו חזרה אל ה- DMA לכתובה בזיכרון. לסנכרון מושלם מול המערכת הנדגמת, חיווטנו את אותות השעון clk והאיפוס resetn של הרכיב אל תשתית ה- 125 MHz ובלוק ה- Processor System Reset המרכזיים.

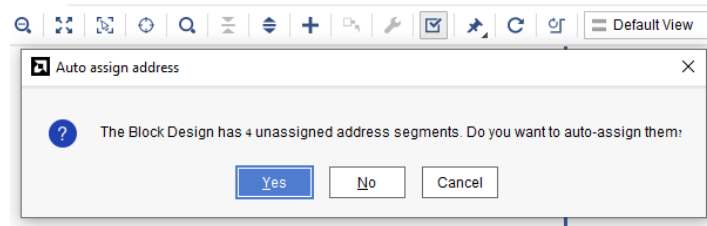




שלב 21: אימות המערכת יצירת מעטפת ראשי והוספת קובץ אילוצים

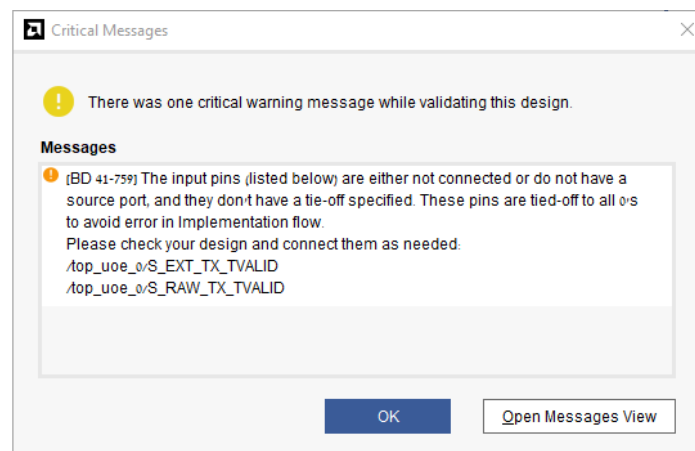
בשלב זה נסיים רשמית את העבודה על שרטוט החומרה. נוודא שהתכנון תקין מבחינה לוגית וניתן לו את הכתובות הנדרשות. לאחר מכן ניצור עבורו קובץ מעטפת המתרגם את השרטוט לקוד VHDL ולסיום נסיף את קובץ האילוצים שיקשר בין השרטוט לפינים הפיזיים על לוח הפיתוח.

לאחר סיום כלל החיבורים במערכת הרצנו אימות על ידי לחיצה על כפתור Validate Design. במהלך האימות קפצה הודעת Auto assign address ולחצנו בה על Yes.

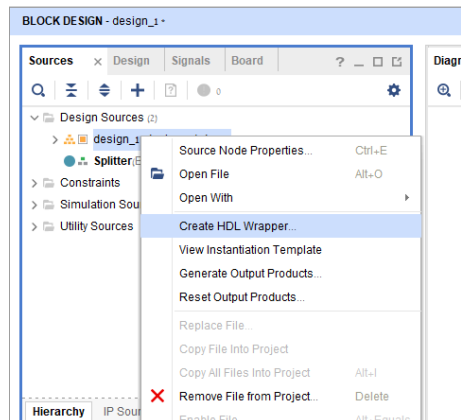


פעולה זו מקצה אוטומטית כתובות זיכרון לרכיבים החדשים שהוספנו כמו רכיב התקשורת הטורית והטיימר כדי שהמעבד יוכל לגשת אליהם בהמשך.

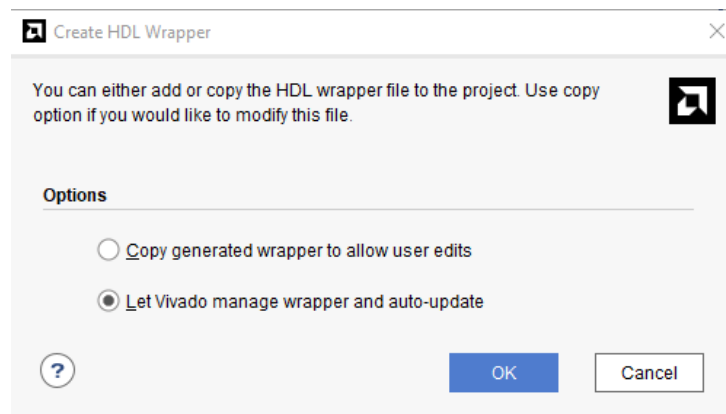
בנוסף קיבלנו הודעת אזהרה המודיעה כי פיני ה-TVALID של כניסות השידור החיצוני והגולמי בליבת ה-UOE חוברו לאפס. הודעה זו תקינה לחלוטין ואף רצויה מכיוון שאיננו משתמשים בכניסות אלו וחיבורן לאפס קבוע מונע הכנסת רעשים למערכת.



כדי שתוכנת הפיתוח תוכל לעבוד עם השרטוט הגרפי עליה לארוז אותו לקובץ קוד מרכזי.
בחלונית המקורות לחצנו קליק ימני על קובץ השרטוט הראשי ובחרנו באפשרות Create HDL Wrapper.



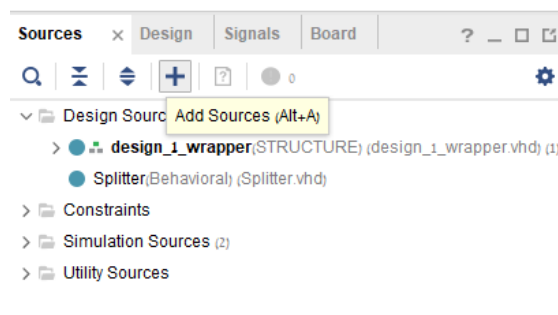
בחלון ההגדרות שנפתח בחרנו באפשרות auto-
update.

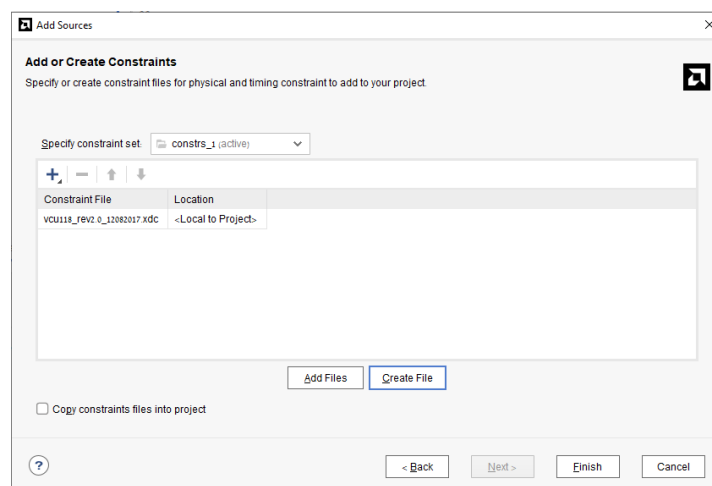
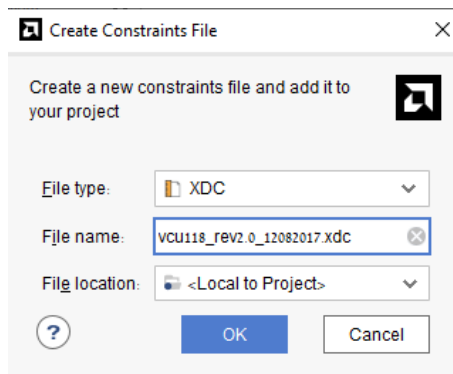
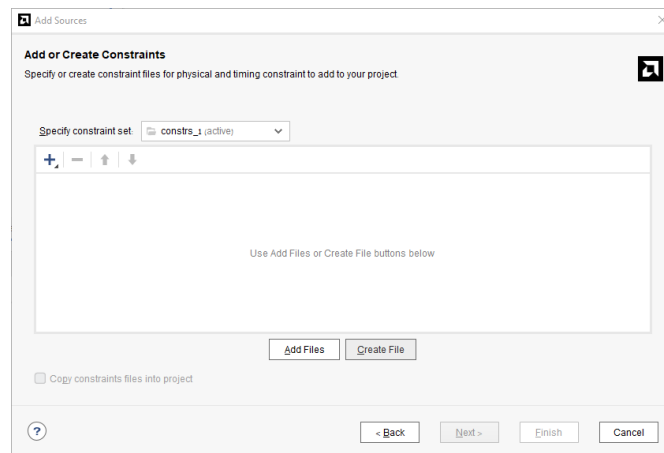
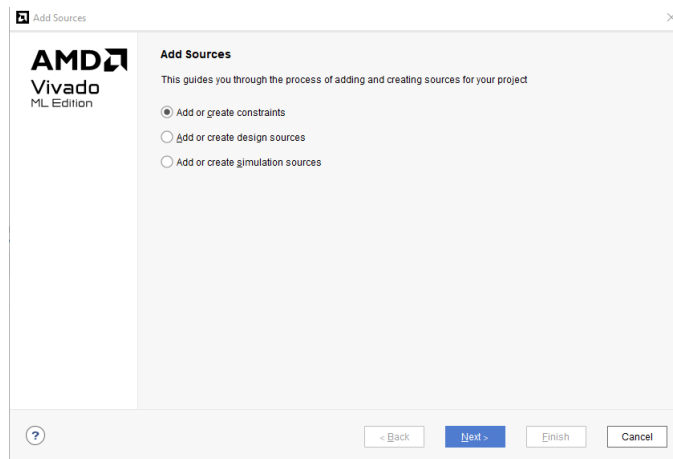


אפשרות זו חוסכת מאיתנו עבודה ידנית ומאפשרת לתוכנה לעדכן את קובץ המעטפת באופן אוטומטי בכל פעם שנבצע שינוי עתידי בשרטוט.

השלב האחרון לפני תחילת הסינתזה הוא הוספת קובץ אילוצים בעל סיומת XDC. קובץ זה מכיל את ההוראות המדויקות לתוכנה לאיזה פין פיזי על גבי כרטיס ה-VCU118 יש לחבר כל ממשק חיצוני שיצרנו.

פתחנו את חלון הוספת המקורות בחרנו ביצירת קובץ אילוצים חדש והענקנו לו שם.





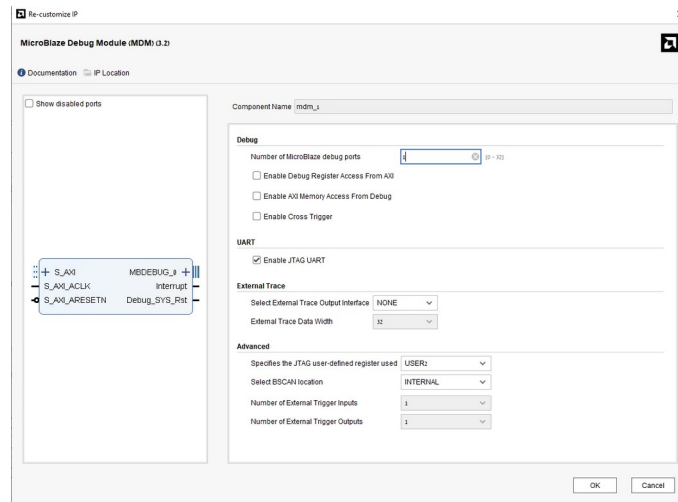
לאחר מכן פתחנו את הקובץ הריק והדבקנו לתוכו את כל פקודות קיבוע הפינים והגדרות המתחים הדרושות לממשקי הרשת השעונים והתקשורת הטורית של הפרויקט.

```
1 #####
2 ## VCU118 Rev2.0 XDC 12/08/2017
3 #####
4 #SGMII
5 set_property PACKAGE_PIN AR23 [get_ports ext_mdio_pcs_pma_0_0_mdio_io]
6 set_property IOSTANDARD LVCMOS18 [get_ports ext_mdio_pcs_pma_0_0_mdio_io]
7 set_property IOSTANDARD LVDS [get_ports sgmii_0_0_rxn]
8 set_property PACKAGE_PIN AU24 [get_ports sgmii_0_0_rxp]
9 set_property PACKAGE_PIN AV24 [get_ports sgmii_0_0_rxn]
10 set_property IOSTANDARD LVDS [get_ports sgmii_0_0_rxp]
11 set_property IOSTANDARD LVDS [get_ports sgmii_0_0_txn]
12 set_property PACKAGE_PIN AU21 [get_ports sgmii_0_0_txp]
13 set_property PACKAGE_PIN AV21 [get_ports sgmii_0_0_txn]
14 set_property IOSTANDARD LVDS [get_ports sgmii_0_0_txp]
15 set_property PACKAGE_PIN AV23 [get_ports ext_mdio_pcs_pma_0_0_mdc]
16 set_property IOSTANDARD LVCMOS18 [get_ports ext_mdio_pcs_pma_0_0_mdc]
17 #set_property PACKAGE_PIN AU23 [get_ports "PHY1_CLKOUT"] ;# Bank 64 VCCO - VCC1V8_FPGA - IO_L14P_T2L_N2_GC_64
18 #set_property IOSTANDARD LVCMOS18 [get_ports "PHY1_CLKOUT"] ;# Bank 64 VCCO - VCC1V8_FPGA - IO_L14P_T2L_N2_GC_64
19 set_property IOSTANDARD LVDS [get_ports refclk125_in_0_clk_n]
20 set_property PACKAGE_PIN AY24 [get_ports refclk125_in_0_clk_p]
21 set_property PACKAGE_PIN AY23 [get_ports refclk125_in_0_clk_n]
22 set_property IOSTANDARD LVDS [get_ports refclk125_in_0_clk_p]
23 set_property PACKAGE_PIN BA21 [get_ports "PHY1_RESET_B"] ;# Bank 64 VCCO - VCC1V8_FPGA - IO_T1U_N12_64
24 set_property IOSTANDARD LVCMOS18 [get_ports "PHY1_RESET_B"] ;# Bank 64 VCCO - VCC1V8_FPGA - IO_T1U_N12_64
25 #CLK
26 set_property IOSTANDARD LVDS [get_ports sysclk1_300_clk_n]
27 set_property PACKAGE_PIN G31 [get_ports sysclk1_300_clk_p]
28 set_property PACKAGE_PIN F31 [get_ports sysclk1_300_clk_n]
29 set_property IOSTANDARD LVDS [get_ports sysclk1_300_clk_p]
30 #UART
31 #set_property PACKAGE_PIN BB22 [get_ports "USB_UART_RTS"] ;# Bank 64 VCCO - VCC1V8_FPGA - IO_L8N_T1L_N5_AD12N_64
32 #set_property IOSTANDARD LVCMOS18 [get_ports "USB_UART_RTS"] ;# Bank 64 VCCO - VCC1V8_FPGA - IO_L8N_T1L_N5_AD12N_64
33 set_property PACKAGE_PIN BB21 [get_ports rs232_uart_txd]
34 set_property IOSTANDARD LVCMOS18 [get_ports rs232_uart_txd]
35 set_property PACKAGE_PIN AW25 [get_ports rs232_uart_rxd]
36 set_property IOSTANDARD LVCMOS18 [get_ports rs232_uart_rxd]
37 #set_property PACKAGE_PIN AY25 [get_ports "USB_UART_CTS"] ;# Bank 64 VCCO - VCC1V8_FPGA - IO_L8P_T1L_N2_AD5P_64
38 #set_property IOSTANDARD LVCMOS18 [get_ports "USB_UART_CTS"] ;# Bank 64 VCCO - VCC1V8_FPGA - IO_L8P_T1L_N2_AD5P_64
39 #RESET
40 set_property PACKAGE_PIN L19 [get_ports reset]
41 set_property IOSTANDARD LVCMOS12 [get_ports reset]
42 # RX SGMII
43 set_property DIFF_TERM_ADV TERM_100 [get_ports sgmii_0_0_rxp]
44 set_property DIFF_TERM_ADV TERM_100 [get_ports sgmii_0_0_rxn]
45 set_property DQS_BIAS TRUE [get_ports sgmii_0_0_rxp]
46 set_property DQS_BIAS TRUE [get_ports sgmii_0_0_rxn]
47 #set_property EQUALIZATION EQ_LEVEL0 [get_ports {sgmii_0_0_rxp sgmii_0_0_rxn} ]
48 # TX SGMII
49 set_property LVDS_PRE_EMPHASIS FALSE [get_ports sgmii_0_0_txp]
50 set_property LVDS_PRE_EMPHASIS FALSE [get_ports sgmii_0_0_txn]
51
52
53 set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
54 set_property OUTPUT_IMPEDANCE {} [get_ports {sgmii_0_0_txp sgmii_0_0_txn} ]
55 set_property ODT {} [get_ports {sgmii_0_0_rxp sgmii_0_0_rxn} ]
56 set_property OFFCHIP_TERM NONE [get_ports sgmii_0_0_txp]
57 set_property C_CLK_INPUT_FREQ_HZ 300000000 [get_debug_cores dbg_hub]
58 set_property C_ENABLE_CLK_DIVIDER false [get_debug_cores dbg_hub]
59 set_property C_USER_SCAN_CHAIN 1 [get_debug_cores dbg_hub]
60 connect_debug_port dbg_hub/clk [get_nets clk]
```

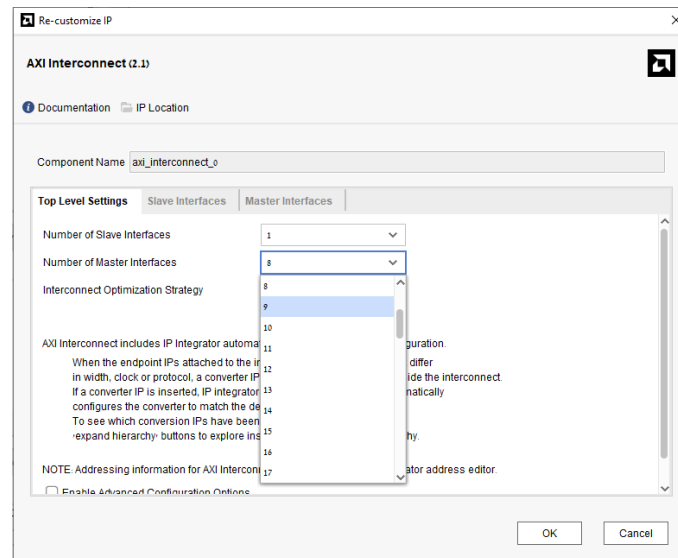
שלב 22:

רגע לפני שניגש להידור הפרויקט, נבצע שינוי קטן בחומרה שיחסוך לנו משאבי עיבוד יקרים בשלב כתיבת התוכנה ב Vitis. בשרטוט שלנו קיים הבלוק MicroBlaze Debug Module (MDM). כדי לאפשר ניתוב של פקודות ההדפסה printf ישירות דרך כבל ה-JTAG במהירות גבוהה וללא תלות ברכיב UART חיצוני, נגדיר אותו כערוץ תקשורת AXI.

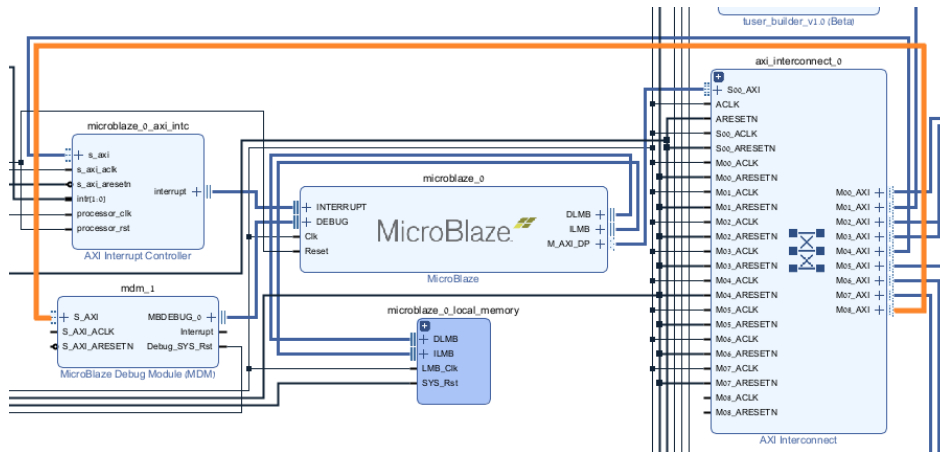
נלחץ לחיצה כפולה על רכיב ה-MDM בשרטוט, ובחלון ההגדרות נסמן ב-V את האפשרות Enable JTAG UART. נלחץ OK. פעולה זו תחשוף פורט בקרה חדש מסוג S_AXI על גבי הבלוק.



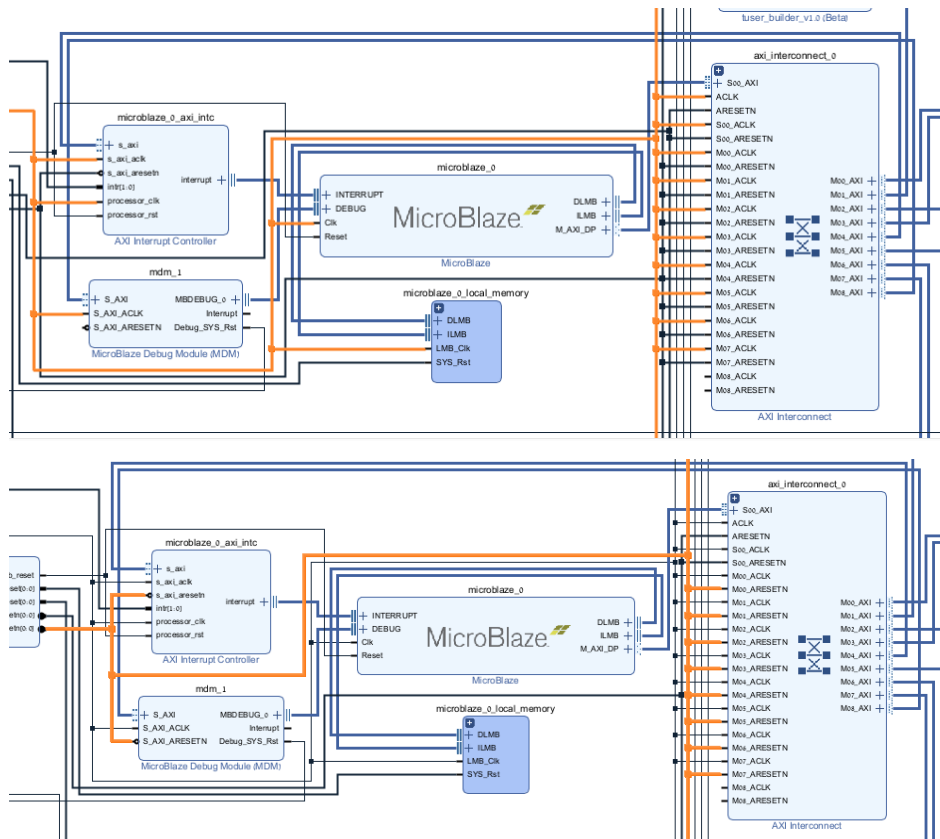
נפתח את חלון ההגדרות של בקר ה-axi_interconnect_0 ונגדיל את הערך של Number of Master Interfaces ב-1, מ-8 ל-9. פעולה זו תייצר עבורנו את הפורט החדש M08_AXI.



נמתח חוט ונחבר בין הפורט החדש M08_AXI ב-Interconnect אל כניסת ה-S_AXI שהתווספה ל-MDM.



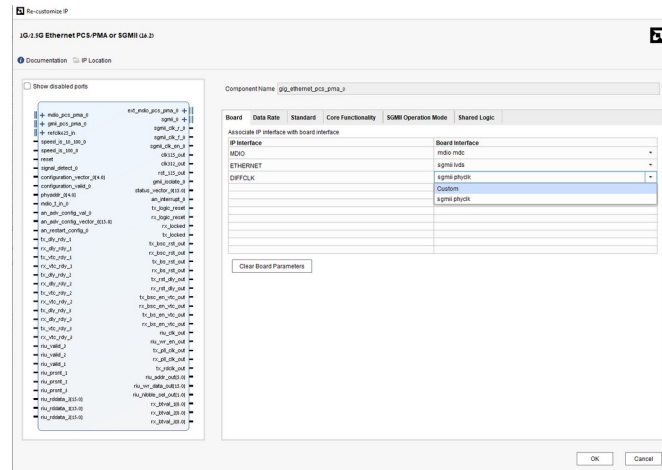
כדי להבטיח סנכרון מושלם, נחבר את כניסות השעון והריסט החדשות שנוצרו ב-M08_ARESETN ו-M08_ACLK ב-Interconnect אל קווי השעון והריסט המרכזיים של המערכת.



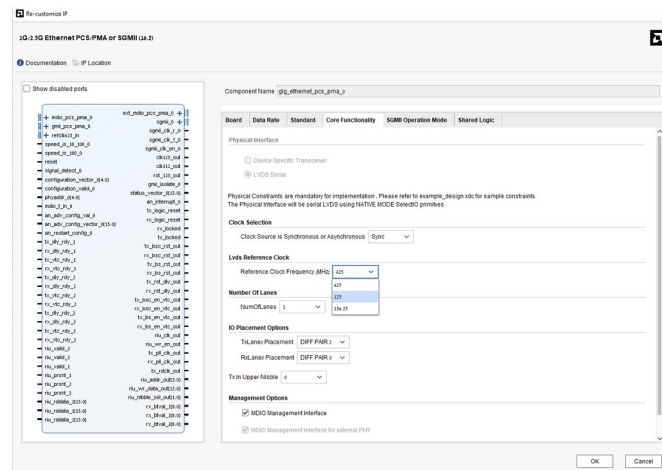
אוטומטית תדר עבודה של 625 MHz עבור ה-SGMII. אולם, מכיוון שפרוטוקול התקשורת שלנו Gigabit Ethernet מבוסס על שעון ליבה של 125 MHz, פער זה ימנע מהרכיב הפיזי להגיע לנעילה (Lock) ולשדר נתונים.

רגע לפני תחילת ה-Synthesis, עלינו לדרוס את ברירת המחדל הזו:

נלחץ לחיצה כפולה על הבלוק gig_ethernet_pcs_pma_0 בשרטוט כדי לפתוח את חלון ההגדרות שלו. נעבור ללשונית Board. בשורה של DIFFCLK נשנה את הערך מהגדרת הלוח sgmmii phyclk לאפשרות Custom.



כעת, נעבור ללשונית Core Functionality. בשדה Lvds Reference Clock נוודא שהתדר מעודכן לערך התקין 125 MHz.



נלחץ OK ונשמור את השרטוט. המערכת כעת מסוכנרת לחלוטין לתדר הרשת.

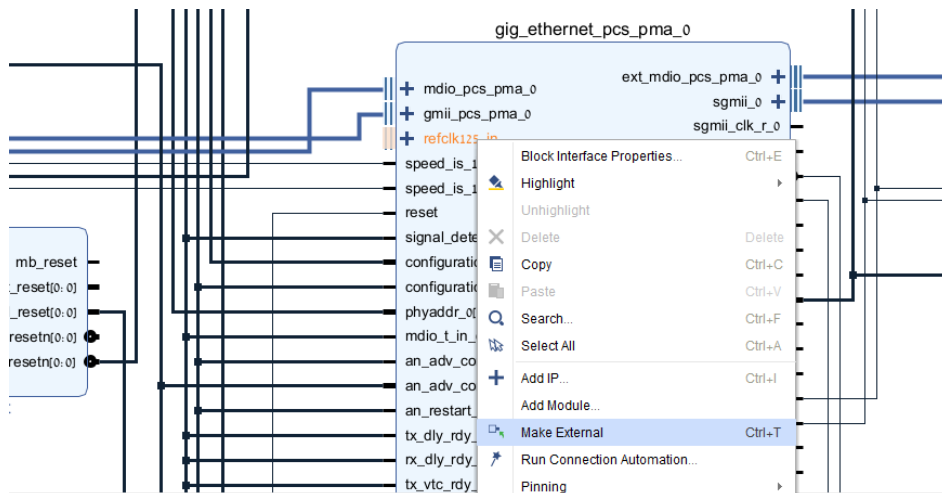
הערה לקורא: זו הסיבה שבחלק מצילומי המסך בשלבים הקודמים של המדריך עשויה להופיע ברירת המחדל הישנה. כעת, בשלב הסופי, אנו מקבעים את התדר הנכון באופן ידני.

כדי לאפשר למערכת לקבל שעון ייחוס ישירות מהמתנד הפיזי שעל הלוח, עלינו לחשוף את כניסת השעון של הרכיב כפורט ראשי בשרטוט:

ניגש לבלוק הרלוונטי ונאתר את הפין refclk125_in.

נלחץ קליק ימני בדיוק על השם או הקצה של הפין.

בתפריט שייפתח, נבחר באפשרות Make External.

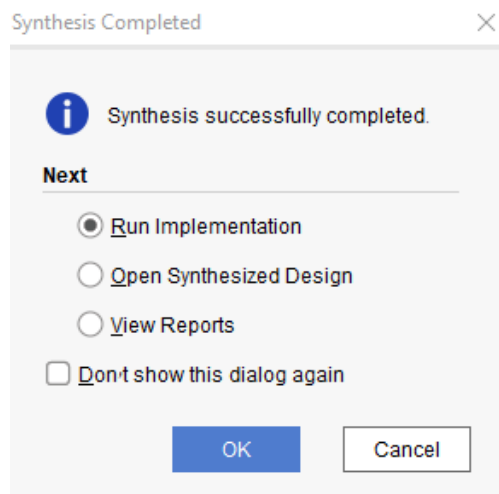


תוכנת Vivado תמתח קו אוטומטי ותייצר פורט חיצוני חדש, אשר יקבל אוטומטית את השם .refclk125_in_0.

בשלב זה המערכת שלנו שלמה, מקומפלת ונבדקה לוגית. כעת נריץ את שרשרת ההידור המלאה של תוכנת Vivado כדי להפוך את השרטוט לקובץ בינארי שניתן לצרוב על רכיב ה-FPGA, ולבסוף נייצא את כלל הגדרות החומרה לטובת כתיבת קוד התוכנה ב-Vitis.

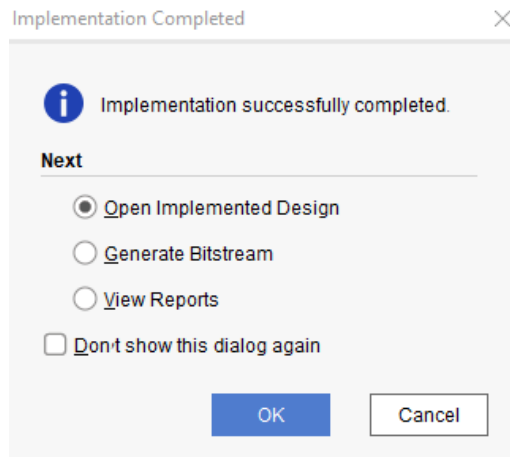
השלב הראשון בהידור הוא הרצת הסינתזה. לחצנו על Run Synthesis בסרגל השמאלי והמתנו לסיום התהליך.

- ▼ SYNTHESIS
- ▶ Run Synthesis
- > Open Synthesized Design

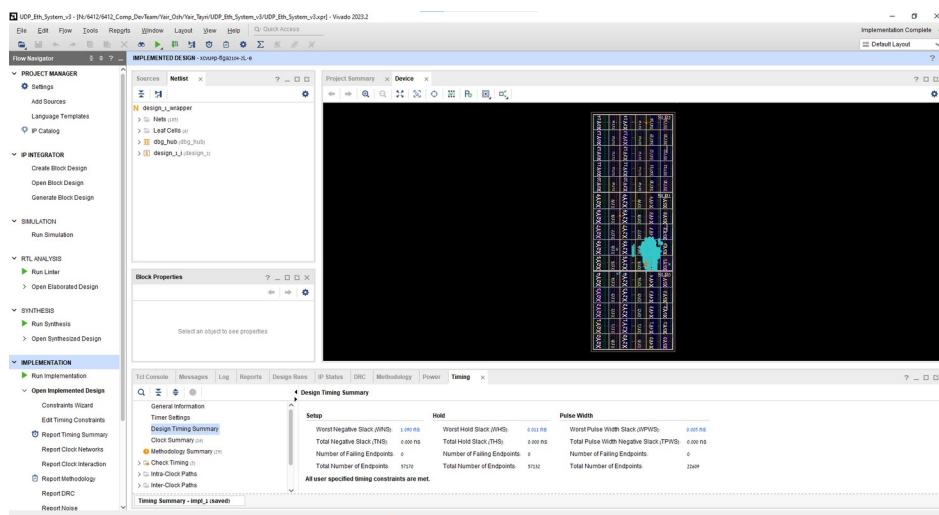


עם קבלת חלון ההודעה המאשר שהסינתזה עברה בהצלחה, בחרנו באפשרות Run Implementation ולחצנו על OK כדי להמשיך לשלב פריסת הרכיבים הפיזית על השבב.

▼ IMPLEMENTATION
 ▶ Run Implementation
 > Open Implemented Design



לאחר מספר דקות התקבל חלון נוסף המאשר כי גם שלב האימפלמנטציה הסתיים בהצלחה. בחלון זה בחרנו באפשרות Open Implemented Design כדי לפתוח את התצוגה הפיזית של הלוח.



(I/O Ports) של השבב לאחר שלב ה-Implementation.
הדוח מפרט את תקני המתח ((I/O Standard השונים המשמשים במערכת, כגון LVCMOS18-ו-LVDS (המשמש לתקשורת ה-SGMII המהירה), וכן פרמטרים קריטיים נוספים כמו קצב שינוי האות (Slew Rate) ושולי רעש (Remaining Margin).
בחינה של דוח זה מאשרת כי אין חריגות חשמליות או חריגות רעש, וכי המערכת עומדת בדרישות ותעבוד ביציבות גם בתדרי עבודה גבוהים ללא פגיעה בשלמות האות.

Noise

Summary

Messages (1)

I/O Bank Details

Links

I/O Bank Details

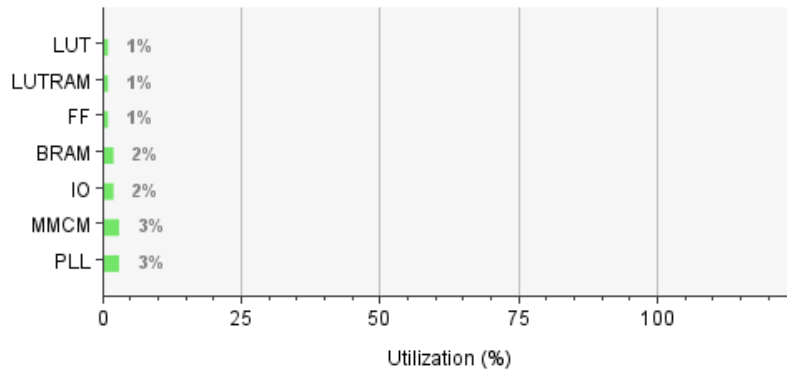
Name	Port	I/O Std	Vcco	Slew	Drive Strength (mA)	OUT
 I/O Bank 0 (0)						
 I/O Bank 40 (0)						
 I/O Bank 41 (0)						
 I/O Bank 42 (0)						
 I/O Bank 43 (0)						
 I/O Bank 45 (0)						
 I/O Bank 46 (0)						
 I/O Bank 47 (0)						
 I/O Bank 48 (0)						
 I/O Bank 64 (6)		multiple	1.80	multiple	multiple	
 BA21	PHY1_RESET_B[0]	LVC MOS18	1.80	SLOW		12
 AV23	ext_mdio_pcs_pma_0_0_mdc	LVC MOS18	1.80	SLOW		12
 AR23	ext_mdio_pcs_pma_0_0_mdio_io	LVC MOS18	1.80	SLOW		12
 BB21	rs232_uart_bxd	LVC MOS18	1.80	SLOW		12
 AU21	sgmii_0_0_txp	LVDS	1.80			
 AV21	sgmii_0_0_bxn	LVDS	1.80			
 I/O Bank 65 (0)						
I/O Bank 66 (0)						

OUTPUT_IMPEDA...	Remaining Margin (%)	PRE_EMPHA...	LVDS_PRE_EMPHA...	Off-Chip Termina...	Notes
				multiple	
	95.29			FP_VTT_50	
	93.72			FP_VTT_50	
	95.58			FP_VTT_50	
	94.97			FP_VTT_50	
	97.23			NONE	
	97.26			NONE	

להלן דוח ניצול המשאבים הכולל Summary של המערכת. מתוך הדוח עולה כי הפרויקט צורך אחוזים בודדים בלבד ממשאבי לוח ה- VCU118 (כ- 1.36% מה- LUTs וכ- 2.43% בלוקי זיכרון (BRAM). נתון זה מוכיח כי הארכיטקטורה שמומשה – הכוללת את מעבד ה-MicroBlaze, מנוע ה-UOE המותאם אישית ורכיבי התקשורת – היא חסכונית ויעילה. תצורה זו מותירה מרחב משאבים עצום המאפשר הרחבות עתידיות ושילוב מאיצי חומרה נוספים על גבי השבב ללא חשש מחריגת משאבים.

Summary

Resource	Utilization	Available	Utilization %
LUT	16067	1182240	1.36
LUTRAM	1742	591840	0.29
FF	23542	2364480	1.00
BRAM	52.50	2160	2.43
IO	14	832	1.68
MMCM	1	30	3.33
PLL	2	60	3.33



הטבלה המפורטת מציגה את צריכת המשאבים בחלוקה למודלים הפנימיים של המערכת. מניתוח הנתונים עולה כי מנוע UOE המותאם אישית ורכיב ניתוב התקשורת SmartConnect הם הצרכנים העיקריים של משאבי הלוגיקה. לעומת זאת, בלוקי הזיכרון מוקצים בעיקר לזיכרון המקומי של המעבד ולמערכת התקשורת הפיזי. פירוט זה מאפשר בקרת תכנון ומעקב מדויק אחר רמת היעילות של כל בלוק חומרה בפרויקט.

Resource	Utilization	Available	Utilization %
LUT	16067	1182240	1.36
LUTRAM	1742	591840	0.29
FF	23542	2364480	1.00
BRAM	52.50	2160	2.43
IO	14	832	1.68
MMCM	1	30	3.33
PLL	2	60	3.33

הטבלה הבאה מציגה את אבני הבניין הבסיסיות ביותר המרכיבות את המערכת לאחר שלבי הסינתזה והמימוש. ניתן לראות שימוש נרחב ברכיבי אוגרים מסוג FDRE ו-LUTs מסוגים שונים (בעיקר LUT6 המשמשים למימוש פונקציות לוגיות מורכבות). התפלגות זו משקפת את המימוש החומרתי הישיר של מנוע ה-UDP ומעבד ה-MicroBlaze, ומאשרת שהכלי ביצע אופטימיזציה נכונה בבחירת הרכיבים כדי לחסוך בשטח סילקון.

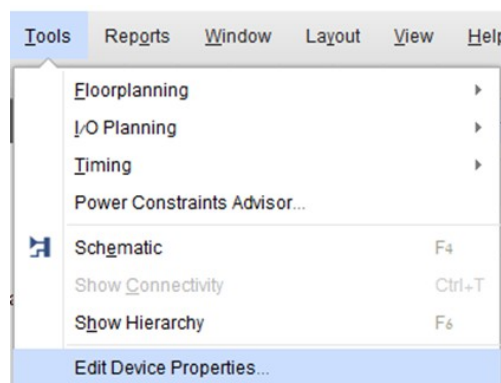
Primitives		
Ref Name	Used	Functional Category
FDRE	22175	Register
LUT6	6481	CLB
LUT3	3253	CLB
LUT5	2910	CLB
LUT4	2798	CLB
LUT2	1928	CLB
SRL16E	913	CLB
RAMD32	870	CLB
FDSE	779	Register
SRLC32E	466	CLB
LUT1	430	CLB
FDCE	421	Register
RAMD64E	368	CLB
CARRY8	257	CLB
MUXF7	186	CLB
FDPE	166	Register
RAMS32	134	CLB
RAMB36E2	50	BLOCKRAM
SRLC16E	17	CLB
BUFGCE	8	Clock
IBUFCTRL	7	Others
RAMB18E2	5	BLOCKRAM
OBUF	3	I/O
INBUF	3	I/O
DIFFINBUF	3	I/O
RX_BITSLICE	2	I/O
PLLE4_ADV	2	Clock
BSCANE2	2	Configuration
BITSLICE_CONTROL	2	I/O
TX_BITSLICE	1	I/O
RIU_OR	1	I/O
OBUFT	1	I/O
OBUFDS	1	I/O
MMCME4_ADV	1	Clock
BUFGCE_DIV	1	Clock
AND2B1L	1	Others

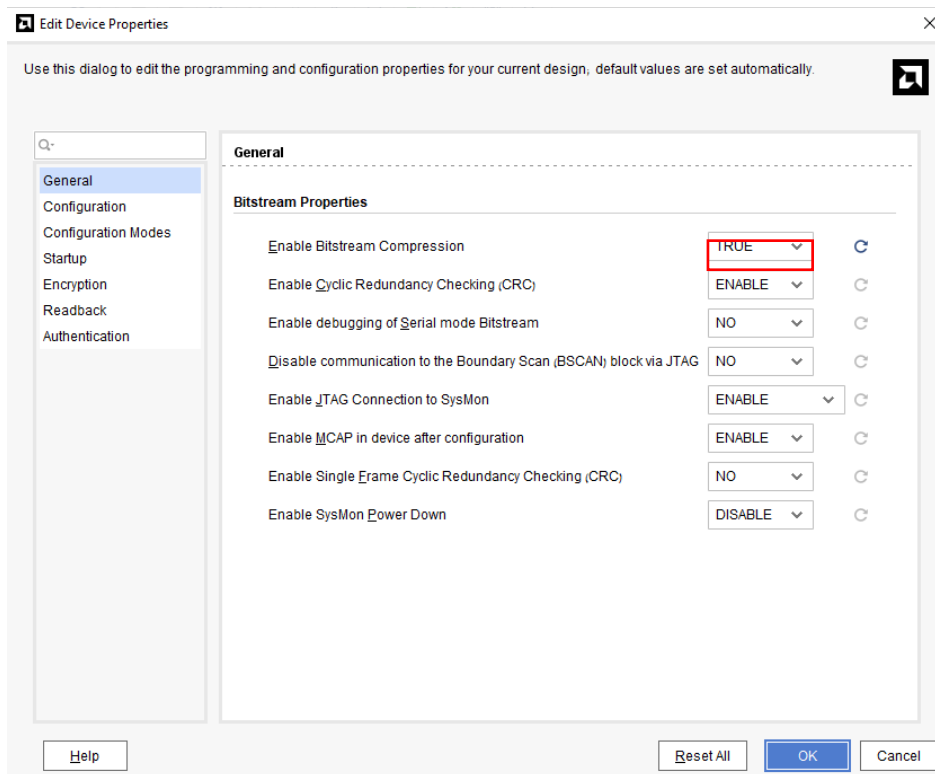
רכיב ה-FPGA שעל לוח ה- VCU118 מורכב ממספר אזורי סיליקון נפרדים. טבלאות אלו מציגות את פיזור הלוגיקה ומשאבי הקלט והפלט על פני השבב. ניתן לראות כי מרבית המערכת ממוקמת באזור סיליקון אחד מרכזי, SLR1. ריכוז התכנון באזור יחיד מקטין את הצורך בחציית גבולות בין אזורים שונים, דבר התורם משמעותית לעמידה באילוצי התזמון ומפחית את השיעור הכללי בחומרה.

SLR IO Utilization							
SLR Index	Used IOBs	(%)IOBs	Used IPADs	(%)IPADs	Used OPADs	(%)OPADs	GTs
SLR2	1	0.48	0	0.00	0	0.00	0
SLR1	13	3.13	0	0.00	0	0.00	0
SLR0	0	0.00	0	0.00	0	0.00	0
Total	14		0		0		0

SLR CLB Logic and Dedicated Block Utilization				
Site Type	SLR0	SLR1	SLR2	
> Block RAM Tile	0	52.5	0	
CARRY8	0	257	0	
> CLB	5	3773	0	
> CLB LUTs	13	16054	0	
CLB Registers	6	23536	0	
DSPs	0	0	0	
F7 Muxes	0	186	0	
F8 Muxes	0	0	0	
F9 Muxes	0	0	0	
Unique Control Sets	1	1487	0	
URAM	0	0	0	

כדי לייעל את תהליך הצריבה ולחסוך בנפח זיכרון, חשוב לוודא שקובץ ה-Bitstream נידחס. בתפריט העליון של התצוגה הפיזית לחצנו על Tools ובחרנו ב- Edit Device Properties. בחלון שנפתח, תחת קטגוריית General, ווידאנו שהשורה Enable Bitstream Compression מוגדרת כ-TRUE.





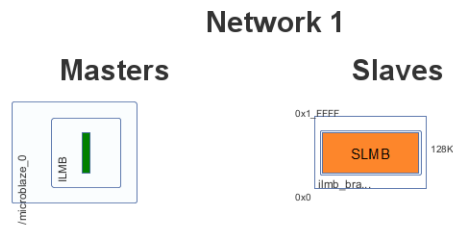
טרם ייצור קובץ ה-Bitstream, בוצעה בדיקה וחלוקה של מפת הכתובות ב-Vivado.

מרחב הנתונים (Network 0):

מרכז את כלל רכיבי ה-AXI (בקר ה-DMA, הטיימר, מנוע ה-UDP ההיקפי, זיכרון ה-BRAM וכו'), לכל אחד מהם הוקצתה כתובת בסיס (Base Address) ונפח זיכרון ייעודי.

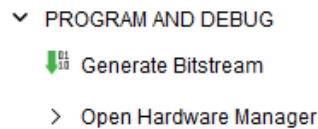


מרחב ההוראות (Network 1): מכיל את הגישה הישירה לזיכרון המקומי (Local Memory) ממנו המעבד שולף את קוד התוכנה

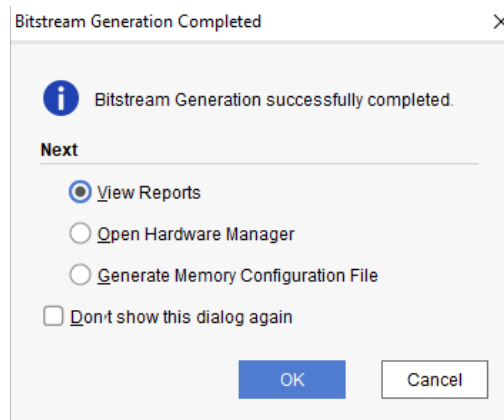


מיפוי תקין זה מבטיח שללא התנגשויות כתובות (Address Overlap), המעבד יוכל לגשת לכל הרגיסטרים במערכת.

לאחר שווידאנו את הגדרות הדחיסה, ניגשנו לחלק התחתון של סרגל הכלים השמאלי PROGRAM AND DEBUG ולחצנו על Generate Bitstream.

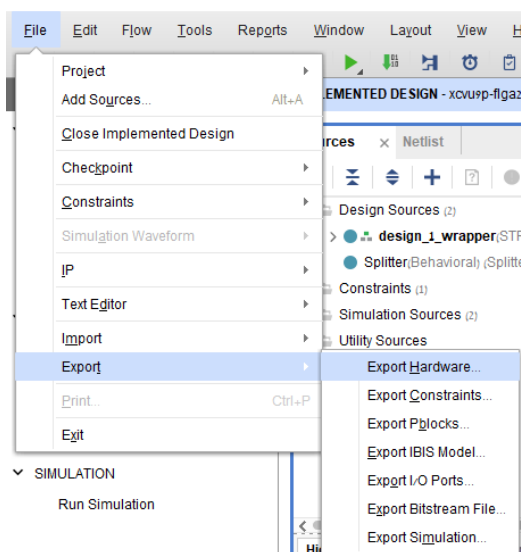


המתנו עד שהתוכנה סיימה לקמפל את קובץ הצריבה הסופי, ועם קפיצת חלון אישור הסיום Bitstream Generation successfully completed לחצנו על Cancel מאחר שאין לנו צורך בפתיחת חלונות נוספים.

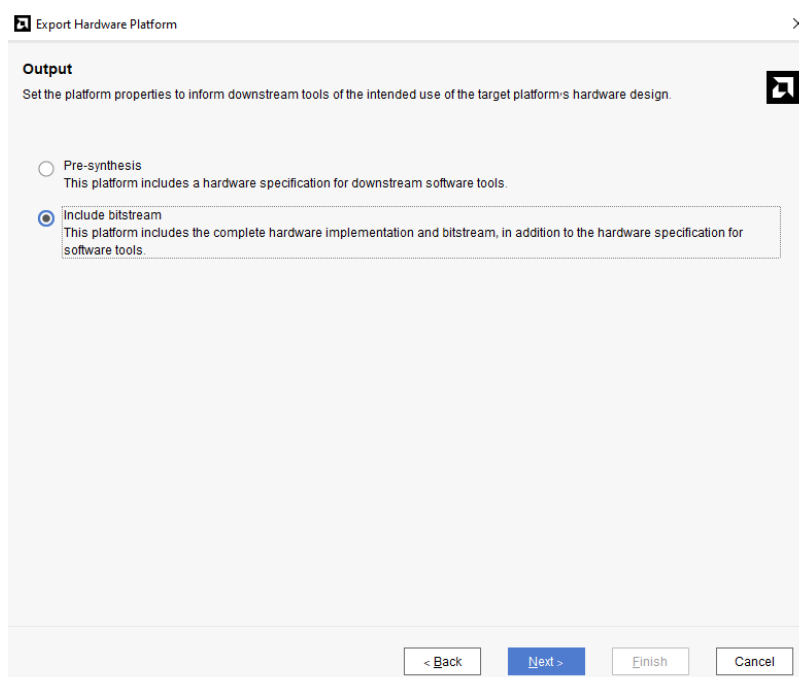


הצעד האחרון ב-Vivado הוא יצירת קובץ XSA המאגד את הגדרות החומרה וקובץ הצריבה עבור סביבת הפיתוח של המעבד.

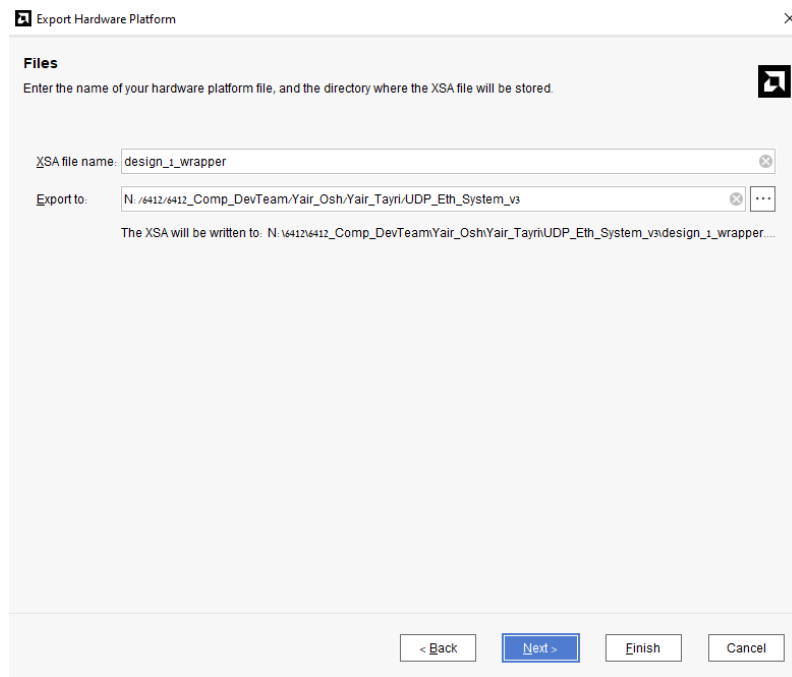
בתפריט העליון מצד שמאל לחצנו על File, בחרנו ב-Export, ולאחר מכן ב-Export Hardware.



באשף הייצוא שנפתח עברנו למסך Output ודאגנו לסמן את האפשרות Include bitstream.



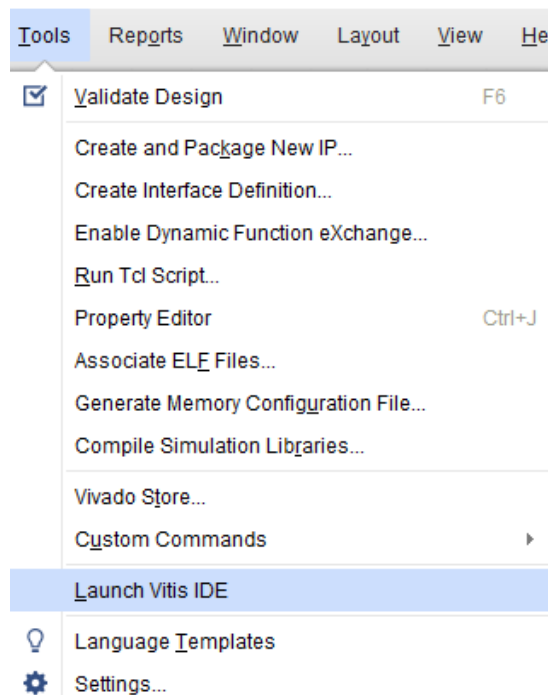
שלב זה הוא קריטי כדי שקובץ הצריבה ייכלל בתוך הייצוא. המשכנו למסך הבא, השארנו את שם הקובץ ומיקומו כברירת מחדל, ולחצנו על Finish לסיום התהליך.



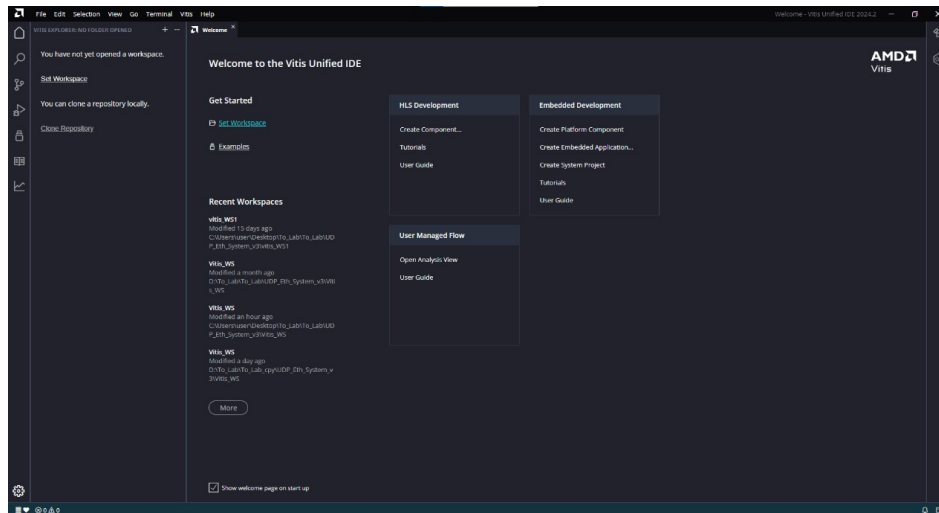
שלב 23:

בשלב זה נייבא את החומרה שיצרנו אל סביבת הפיתוח ונכתוב את קוד ניהול הרשת של המעבד.

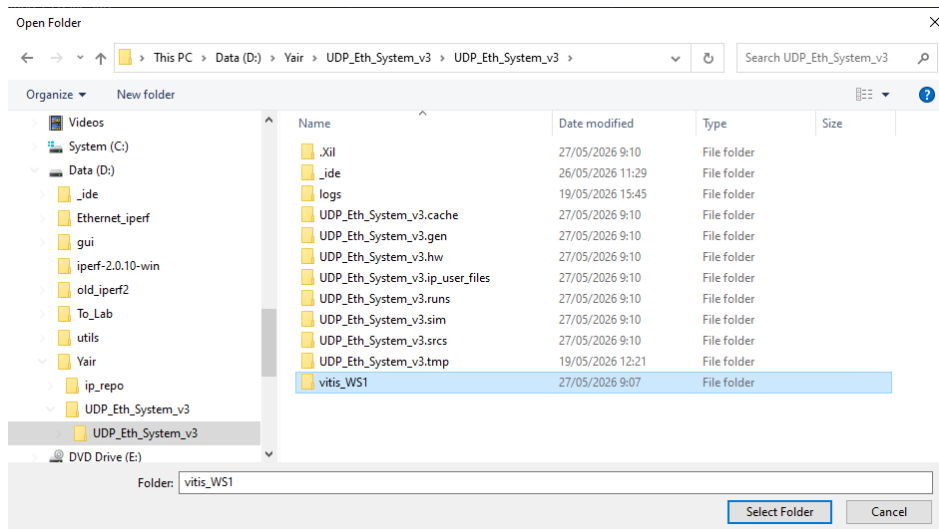
תחילה נפתח את סביבת הפיתוח לתוכנה ונגדיר את תיקיית העבודה של הפרויקט. נלחץ על תפריט Tools בתוכנת Vivado ונבחר באפשרות Launch Vitis IDE.



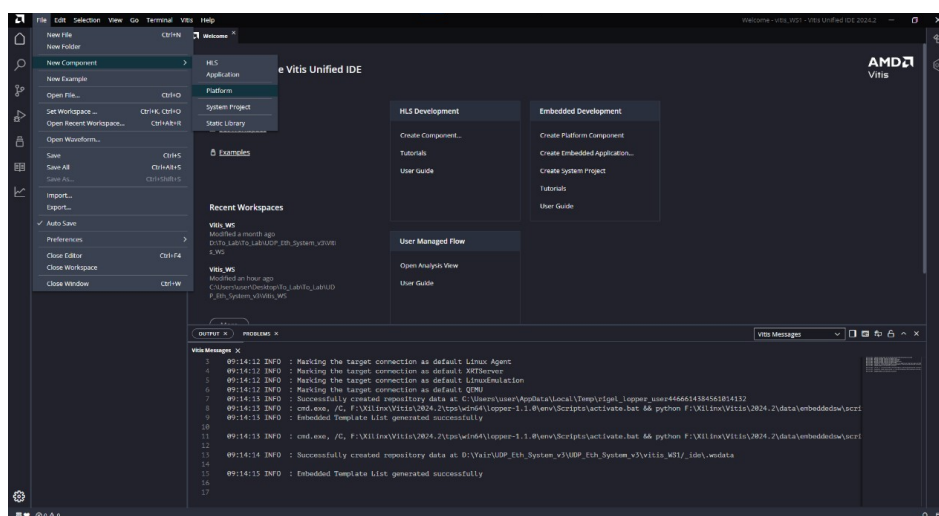
נלחץ על Set Workspace.



בחלון שנפתח נבחר את התיקיה vitis_WS1 כסביבת העבודה המרכזית.



כעת ניצור את שכבת התשתית המקשרת בין החומרה לתוכנה. נבחר ביצירת רכיב פלטפורמה חדש.



נעניק לו את השם UDP_Platform.

The screenshot shows the 'Create Platform Component' dialog box with the 'Name and Location' tab selected. The 'Component name' field contains 'UDP_Platform'. The 'Component location' dropdown shows 'D:\Yair\UDP_Eth_System_v3\UDP_Eth_System_v3\vitis_WS1'. A 'Browse' button is next to the location field. Below the fields, it states: 'Component will be created at D:\Yair\UDP_Eth_System_v3\UDP_Eth_System_v3\vitis_WS1\UDP_Platform'. A note at the bottom says: 'Note: Avoid using excessively deep paths, as you may encounter issues on Windows systems due to the character limit.' At the bottom of the dialog are 'Cancel' and 'Next' buttons.

נטען את קובץ ה-XSA שייצאנו בשלב הקודם design_1_wrapper.xsa.

The screenshot shows the 'Create Platform Component' dialog box with the 'Select Platform Creation Flow' tab selected. The 'Hardware Design (XSA)' radio button is selected. The 'Hardware Design (XSA) For Implementation' dropdown shows 'Select hardware design'. A 'Browse' button is next to it. An overlay window titled 'Select Hardware Design (XSA)' is open, showing a file explorer view of the directory 'D:\Yair\UDP_Eth_System_v3\UDP_Eth_System_v3'. The file 'design_1_wrapper.xsa' is selected in the file list. The 'File name' field at the bottom of the overlay contains 'design_1_wrapper.xsa' and the file type is set to 'XSA (*.xsa)'. 'Open' and 'Cancel' buttons are at the bottom of the overlay. The 'Next' button is visible in the background dialog.

נגדיר את מערכת ההפעלה כ-standalone ואת המעבד כ-microblaze_0.

The screenshot shows the 'Create Platform Component' dialog box with the 'OS and Processor' tab selected. The breadcrumb trail is 'Name and Location > Flow > OS and Processor > Summary'. The section title is 'Select Operating System and Processor'. Below it, a text description states: 'Specify the details for the initial domain to be added to the platform component. The platform can be modified later to add new domains or change settings by opening the platform editor.' There are two dropdown menus: 'Operating system:' with 'standalone' selected, and 'Processor:' with 'microblaze_0' selected. At the bottom, there are three buttons: 'Cancel', 'Back', and 'Next'.

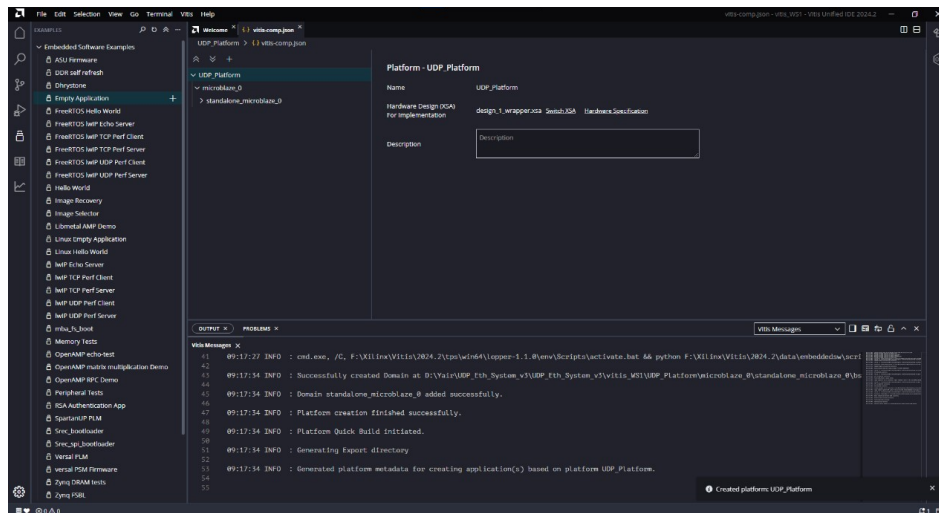
ולסיום נלחץ Finish.

The screenshot shows the 'Create Platform Component' dialog box with the 'Summary' tab selected. The breadcrumb trail is 'Name and Location > Flow > OS and Processor > Summary'. The section title is 'Summary'. Below it, a text description states: 'The following new Platform component will be created.' There is a table with the following data:

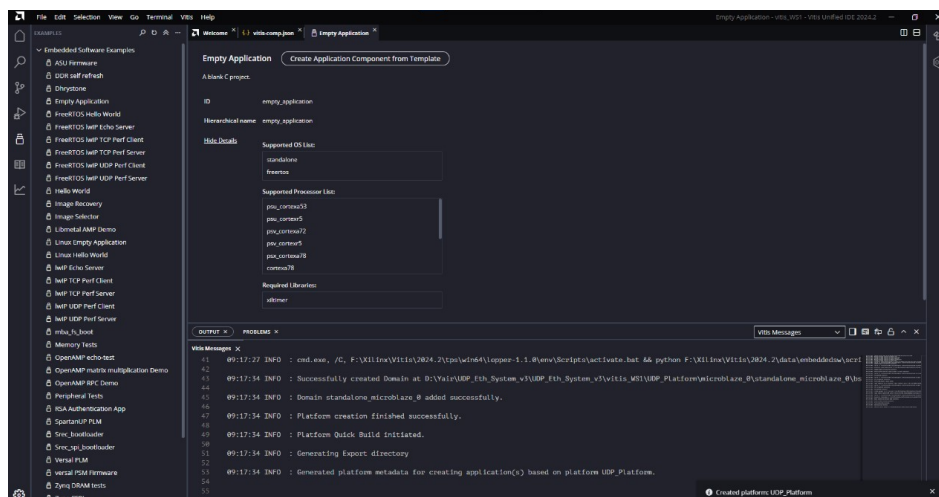
Name	UDP_Platform
Workspace	D:\Vair\UDP_Eth_System_v3\UDP_Eth_System_v3\witis_WS1
Component directory	D:\Vair\UDP_Eth_System_v3\UDP_Eth_System_v3\witis_WS1\UDP_Platform
Operating system	standalone
Processor	microblaze_0

Below the table, there is a note with a yellow warning icon: 'Note: A domain with selected operating system and processor will be added to the platform. The platform can be modified later to add new domains or change settings.' At the bottom, there are three buttons: 'Cancel', 'Back', and 'Finish'.

לאחר שהפלטפורמה מוכנה, ניצור את שכבת היישום. נפתח פרויקט אפליקציה חדש ונבחר בתבנית פרויקט C ריק בשם Empty Application.



נלחץ על 'Create Application Component from Template'.



נעניק לו את השם UDP_app.

The screenshot shows the 'Create Application Component - Empty Application' dialog box with the 'Name and Location' tab selected. The 'Component name' field contains 'UDP_app'. The 'Component location' dropdown shows 'D:\Yair\UDP_Eth_System_v3\UDP_Eth_System_v3\witis_WS1'. A 'Browse' button is next to it. Below the fields, it states 'Component will be created at D:\Yair\UDP_Eth_System_v3\UDP_Eth_System_v3\witis_WS1\UDP_app'. A note at the bottom says 'Note: Avoid using excessively deep paths, as you may encounter issues on Windows systems due to the character limit.' At the bottom of the dialog are 'Cancel' and 'Next' buttons.

Create Application Component - Empty Application

Name and Location > Hardware > Domain > Summary

Name and Location

Choose a name for your component and specify a directory where the component data files will be stored.

Component name:

Component location: [Browse](#)

Component will be created at D:\Yair\UDP_Eth_System_v3\UDP_Eth_System_v3\witis_WS1\UDP_app

Note: Avoid using excessively deep paths, as you may encounter issues on Windows systems due to the character limit.

[Cancel](#) [Next](#)

נקשר את האפליקציה לפלטפורמה שיצרנו, נבחר בדומיין המעבד הקיים, ונבחר בתבנית פרויקט C ריק בשם Empty Application.

נקשר את האפליקציה לפלטפורמה שיצרנו.

The screenshot shows the 'Create Application Component - Empty Application' dialog box with the 'Select Platform' tab selected. It displays a table of platforms from repositories. The table has columns: NAME, BOARD, FLOW, VENDOR, and PATH. One platform is listed: 'UDP_Platform' with board 'vcu118', flow 'Embedded', vendor 'xilinx.com', and path '...S1\UDP_Platform\export\UDP_Platform\UDP_Platform.xpfm'. There is an 'Info' link next to the path. At the bottom are 'Cancel', 'Back', and 'Next' buttons.

Create Application Component - Empty Application

Name and Location > Hardware > Domain > Summary

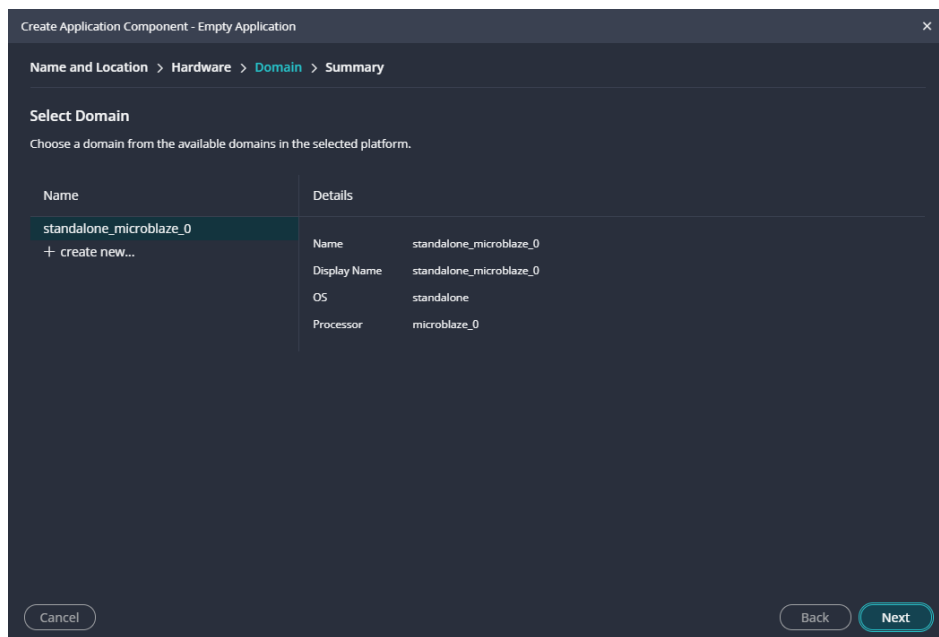
Select Platform

Platforms from your repositories that support the selected example. To create a new platform, use "File -> New Component -> Platform"

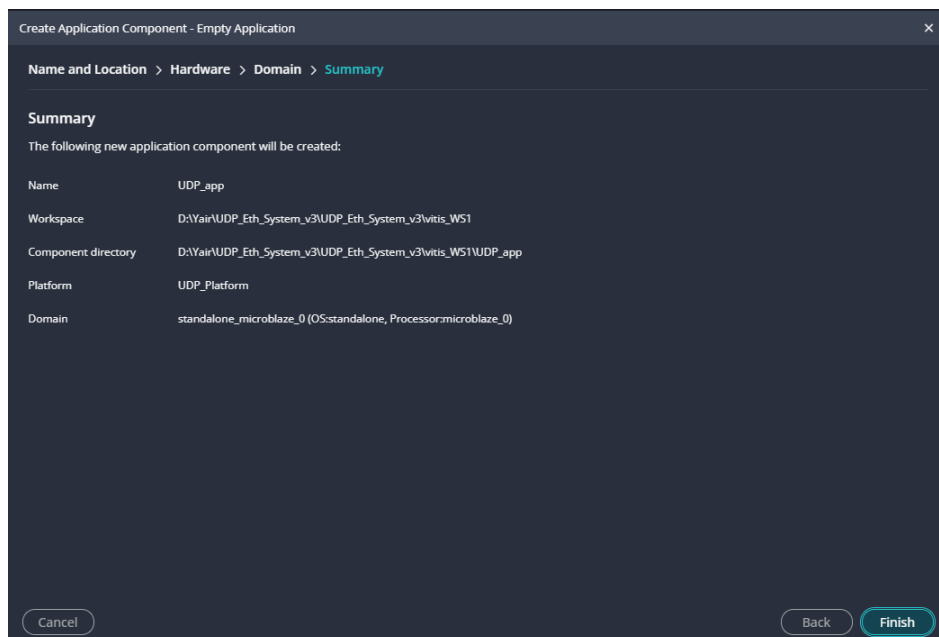
NAME	BOARD	FLOW	VENDOR	PATH
UDP_Platform	vcu118	Embedded	xilinx.com	...S1\UDP_Platform\export\UDP_Platform\UDP_Platform.xpfm Info

[Cancel](#) [Back](#) [Next](#)

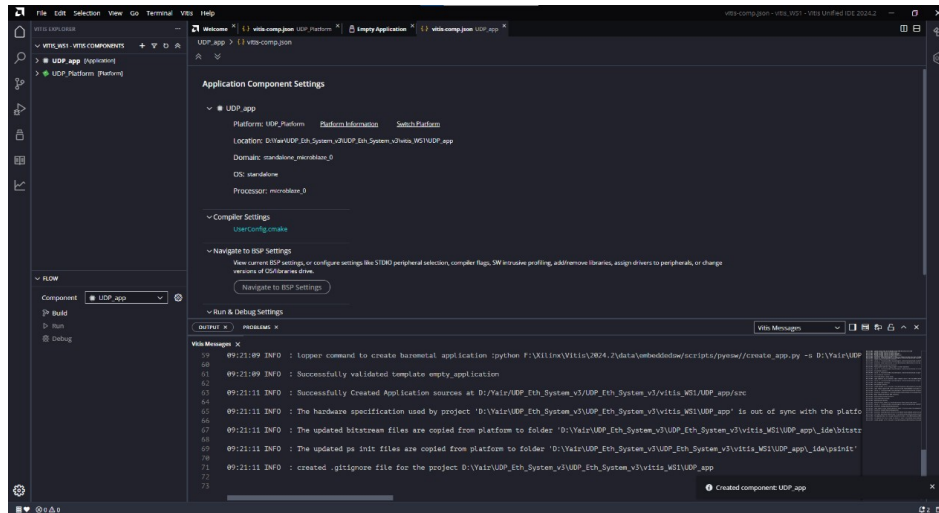
נלחץ. Next.



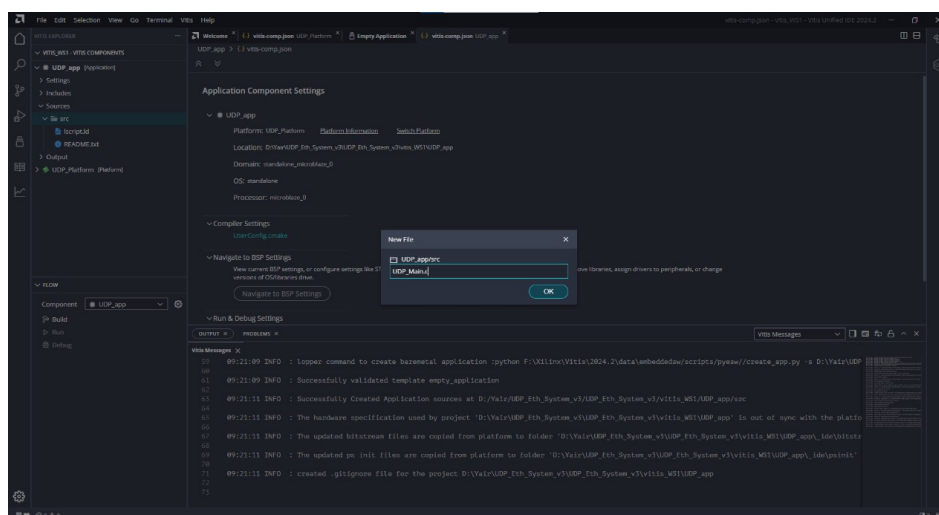
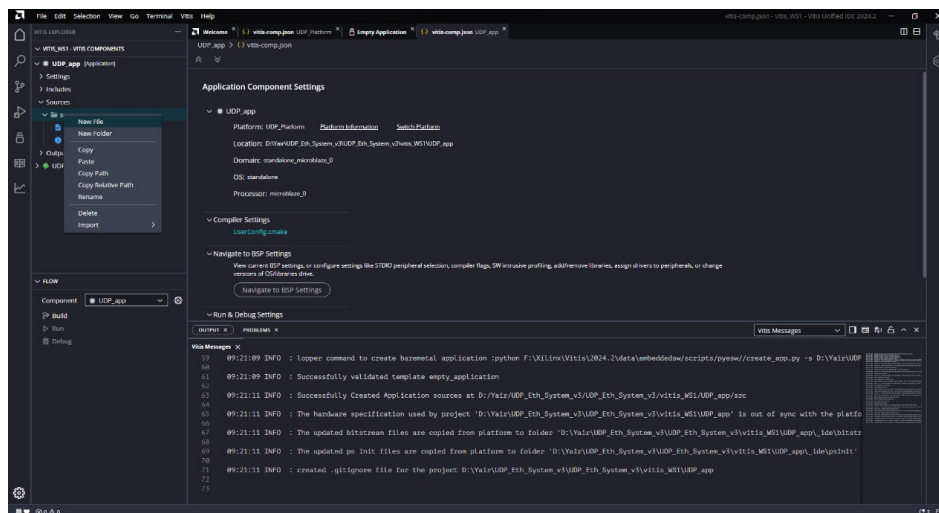
ולסיום נלחץ. Finish.



נקבל הודעה שאפליקציית התוכנה נוצרה.



לסיום, נוסיף קובץ מקור חדש בשם UDP_Main.c בתוך תיקיית src.



ניזן לקובץ את קוד התוכנה הכולל את הגדרות כתובת ה-MAC וכתובת ה-IP.

```
1 #include "xil_printf.h"
2 #include "xparameters.h"
3 #include "xaxidma.h"
4 #include "xintc.h"
5 #include "xil_types.h"
6 #include "xstatus.h"
7 #include "xil_exception.h"
8 #include <xil_io.h>
9 #include "sleep.h"
10 #include "xtmrctr.h"
11
12 /* Global Definitions */
13 #define DMA_BASEADDR XPAR_XAXIDMA_0_BASEADDR
14 #define INTC_BASEADDR XPAR_XINTC_0_BASEADDR
15 #define UOE_BASEADDR XPAR_TOP_UOE_0_BASEADDR
16 #define TUSER_BASEADDR XPAR_TUSER_BUILDER_0_BASEADDR
17 #define AXI_BRAM_BASEADDR XPAR_XBRAM_0_BASEADDR
18 #define DMA_MM2S_INTR_NUM XPAR_FABRIC_AXI_DMA_0_INTR
19 #define DMA_S2MM_INTR_NUM XPAR_FABRIC_AXI_DMA_0_INTR_1
20 #define RX_DMA_MEM_ADDR (AXI_BRAM_BASEADDR + 0x0000)
21 #define TX_DMA_MEM_ADDR (AXI_BRAM_BASEADDR + 0x0400)
22 #define ECHO_LENGTH_BYTES 1024
23 #define HEADER_SIZE_BYTES 8
24 #define NUM_TX_PACKETS 10000
25 /* Local MAC = 00:0A:35:01:02:03 */
26 #define LOCAL_MAC_MSB_VALUE 0x0000000A
27 #define LOCAL_MAC_LSB_VALUE 0x35010203
28 /* FPGA IP = 192.168.1.10 */
29 #define LOCAL_IP_VALUE 0xC0A8010A
30 /* PC IP = 192.168.1.20 */
31 #define DEST_IP_VALUE 0xC0A80114
32 #define FPGA_SRC_PORT 5002
33 #define PC_DST_PORT 5002
34 #define UOE_REG_LOCAL_MAC_ADDR_LSB 0x04
35 #define UOE_REG_LOCAL_MAC_ADDR_MSB 0x08
36 #define UOE_REG_LOCAL_IP_ADDR 0x0C
37 #define UOE_REG_CONFIG_DONE 0x38
38 #define TUSER_REG_DST_IP 0x00
39 #define TUSER_REG_LENGTH 0x04
40 #define TUSER_REG_SRC_PORT 0x08
41 #define TUSER_REG_DST_PORT 0x0C
42 #define MDIO_SETUP_REG 0x0500
43 #define MDIO_CTRL_REG 0x0504
44 #define MDIO_WRITE_REG 0x0508
45 #define MDIO_READ_REG 0x050C
46 #define MDIO_READY_BIT 0x00010000 // Bit 16
47 #define MAC_BASEADDR XPAR_TRI_MODE_ETHERNET_MAC_0_BASEADDR
48 #define TIMER_BASEADDR XPAR_AXI_TIMER_0_BASEADDR
49
50 #define TIMER_COUNTER_NUMBER 0
51
52 void mdio_init();
53 void mdio_write(u8 phy_addr, u8 reg_addr, u16 data);
54 u16 mdio_read(u8 phy_addr, u8 reg_addr);
55
56 /*Instance for DMA & Interrupt Controller*/
57 static XAxiDma AxiDma;
58 static XIntc Intc;
59 static XTmrCtr Timer;
60
61 /*ISR Flags*/
62 static volatile int RxDone = 0;
63 static volatile int TxDone = 0;
64
65 /*UOE Configuration Function*/
66 static void config_uoe(void)
67 {
68     xil_printf("Configuring UOE...\r\n");
69     Xil_Out32(UOE_BASEADDR + UOE_REG_LOCAL_MAC_ADDR_LSB, LOCAL_MAC_LSB_VALUE);
70     Xil_Out32(UOE_BASEADDR + UOE_REG_LOCAL_MAC_ADDR_MSB, LOCAL_MAC_MSB_VALUE);
71     Xil_Out32(UOE_BASEADDR + UOE_REG_LOCAL_IP_ADDR, LOCAL_IP_VALUE);
72     Xil_Out32(UOE_BASEADDR + UOE_REG_CONFIG_DONE, 1);
73     xil_printf("UOE configured\r\n");
74 }
75
76 /*Tuser_builder Configuration Function*/
77 static void config_tuser_builder(u32 length_bytes)
78 {
79     xil_printf("Configuring TUSER builder...\r\n");
80     Xil_Out32(TUSER_BASEADDR + TUSER_REG_DST_IP, DEST_IP_VALUE);
81     Xil_Out32(TUSER_BASEADDR + TUSER_REG_LENGTH, length_bytes);
82     Xil_Out32(TUSER_BASEADDR + TUSER_REG_SRC_PORT, FPGA_SRC_PORT);
83     Xil_Out32(TUSER_BASEADDR + TUSER_REG_DST_PORT, PC_DST_PORT);
84     xil_printf("TUSER builder configured\r\n");
85 }
86
87 /*TIMER Init*/
88 static int init_timer(void)
89 {
90     int status;
91     status= XTmrCtr_Initialize(&Timer, TIMER_BASEADDR);
92     if (status != XST_SUCCESS){
93         xil_printf("Timer Init Failed\r\n");
94         return XST_FAILURE;
95     }
96     XTmrCtr_Reset(&Timer, TIMER_COUNTER_NUMBER);
```

```

97     XlmrCtr_Start(&Timer, TIMER_COUNTER_NUMBER);
98     xil_printf("Timer initialized\r\n");
99     return XST_SUCCESS;
100 }
101
102 /*DMA Configuration*/
103 static int init_dma(void)
104 {
105     XAxiDma_Config *DMACon;
106     xil_printf("Initializing DMA...\r\n");
107     DMACon = XAxiDma_LookupConfig(DMA_BASEADDR);
108     if (DMACon == NULL)
109         return XST_FAILURE;
110     if (XAxiDma_CfgInitialize(&AxiDma, DMACon) != XST_SUCCESS)
111         return XST_FAILURE;
112     XAxiDma_IntrEnable(&AxiDma, XAXIDMA_IRQ_ALL_MASK, XAXIDMA_DMA_TO_DEVICE);
113     XAxiDma_IntrEnable(&AxiDma, XAXIDMA_IRQ_ALL_MASK, XAXIDMA_DEVICE_TO_DMA);
114     xil_printf("DMA initialized\r\n");
115     return XST_SUCCESS;
116 }
117
118 /*Start RX DMA */
119 static int start_rx_dma(u32 length_bytes)
120 {
121     int Status;
122     RxDone = 0;
123     xil_printf("Starting RX DMA...\r\n");
124     Status = XAxiDma_SimpleTransfer(&AxiDma, (UINTPTR)RX_DMA_MEM_ADDR, length_bytes, XAXIDMA_DEVICE_TO_DMA);
125     if (Status != XST_SUCCESS) {
126         xil_printf("RX DMA start failed\r\n");
127         return XST_FAILURE;
128     }
129     return XST_SUCCESS;
130 }
131
132 /*Start TX DMA */
133 static int start_tx_dma(u32 length_bytes)
134 {
135     int Status;
136     TxDone = 0;
137     //xil_printf("Starting TX DMA...\r\n");
138     Status = XAxiDma_SimpleTransfer(&AxiDma, (UINTPTR)TX_DMA_MEM_ADDR, length_bytes, XAXIDMA_DMA_TO_DEVICE);
139     if (Status != XST_SUCCESS) {
140         //xil_printf("TX DMA start failed\r\n");
141         return XST_FAILURE;
142     }
143     return XST_SUCCESS;
144 }
145
146 /*DMA RX Interrupt*/
147 static void dma_s2mm_isr(void *CallBackRef)
148 {
149     u32 IrqStatus;
150     (void)CallBackRef;
151     IrqStatus = XAxiDma_IntrGetIrq(&AxiDma, XAXIDMA_DEVICE_TO_DMA);
152     XAxiDma_IntrAckIrq(&AxiDma, IrqStatus, XAXIDMA_DEVICE_TO_DMA);
153     if (IrqStatus & XAXIDMA_IRQ_IOC_MASK) {
154         RxDone = 1;
155     }
156 }
157
158 /*DMA TX Interrupt*/
159 static void dma_mm2s_isr(void *CallBackRef)
160 {
161     u32 IrqStatus;
162     (void)CallBackRef;
163     IrqStatus = XAxiDma_IntrGetIrq(&AxiDma, XAXIDMA_DMA_TO_DEVICE);
164     XAxiDma_IntrAckIrq(&AxiDma, IrqStatus, XAXIDMA_DMA_TO_DEVICE);
165     if (IrqStatus & XAXIDMA_IRQ_IOC_MASK) {
166         TxDone = 1;
167     }
168 }

```

```

165 }
166
167 /*clear DMA Memory Area*/
168 static void clear_DMAArea(u32 base_addr, u32 length_bytes)
169 {
170     u32 i;
171     for (i = 0; i < length_bytes; i += 4) {
172         Xil_Out32(base_addr + i, 0);
173     }
174 }
175
176
177 /*copy RX Area to TX Area*/
178 static void copy_rx_to_tx(u32 length_bytes)
179 {
180     u32 i;
181     u32 data;
182     for (i = 0; i < length_bytes; i += 4) {
183         data = Xil_In32(RX_DMA_MEM_ADDR + i);
184         Xil_Out32(TX_DMA_MEM_ADDR + i, data);
185     }
186 }
187
188
189 /*Interrupt Controller Init*/
190 static int init_interrupt_controller(void)
191 {
192     xil_printf("Initializing interrupt controller...\r\n");
193     if (XIntc_Initialize(&Intc, INT_C_BASEADDR) != XST_SUCCESS)
194         return XST_FAILURE;
195     if (XIntc_Connect(&Intc, DMA_MM2S_INTR_NUM, dma_mm2s_isr, &AxiDma) != XST_SUCCESS)
196         return XST_FAILURE;
197     if (XIntc_Connect(&Intc, DMA_S2MM_INTR_NUM, dma_s2mm_isr, &AxiDma) != XST_SUCCESS)
198         return XST_FAILURE;
199     if (XIntc_Start(&Intc, XIN_REAL_MODE) != XST_SUCCESS)
200         return XST_FAILURE;
201     XIntc_Enable(&Intc, DMA_MM2S_INTR_NUM);
202     XIntc_Enable(&Intc, DMA_S2MM_INTR_NUM);
203     Xil_ExceptionInit();
204     Xil_ExceptionRegisterHandler(XIL_EXCEPTION_ID_INT, (Xil_ExceptionHandler)XIntc_InterruptHandler, &Intc);
205     Xil_ExceptionEnable();
206     xil_printf("Interrupt controller initialized\r\n");
207     return XST_SUCCESS;
208 }
209

```

```

210 /*Wait for the process to be done*/
211 static int wait_for_rx_done(void)
212 {
213     xil_printf("Waiting for RX packet...\r\n");
214     while (RxDone == 0) {
215     }
216     xil_printf("RX done\r\n");
217     return XST_SUCCESS;
218 }
219
220 static int wait_for_tx_done(void)
221 {
222     //xil_printf("Waiting for TX done...\r\n");
223     while (TxDone == 0) {
224     }
225     //xil_printf("TX done\r\n");
226     return XST_SUCCESS;
227 }
228
229 /*Prep*/
230 static int prepare_receive(void)
231 {
232     clear_DMAArea(RX_DMA_MEM_ADDR, ECHO_LENGTH_BYTES);
233     return start_rx_dma(ECHO_LENGTH_BYTES);
234 }
235
236 static int prepare_echo_transmit(void)
237 {
238     copy_rx_to_tx(ECHO_LENGTH_BYTES);
239     return start_tx_dma(ECHO_LENGTH_BYTES);
240 }
241

```

```

242 /* Build TX Packet
243 * bytes 0-3 : sequence number
244 * bytes 4-7 : timestamp, lower 32 bits
245 * bytes 8- : A-Z pattern
246 */
247 static void build_payload(u32 length_bytes)
248 {
249     u32 i;
250     for (i = HEADER_SIZE_BYTES; i < length_bytes; i++) {
251         Xil_Out8(TX_DMA_MEM_ADDR + i, (u8)('A' + ((i - HEADER_SIZE_BYTES) % 26)));
252     }
253 }
254
255

```

```

256 static void update_packet_header(u32 seq)
257 {
258     u32 timestamp;
259     timestamp = XlMrCtr_GetValue(&Timer, TIMER_COUNTER_NUMBER);
260     Xil_Out32(TX_DMA_MEM_ADDR + 0, seq);
261     Xil_Out32(TX_DMA_MEM_ADDR + 4, (u32)timestamp);
262 }
263
264 /*Send*/
265 static int send_packet(void)
266 {
267     int Status;
268     u32 seq;
269     xil_printf("Start TX\n");
270     build_payload(ECHO_LENGTH_BYTES);
271     for (seq = 0; seq < NUM_TX_PACKETS; seq++) {
272         update_packet_header(seq);
273         Status = start_tx_dma(ECHO_LENGTH_BYTES);
274         if (Status != XST_SUCCESS) {
275             xil_printf("TX DMA failed\n");
276             return XST_FAILURE;
277         }
278         Status = wait_for_tx_done();
279         if (Status != XST_SUCCESS) {
280             xil_printf("TX wait failed\n");
281             return XST_FAILURE;
282         }
283     }
284     xil_printf("Finished TX\n");
285     return XST_SUCCESS;
286 }
287
288 /*MDIO Config*/
289 void mdio_init() {
290     // Bit 6 (0x40) enables the MDIO bus.
291     // Bits [5:0] are the clock divisor.
292     u32 setup_word = 0x40 | 0x13; // Enable = 1, Divisor = 19
293
294     Xil_Out32(MAC_BASEADDR + MDIO_SETUP_REG, setup_word);
295 }
296
297
298 /*Write MDIO*/
299 void mdio_write(u8 phy_addr, u8 reg_addr, u16 data) {
300     // 1. Poll the Ready Bit to ensure the bus is not currently busy
301     while ((Xil_In32(MAC_BASEADDR + MDIO_READ_REG) & MDIO_READY_BIT) == 0);
302     // 2. Put the payload into the Write Data Register
303     Xil_Out32(MAC_BASEADDR + MDIO_WRITE_REG, data);
304     // 3. Build the Control Word
305     // Bit 11: Initiate transaction (1)
306     // Bits [15:14]: OP Code (01 = Write) -> (1 << 14)
307     // Bits [20:16]: Register Address
308     // Bits [28:24]: PHY Address
309     u32 ctrl_word = (phy_addr << 24) | (reg_addr << 16) | (1 << 14) | (1 << 11);
310     // 4. Write to Control Register to trigger the hardware transaction
311     Xil_Out32(MAC_BASEADDR + MDIO_CTRL_REG, ctrl_word);
312     // 5. Poll the Ready Bit again to wait for the write to physically finish
313     while ((Xil_In32(MAC_BASEADDR + MDIO_READ_REG) & MDIO_READY_BIT) == 0);
314 }
315
316 /*Read MDIO*/
317 u16 mdio_read(u8 phy_addr, u8 reg_addr) {
318     // 1. Poll the Ready Bit to ensure the bus is not busy
319     while ((Xil_In32(MAC_BASEADDR + MDIO_READ_REG) & MDIO_READY_BIT) == 0);
320
321     // 2. Build the Control Word
322     // Bit 11: Initiate transaction (1)
323     // Bits [15:14]: OP Code (10 = Read) -> (2 << 14)
324     // Bits [20:16]: Register Address
325     // Bits [28:24]: PHY Address
326     u32 ctrl_word = (phy_addr << 24) | (reg_addr << 16) | (2 << 14) | (1 << 11);
327
328     // 3. Write to Control Register to trigger the hardware transaction
329     Xil_Out32(MAC_BASEADDR + MDIO_CTRL_REG, ctrl_word);
330
331     // 4. Poll the Ready Bit to wait for the PHY to respond
332     while ((Xil_In32(MAC_BASEADDR + MDIO_READ_REG) & MDIO_READY_BIT) == 0);
333
334     // 5. Extract and return the lower 16 bits of the Read Data register
335     return (u16)(Xil_In32(MAC_BASEADDR + MDIO_READ_REG) & 0xFFFF);
336 }
337

```

```

338 /*Main*/
339 int main(void)
340 {
341     int Status;
342     u8 my_phy_addr = 0x03; // Verify your PHY strapping
343     /*init PHY*/
344     // Initialize the MDIO bus
345     mdio_init();
346     mdio_write(my_phy_addr, 0x00d3, 0x0000); // SGMII Clock enable - 0x4000 for 6 wire mode. 0x0000 for 4 wire mode.
347     mdio_write(my_phy_addr, 0x0000, 0x1140); // Control register 0 for 1 G use 0x1140
348     mdio_write(my_phy_addr, 0x0010, 0x0800); // SGMII enable
349     mdio_write(my_phy_addr, 0x0014, 0x2bc0); // Configuration register 2
350     mdio_write(my_phy_addr, 0x0032, 0x0050); // RGMII disable
351     xil_printf("reg 0 value is: %x\n",mdio_read(my_phy_addr, 0x0000));
352     while((mdio_read(my_phy_addr, 0x05) & 0x4000) != 0x4000){
353         xil_printf("Link is DOWN!\n");
354         usleep(100000);
355     }
356     xil_printf("Link is UP!\n");
357
358
359     xil_printf("Start UDP Protocole...\n");
360     Status = init_dma();
361     if (Status != XST_SUCCESS) {
362         return XST_FAILURE;
363     }
364     Status = init_interrupt_controller();
365     if (Status != XST_SUCCESS) {
366         return XST_FAILURE;
367     }
368     Status=init_timer();
369     if (Status != XST_SUCCESS) {
370         return XST_FAILURE;
371     }
372     config_uoe();
373     config_tuser_builder(ECHO_LENGTH_BYTES);
374     // For TX only
375     Status = send_packet();
376     if (Status != XST_SUCCESS) {
377         return XST_FAILURE;
378     }

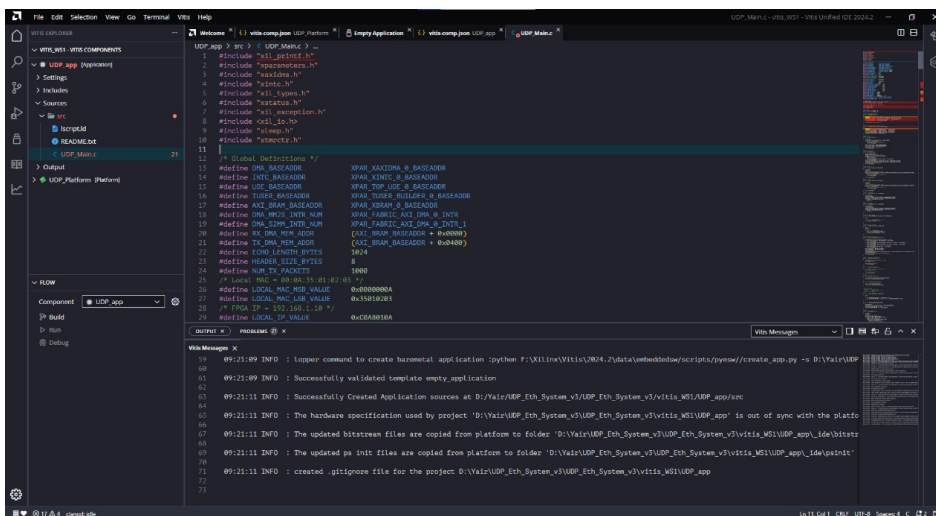
```

```

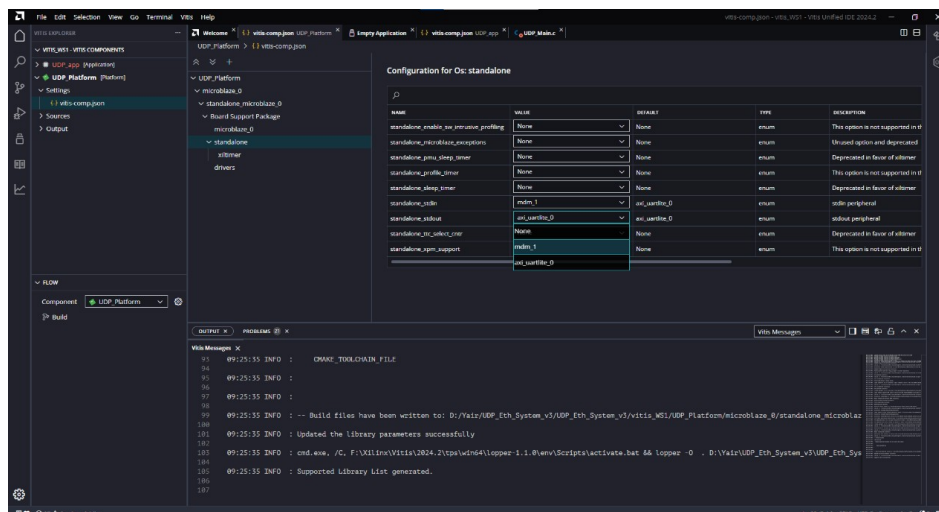
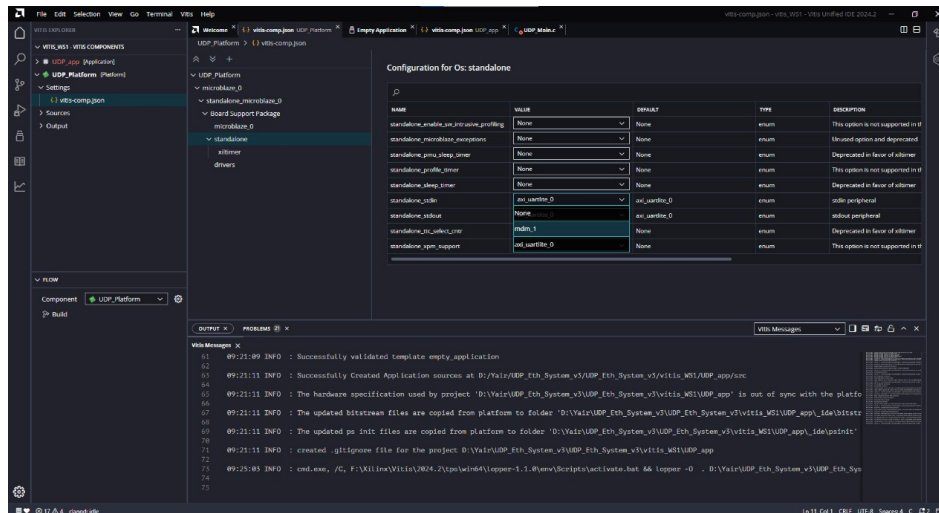
379 //For LoopBack
380
381 while(1){
382
383     Status = prepare_receive();
384     if (Status != XST_SUCCESS) {
385         return XST_FAILURE;
386     }
387     Status = wait_for_rx_done();
388     if (Status != XST_SUCCESS) {
389         return XST_FAILURE;
390     }
391
392     Status = prepare_echo_transmit();
393     if (Status != XST_SUCCESS) {
394         return XST_FAILURE;
395     }
396
397     Status = wait_for_tx_done();
398     if (Status != XST_SUCCESS) {
399         return XST_FAILURE;
400     }
401
402     xil_printf("END UDP Protocole...\n");
403 }
404
405 while(1){
406 }
407 return 0;
408 }

```

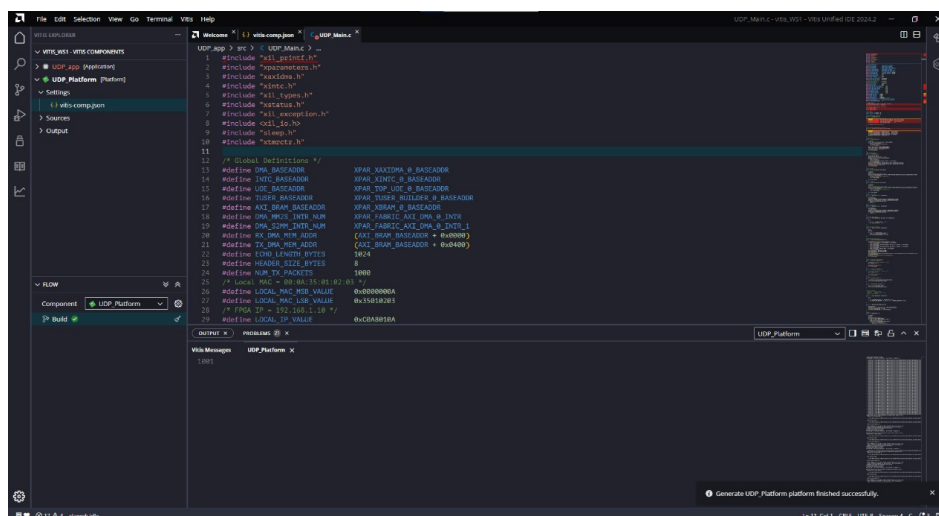
נשמור את הקובץ.



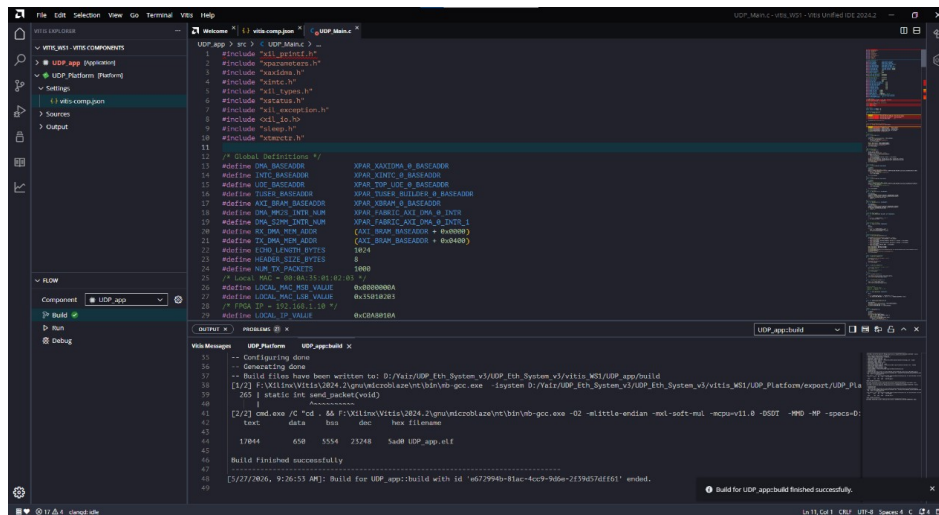
על מנת לאפשר למעבד לשלוח הדפסות למסך דרך כבל הצריבה, ניגש להגדרות ה-Board Support Package. נשנה את ערוצי הקלט והפלט stdout ו-stdin כך שיעבדו מול רכיב mdm_1.



נלחץ על כפתור Build לקימפול הפלטפורמה ונמתין לקבלת הודעת ההצלחה.



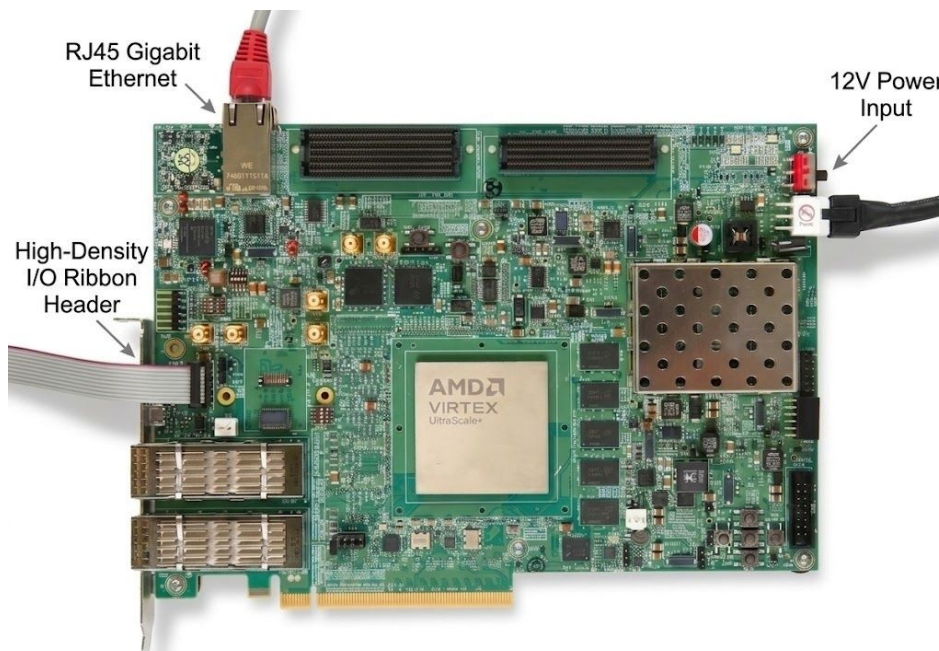
נלחץ על כפתור Build לאפליקציה. נוודא כי מתקבלת הודעת Build Finished successfully המאשרת שקובץ הריצה מוכן לצריבה.



שלב 24:

כדי לאפשר תקשורת ישירה בין המחשב ללוח הפיתוח, נדרש חיבור פיזי והגדרת כתובת רשת סטטית במחשב.

תחילה, נחבר את לוח ה- 118VCU פיזית לסביבת העבודה.

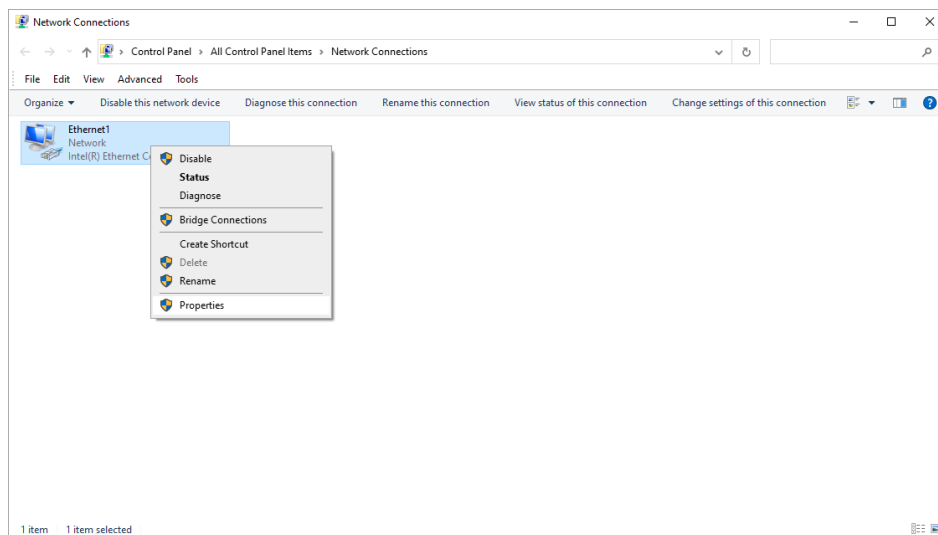


נוכל לראות כי ספק הכוח מחובר בכבל שחור ומזין V12 ללוח, כבל ה- JTAG האפור מחובר לצורך צריבת המערכת וניטור האותות וכבל הרשת האדום מחובר ישירות מיציאת ה- RJ45 בלוח אל כרטיס הרשת במחשב הבדיקה ליצירת ערוץ תקשורת ישיר.

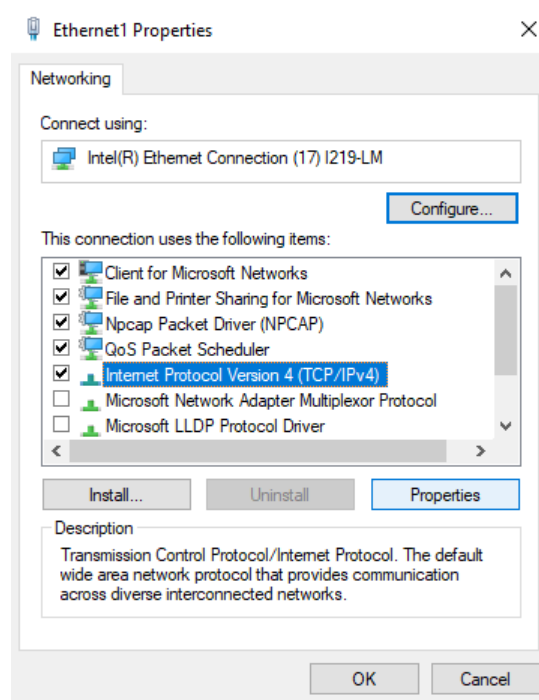
ניגש להגדרות הרשת במערכת ההפעלה של המחשב. נפתח את לוח הבקרה וניכנס לניהול חיבורי הרשת. נאתר את כרטיס הרשת המחובר ללוח.



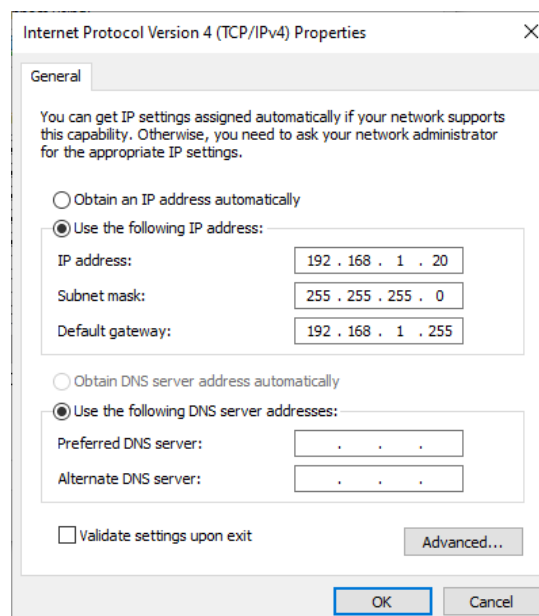
נלחץ עליו קליק ימני ונבחר במאפיינים.



בחלון המאפיינים, נסמן את פרוטוקול האינטרנט גרסה 4 ונלחץ על מאפיינים.



נגדיר את כרטיס הרשת לעבודה עם כתובת רשת סטטית. זיין את הכתובת 192.168.1.20 ואת מסיכת הרשת 255.255.255.0 כדי להימצא באותה רשת משנה שהוגדרה בקוד התוכנה של המעבד. נשמור את ההגדרות.



שלב 25:

טרם המעבר לבדיקות העומס והשיהוי באמצעות תוכנת הבדיקה הייעודית שסופקה על ידי מנחה הפרויקט, אייל (ה-Tester), ביצענו שלב אימות מקדים. תוכנת ה-Tester יעילה מאוד למבחני מאמץ, אך היא מייצרת מטעני נתונים (Payloads) אקראיים, עובדה המקשה על מעקב ויזואלי והפרדה ברורה בין כותרות הפרוטוקול לבין המידע עצמו.

לכן, על מנת לאמת את תקינות פענוח החבילות במנוע ה-UDP בצורה שקופה, יצרנו מטען נתונים מותאם אישית הכולל 1024 בתים של התו 'B' (שערכו ההקסדצימלי '0x42'). שידור

תבנית נתונים צפויה ורציפה זו דרך סקריפט Python, אפשר לנו לזהות בבירור בצגי הניטור את נקודת התפר המדויקת שבין כותרות התקשורת לבין המידע נטו.

לשם ביצוע הבדיקה הממוקדת, כתבנו סקריפט המייצר מנת נתונים בגודל המקסימלי המוגדר – 1024 בתים. כפי שמופיע בקוד, המטען מורכב משכפול של התו 'B'. החבילה משודרת אל פורט 5001 בלוח ה-FPGA תוך פתיחת Socket ישיר, מודדת את זמני היציאה והחזרה, ומוודאת שהנתיב פתוח.

 PC_Code.py - C:\Users\user\Desktop\PC_Code.py (3.7.6)

File Edit Format Run Options Window Help

```
import socket
import time

sock= socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
fpga_ip="192.168.1.10"
port=5001
payload= b'B' *1024
start= time.time()
sock.sendto(payload, (fpga_ip,port))
sock.recvfrom(1024)
end=time.time()
rtt= (end-start) *1000
print("latency=" , rtt/2)
```

לכידת התעבורה בתוכנת Wireshark מספקת את ההוכחה הוויזואלית להצלחת המנגנון.

Info	Length	Protocol	Destination	Time	Source	No.
Domain/Workgroup Announcement WORKGROUP, NT Workstation, Domain Enum 258		BROWSER	192.168.1.255	0.000000000	192.168.1.20 1	
Len=1024 5001 → 49857 1066		UDP	192.168.1.10	6.655138000	192.168.1.20 2	
		UDP	192.168.1.20	6.655548000	192.168.1.10 3	
Who has 192.168.1.10? Tell 192.168.1.20 42		ARP	Xilinx_01:02:03	11.369446000	HewlettP_f9:0d:7	4
is at 00:0a:35:01:02:03 192.168.1.10 60		ARP	HewlettP_f9:0d:9f	11.369642000	Xilinx_01:02:03 5	
Who has 192.168.1.254? Tell 192.168.1.20 42		ARP	Broadcast	21.022704000	HewlettP_f9:0d:6	6
Who has 192.168.1.254? Tell 192.168.1.20 42		ARP	Broadcast	21.873188000	HewlettP_f9:0d:7	
Who has 192.168.1.254? Tell 192.168.1.20 42		ARP	Broadcast	22.875169000	HewlettP_f9:0d:8	

בניתוח חבילת ה-UDP החוזרת מה-FPGA, ניתן לראות בבירור את החלוקה: היכן מסיימות כותרות התקשורת הסטנדרטיות, והיכן מתחיל המידע הנקי. תצוגת הנתונים הגולמית (Hex Dump) מאשרת כי הזיכרון ורכיבי ה-UOE עיבדו והחזירו את המידע ללא שגיאות או עיוותים – כל 1024 הבתים נשמרו כערך '0x42' (המוצגים כאותיות 'BBBBBBBB'), מה שמוכיח את שלמות נתיב הנתונים טרם המעבר לתוכנת המנחה.

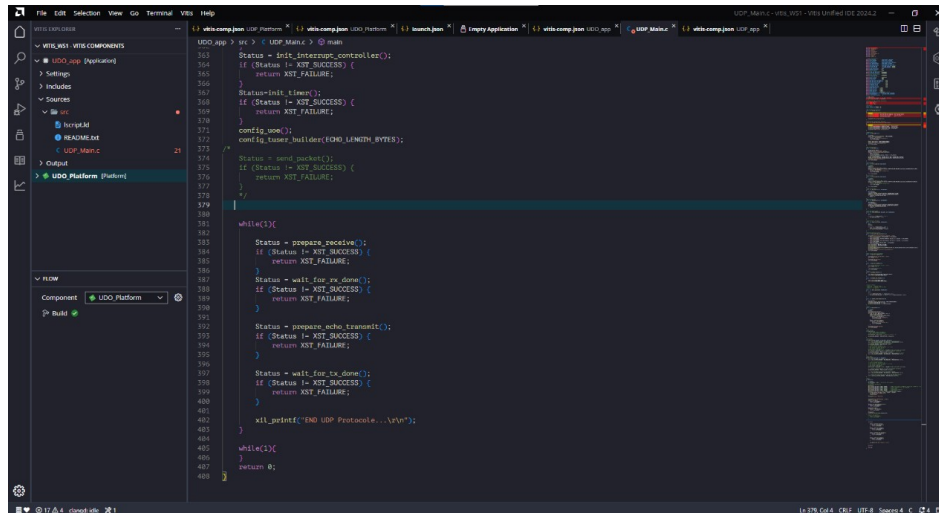
Frame 2: 1066 bytes on wire (8528 bits), 1066 bytes captured (8528 bits) on interface 0 <
(Ethernet II, Src: HewlettP_f9:0d:9f (ec:9a:74:f9:0d:9f), Dst: Xilinx_01:02:03 (00:0a:35:01:02:03) <
Internet Protocol Version 4, Src: 192.168.1.20, Dst: 192.168.1.10 <
User Datagram Protocol, Src Port: 49857, Dst Port: 5001 <
(Data (1024 bytes) <

0000	00 0a 35 01 02 03 ec 9a 74 f9 0d 9f 08 00 45 00	--5-----t-----E-
0010	04 1c 49 ce 00 00 80 11 00 00 c0 a8 01 14 c0 a8	--I-----
0020	01 0a c2 c1 13 89 04 08 87 88 42 42 42 42 42 42	-----BBBBBB
0030	42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42	BBBBBBBBBBBBBBBB
0040	42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42	BBBBBBBBBBBBBBBB
0050	42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42	BBBBBBBBBBBBBBBB
0060	42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42	BBBBBBBBBBBBBBBB
0070	42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42	BBBBBBBBBBBBBBBB
0080	42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42	BBBBBBBBBBBBBBBB
0090	42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42	BBBBBBBBBBBBBBBB
00a0	42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42	BBBBBBBBBBBBBBBB
00b0	42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42	BBBBBBBBBBBBBBBB
00c0	42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42	BBBBBBBBBBBBBBBB
00d0	42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42	BBBBBBBBBBBBBBBB
00e0	42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42	BBBBBBBBBBBBBBBB

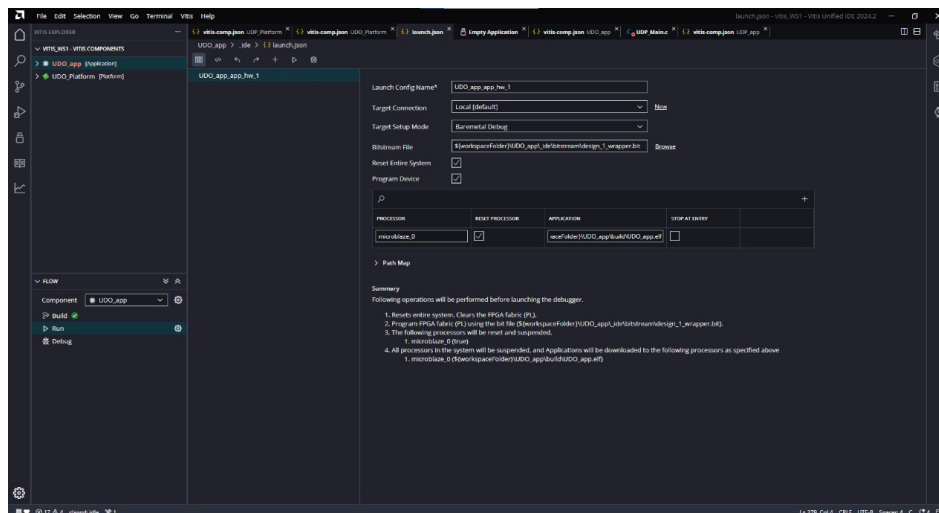
שלב 26:

תחילה נבצע את בדיקות העומס הראשוניות עם חוצץ נתונים BRAM בנפח של 8 KB שזהו ערך ברירת המחדל.

נבחן את תצורת ה-Loopback. לשם כך, ננטרל זמנית את פונקציית השידור היזום על ידי סימונה בקוד המקור, ובבצע קימפול מחדש (Build) לאפליקציה.



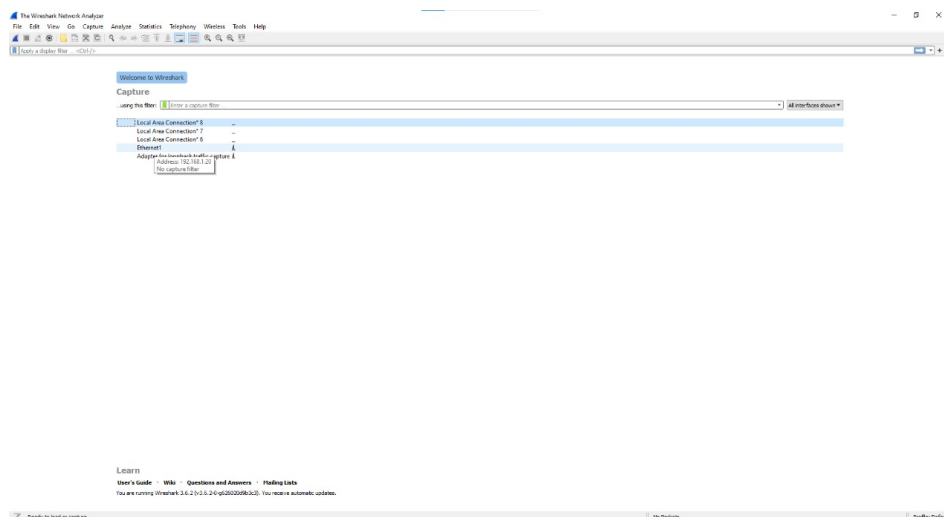
כעת נעבור להרצת התוכנה העדכנית על גביי הלוו. נפתח את חלון תצורת ההרצה (Run Configuration) בסביבת Vitis. תחת לשונית הגדרות היעד (Target Setup), חובה לוודא כי מסומנת בוי (V) האפשרות Program FPGA. סימון זה הכרחי על מנת להבטיח שמערכת Vitis תצורב תחילה את קובץ החומרה המעודכן (Bitstream) אל תוך השבב, ורק לאחר מכן תטען אליו את קוד התוכנה של המעבד. פעולה זו מבטיחה סנכרון מוחלט בין שכבת החומרה לשכבת התוכנה טרם תחילת בדיקות התקשורת.



נחכה לצריבה, במצב הזה נראה שהנורה הכחולה כבתה לכמה שניות ולאחר מכן נדלקה מחדש..

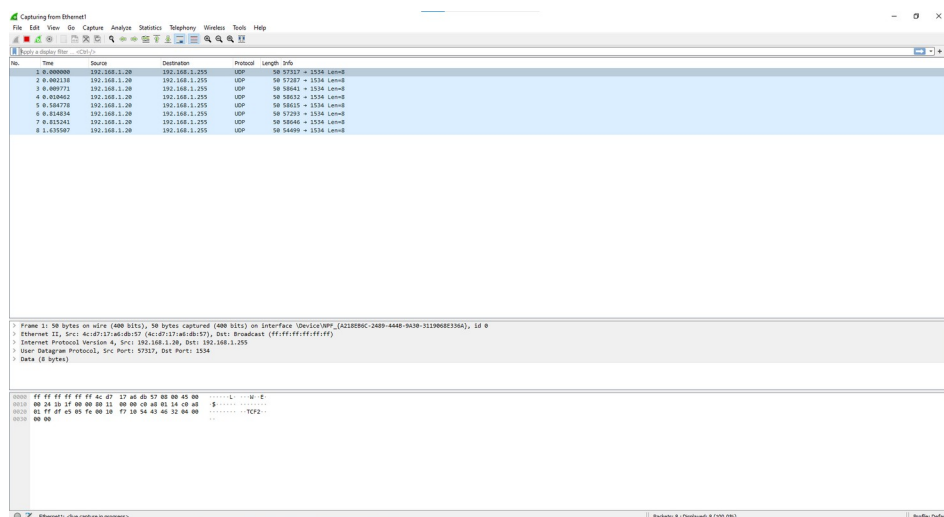
נפתח את הווירשארק ונלחץ על אטרנט:

כדי לאמת באופן בלתי תלוי את תקינות חבילות ה-UDP המועברות, השתמשנו בתוכנת הניטור Wireshark. בחרנו בממשק הרשת המחובר פיזית ללוח הפיתוח (במקרה זה, Ethernet 1), פעולה המאפשרת ללכוד בזמן אמת את כל חבילות המידע הנכנסות והיוצאות מהמחשב אל ה-FPGA. הגדרה זו היא קריטית כדי לוודא שהפרוטוקול המיושם על הלוח עומד בתקני הרשת וכי אין אובדן נתונים בדרך (Packet Loss).

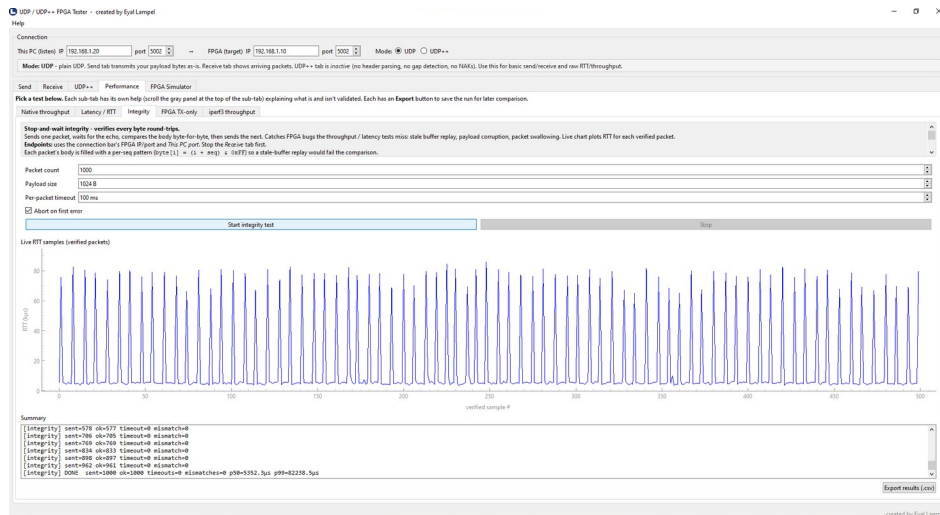


נזהה כי האפפיגיא"י נמצא במצב האזנה:

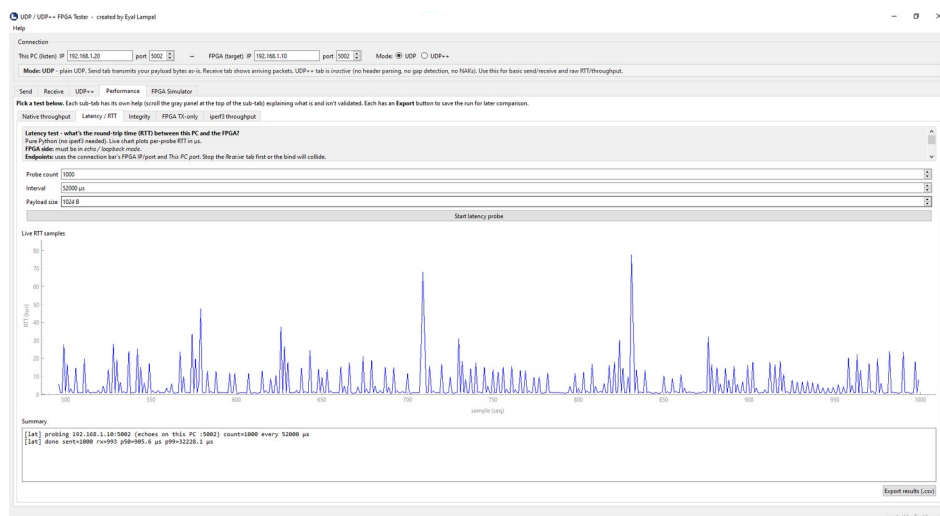
ניתן להבחין בבירור ברצף החבילות הנשלחות (Data Packets) מהמחשב אל כתובת ה-IP המוגדרת של ה-FPGA. לכל חבילה מוצגים המאפיינים הקריטיים: אורך המטען (Payload Length), כתובות המקור והיעד, וזמני ההגעה המדויקים. רצף זה מאשר כי המערכת משדרת את החבילות בקצב יציב ומתוזמן, וכי אין שיבושים או השמטות בתקשורת הדו-כיוונית.



לטובת אימות אמינות ערוץ התקשורת, ביצענו סדרת בדיקות מאמץ באמצעות סביבת ה-Tester המקומית. תחילה, הרצנו בדיקת Integrity Test בתצורת Loopback, כאשר הגדרנו גודל חבילה (Payload) של 1024 בתים. בדיקה זו מוודאת כי הנתונים הנשלחים מהמחשב אל ה-FPGA וחוזרים ממנו, נשמרים ללא שינוי או שגיאות בייט (Bit Mismatch). הגרף המצורף מתעד תקינות ההעברה לאורך רצף דגימות, המעיד על זרימת נתונים יציבה.



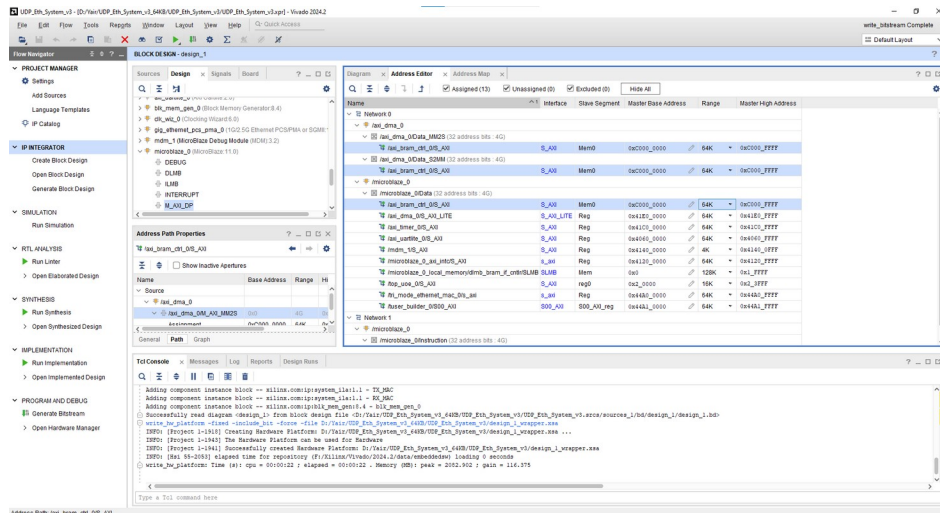
כשלב המשך להערכת ביצועי זמן אמת, עדכנו את פרמטרי הבדיקה: שינינו את מרווח הזמן (Interval) ל- 52,000 מיקרו-שניות ואת גודל המטען ל- 1024 בתים, לצורך ביצוע מדידת Latency.



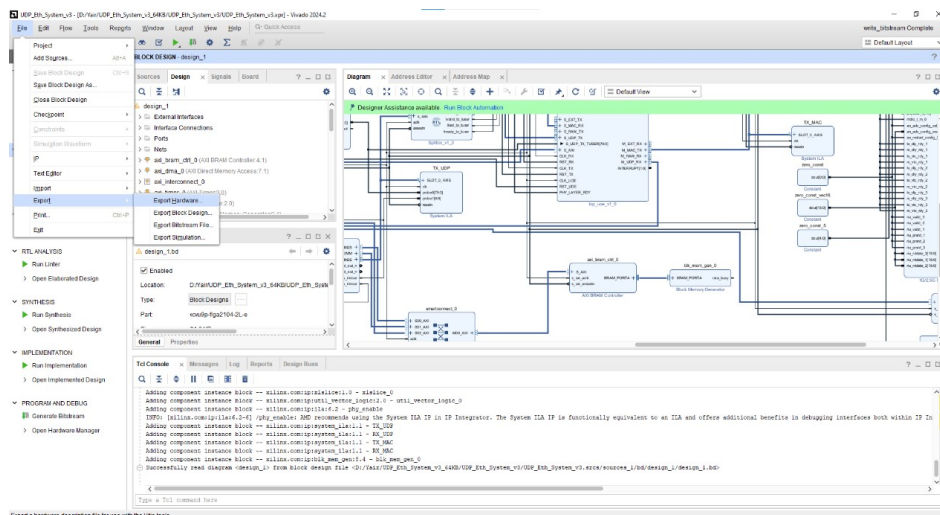
נוכל לראות כי בבדיקה זו, המערכת תחת תצורת ה- 8KB המקורית הציגה מגבלת ביצועים: מתוך רצף של 1000 חבילות שנשלחו, חזרו 993 חבילות בלבד. אובדן זה של 7 חבילות נובע מכך שנפח החוצץ 8KB היה קטן מדי, מה שגרם לעומס תעבורה שחרג מיכולת הריקון של המעבד והוביל לנפילת חבילות (Packet Loss). תוצאה זו מהווה את הבסיס ההנדסי לצורך באופטימיזציית נפח הזיכרון הזיכרון שבוצעה בשלבים הבאים.

על מנת למצוא את נפח הזיכרון האופטימלי שיבטיח אמינות תעבורה מושלמת ללא בזבז משאבי חומרה יקרים בשבב, בחנו באופן שיטתי שלושה נפחי זיכרון שונים: KB16, KB32, ו- KB 64.

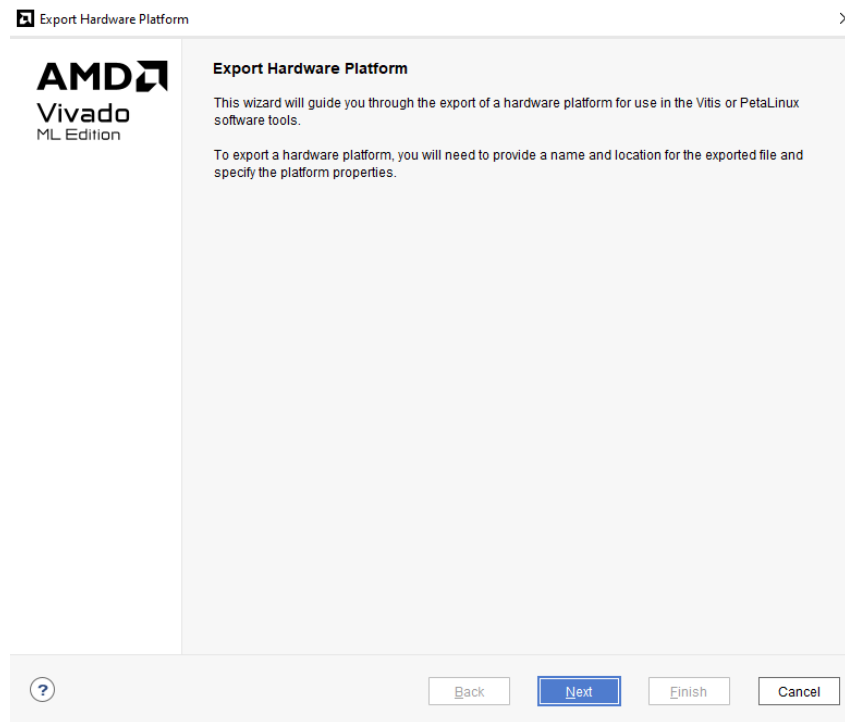
עבור כל אחד מגדלי הזיכרון הנבחרים חזרנו לשרטוט החומרה המרכזי. דרך חלון Address Editor עדכנו את טווח הכתובות של רכיב axi_bram_ctrl. להלן תמונת חלון ה- Address Editor המציגה את שינוי טווח הכתובות ל- 64 KB.



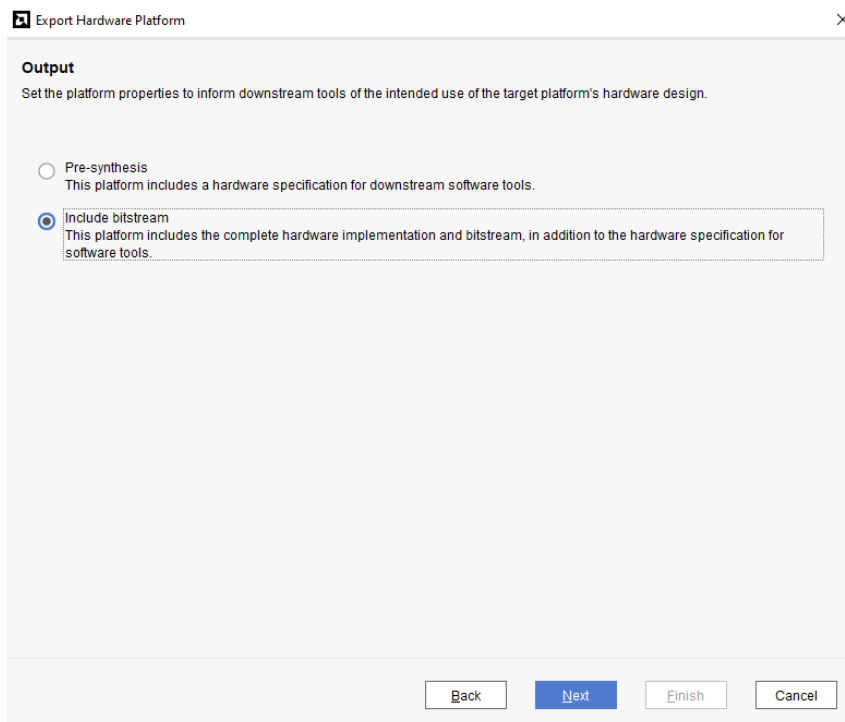
נפתח את תפריט ה- File ונבחר באפשרות Export Hardware.



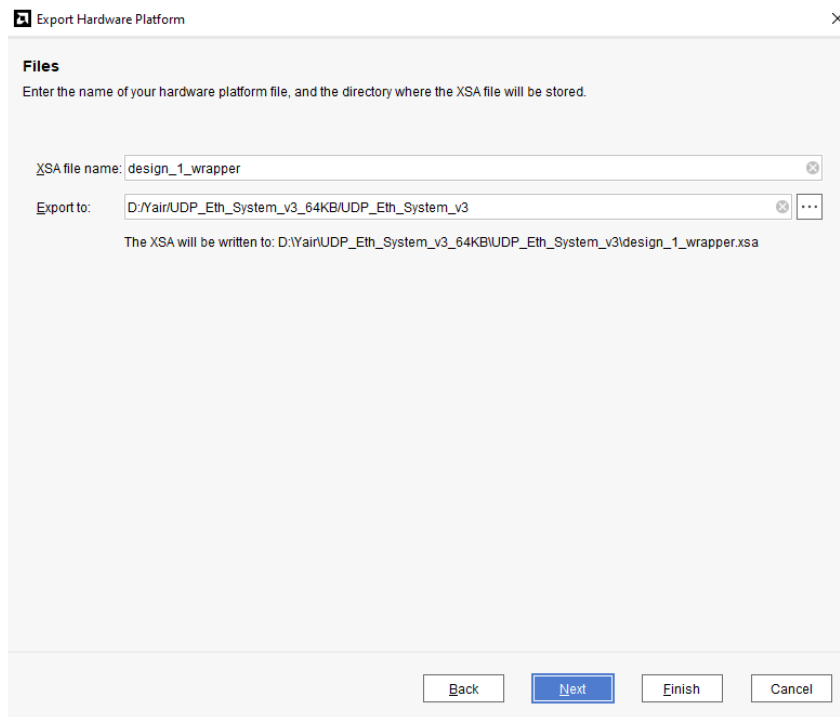
נלחץ Next.



בחלון אשף הייצוא נסמן אפשרות Include bitstream.



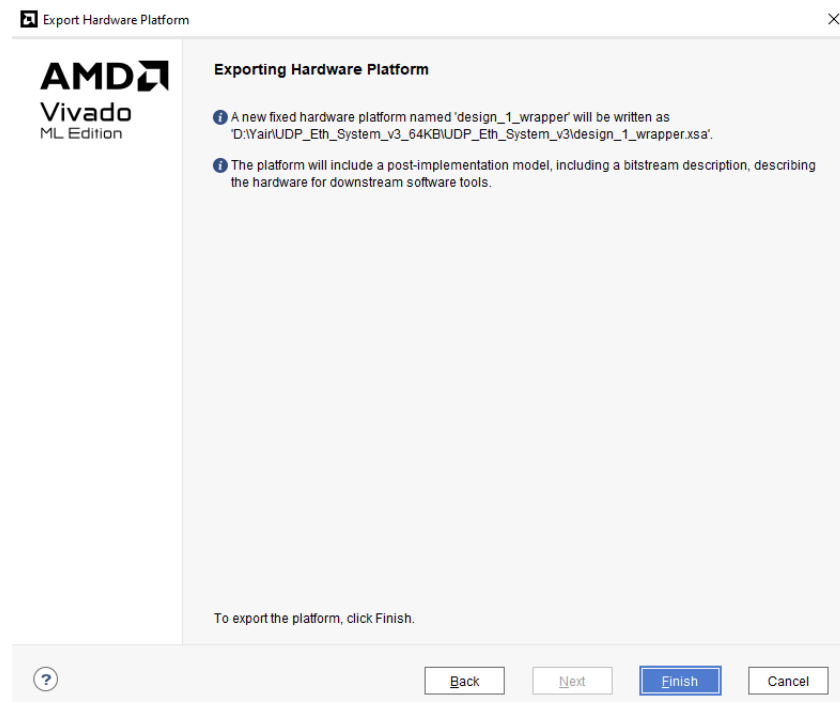
נניצא את קובץ design_1_wrapper.xsa לתיקייה הייעודית.



The dialog box is titled "Export Hardware Platform" and contains the following elements:

- Files** section with the instruction: "Enter the name of your hardware platform file, and the directory where the XSA file will be stored."
- XSA file name:** A text field containing "design_1_wrapper".
- Export to:** A text field containing "D:\Yair\UDP_Eth_System_v3_64KB\UDP_Eth_System_v3".
- A confirmation line: "The XSA will be written to: D:\Yair\UDP_Eth_System_v3_64KB\UDP_Eth_System_v3\design_1_wrapper.xsa".
- Navigation buttons at the bottom: "Back", "Next" (highlighted in blue), "Finish", and "Cancel".

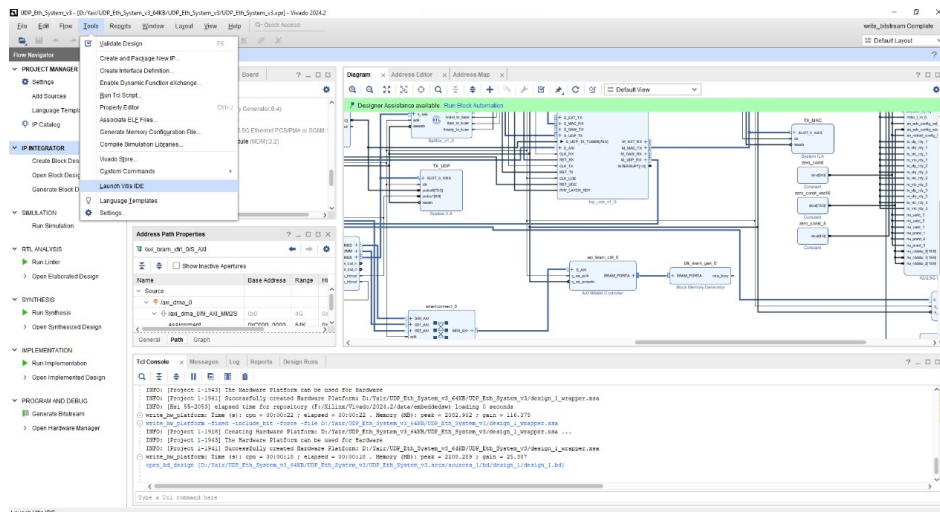
נלחץ Finish.



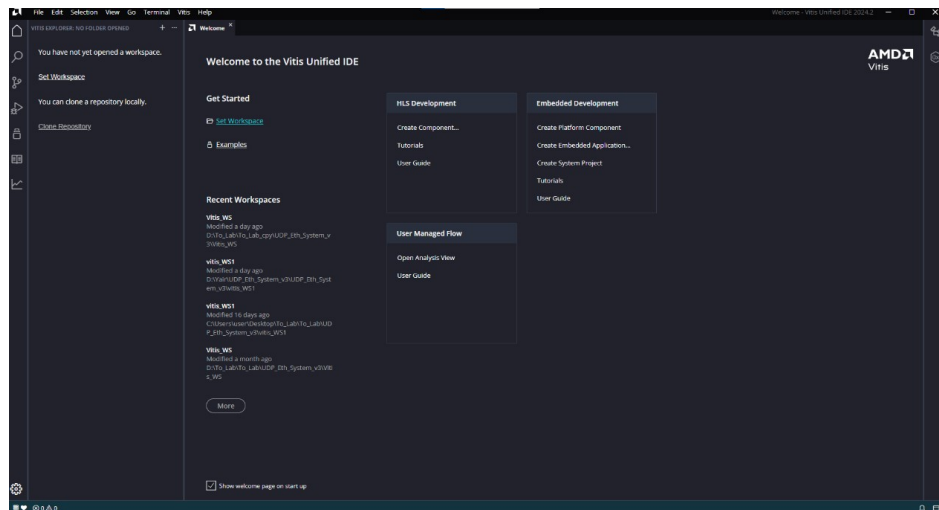
The dialog box is titled "Exporting Hardware Platform" and contains the following elements:

- AMD Vivado ML Edition** logo on the left.
- Exporting Hardware Platform** section with two informational messages:
 - "A new fixed hardware platform named 'design_1_wrapper' will be written as 'D:\Yair\UDP_Eth_System_v3_64KB\UDP_Eth_System_v3\design_1_wrapper.xsa'."
 - "The platform will include a post-implementation model, including a bitstream description, describing the hardware for downstream software tools."
- A note at the bottom: "To export the platform, click Finish."
- Navigation buttons at the bottom: "Back", "Next", "Finish" (highlighted in blue), and "Cancel".

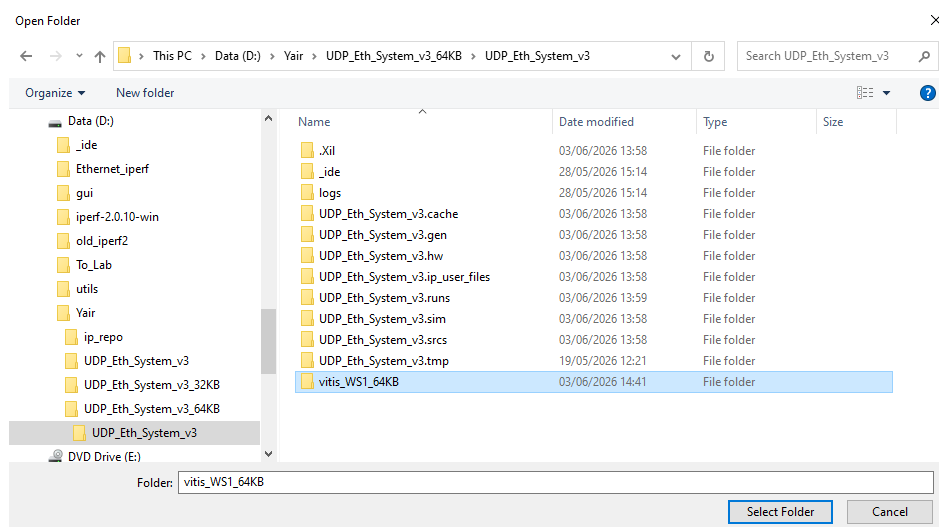
נלחץ על תפריט Tools בתוכנת Vivado ובחר באפשרות Launch Vitis IDE.



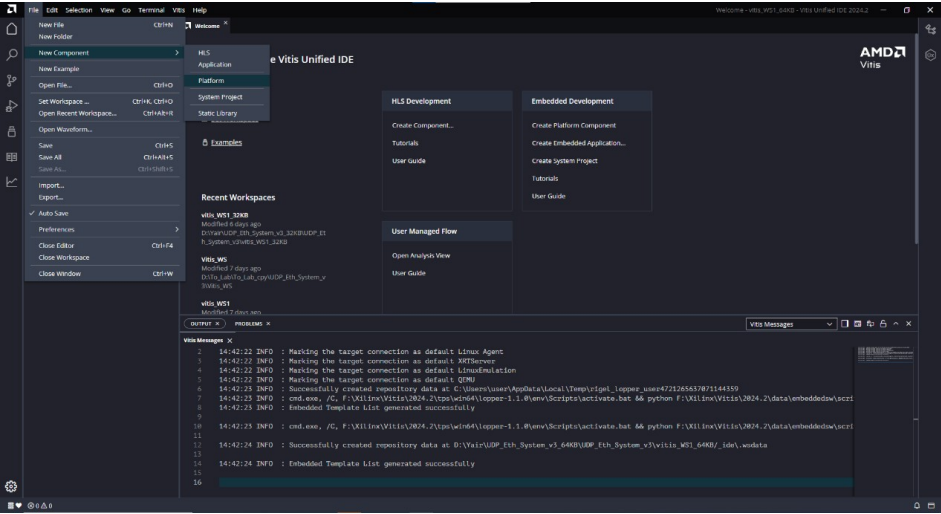
נלחץ על Set Workspace.



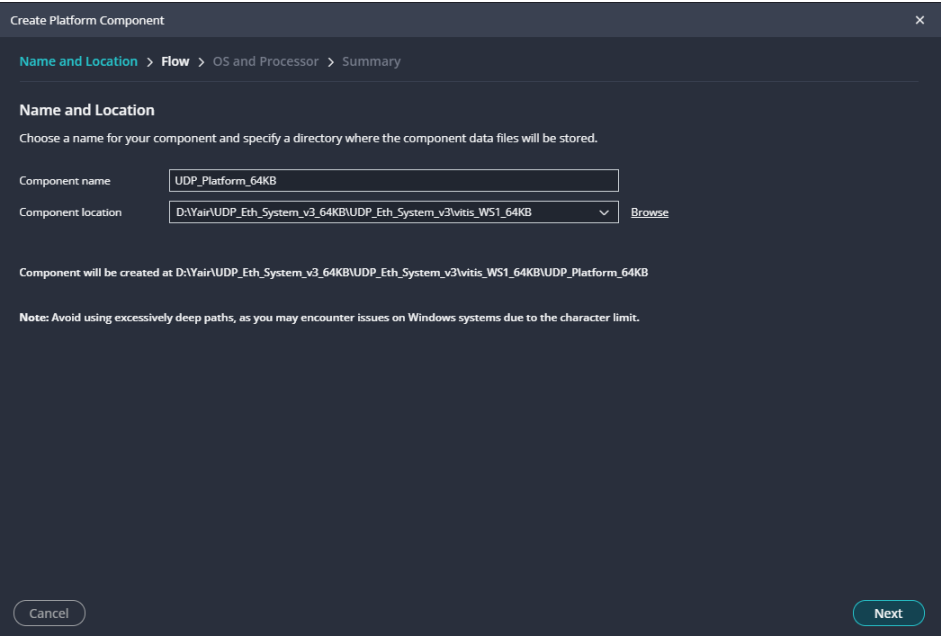
בחלון שנפתח נבחר את התיקייה vitis_WS1_64KB כסביבת העבודה המרכזית.



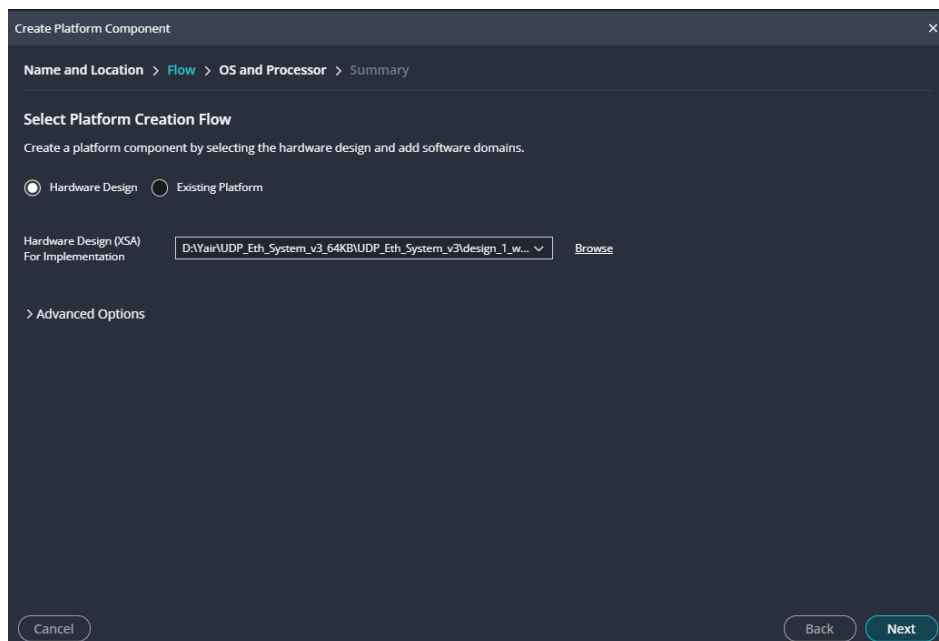
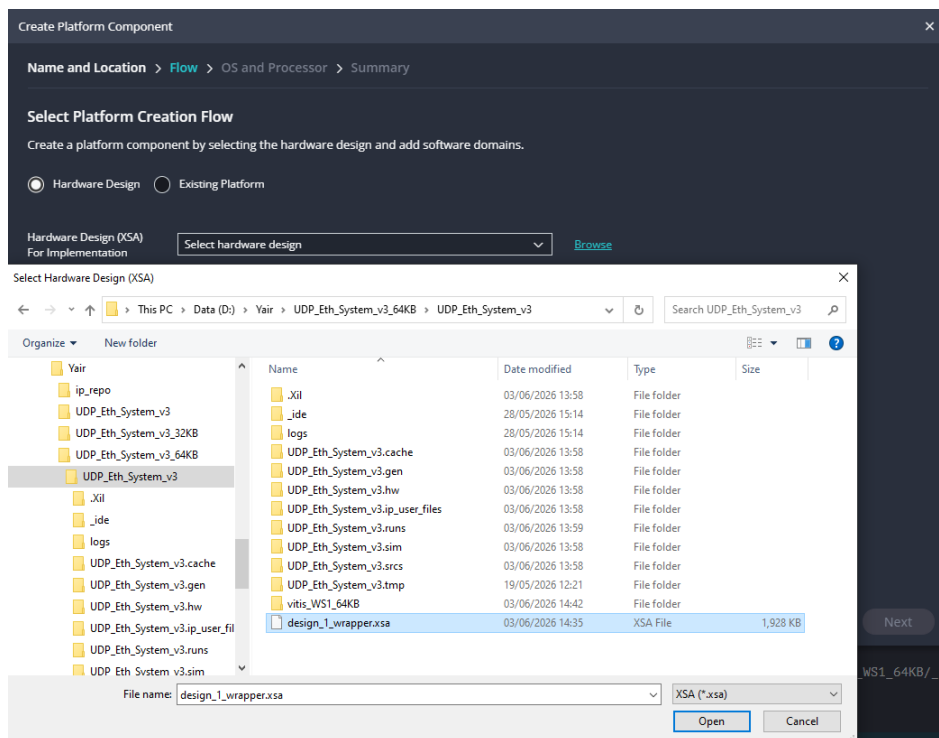
כעת ניצור את שכבת התשתית המקשרת בין החומרה לתוכנה. נבחר ביצירת רכיב פלטפורמה חדש.



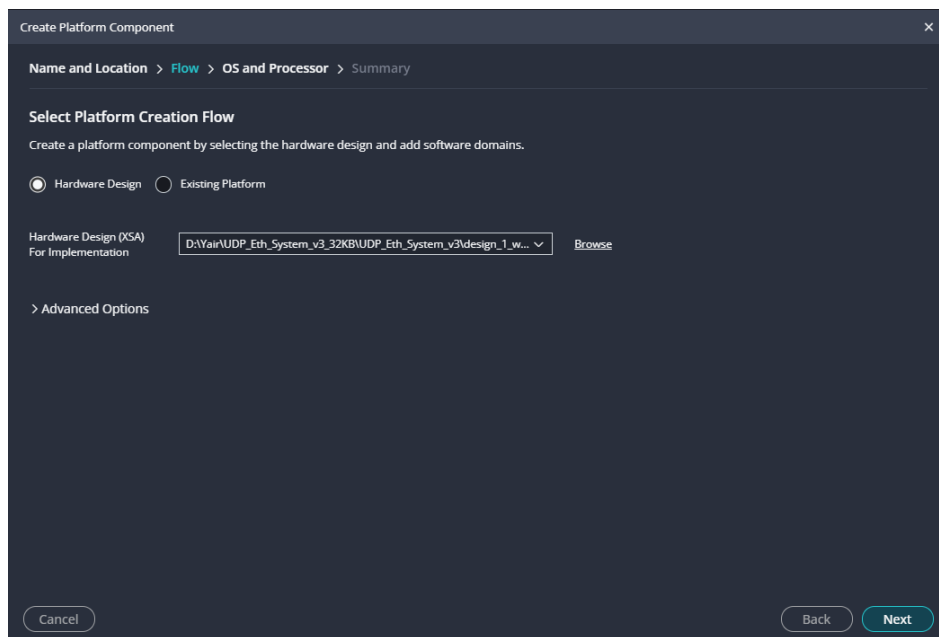
נעניק לו את השם UDP_Platform_64KB.



נטען את קובץ ה-XSA שייצאנו בשלב הקודם design_1_wrapper.xsa.

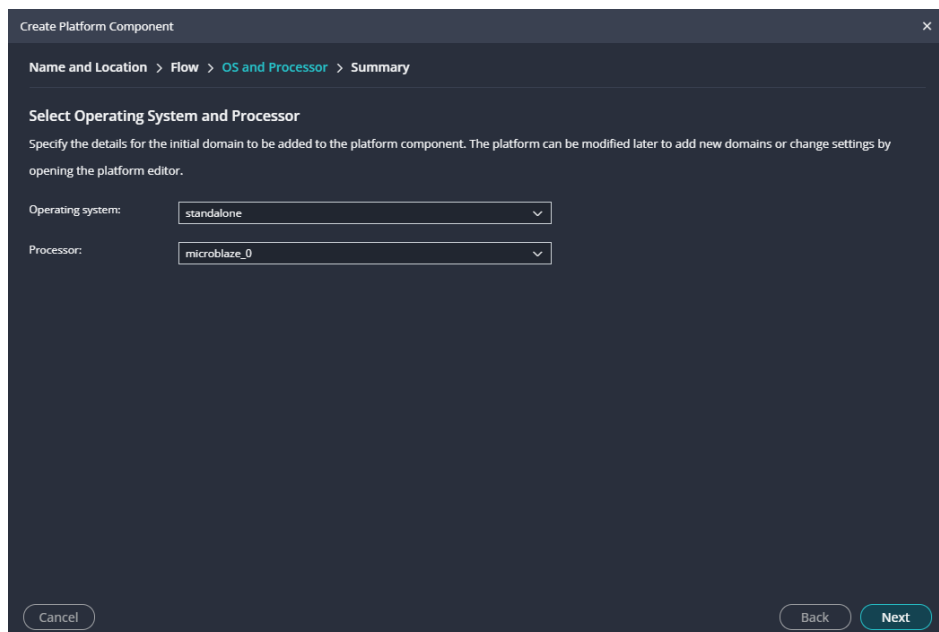


נגדיר את מערכת ההפעלה כ-standalone ואת המעבד כ-microblaze_0.



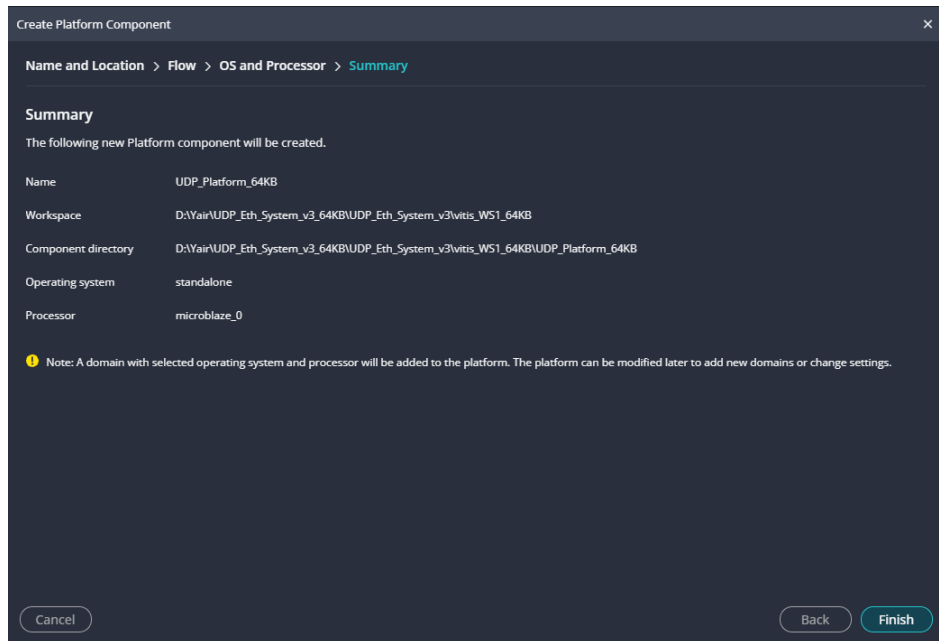
The screenshot shows the 'Create Platform Component' dialog box with the 'Flow' tab selected. The breadcrumb navigation is 'Name and Location > Flow > OS and Processor > Summary'. The section 'Select Platform Creation Flow' contains two radio buttons: 'Hardware Design' (selected) and 'Existing Platform'. Below this, the 'Hardware Design (XSA) For Implementation' section shows a dropdown menu with the path 'D:\Yair\UDP_Eth_System_v3_32KB\UDP_Eth_System_v3\design_1_w...' and a 'Browse' button. An '> Advanced Options' link is also present. At the bottom, there are 'Cancel', 'Back', and 'Next' buttons.

נלחץ Next.

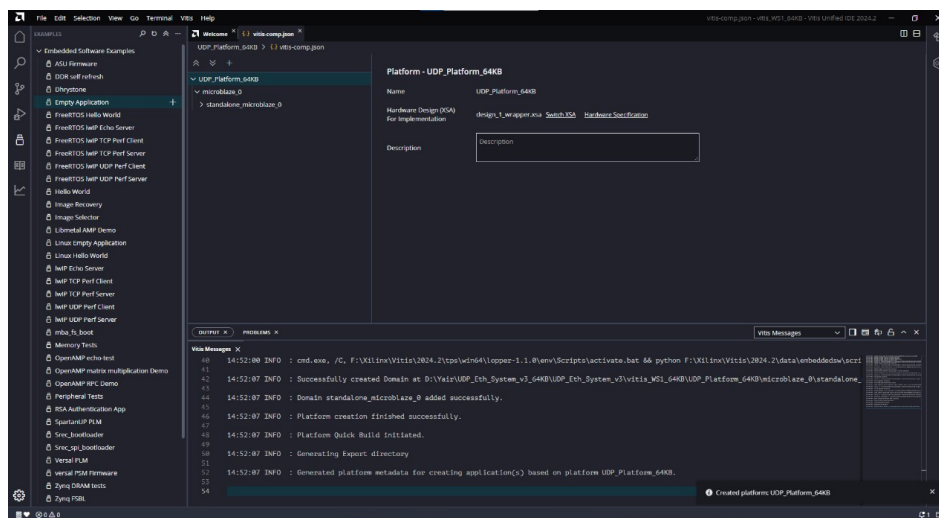


The screenshot shows the 'Create Platform Component' dialog box with the 'OS and Processor' tab selected. The breadcrumb navigation is 'Name and Location > Flow > OS and Processor > Summary'. The section 'Select Operating System and Processor' contains instructions: 'Specify the details for the initial domain to be added to the platform component. The platform can be modified later to add new domains or change settings by opening the platform editor.' Below this, the 'Operating system:' dropdown is set to 'standalone' and the 'Processor:' dropdown is set to 'microblaze_0'. At the bottom, there are 'Cancel', 'Back', and 'Next' buttons.

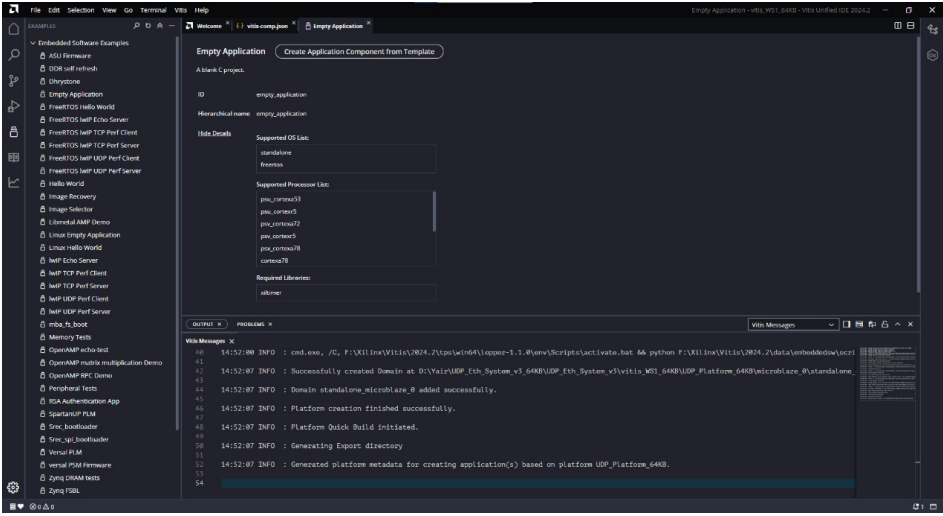
ולסיום נלחץ Finish.



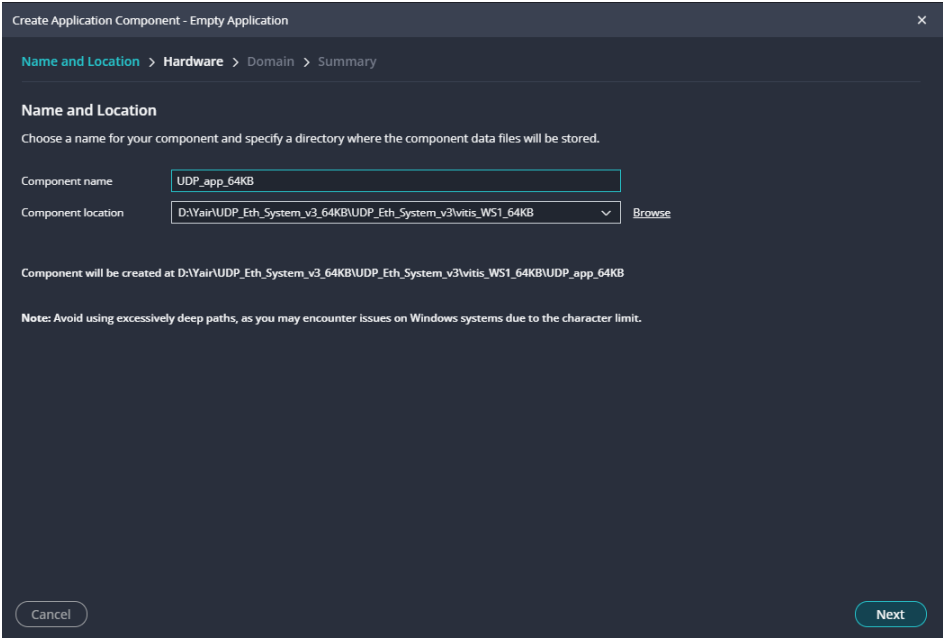
לאחר שהפלטפורמה מוכנה, ניצור את שכבת היישום. נפתח פרויקט אפליקציה חדש ונבחר בתבנית פרויקט C ריק בשם Empty Application.



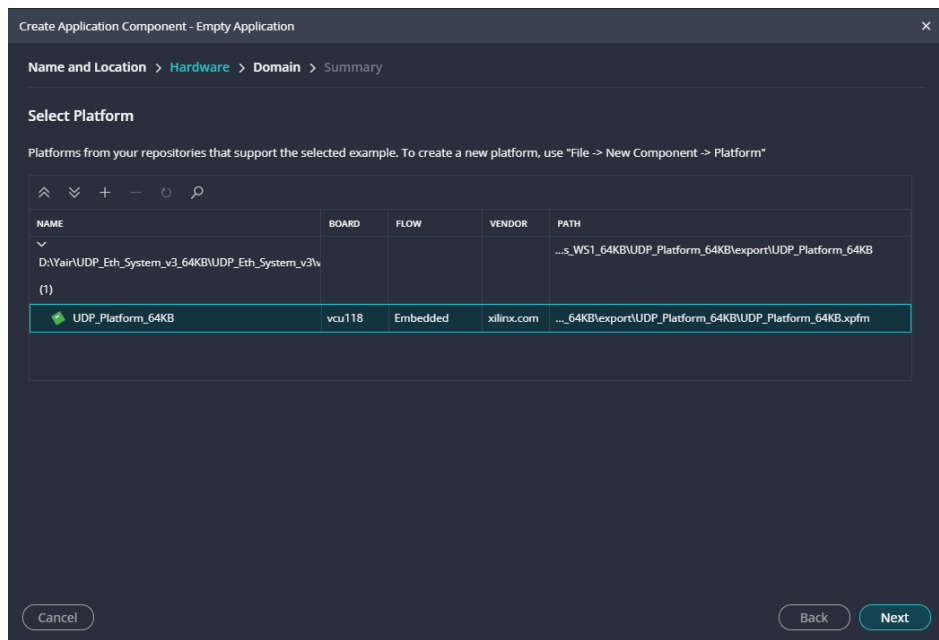
בניית Application Component מ-Template



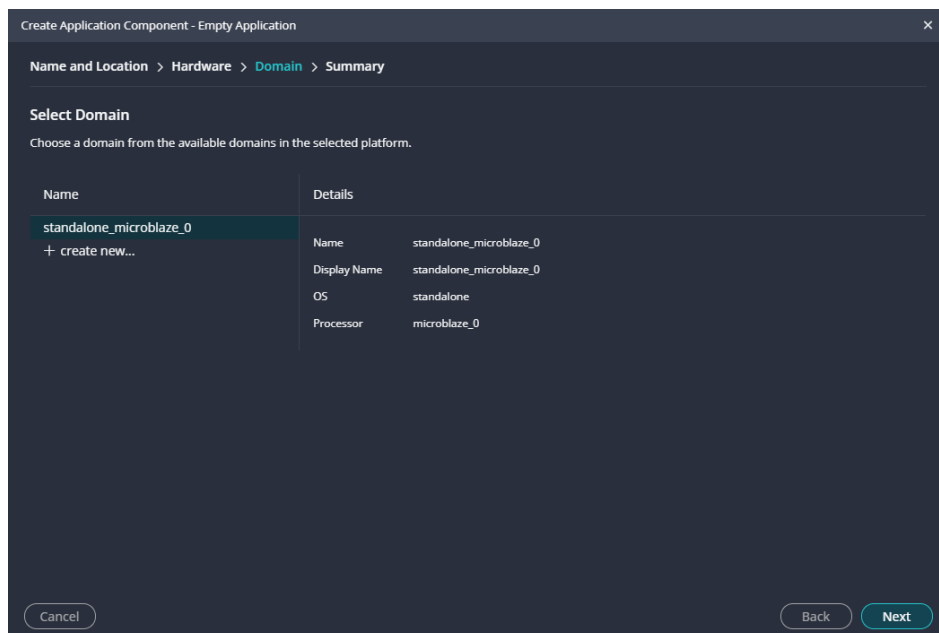
נעניק לו את השם UDP_app_64KB.



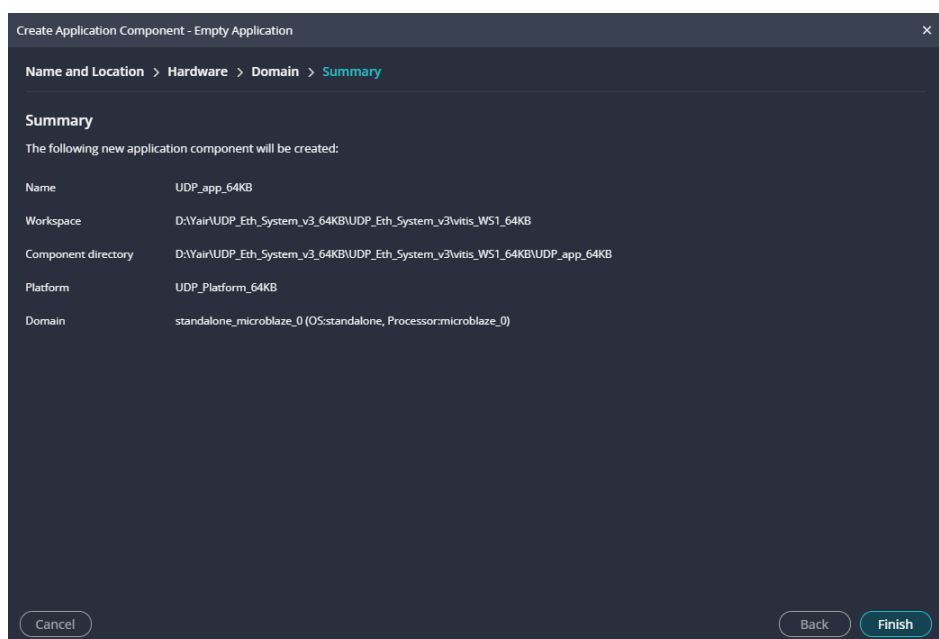
נקשר את האפליקציה לפלטפורמה שיצרנו.



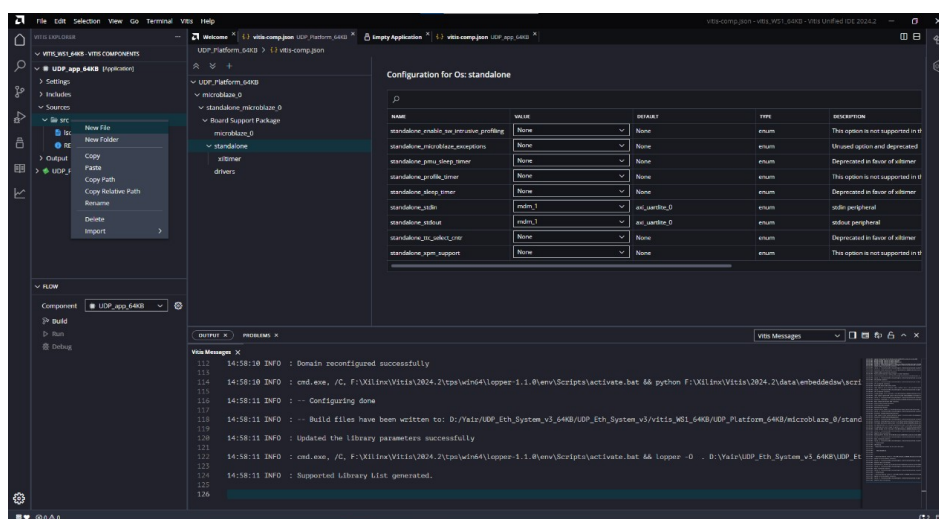
נלחץ. Next.



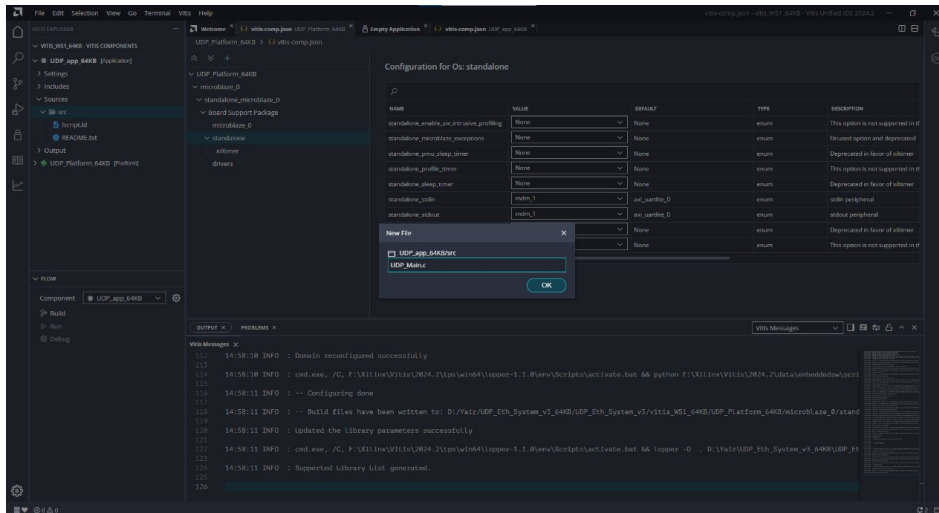
Finish. ולסיום נלחץ



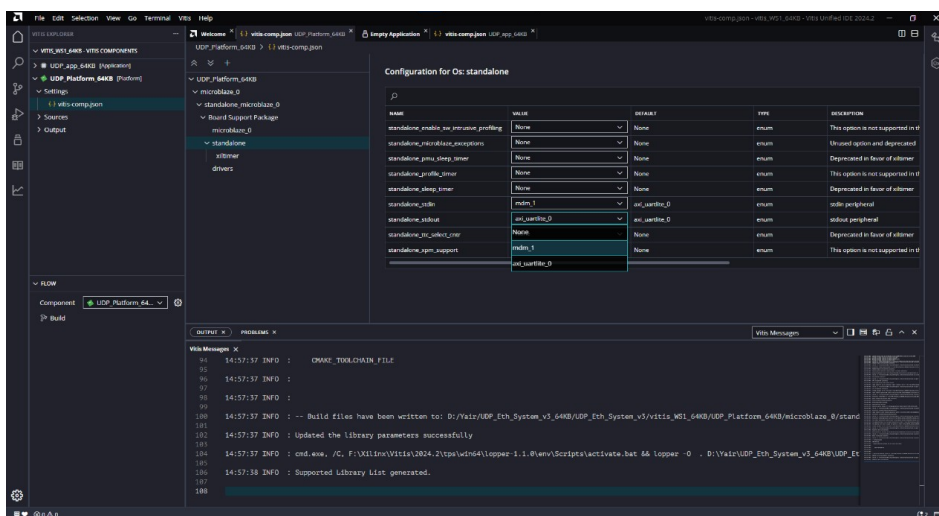
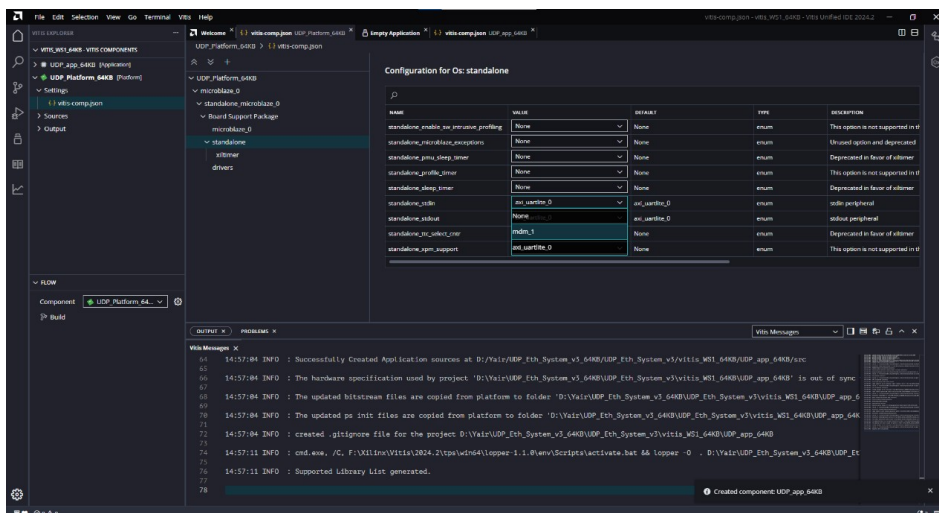
נוסיף קובץ מקור חדש בשם UDP_Main.c בתוך תיקיית src.



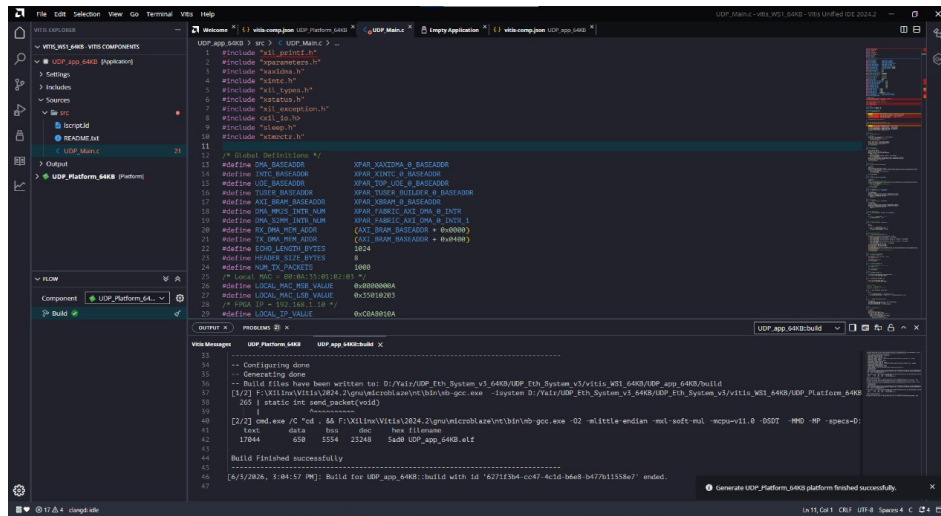
ניזן לתוך הקובץ את קוד התוכנה שכתבנו בעבר ונשמור אותו.



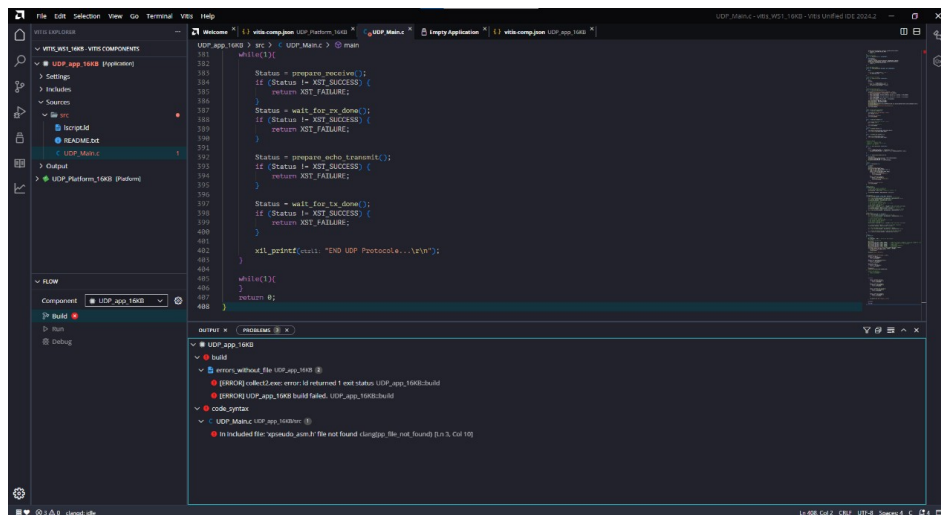
על מנת לאפשר למעבד לשלוח הדפסות למסך דרך כבל הצריבה, ניגש להגדרות ה-Board Support Package. נשנה את ערוצי הקלט והפלט stdout ו-stdin כך שיעבדו מול רכיב mdm_1.



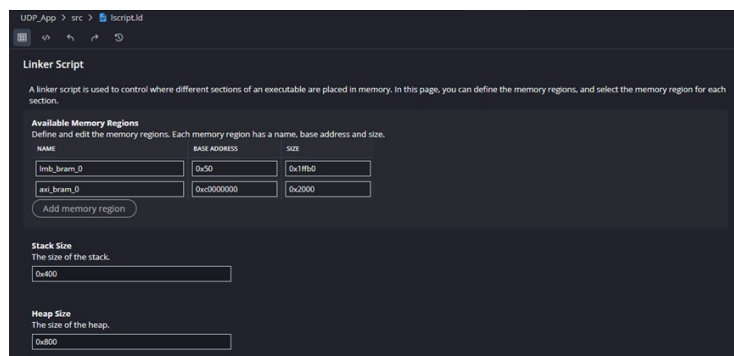
נלחץ על כפתור Build לקימפול הפלטפורמה ונמתין לקבלת הודעת ההצלחה.



הניסיון לרדת לנפח חוצץ של 16 KB נכשל. נפח זה התברר כקטן מדי בכדי להכיל את קוד התוכנה של המעבד, פונקציות התקשורת ומרחב המחסנית. מנגנון הקישור של הקומפיילר דיווח על חוסר מקום וקרא עם שגיאה עוד בטרם נבדקה תעבורת הרשת.



על מנת להבין לעומק את דרישות הזיכרון של קוד המעבד וכן את הסיבה לקריסת תצורת ה- 16 KB, בחנו את הגדרות קובץ ה- (lscript.ld) Linker Script בסביבת Vitis. חלון זה מציג את הגדרת גבולות הזיכרון הזמינים, כולל הקצרות ה- Stack וה- Heap. בדיקה זו נועדה לוודא כי נפח התוכנה הכולל אינו חורג מזיכרון ה- LMB הפיזי שהוקצה למעבד בחומרה.

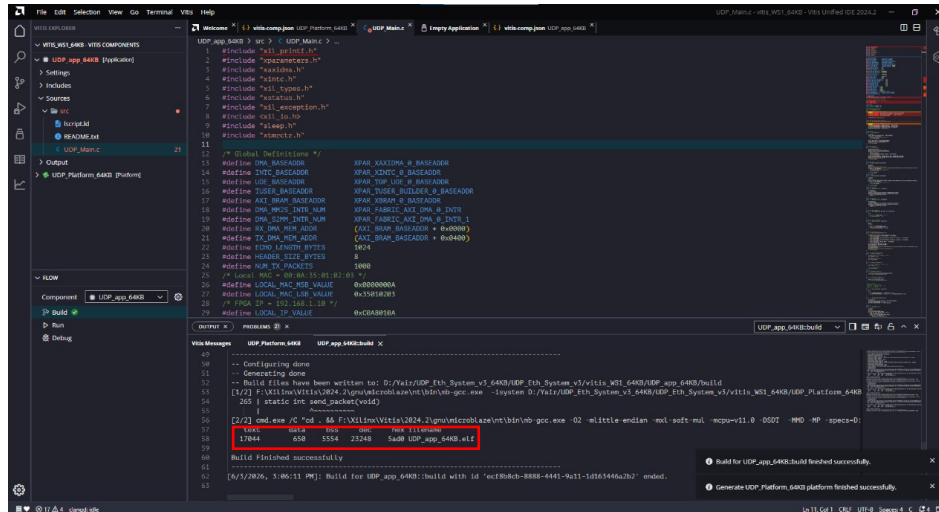


בנוסף, מתבצע שיוך של מקטעי התוכנה אל אזורי הזיכרון הפיזיים. כולל המקטעים המקומפלים, כגון מקטע הקוד text ומקטעי המשתנים data – bss, מנותבים באופן מפורש אל אזור ה- lmb_bram_0. מיפוי מרוכז זה מבטיח כי האפליקציה כולה תרוץ ישירות מתוך הזיכרון המקומי והמהיר של המעבד, ללא גלישה לכתובות זיכרון שאינן מוקצות לה.

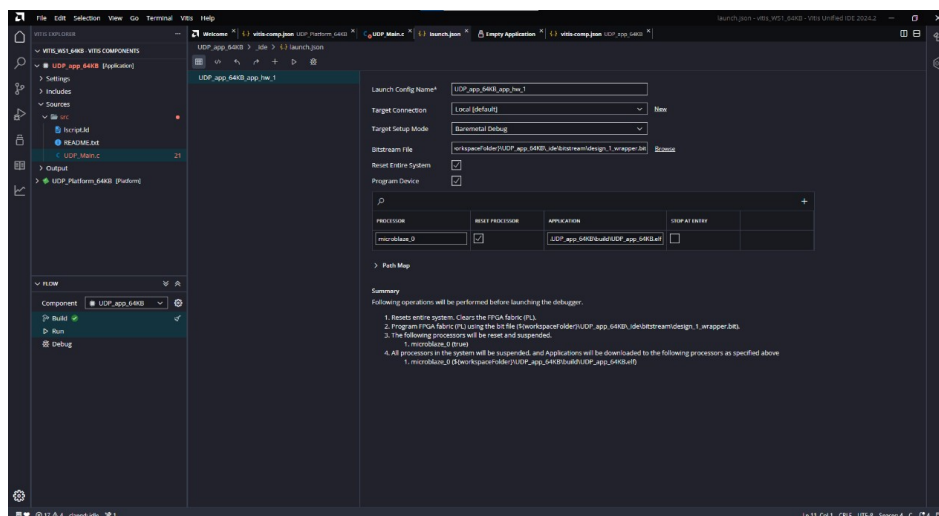
UDP_App > src > lscript.ld

SECTION	MEMORY REGION
.text	lmb_bram_0
.init	lmb_bram_0
.fini	lmb_bram_0
.ctors	lmb_bram_0
.dtors	lmb_bram_0
.drvcfg_sec	lmb_bram_0
.rodata	lmb_bram_0
.sdata2	lmb_bram_0
.sbss2	lmb_bram_0
.data	lmb_bram_0
.got	lmb_bram_0
.got1	lmb_bram_0
.got2	lmb_bram_0
.eh_frame	lmb_bram_0
.jcr	lmb_bram_0
.gcc_except_table	lmb_bram_0
.sdata	lmb_bram_0
.sbss	lmb_bram_0
.tdata	lmb_bram_0
.tbss	lmb_bram_0
.bss	lmb_bram_0
.heap	lmb_bram_0
.stack	lmb_bram_0

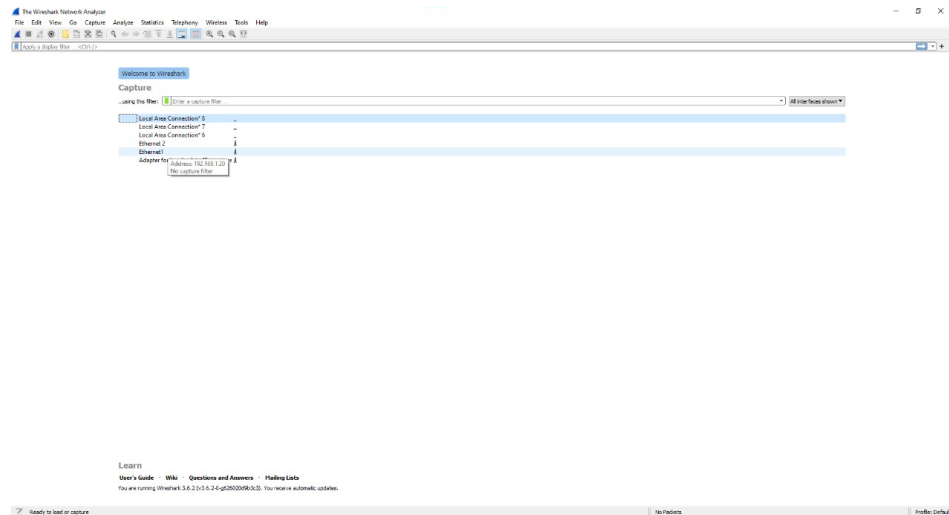
נלחץ על כפתור Build לאפליקציה. נוודא כי מתקבלת הודעת Build Finished successfully המאשרת שקובץ הריצה מוכן לצריבה. ניתן לראות בקונסולת הפלט את החלוקה המדוייקת של זיכרון התוכנה. גודלו הכולל של הקובץ עומד על 23,248 בתים (כ- 23 KB). נתון זה מוכיח בצורה מובהקת מדוע תצורת הזיכרון המקורית של 16 KB ב- MicroBlaze קרסה בעבר, ומאשר כי הגדלת זיכרון ה-BRAM המקומי ל- 64 KB הייתה פעולה הכרחית ומוצלחת.



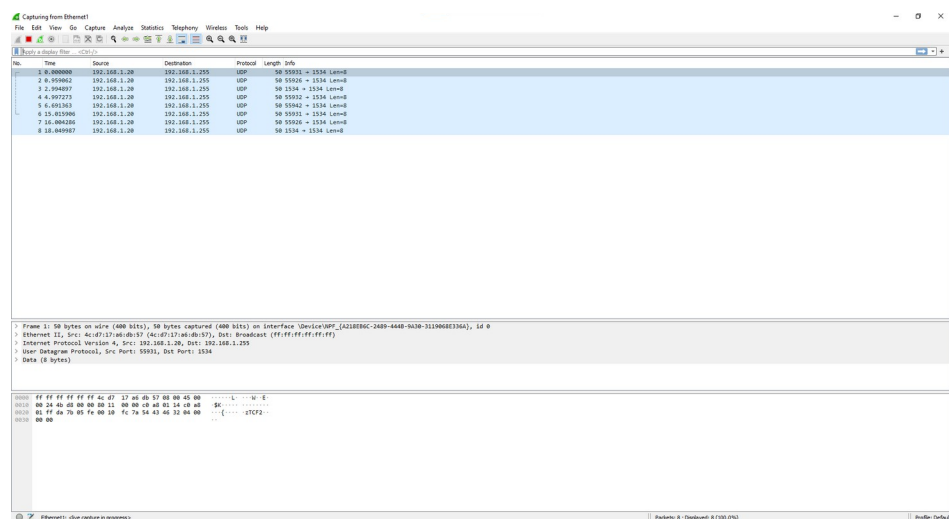
כעת נעבור להרצת התוכנה העדכנית על גביי הלוח. נפתח את חלון תצורת ההרצה (Run Configuration) בסביבת Vitis. תחת לשונית הגדרות היעד (Target Setup), נוודא כי מסומנת בוי (V) האפשרות Program FPGA.



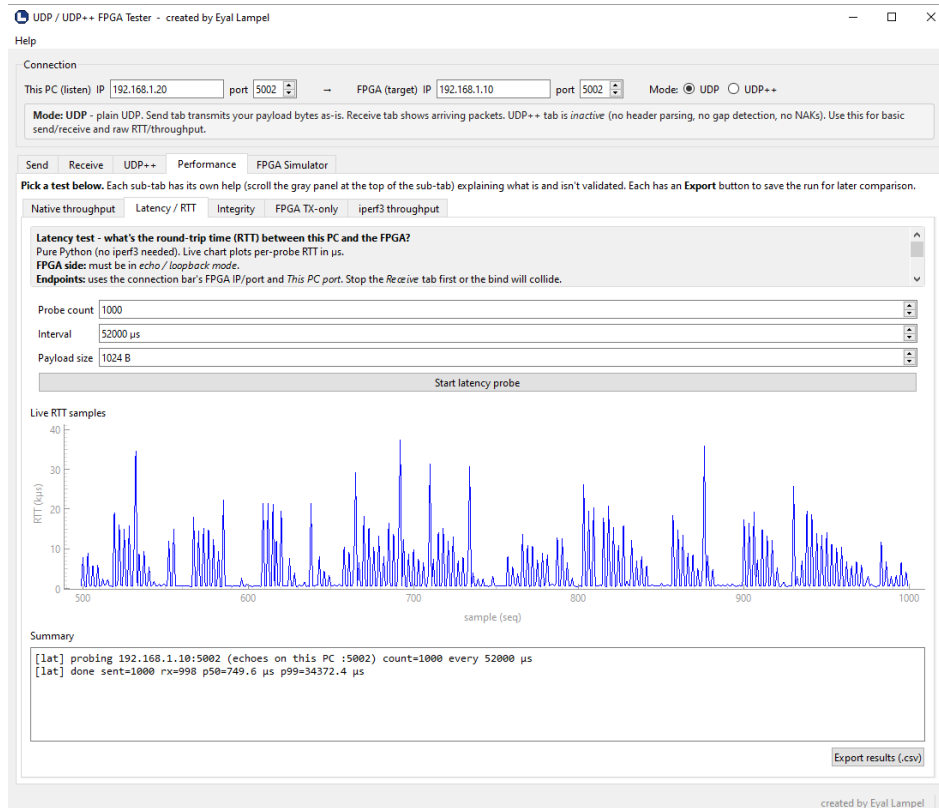
נשתמש בתוכנת הניטור Wireshark. נבחר בממשק הרשת המחובר פיזית ללוח הפיתוח (במקרה זה, Ethernet 1),



נבחין בבירור ברצף החבילות הנשלחות (Data Packets) מהמחשב אל כתובת ה- IP המוגדרת של ה- FPGA.

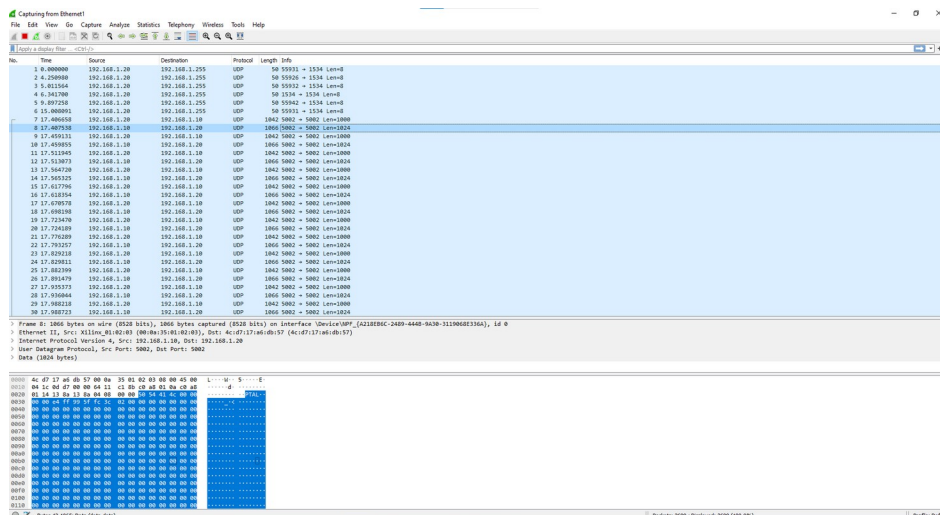
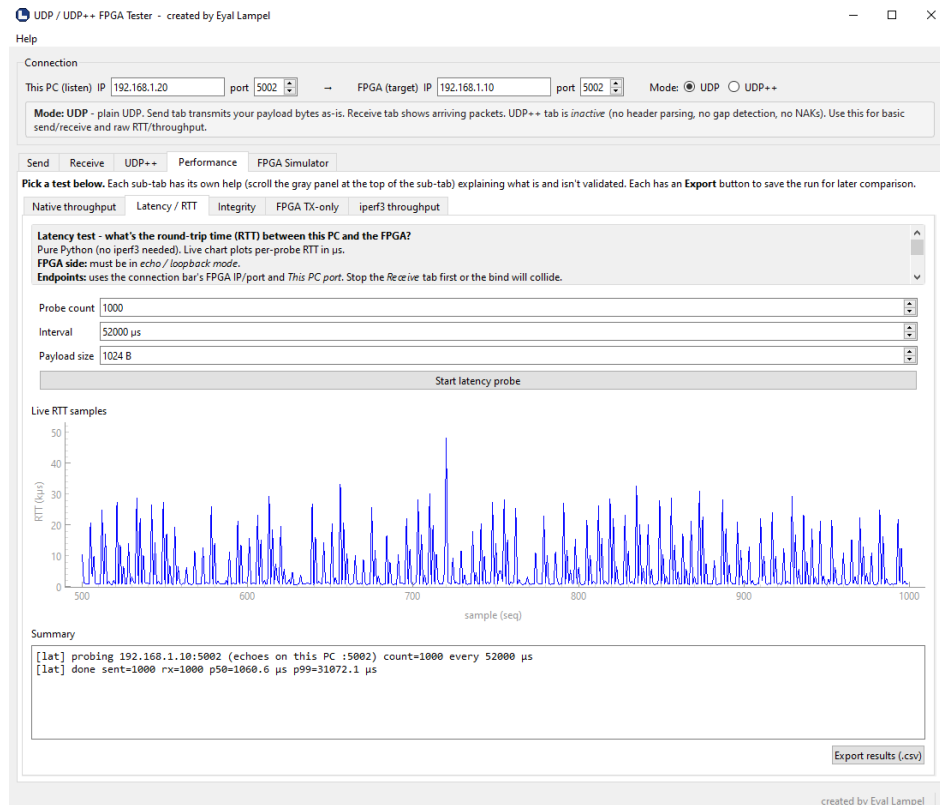


כעת לצורך ביצוע מדידת Latency, נעדכן את פרמטרי הבדיקה:
 מרווח הזמן (Interval) ל- 52,000 מיקרו-שניות ואת גודל המטען ל- 1024 בתים.
 לסיום, נלחץ על Start latency probe.



תצורת ה- KB32 עברה את תהליכי הקימפול והצריבה בהצלחה. עם זאת, הרצת מדידת Latency הכוללת שליחת 1000 חבילות רציפות בגודל 1024 בתים חשפה חוסר יציבות גבולית. בעת עומס תעבורה מתפרץ קצב השליפה של המעבד לא הספיק לרוקן את החוצץ בזמן. כתוצאה מכך הזיכרון התמלא וחלק קטן מהחבילות נשמט בדרך, כלומר מתוך 1000 חבילות שנשלחו, חזרו 998 חבילות בלבד. בהשוואה לתצורת ה- KB8 קיבלנו שיותר חבילות חזרו, 998 לעומת 993 אבל עדיין נשמטו 2 חבילות בדרך.

מעבר לתצורת ה- KB64 והרצת אותה מדידת Latency הניבו תפקוד בסנכרון מושלם. הבדיקה הסתיימה בהצלחה מוחלטת עם קבלה ושידור של 1000 מתוך 1000 חבילות ללא כל אובדן נתונים. במקביל השתמשנו בתוכנת Wireshark לניטור התעבורה הפיזית ולוודא הגעת חבילות התקשורת באורך המלא. מסקנת הניסוי היא שנפח של 64 KB מעניק למערכת את שולי הביטחון הנדרשים לספיגת עומסים קיצוניים תוך חיסכון משמעותי במשאבי לוח ביחס לתכנון מופרז.



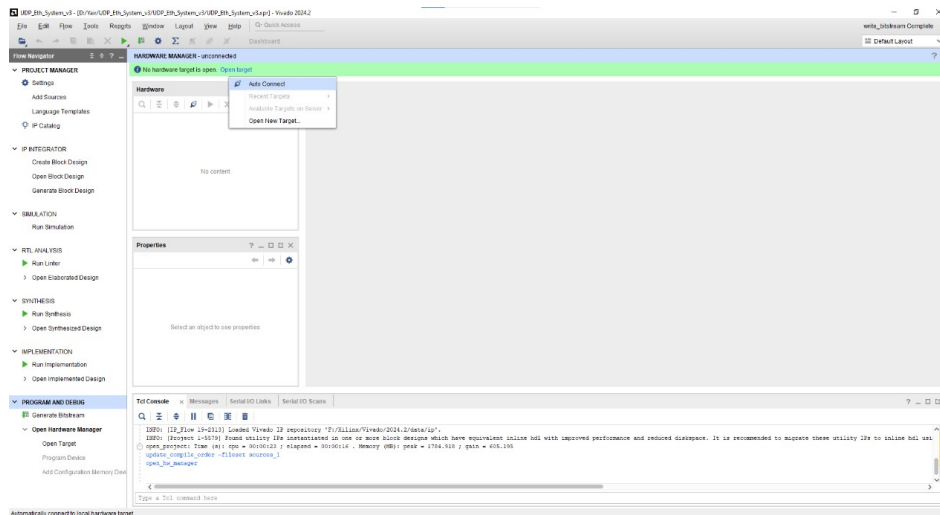
שלב 27:

השלב הנוכחי מתמקד בהטמעת רכיבי בדיקה פנימיים בחומרה, במטרה לבצע ניטור דינמי של אותות המפתח במערכת ולאמת את תקינות הפרוטוקולים בעת עבודה פיזית מול הלוח.

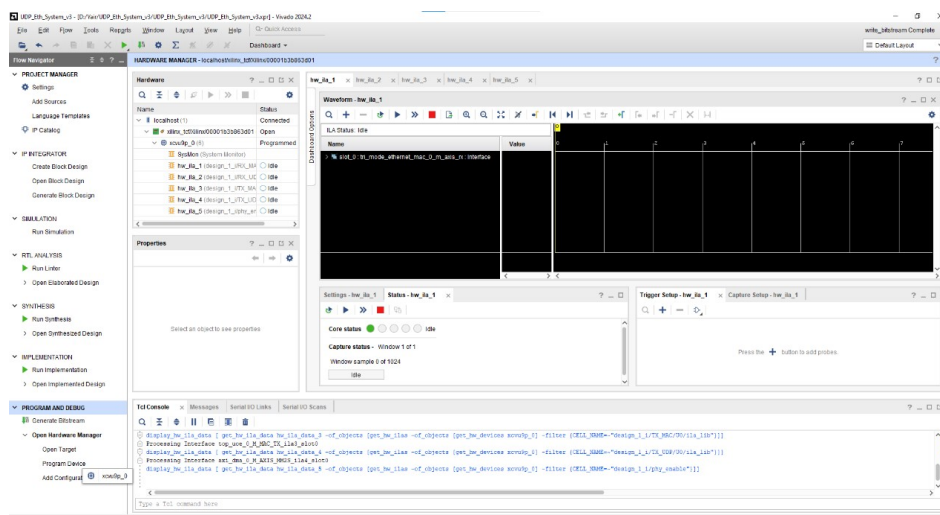
תחילה נציין כי הפעלת מערך הניטור בוצעה עבור חבילות הנתונים שיצרנו בסקריפט הפייתון (רצף התו 'B'), על מנת ללכוד ולוודא את מבנה החבילה בצורה שקופה. בהמשך התהליך,

נשתמש באותה סביבת ניטור בדיוק כדי לנטר את האותות תחת ומס התעבורה שמייצרת תוכנת ה- Tester של המנחה אייל.

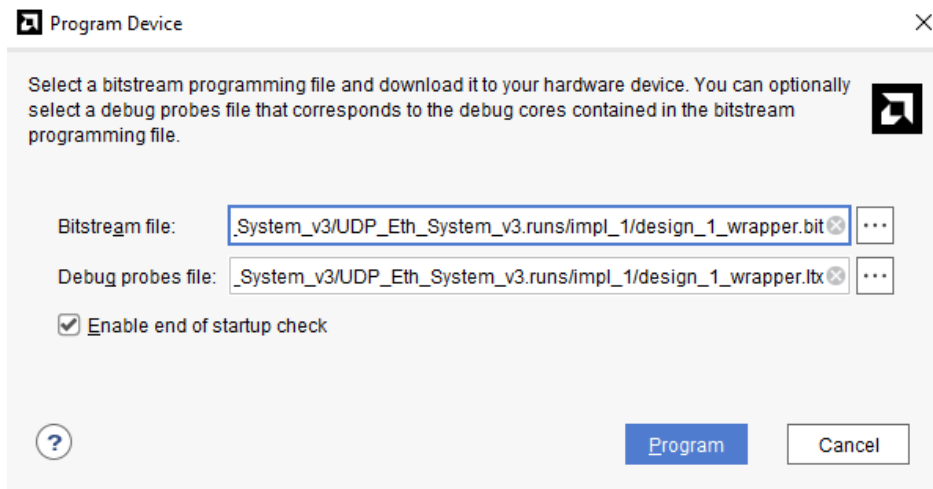
לשם התחלת הניטור, נפתח ממשק ה- Vivado Hardware Manager ב- Vivado ונבצע התחברות אוטומטית אל רכיב ה- Target (Auto Connect).



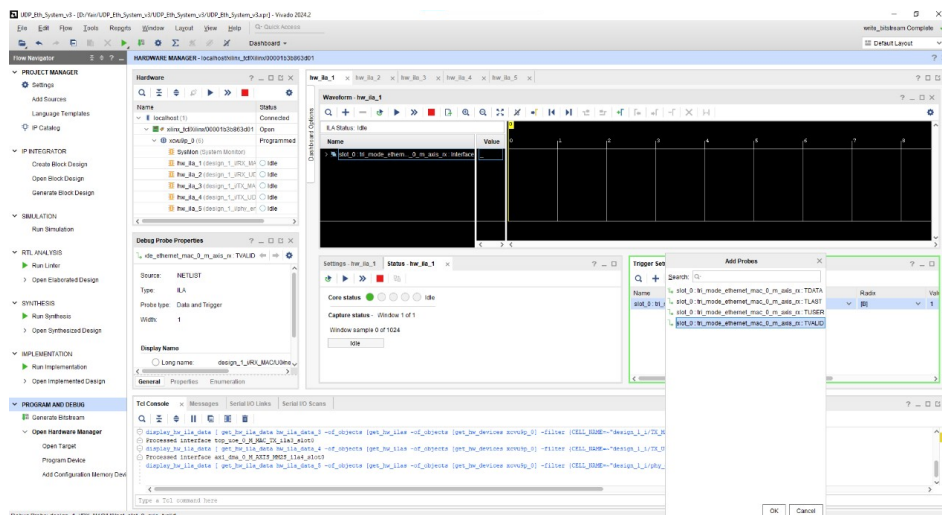
נעת נבצע צריבה של השבב (Program Device).



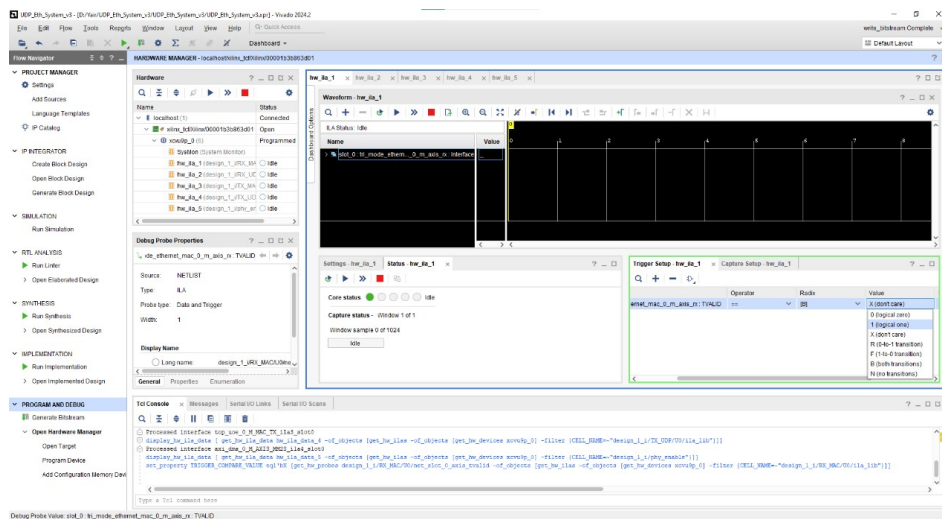
בעת הטעינה, נבחר קובץ ה- Bitstream המתאים ובצמוד אליו נשייך את קובץ ה- ltx כדי שהתוכנה תדע לשייך את הגרפים לחומרת הפרויקט.



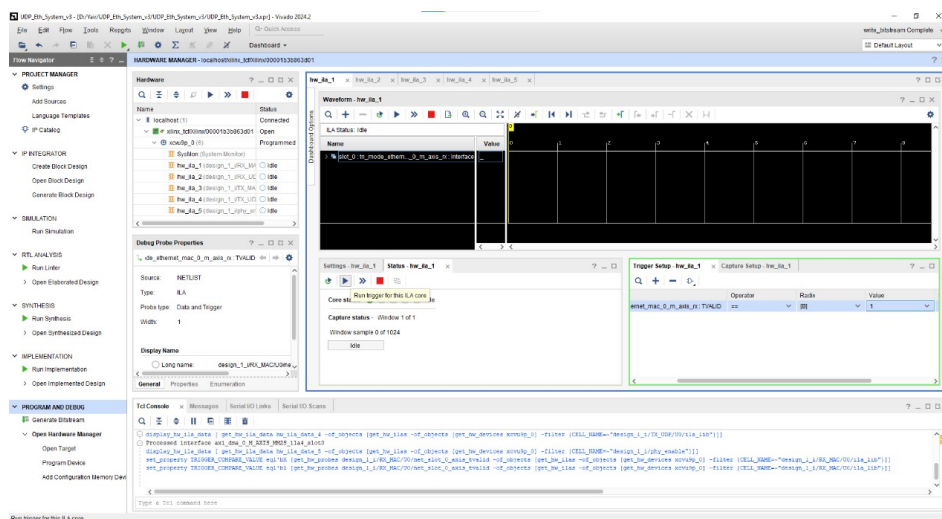
עם השלמת הצריבה, נפתחו אוטומטית חלוניות ה-Dashboard של ה-ILA. על מנת ללכוד במדויק את רגע הגעת מנת הרשת ולמנוע הקלטה של מצבי סרק, הוגדר תנאי הפעלה (Trigger) - TVALID של ממשק הקליטה/שידור נגרר אל חלון ה-Trigger Setup.



נקבע את ערכו ל-1 (או Rising Edge).

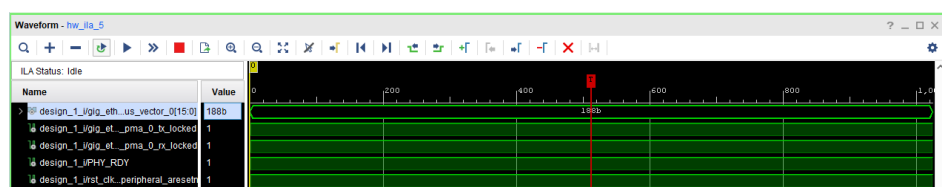


במצב זה המערכת נדרכת ועוברת למצב Waiting for Trigger.



בדיקת תקינות הקישור הפיזי (PHY):

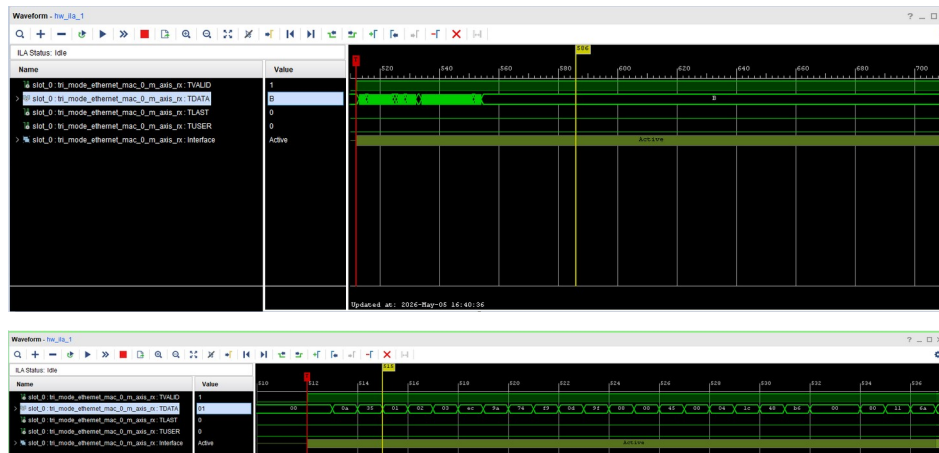
לפני ניתוח זרימת הנתונים במסלול הקליטה, חובה לוודא כי הקישור הפיזי יציב ופעיל. ניתן לראות כי אותות הנעילה של השידור והקליטה יחד עם אות המוכנות הכללי נמצאים במצב לוגי '1'. מצב זה מאשר שהתקשורת הפיזית מול מחשב הבדיקה הוקמה בהצלחה וניתן להתחיל לקלוט חבילות רשת.



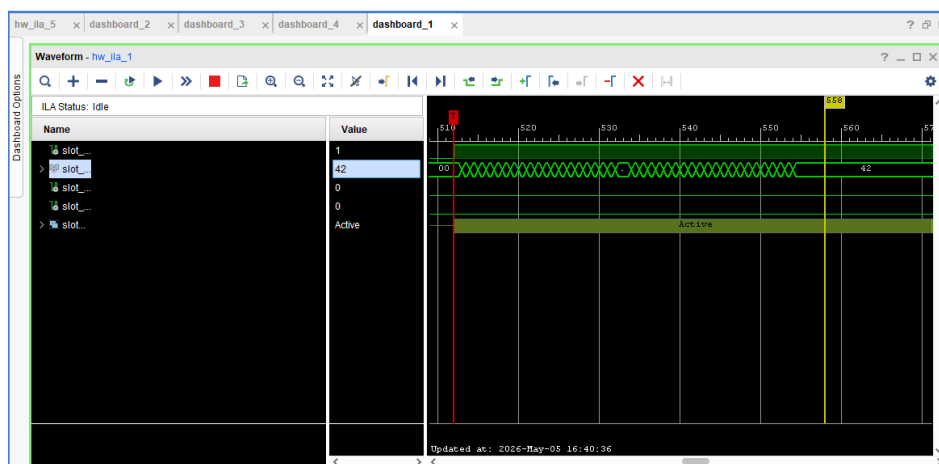
תחנה ראשונה: כניסה למערכת (מסלול RX_MAC):

ברגע שהחבילה מתקבלת מהרכיב הפיזי (PHY), היא נדגמת בכניסה למערכת התקשורת. בנקודה זו, בקר ה-MAC מסיר את מעטפת ה-Ethernet החיצונית ומתחיל להזרים את החבילה (הכוללת את כותרות ה-IP וה-UDP) אל מנוע ה-UOE לטובת פענוח.

בצורות הגל של ערוץ ה-RX_MAC ניתן לראות את תחילת זרימת הנתונים. האות TVALID עולה ללוגי '1', ועל אפיק ה-TDATA מופיעות כותרות החבילה (4c, d7, ff וכו') המוזרמות לבדיקה בתוך מנוע ה-UDP.



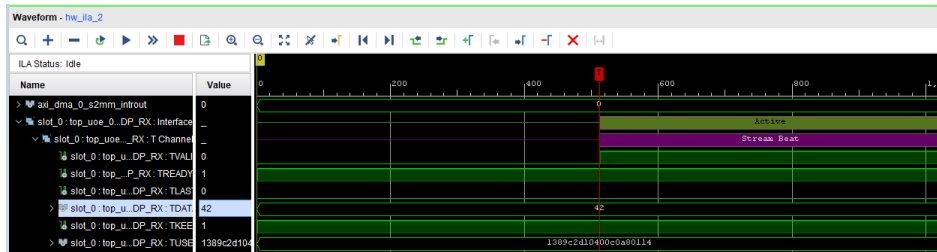
להלן התצוגה הכללית של קליטת החבילה במלואה, מבט רחב (Zoom Out) על חלון הזמן של קליטת החבילה בערוץ ה-MAC_RX. בתצוגה זו ניתן לראות את משך הזמן הכולל של העברת הנתונים (האזור שבו אות ה-TVALID נשאר גבוה). בסיום רצף הנתונים המרכיבים את מעטפת התקשורת, ניתן להבחין בקצה הימני בערך 0x42 (המידע עצמו), המלווה באופן צמוד למחזור הסיום. תמונה זו מאשרת את רציפות זרימת החבילה כולה מה-MAC אל מנוע ה-UOE ללא קיטעון.



תחנה שנייה: חילוץ המידע הנקי (מסלול RX_UDP):

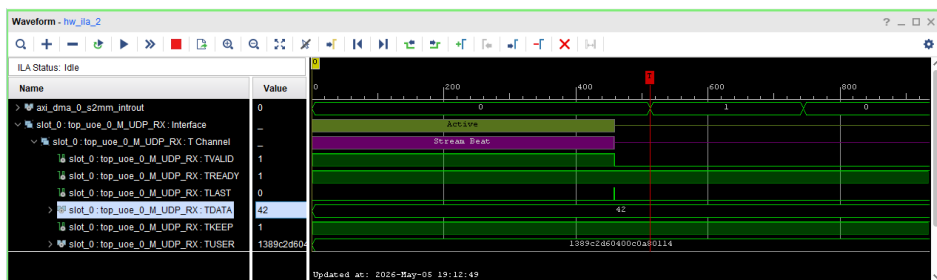
לאחר שמנוע ה-UOE פענח בהצלחה את כותרות ה-IP וה-UDP ואימת את תקינותו, הוא "חותך" אותן ומותיר רק את המידע הנקי (ה-Payload). המידע נשלח ישירות לבקר ה-DMA כדי שייכתב לזיכרון ה-BRAM.

הניטור בערוץ ה-RX_UDP מציג בצורה מושלמת את חילוץ הנתונים. מנוע ה-UOE סינן את כל הכותרות, ועל קו ה-TDATA מופיע מחזור שעון בודד בלבד המכיל את הערך 0x42 באותו מחזור בדיוק, האות TLAST עולה ל-1, מה שמעיד על כך שהחבילה הסתיימה כהלכה והמידע הועבר בשלמותו.



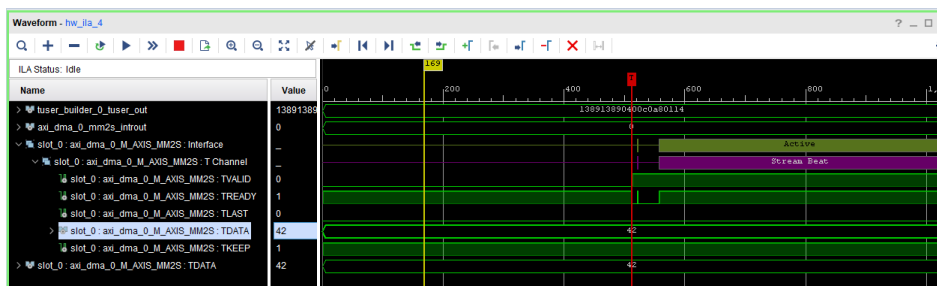
מנגנון הפסיקות בקליטה (RX Interrupt):

על מנת ליידע את המעבד כי התקבלה חבילה חדשה בזיכרון, בקר ה-DMA מקפיץ פסיקת חומרה. דגימת האות s2mm_introut מאשרת כי בקר ה-DMA הפעיל פסיקת קליטה תקינה. המעבד (MicroBlaze) קולט פסיקה זו, ניגש לזיכרון, קורא את התו 0x42, ומכין אותו לשידור חוזר (Echo).

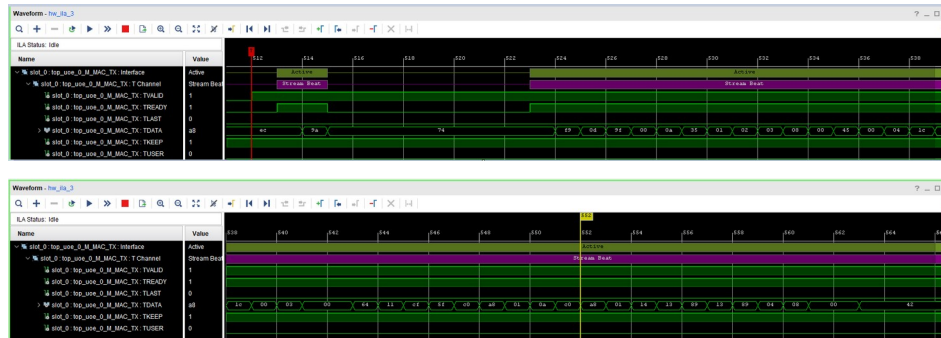


תחנה שלישית: שידור חוזר מהזיכרון (מסלול TX_UDP):

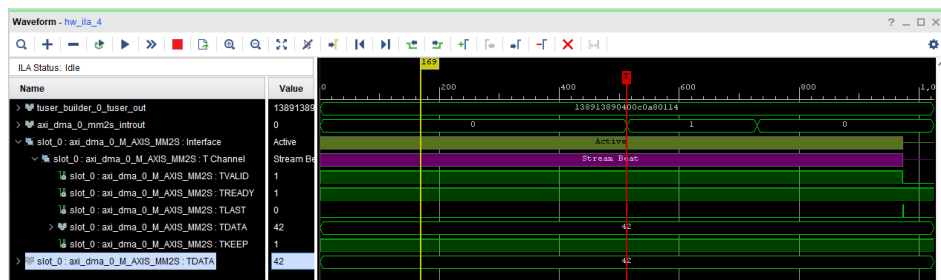
מעבד ה-MicroBlaze נותן הוראה ל-DMA לשדר את המידע חזרה. ה-DMA שולף את הערך 0x42 מהזיכרון ודוחף אותו חזרה אל מנוע ה-UOE לצורך בניית חבילת רשת יוצאת. במסלול השידור, ה-DMA יוזם לחיצת יד תקינה מול ה-UOE. האותות TREADY ו-TVALID נמצאים יחד במצב לוגי גבוה, והערך 0x42 נדחף אל ערוץ ה-TDATA בדרכו לעיטוף.



תחנה רביעית: הרכבת כותרות ויציאה ל-MAC (מסלול TX_MAC):
 מנוע ה-UOE מקבל את הביט הבודד, מייצר עבורו מחדש כותרות UDP, IPv4 ו-MAC (תוך החלפת כתובות המקור והיעד), ומשדר את החבילה השלמה אל בקר ה-MAC בדרכה החוצה. התצוגה בערוץ ה-TX_MAC מאשרת את בניית החבילה על ידי מנוע ה-UOE: הלוגיקה עטפה את התו 420x מחדש בכותרות התקשורת הנדרשות, והחבילה המלאה מוזרמת אל בקר הרשת. מחזור השעון האחרון המכיל את התו שלנו מגובה בפולס של TLAST.

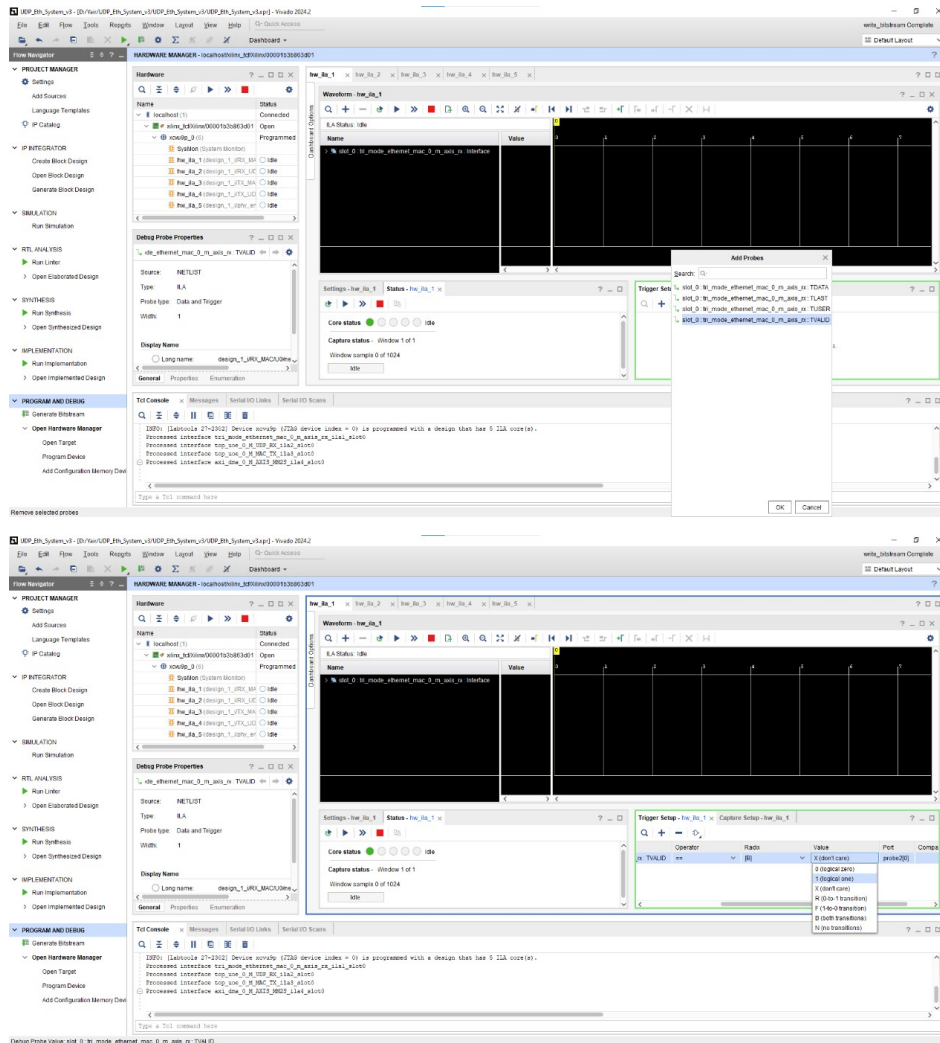


מנגנון הפסיקות בשידור (TX Interrupt):
 בסיום דחיפת הנתונים מהזיכרון החוצה, ה-DMA מתריע למעבד כי משימת השידור הושלמה. פסיקת השידור עולה ללוגי '1', מה שמעיד כי מחזור הטיפול של ה-DMA בחבילה (Transaction) הסתיים בהצלחה, והמערכת פנויה לקבלת החבילה הבאה.

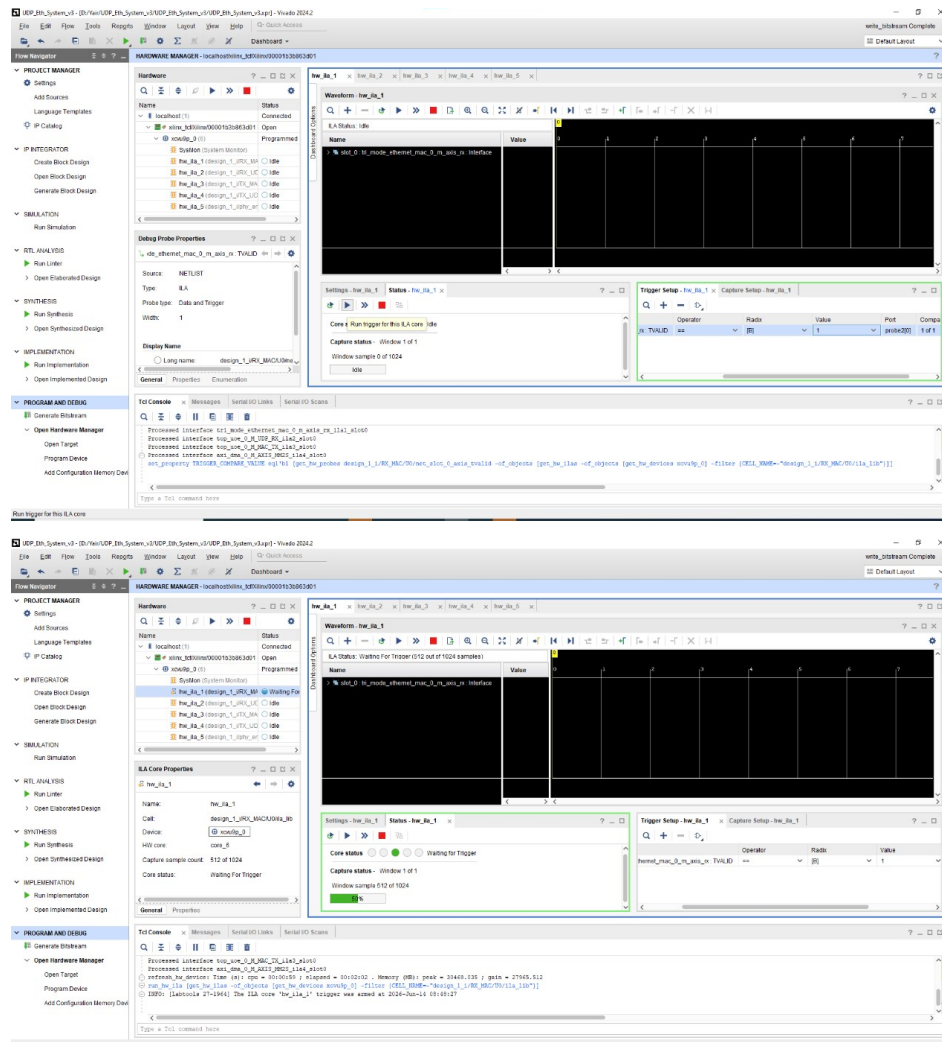


כפי שהגדרנו בתחילת פרק הניטור, לאחר אימות מסלול הנתונים מול חבילות בודדות מסקריפט הפייתון, אנו עוברים כעת לשלב השני, בדיקת עומס. נשתמש באותה סביבת ILA בדיוק על מנת לנטר ולנתח את התנהגות המערכת תחת זרם הנתונים הרציף שמייצרת תוכנת ה-Tester. המטרה היא לוודא שהמערכת עומדת בקצב הגבוה ללא אובדן מידע או חסימות בנתיב הנתונים.

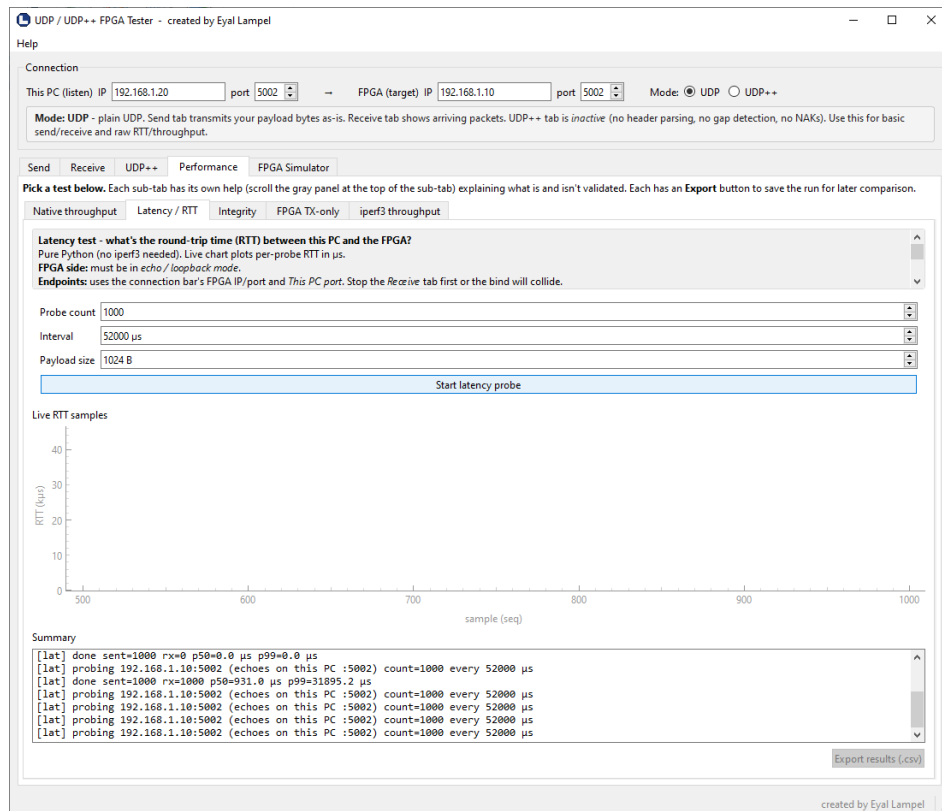
בדומה לבדיקה הקודמת, הוספנו את אות ה- TVALID של ממשק הקליטה לחלון ה- Trigger Setup וקבענו את ערכו ל- '1' לוגי. הגדרה זו מבטיחה שה- ILA יתחיל להקליט את הנתונים בדיוק ברגע שתוכנת ה- Tester תחל בשידור רצף החבילות אל הלוח.



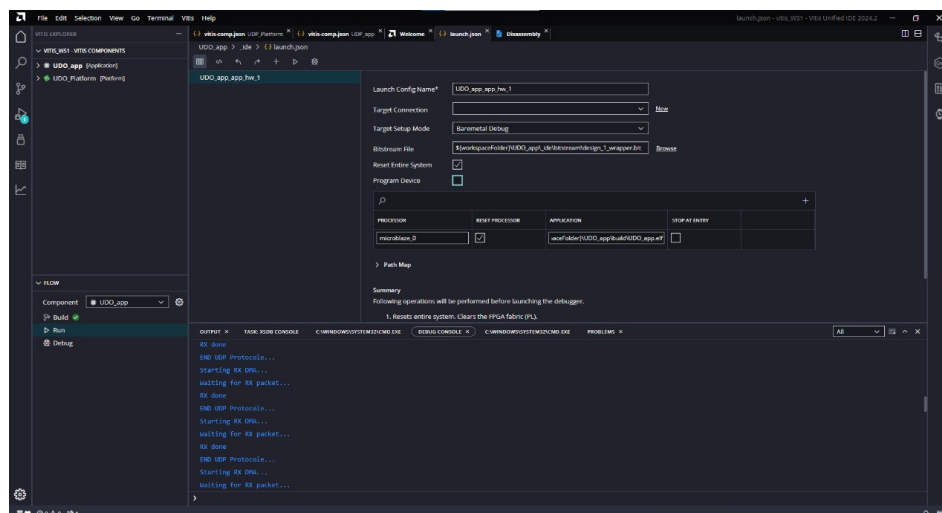
לאחר הגדרת התנאי, הפעלנו את פעולת הדיגום (Run Trigger). בשלב זה, מערך ה-ILA נדקר ועובר למצב "Waiting for Trigger". החומרה נמצאת כעת בהאזנה פעילה וממתינה לתחילת שידור זרם החבילות מתוכנת ה-Tester כדי להחל את הקלטת האותות.



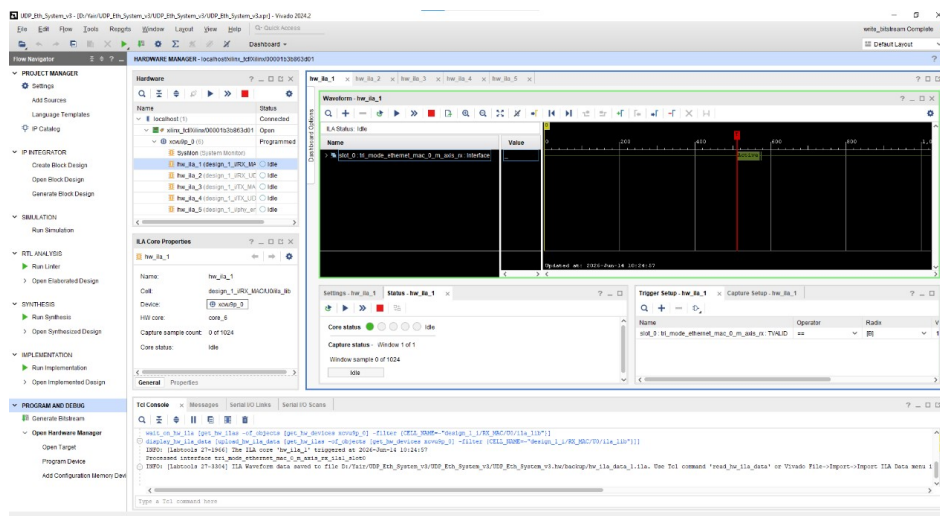
בשלב זה, הפעלנו את תוכנת ה- Tester כדי לייצר את העומס. כפי שניתן לראות בתמונה. הגדרנו בדיקת Latency/RTT הכוללת שידור רציף של 1,000 חבילות נתונים (Probes) לעבר הלוח. פעולה זו מייצרת עומס תעבורה מהיר ורציף על ממשקי הרשת והחומרה של המערכת.



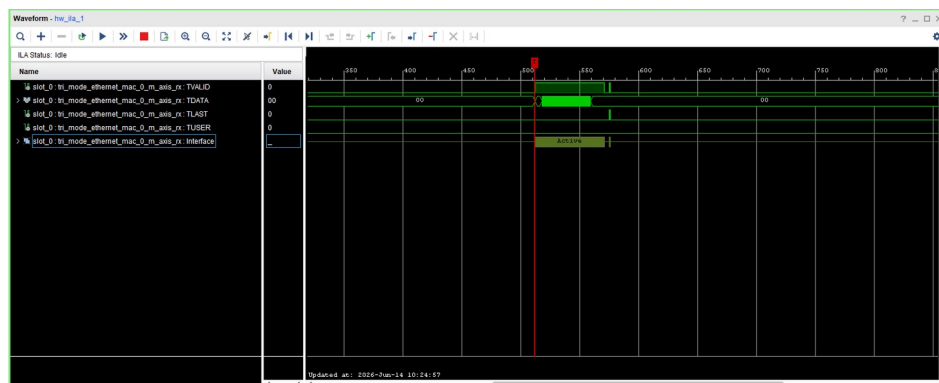
כדי שהחומרה תקלוט את עומס הנתונים, יש לאתחל תחילה את התוכנה. הרצת הקוד ב- Vitis מקנפגת את בקר ה- DMA ופותחת את ערוץ התקשורת לקבלת חבילות. ללא שלב זה, נתיב הנתונים חסום ומערך ה- ILA לא יזהה תעבורה באפיק.



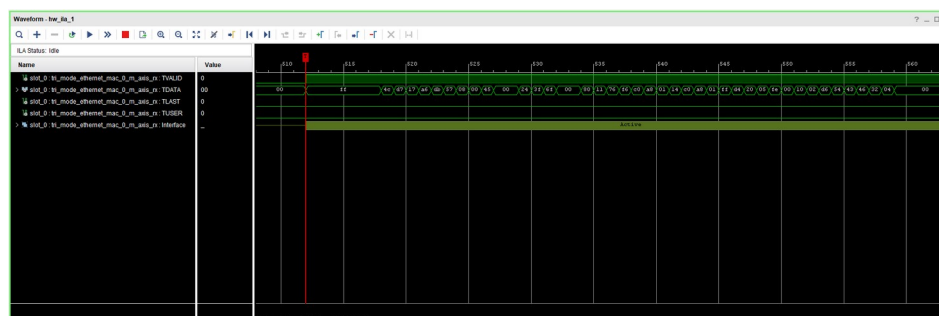
קליטת חבילות (RX) תחת עומס:
 כדי ללכוד את זרם הנתונים בצורה מדויקת, הוגדר אות ה-TVALID כטריגר ברמת לוגי '1'.
 הגדרה זו מוודאת כי ה-ILA יתחיל את הדגימה ברגע זיהוי תחילת העברת חבילה מה-Tester,
 תוך התעלמות ממצבי המתנה (Idle) באפיק.



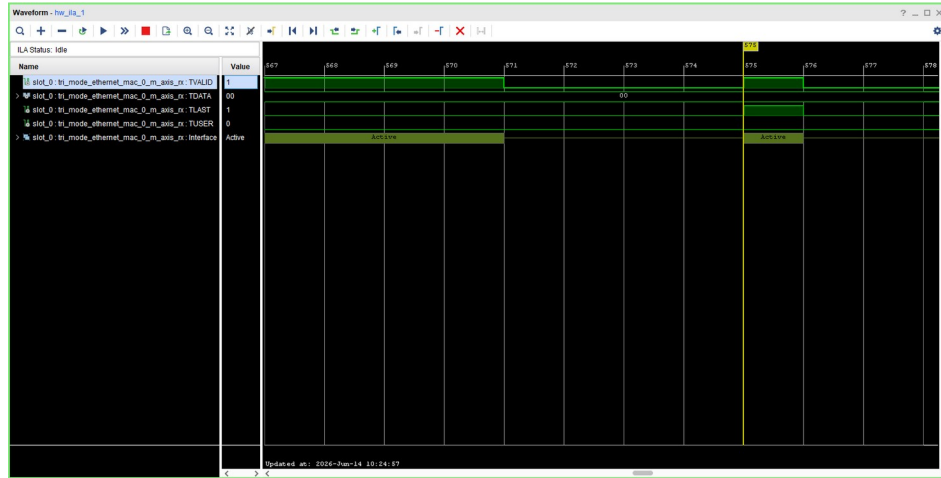
ניתן לראות את עליית אות ה-TVALID בממשק ה-AXI-Stream של ה-RX. עם עליית האות, ניתן להבחין בתחילת העברת המידע באפיק ה-TDATA. אות זה מהווה אינדיקציה ברורה לכך שהחומרה החלה לעבד את החבילה שהגיעה מה-Tester.



התצוגה מציגה את זרם הנתונים הרציף בתוך החבילה. ניתן לראות כיצד המידע עובר במחזורי שעון עוקבים, כאשר אות ה-TVALID נשאר גבוה לאורך כל שלב ה-Payload, מה שמעיד על עבודה רציפה וקצבית ללא עצירות בנתיב הנתונים.

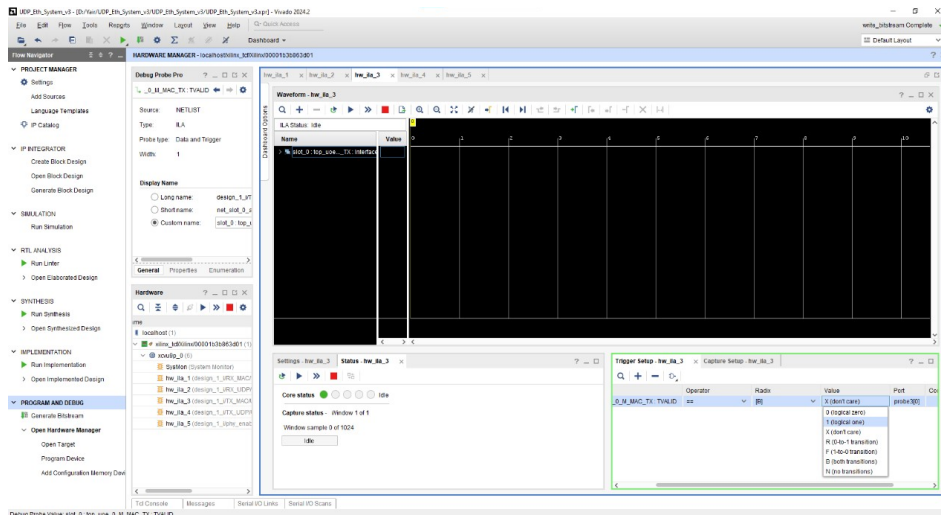
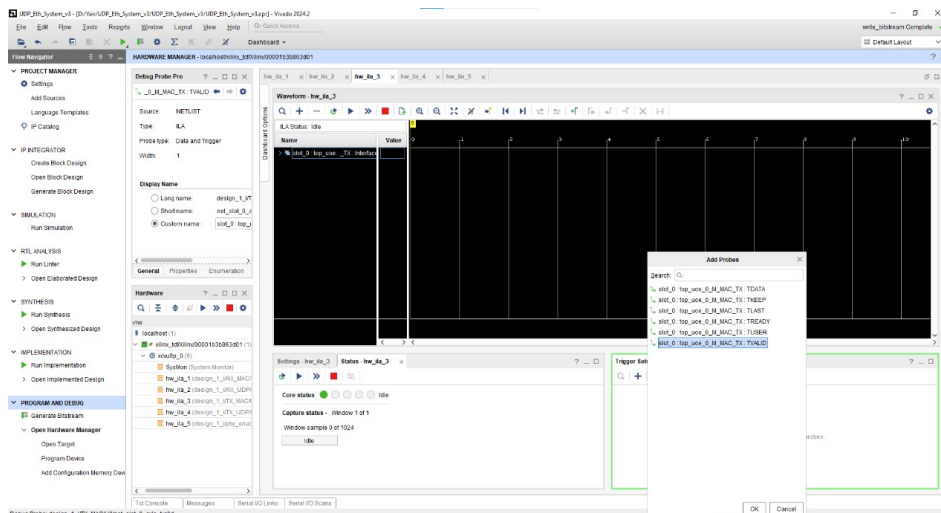


ברגע סיום החבילה, ניתן לראות את עליית אות ה- TLAST ללוגי '1' במחזור שעון מסונכרן. עלייה זו מסמנת את סוף מסגרת המידע הנוכחית ומאפשרת לבקר ה- DMA לזהות שהחבילה התקבלה בשלמותה ולהפעיל את פסיקת הטיפול במידע (Interrupt).

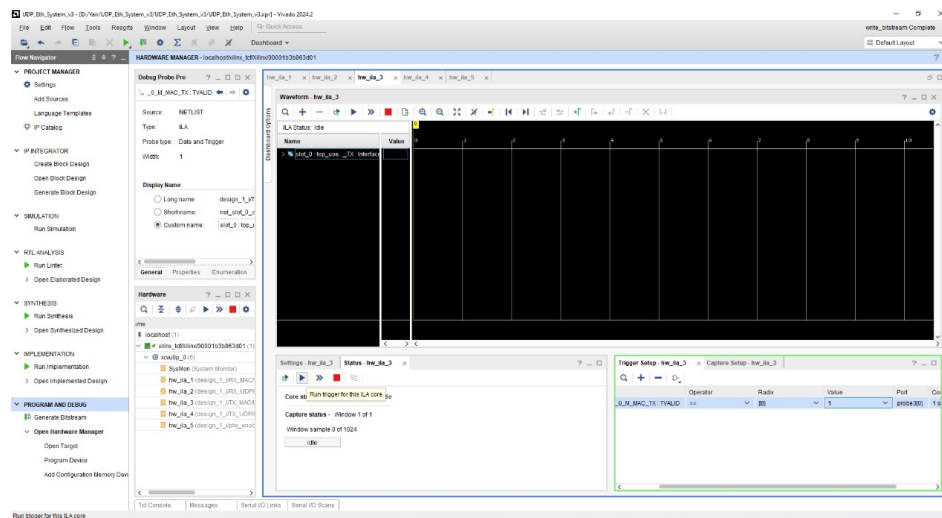


ניתוח מסלול השידור (TX) תחת עומס:

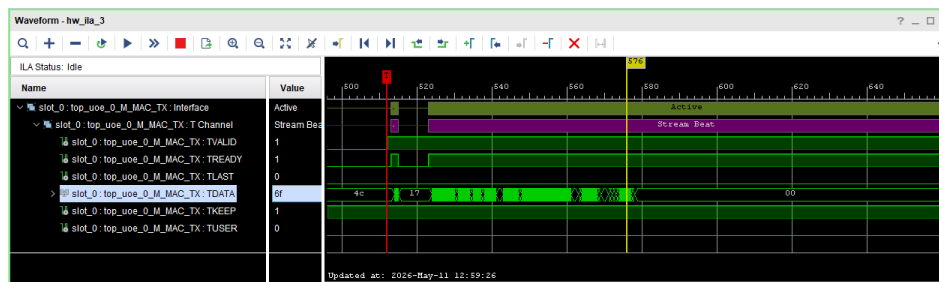
לצורך ניטור יציאת הנתונים מהמערכת, הוגדר אות ה- TVALID של ממשק ה- MAC TX כטריגר ברמת לוגי '1'. הגדרה זו מאפשרת ללכוד את רגע תחילת הוצאת החבילות אל רכיב ה- PHY ואת יציאתן התקינה מהחומרה.



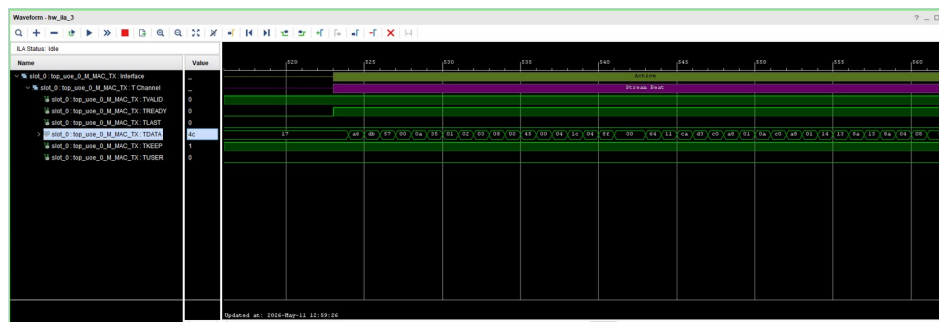
לאחר הגדרת תנאי הטריגר, הפעלנו את פעולת הדיגום (Run Trigger) על רכיב ה-ILA של ממשק ה- MAC TX. ניתן לראות כי המערכת נכנסה למצב המתנה פעיל, בו היא "דרכה" וממתנת לעליית אות ה- TVALID מצד החומרה כדי ללכוד את הנתונים המשודרים ולשמר את חלון הזמן המדויק של השידור.



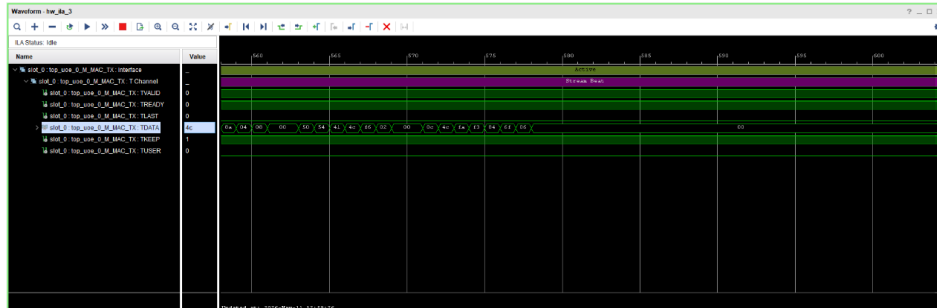
עם הפעלת הדיגום, נתפס הרגע שבו ה- MAC TX מתחיל להוציא נתונים. ניתן לראות את עליית אות ה- TVALID יחד עם הנתונים הראשונים המופיעים באפיק ה- TDATA. אות ה- TREADY הגבוה מעיד על כך שהממשק החיצוני מוכן לקבל את המידע ללא עיכובים.



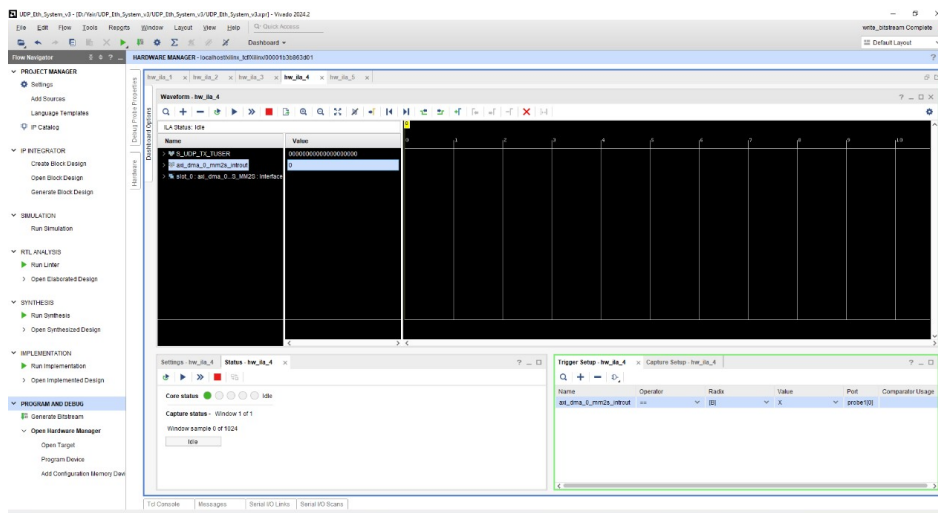
ניתן לראות את זרימת הנתונים הרציפה ב- TX. האותות TVALID ו- TREADY נשארים גבוהים, מה שמאשר העברה יציבה וקצבית של ה- Payload אל ה- MAC. זהו אישור חומרתי לכך שהמערכת מצליחה לעמוד בעומס השידור שנוצר על ידי ה- Tester.



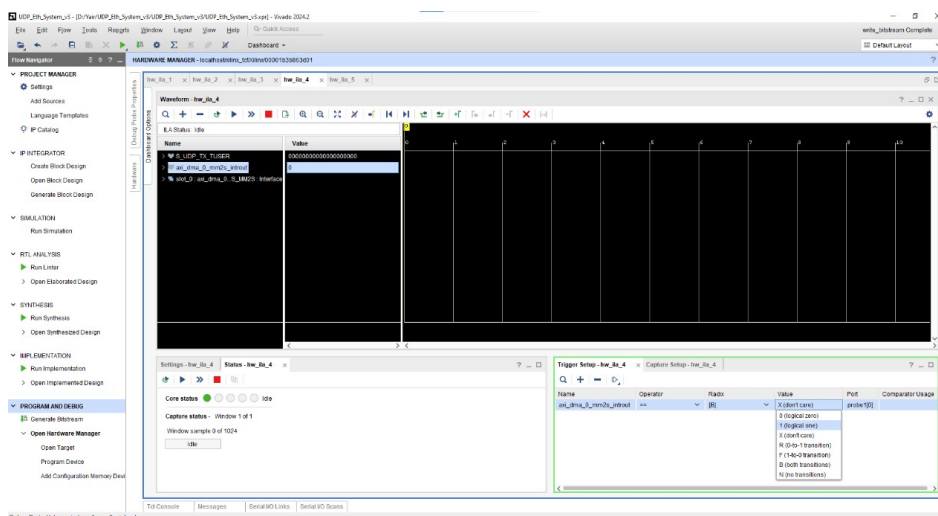
נציין כי סיום החבילה (עליית אות ה-TLAST) אינו מופיע בחלון הדיגום הנוכחי. מאחר שחבילות הנתונים בבדיקה הן בנות 1024 בתים, ורכיב ה-ILA הוגדר ללכידת 1024 דגימות (Samples) בלבד, סיום החבילה מתרחש לאחר מיצוי זיכרון ה-ILA. כתוצאה מכך, לא ניתן לראות את תחילת וסוף החבילה באותה התצוגה, אך המידע המוצג מספק אימות מלא לתקינות זרם הנתונים המשודר.



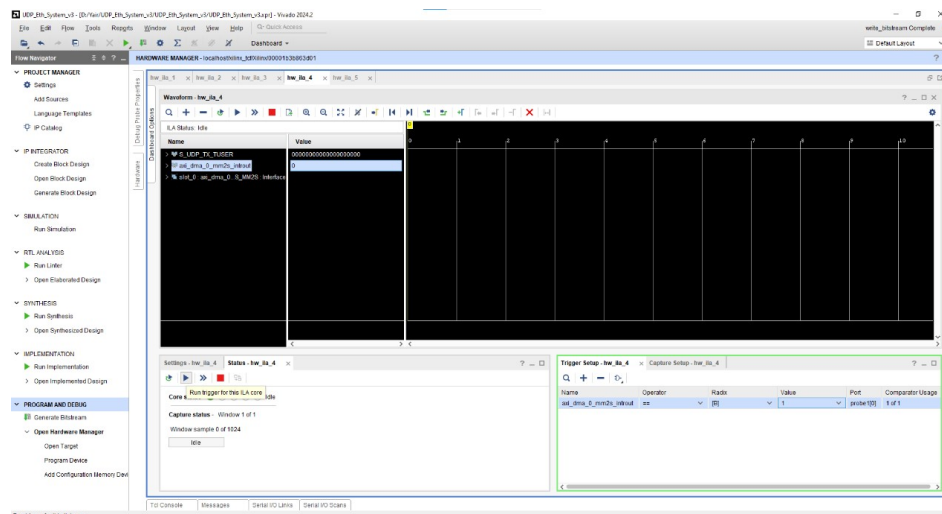
ניתוח מנגנון הפסיקות (Interrupt Analysis):
 כדי לאמת את פעולת מנגנון הפסיקות, הוספנו את אות ה- s2mm_introut (הפסיקה של ה-DMA המציינת סיום כתיבת נתונים לזיכרון) לחלון ה- Trigger Setup.



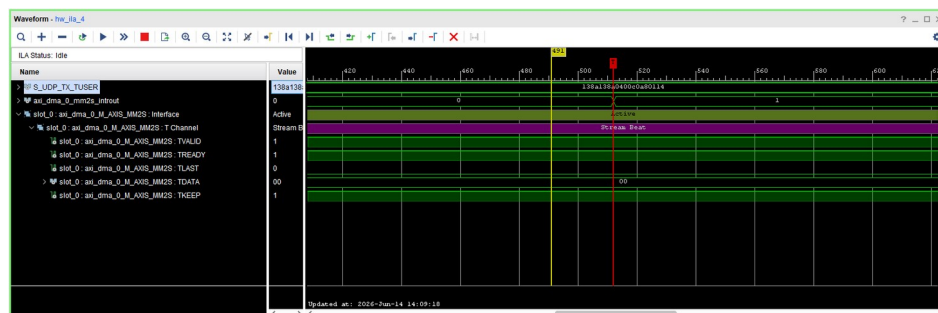
הגדרנו את הטריגר לזיהוי ערך '1' לוגי, כדי לתפוס את הרגע המדויק שבו ה-DMA מתריע למעבד על סיום קליטת חבילה.



לאחר הגדרת הטריגר, הפעלנו את פעולת הדיאגנוסטיקה (Run Trigger). המערכת נמצאת כעת במצב המתנה ("Waiting for Trigger"), מוכנה ללכוד את אות הפסיקה ברגע שיעלה לוגית על ידי בקר ה-DMA, ובכך לאשר את תקינות תהליך העדכון למעבד.

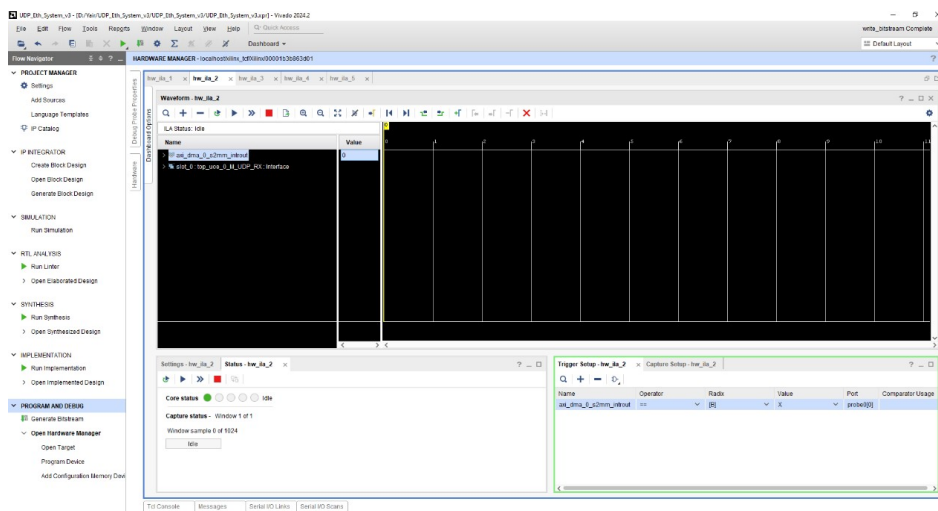


כאן ניתן לראות את אות ה- s2mm_introut במצב לוגי '1'. עליית האות מעידה על כך שה-DMA סיים את העברת החבילה מהממשק לזיכרון ה-BRAM בהצלחה. הפסיקה מתקבלת במעבד, המעבד מאתחל את תהליך הטיפול במידע (Echo), ובכך נסגר המעגל בין קליטת הנתונים בחומרה לבין תגובת התוכנה.

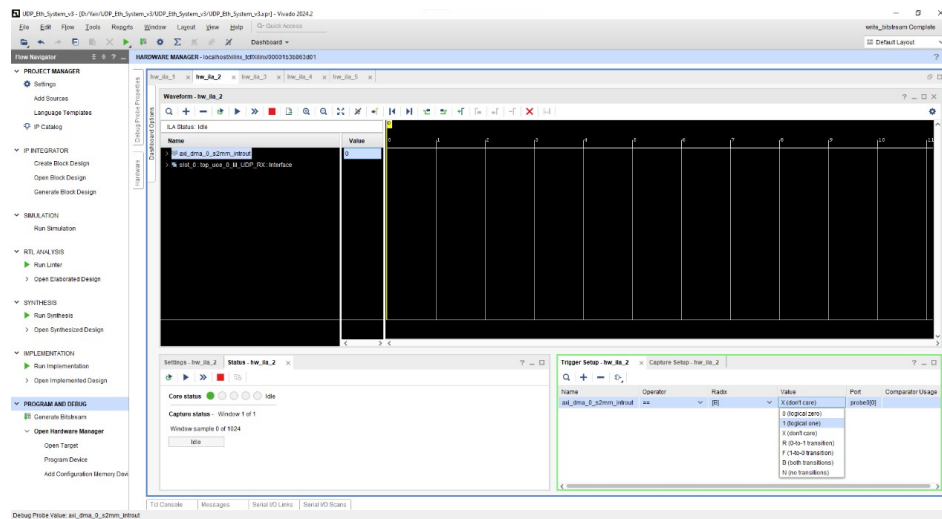


ניתוח מנגנון הפסיקות (Interrupt Analysis):

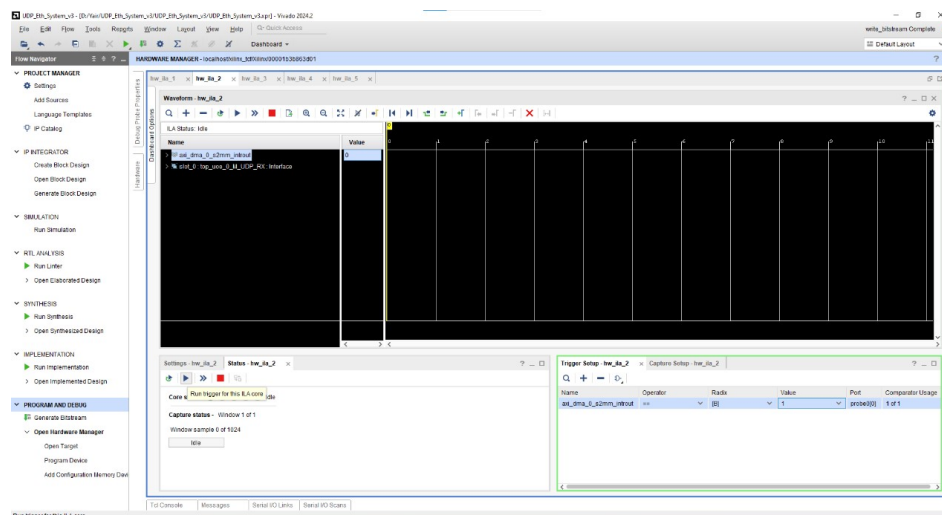
כדי לאמת שהמערכת מעדכנת את המעבד במועד הנכון, הוספנו את אות ה-s2mm_introut למערך ה-ILA. אות זה מהווה את האינדיקציה החומרית לכך שבקר ה-DMA סיים לכתוב את החבילה לזיכרון וממתין לטיפול התוכנה.



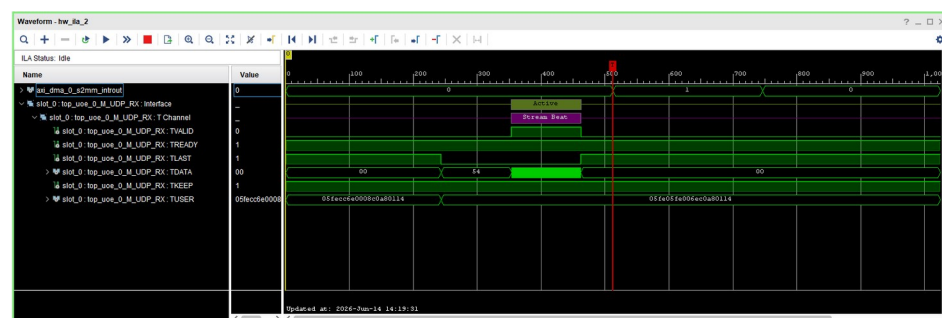
קבענו את ערך הטריגר לאות ה-s2mm_introut להיות '1' לוגי. הגדרה זו מכוונת את ה-ILA "להתעורר" וללכוד את הנתונים בדיוק ברגע שבו עולה הפסיקה, מה שמאפשר לנו לחקור את הקשר הזמני בין פעולת ה-DMA לקבלת האות במעבד.



לאחר הגדרת הטריגר, המערכת נדרכה (Run Trigger) ועברה למצב המתנה פעיל. בשלב זה, ה-ILA דוגם את האפיקים וממתין לעליית אות הפסיקה כדי "להקפיא" את מצב האותות ולשמור את נתוני הדיגום לניתוח.



התמונה מציגה את אות ה-s2mm_introut במצב לוגי '1'. עליית האות מאשרת כי נתוני החבילה נכתבו בהצלחה לזיכרון. ניתן לראות כיצד המערכת משלימה את תהליך הקליטה במקביל ליהוי הפסיקה, מה שמאפשר לתוכנה להפעיל את פונקציית ה-Echo ושלווח את הנתונים חזרה למקור.



שלב 28:

כעת, לאחר שווידאנו את תקינות הפרוטוקול ושלמות נתיבי הנתונים באמצעות כלי הניטור (ILA), נבחן את ביצועי הקצה של המערכת תחת עומס.

בגרף זה רואים את ביצועי המערכת בזמן אמת בזמן הזרמת חבילות UDP. הגענו לקצב ממוצע של כ-850 Mbps, שזה מאוד קרוב למקסימום האפשרי של 1 Gbps. בבדיקה מדדנו קצב שידור (TX) של 860 Mbps וקצב קליטה (RX) של 851 Mbps, עם איבוד מנות יציב של 1%. התוצאות מראות שהמערכת עובדת כמו שצריך ושה-UDP מעביר את הנתונים בצורה חלקה בלי לעצור בדרך.

